

Aula 2 – SBD

Introdução

Profa. Elaine Faria
UFU

Agradecimentos

Este material é baseado

- No livro R. Ramakrishnan e J. Gehrke, *Database Management Systems*
- Nos slides do prof. Bruno Nassif

- Agradecimentos

- Ao professor Bruno Travençolo que gentilmente cedeu seus slides

Motivação

- A quantidade de informação disponível está crescendo exponencialmente
- Os dados e as informações tem um papel importante para as organizações
 - Ex: auxílio na tomada de decisão
- É importante o uso de ferramentas que simplifiquem a tarefa de gerenciar os dados e extrair informações úteis

O que é um SGBD

- SGBD
 - Sistema Gerenciador de Banco de Dados – *Database Management System* (DBMS)
 - Software projetado para armazenar e manipular de forma eficiente grandes quantidades de dados
 - Camada existente entre os dados e os usuários
 - Isola os usuários dos detalhes de *hardware*

O que é um BD

- BD
 - Banco de dados – *Database*
 - Coleção de dados relacionados
 - Tipicamente descrevem as atividades de uma ou mais organizações relacionadas
 - Ex: banco de dados de uma universidade
 - Entidades: alunos, professores, disciplinas, etc.
 - Relacionamento: disciplinas que um professor ministra

SGBD

- Objetivo de um SGBD
 - Gerenciar grandes blocos de informação
 - Definir estruturas para o armazenamento
 - Fornecer mecanismos para a manipulação de informações
 - Garantir a segurança das informações

Aplicações do Sistema de Banco de Dados

- Banco
 - Informações de contas, clientes, transações bancárias, empréstimos, etc.
- Linhas Aérea
 - Reservas e informações de horários
- Universidade
 - Informações de alunos, registros de curso e notas

Aplicações do Sistema de Banco de Dados

- Vendas
 - Informações de clientes, produto e compra
- Indústria
 - Gerenciamento da cadeia de suprimentos e controle da produção de itens na fábrica, estoque e pedidos
- Recursos Humanos
 - Funcionários, salários, descontos, benefícios, etc.

Dado X Informação

- *Dado*
 - É o valor do campo quando é armazenado no Banco de Dados
 - Ex. O valor do campo "nome do cliente" para quem está fazendo a entrada de dados
- *Informação*
 - É o valor que este campo representa para as atividades da empresa
 - Ex. Resposta a uma consulta. Quais os nomes dos clientes localizados no Rio de Janeiro?

Sistema de Arquivos x SGBD

- Arquivos
 - Redundância e inconsistência de dados
 - Dificuldade de acesso a dados
 - Isolamento de dados
 - Problemas de integridade
 - Problemas de atomicidade
 - Anomalias de acesso concorrente
 - Escalabilidade
 - Problemas de segurança

Inconsistência e redundância de dados

- Arquivos e aplicações criadas por diferentes programadores
 - Arquivos com formatos diferentes
 - Programas escritos em diversas linguagens de programação
- Redundância de dados
 - Ex: o telefone de um cliente pode aparecer em mais de um arquivo
 - Aumento no custo de armazenamento e acesso
- Inconsistências de dados
 - As cópias podem divergir ao longo do tempo
 - Ex: a mudança do telefone de um cliente pode ocorrer somente em um dos arquivos

Dificuldade de acesso aos dados

- Cada nova consulta diferente das previstas inicialmente envolve a criação de novos programas para realizá-las
 - Ex: encontre os cliente que moram em uma cidade cujo CEP é 12345-234
 - Ex: encontre os cliente que possuem saldo > 10.000,00
 - ...
 - Ou seja, cada nova requisição exige uma nova implementação, um novo programa

Isolamento de dados

- Dados dispersos em vários arquivos com diferentes formatos aumenta a dificuldade de escrever novas aplicações para recuperação apropriada dos dados

Problemas de integridade

- Os valores dos dados devem satisfazer certas restrições para manutenção da *consistência*.
 - Ex:
 - Restrições feitas adicionando-se códigos aos vários programas de aplicações
 - Difícil a implementação de novas restrições – podem envolver vários programas ou diversos itens de dados em diferentes arquivos

Problemas de atomicidade

- Os sistemas estão sujeitos as falhas
- As aplicações devem assegurar após a detecção de uma falha os dados sejam salvos em seu último estado consistente, anterior a ela.
 - Ex: Transferir R\$ 50,00 da conta A para a conta B
 - É possível que seja feito o débito em A e que o crédito em B não se realize por causa de uma falha, criando assim um estado inconsistente
- As operações devem ser atômicas – deve ocorrer por completo ou não ocorrer
- Difícil garantir essa propriedade em um sistema convencional de processamento de arquivos

Acesso concorrente

- Vários sistemas permitem a manipulação simultânea (concorrente) aos dados
- Interação entre atualizações concorrentes pode resultar em inconsistência dos dados
 - Ex: Saldo de uma conta: R\$ 500,00
 - Dois cliente retiram, ao mesmo tempo, 50 e 100 reais.
 - O sistema lê, nos dois casos, que o saldo é R\$ 500,00
 - Após as retiradas, o saldo pode fica em R\$ 450,00 ou R\$ 400,00 ao invés de R\$ 350,00
- O sistema deve supervisionar esse tipo de operação – o que é difícil caso diferentes programas acessem o mesmo dado

Escalabilidade

- Crescimento da quantidade de dados pode tornar o sistema ineficaz
- Dificuldade de implementação de novas consultas e restrições

Segurança

- Nem todos os usuários de um BD estão autorizados a acessar todos os dados nele contido
 - RH de um banco deve ter acesso somente aos dados relativos aos funcionários, não aos clientes
- Difícil garantir a efetividade das regras de segurança

Solução: SGBD

- Utilizar um SGBD para gerenciamentos dos dados oferece diversas vantagens:
 - Independência dos dados
 - O SGBD provê uma visão abstrata dos dados, de forma que um programa aplicativo não é exposto a detalhes de representação e armazenamento de dados
 - Acesso eficiente aos dados
 - O SGBD dispõe de uma variedade de técnicas sofisticadas para armazenamento e recuperação eficiente de dados, incluindo dados armazenados em dispositivos externos
 - Integridade dos dados e segurança
 - Se os dados forem acessados pelo SGBD é possível garantir restrições de integridade e também o controle de acesso aos dados para diferentes classes de usuários

Solução: SGBD

- Utilizar um SGBD para gerenciamentos dos dados oferece diversas vantagens (cont.):
 - Acesso concorrente e recuperação de falhas
 - Um SGBD garante a consistência no acesso concorrente e recuperação de falhas por meio da atomicidade das operações
 - Administração dos dados
 - Centralização da administração do banco de dados – profissional responsável para organização da representação dos dados para minimizar a redundância e fazer ajustes finos para melhorar o desempenho
 - Redução do tempo de desenvolvimento de aplicativos
 - Disponibilidade de diversas funções para acessos aos dados bem como uma interface de alto nível

SGBD

- Oracle; PostgreSQL; MySQL; Sybase; MS SQL Server; MS Access; Firebird
- Sistema de Banco de Dados (SBD)
 - $SBD = BD + SGBD + \text{Aplicação}$

- Há alguma razão para não usar um SGBD?
 - Sim!

Seu desempenho pode não ser adequado para aplicações em tempo real

Manipular dados em um formato não suportado por uma linguagem de consulta

Por que estudar Banco de Dados?

- Banco de dados estão crescendo em diversidade e volume
 - Bibliotecas digitais, vídeos interativos, projeto genoma,...
- SGBDs envolvem pesquisas de diferentes áreas da Ciência da Computação
 - Sistemas Operacionais, Linguagens, Teoria da Computação, Lógica, Inteligência Artificial e Multimídia

Usuários de um SGBD

- Administrador do BD (*Database Administrator – DBA*)
 - coordena e monitora o uso do BD (segurança, tempo de resposta ruim, etc.)
 - autoriza o acesso ao BD
 - *ajusta* projeto físico quando necessário
 - *adquire software e hardware* necessários
- Projetista do BD
 - identifica os dados a serem armazenados no BD
 - escolhe as estruturas apropriadas para representar e armazenar esses dados
 - levanta necessidades dos usuários (requisitos)

Usuários de um SGBD

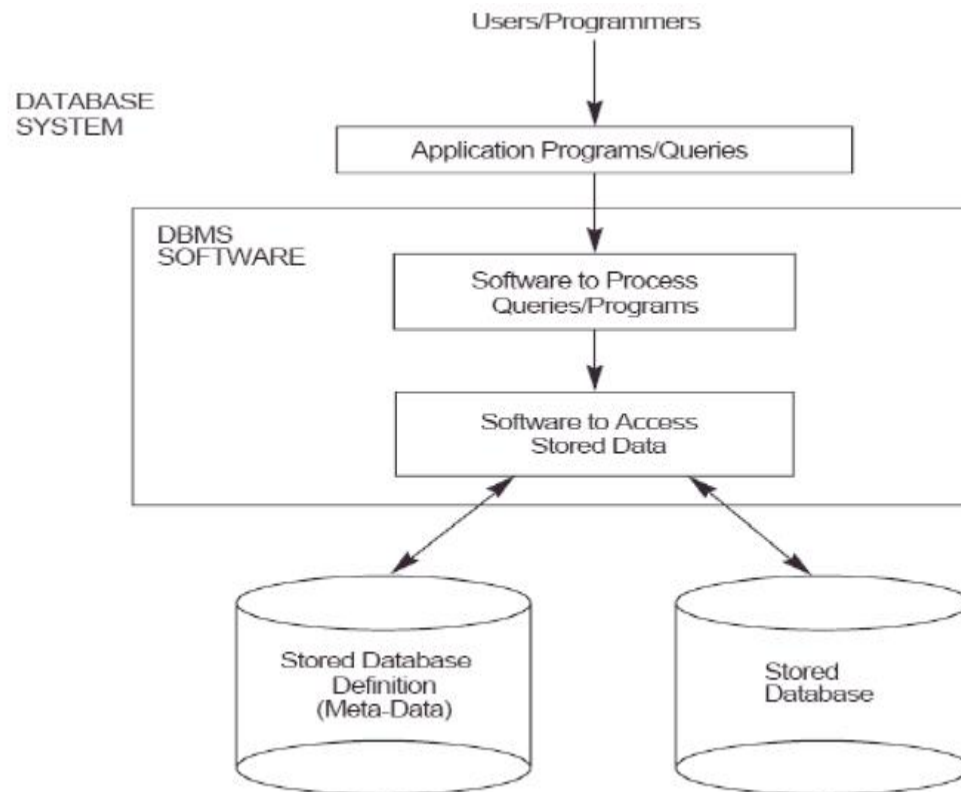
- Programador de aplicações
 - escreve os programas aplicativos
 - realiza requisições ao SGBD
- Usuário final
 - manipula o BD através de
 - linguagens de consulta
 - programas previamente desenvolvidos
 - tipos de usuários
 - Leigos (operador de terminal) X sofisticados (engenheiros)
 - Casuais (gerentes) X freqüentes (caixas de bancos)

Banco de Dados na era da Internet

- Banco de dados multimídia
- Vídeos interativos
- Dados em fluxo
- Bibliotecas digitais
- Dados de projetos científicos (projeto genoma)
- Ferramentas de auxílio à tomada de decisão → *Data Mining*

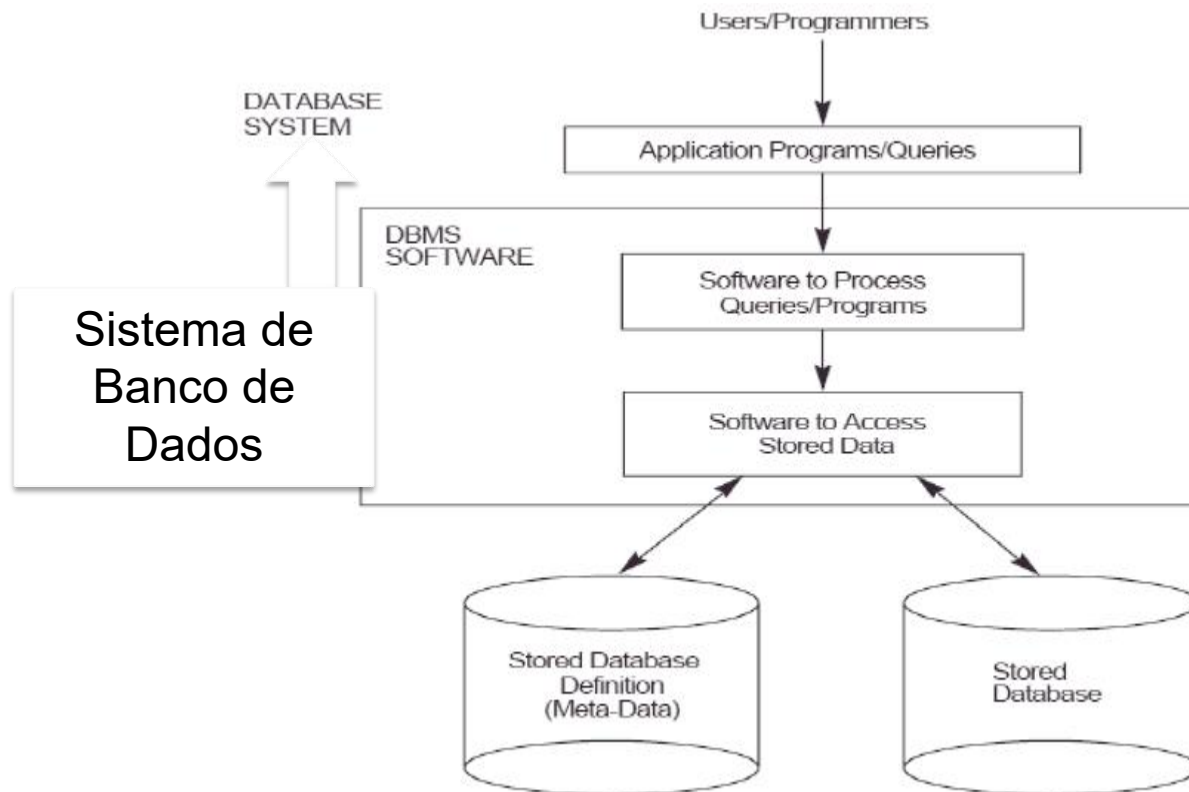
Fundamentos

- Sistema de Banco de Dados (SBD)



Fundamentos

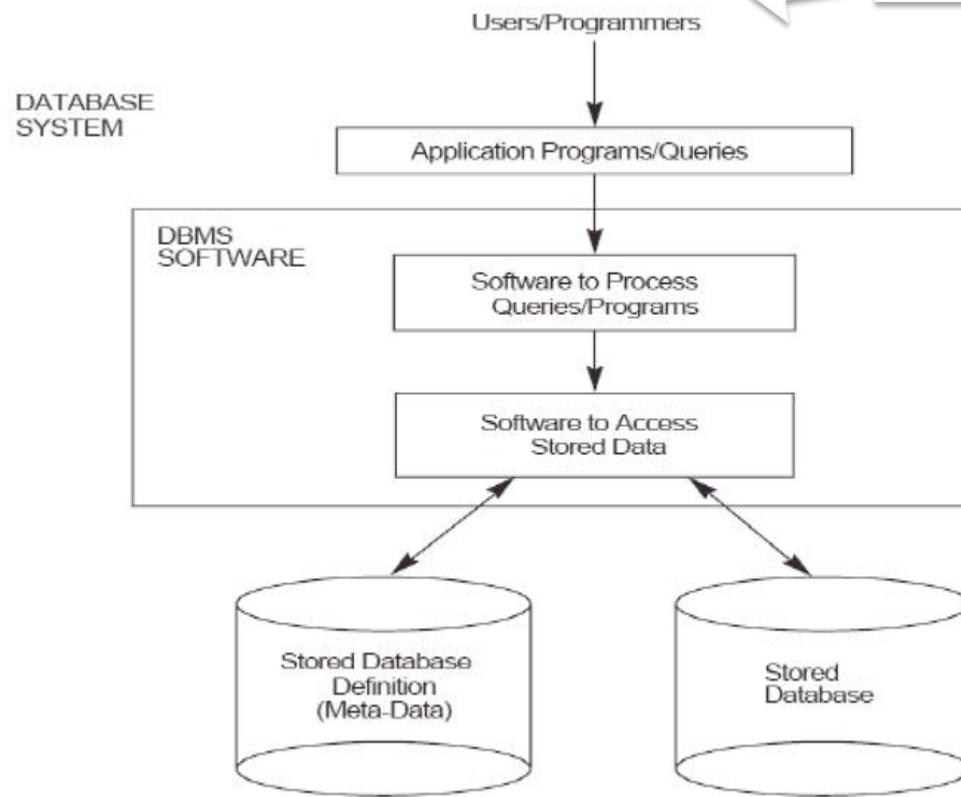
- Sistema de Banco de Dados (SBD)



Fundamentos

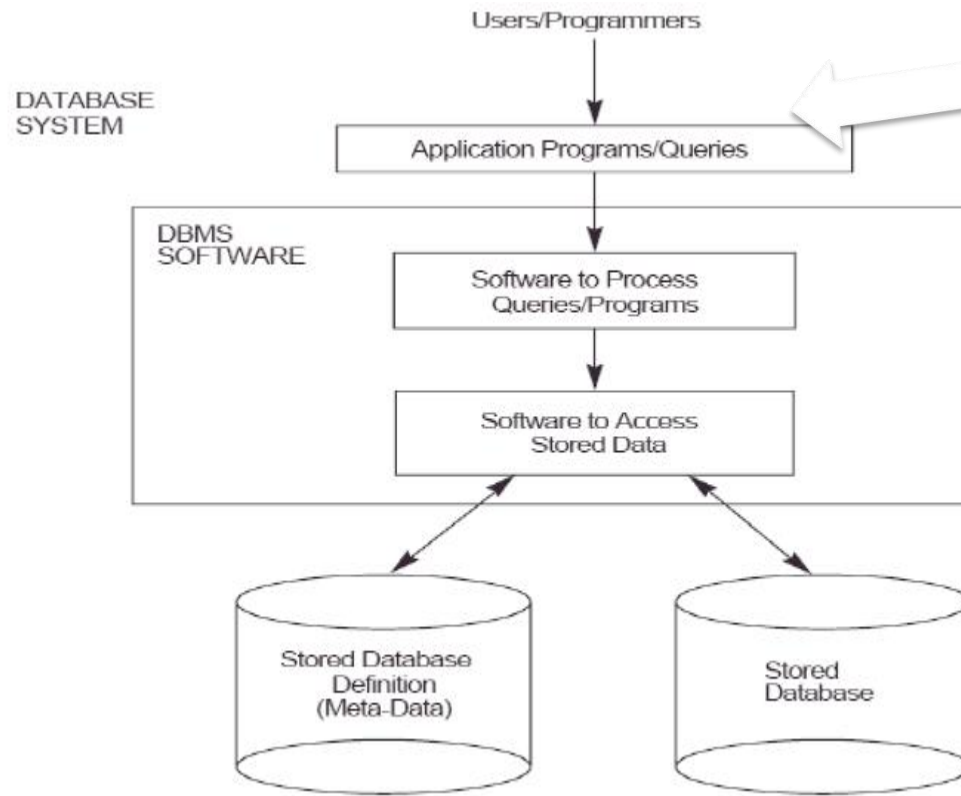
- Sistema de Banco de Dados

Usuários ou programadores



Fundamentos

- Sistema de Banco de Dados (S

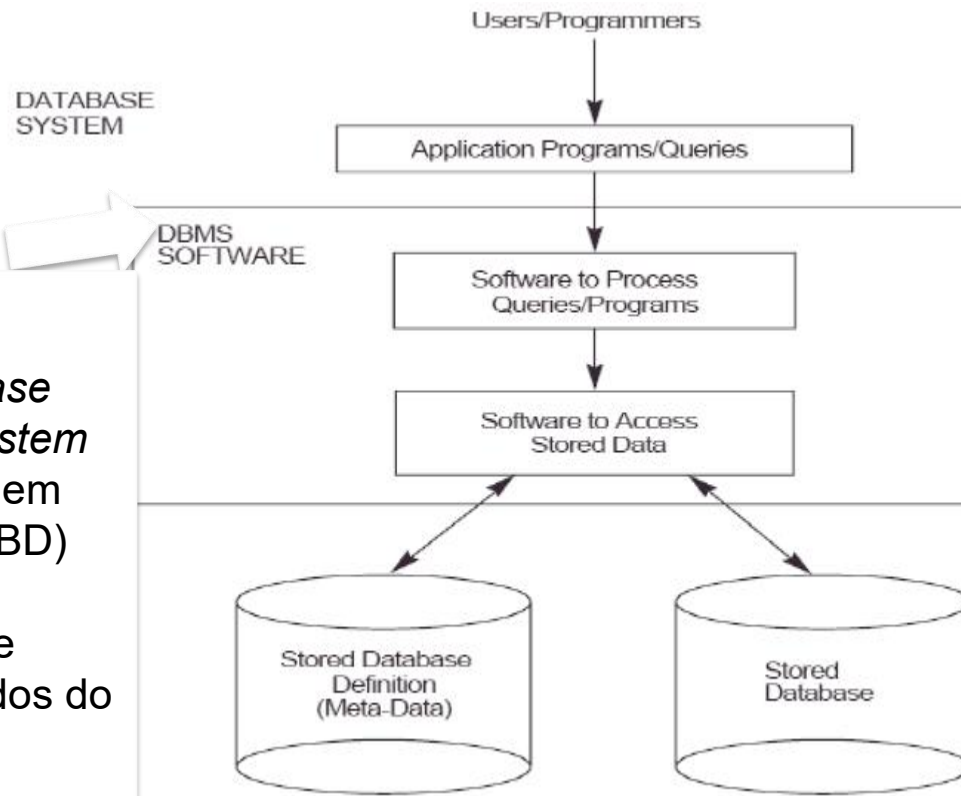


Programas Aplicativos:
software desenvolvidos em
diferentes linguagens

Queries: consultas –
obtenção/modificação de
dados do banco ou da
organização do banco

Fundamentos

- Sistema de Banco de Dados (SBD)

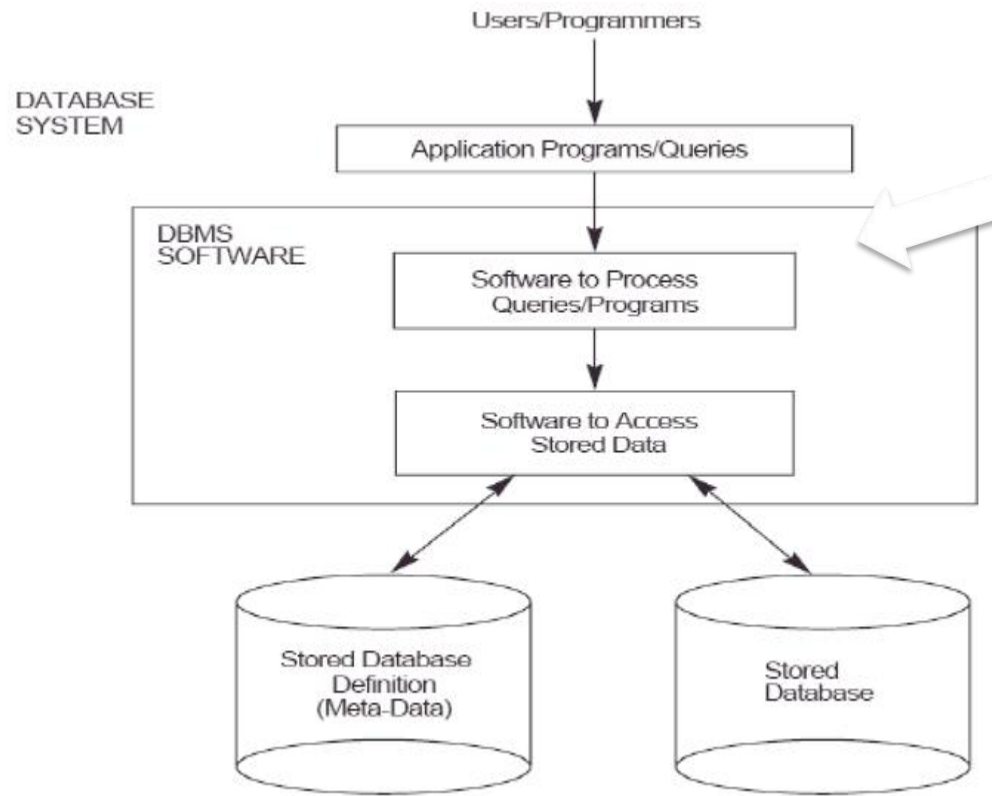


DBMS: *database management system*
(termo original em inglês para SGBD)

Software que gerenciará os dados do sistema

Fundamentos

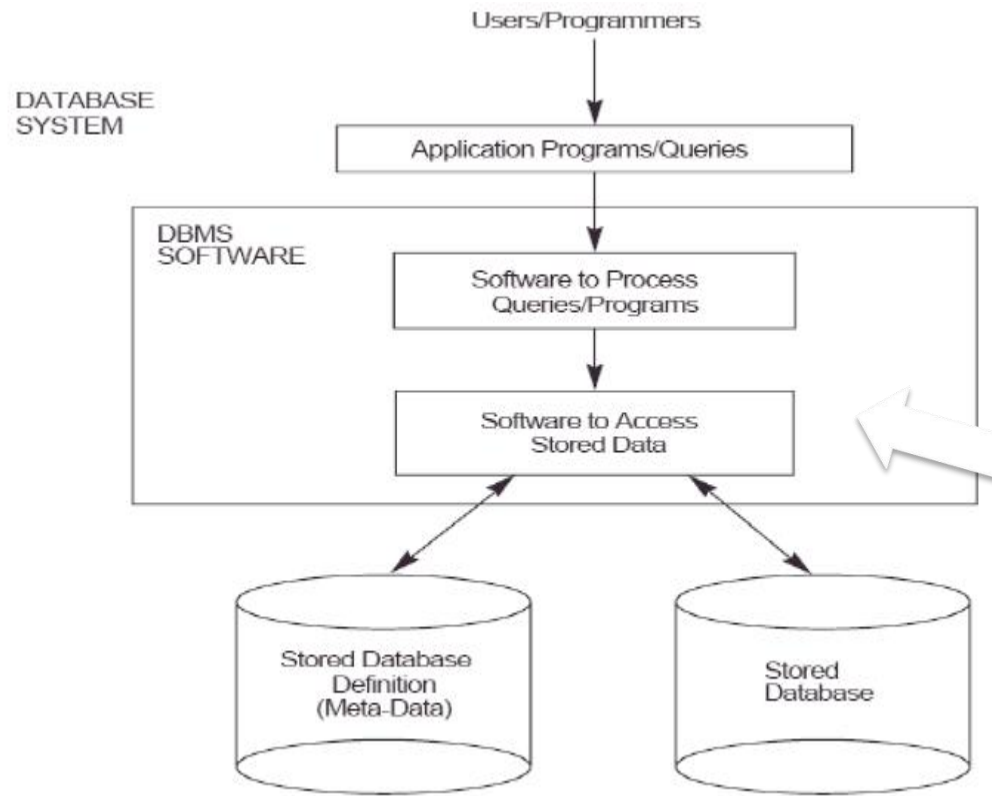
- Sistema de Banco de Dados (SBD)



Uma parte do SGBD é feita para processar as consultas, ou seja, entender o que o software aplicativo está solicitando

Fundamentos

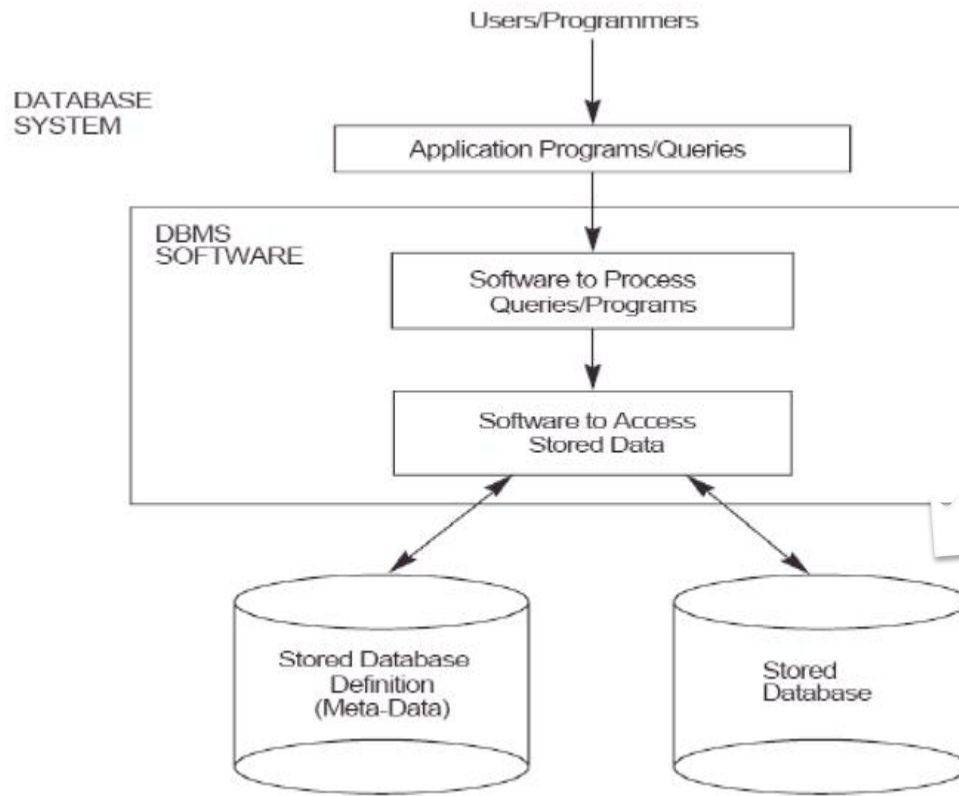
- Sistema de Banco de Dados (SBD)



Outra parte vai conter operações de entrada e saída em arquivos armazenados em disco, utilizando para isso diferentes estruturas de dados

Fundamentos

- Sistema de Banco de Dados (SBD)



Stored Database:
Banco de dados armazenado – são os dados que estão guardados no banco (e.g., “José da Silva”, 222.653.365-95)

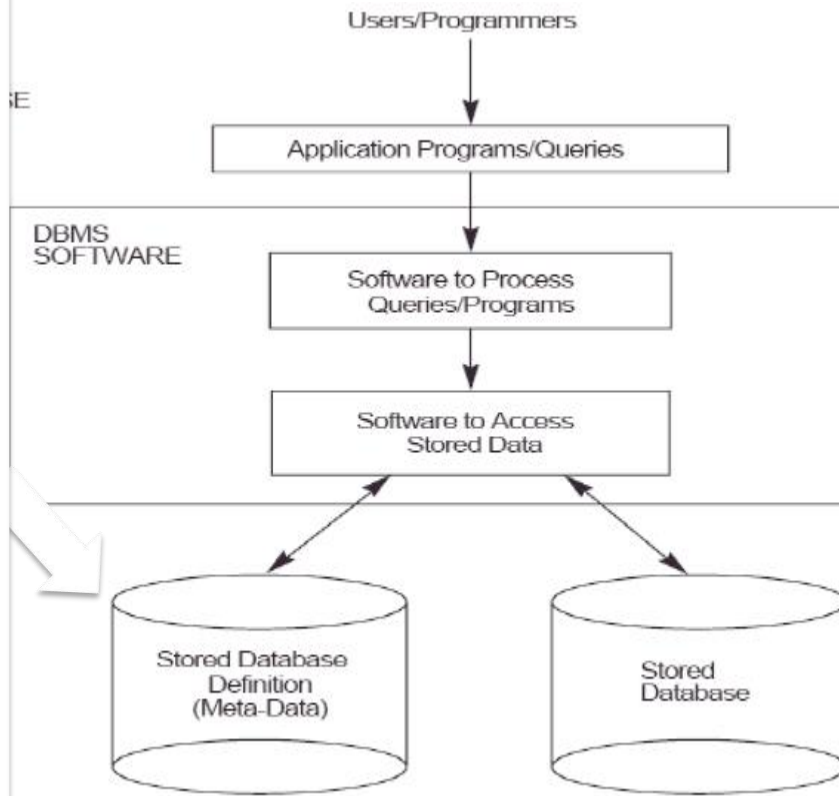
Obs: é comum usar um desenho de um cilindro para indicar armazenamento de dados em diagramas

Fundamentos

Sistema de Banco de Dados (SBD)

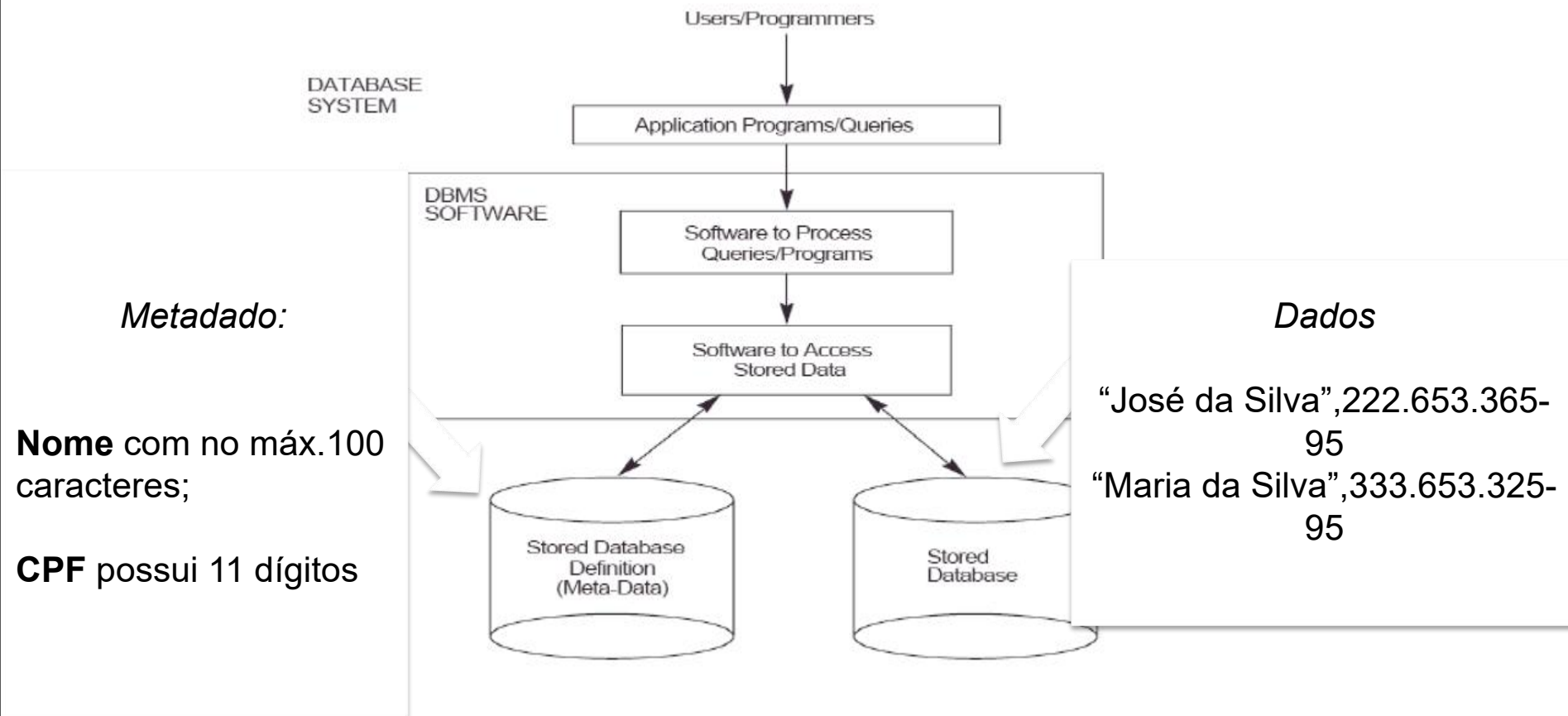
Stored Database Definition (Meta-Data):
Armazenamento da Definição do Banco de dados (metadados).

Metadados são as informações que indicam quais tipos, estruturas e restrições armazenados no banco.
Exemplo: Campos nome com 100 caracteres; CPF possui 11 dígitos



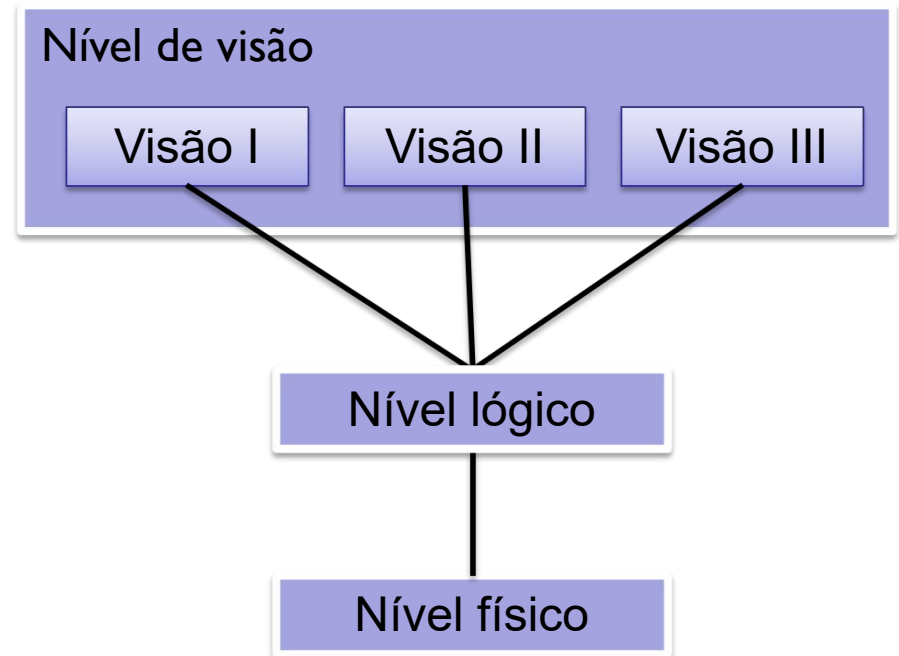
Fundamentos

- Sistema de Banco de Dados (SBD)



Níveis de Abstração

- Nível físico, lógico e de visão
 - Visões descrevem como os usuários vêem os dados
 - O nível lógico descreve quais dados estão descritos no BD e as suas relações
 - O nível físico descrevem os como os dados estão armazenados e suas ED



Independência de dados

- Modificação da definição de um esquema em um nível sem afetar o nível mais alto
- Dois tipos
 - Independência lógica de dados
 - Alterações no nível lógico não afetam os programas aplicativos
 - Essas alterações podem ter o objetivo de alterar a estrutura do BD
 - Independência física de dados
 - Alterações nas estruturas dos arquivos e índices não modificam o nível lógico
 - Essas alterações podem ter o objeto de melhorar o desempenho do sistema

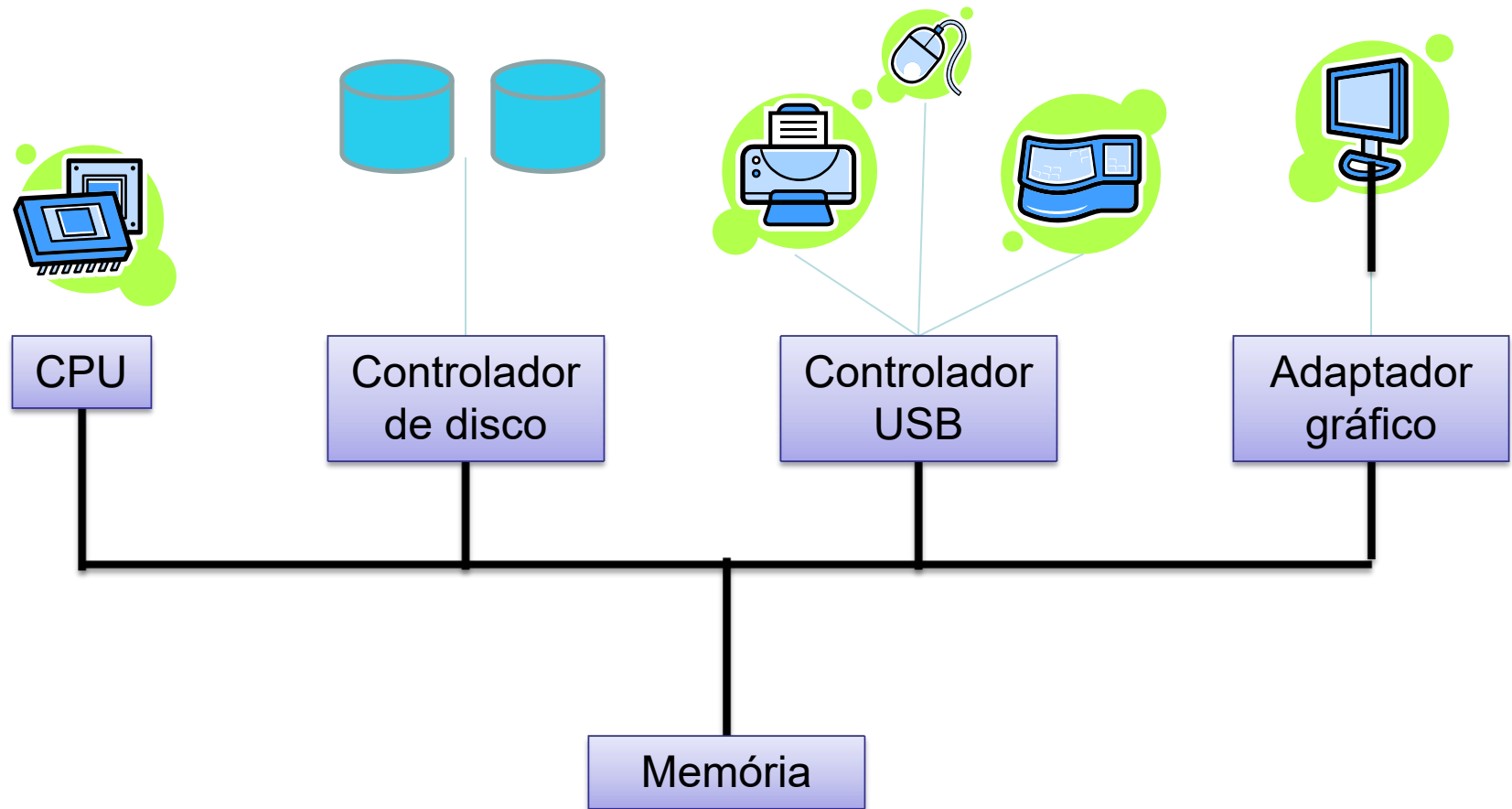
Arquitetura dos sistemas de BD

- A arquitetura é fortemente influenciada pelo sistema básico computacional sobre o qual o sistema de banco de dados é executado
- Aspectos da arquitetura do computador – como rede, paralelismo e distribuição – têm influência na arquitetura do banco de dados

Sistemas Centralizados

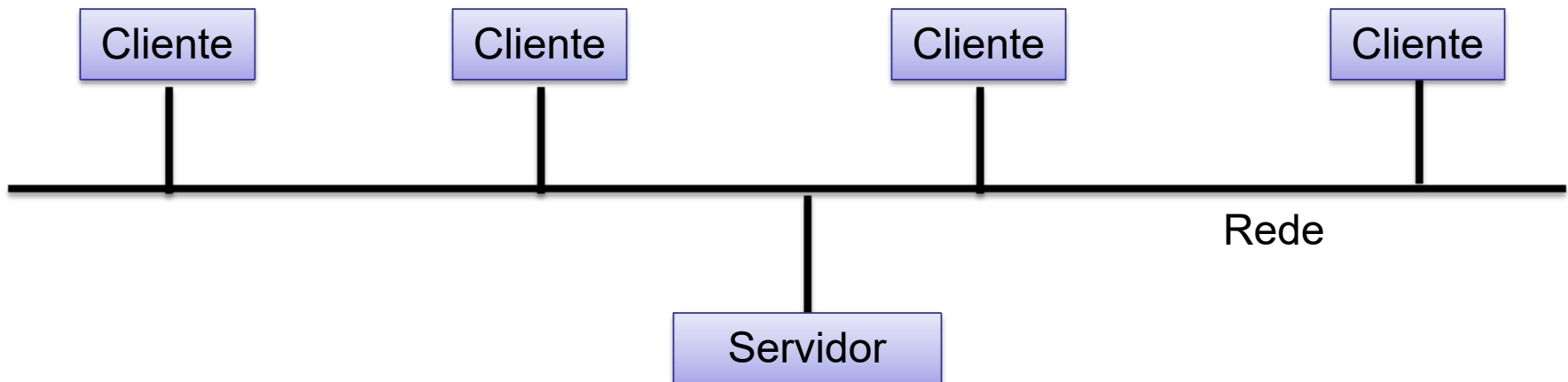
- São executados sobre um único sistema computacional que não interagem com outros sistemas.
- Sistema monousário: uma unidade de trabalho de um única pessoa; única CPU; poucos discos; SO monousuário (ex. PC).
- Sistema multiusuário: mais discos, mais memória, múltiplas CPUs; SO multi-usuário. Serve um alto número de usuários que se conectam via terminais.
Normalmente chamados de sistemas *servidor*

Um Sistemas Computacional Centralizado



Sistema Cliente-Servidor

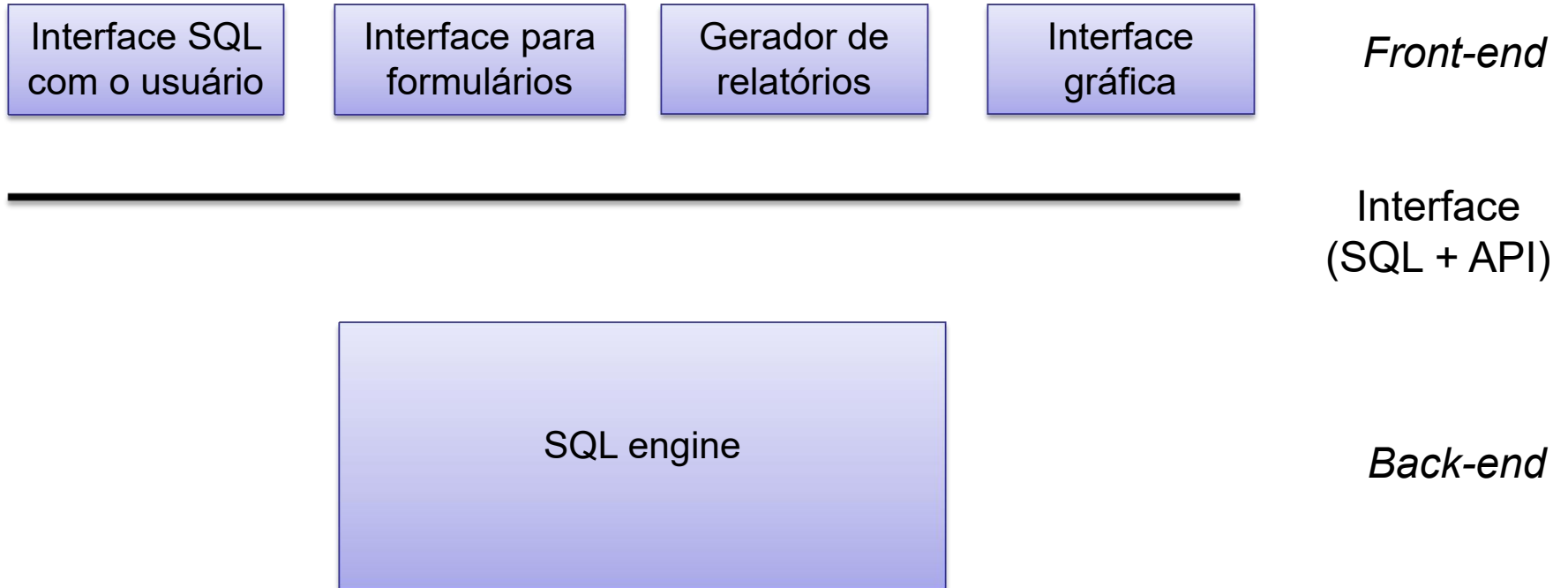
- Sistemas servidores atendem aos requisitos de vários sistemas clientes
- Estrutura geral de um sistema cliente-servidor:



Sistema Cliente-Servidor

- Funcionalidades de um banco de dados podem ser divididas em:
 - *Back-end* – gerencia estruturas de acesso, desenvolvimento e otimização de consultas, controle de concorrência e recuperação
 - *Front-end* – consiste de ferramentas como gerador de relatórios, formulários e recursos de interface gráfica
 - A interface entre essas funcionalidades é feita pela SQL ou de um programa de aplicação

Sistema Cliente-Servidor

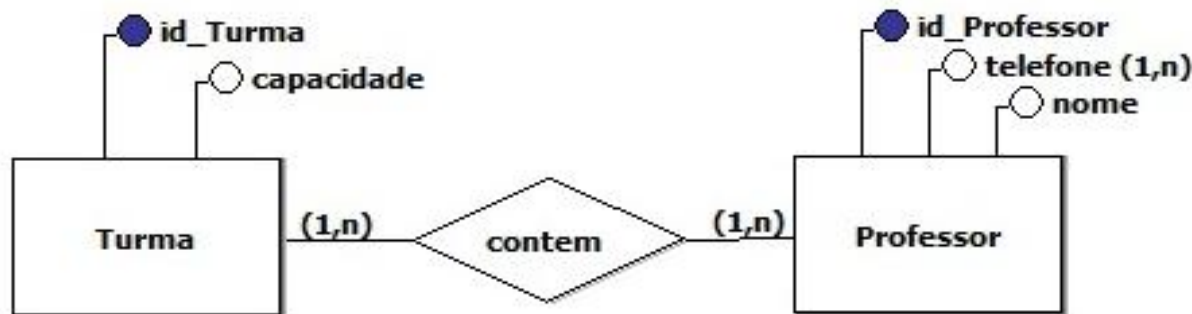


Modelo de dados

- Conjunto de ferramentas conceituais usadas para a descrição dos dados, relacionamento entre os dados, semântica de dados e regras de consistência

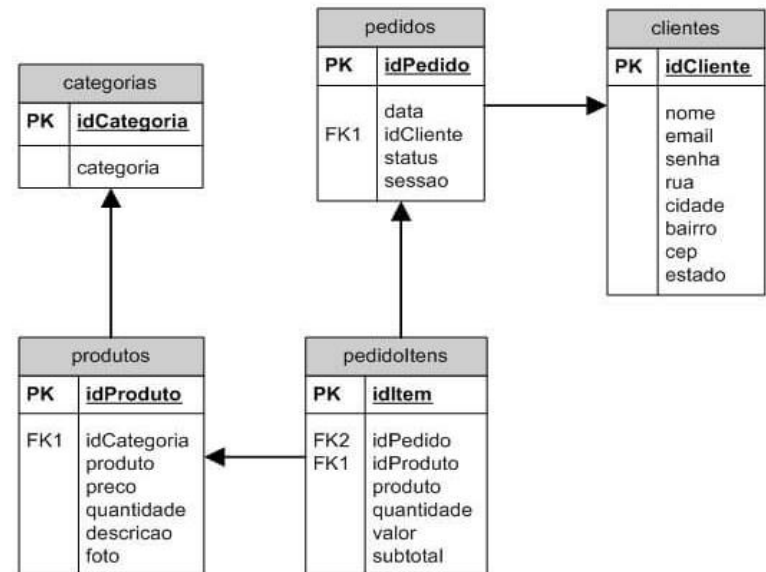
Modelo de dados

- Modelos de Dados (conceitual)
 - Alto nível e deve ser usada para envolver o cliente
 - Coleta de informações sobre as necessidades de negócios
 - Ex:
 - Entidade-Relacionamento (ER)
 - Orientado a Objetos (OO)



Modelo de dados

- Modelos de Dados (lógicos)
 - Leva em conta algumas limitações e implementa recursos como:
 - adequação de padrão e nomenclatura,
 - chaves primárias e estrangeiras,
 - Normalização, integridade referencial, etc;
 - Deve ser criado levando em conta modelo conceitual
 - Ex:
 - Redes
 - Hierárquico
 - **Relacional**
 - Orientado a Objetos



Modelo Relacional

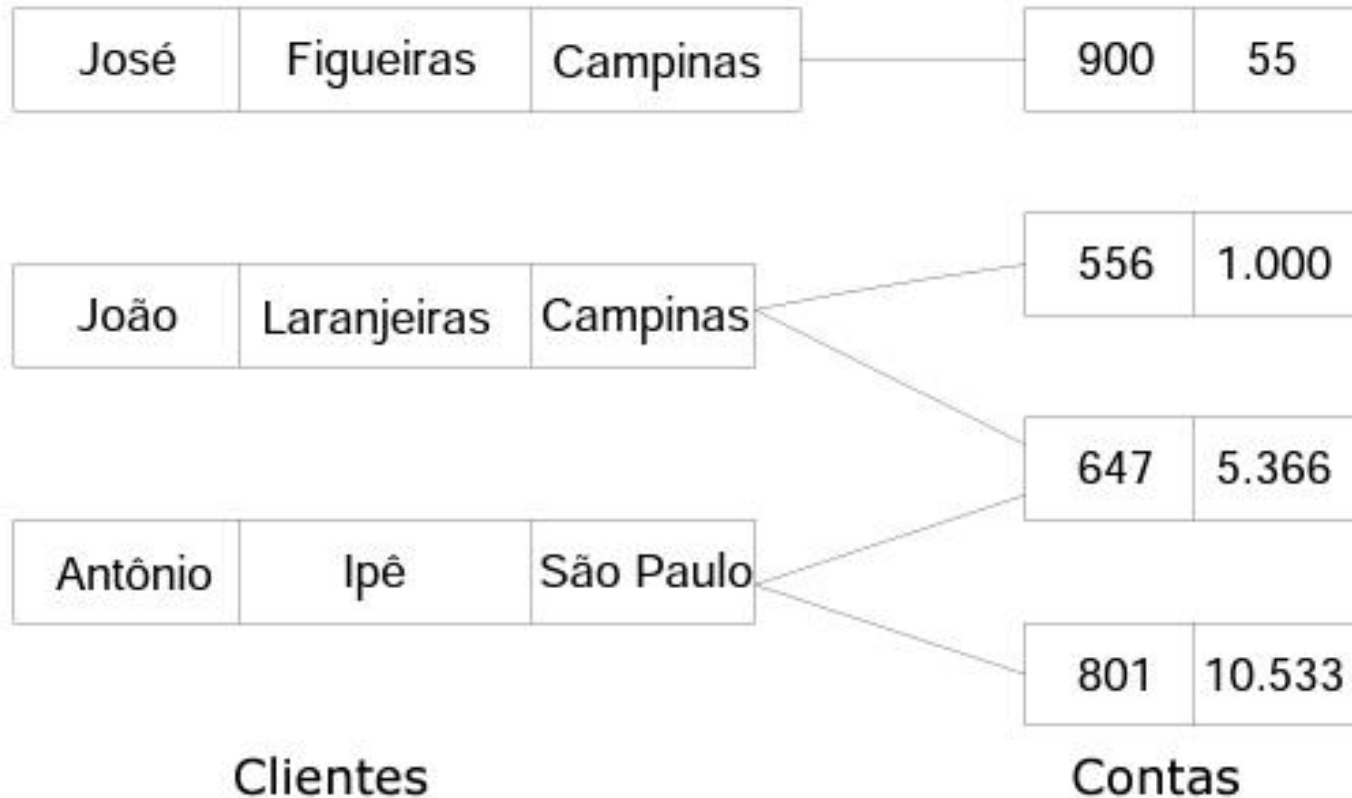
- Conjunto de tabelas são usadas para representar tanto os dados como a relação entre eles

Sigla	Nome	Créditos
SMA0304	Álgebra Linear	4
SCC0503	Algoritmos e Estruturas de Dados II	6
SME0101	Cálculo Numérico II	4
SSC0110	Elementos de Lógica Digital I	4
SCC0240	Banco de Dados	4

Modelo de rede

- Dados são representados como um conjunto de registros (como em Pascal) e as relações entre esses registros são representadas por links (“ponteiros”)
- Os registros são organizados no BD por um conjunto arbitrário de grafos.

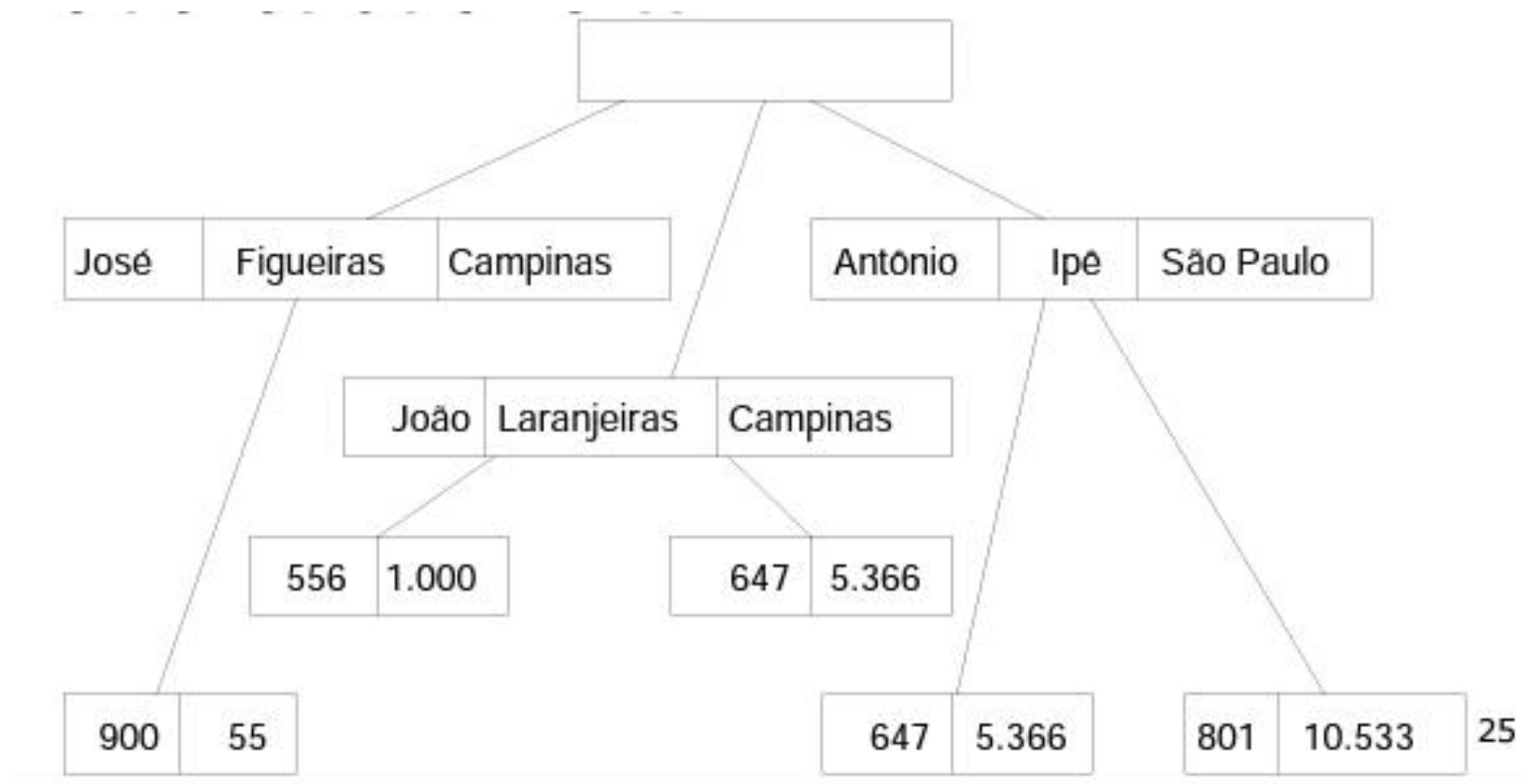
Modelo de rede



Modelo Hierárquico

- Os dados e suas relação são representados por registros e links, respectivamente.
- Registros estão organizados em árvores, ao invés de grafos arbitrários

Modelo Hierárquico



Modelo de dados

- Modelos de Dados (físico)
 - Modelagem física do modelo de banco de dados
 - Leva-se em conta as limitações impostas pelo SGBD escolhido

```
CREATE TABLE turma (  
    idturma INTEGER(4) NOT NULL AUTO_INCREMENT,  
    capacidade INTEGER(2) NOT NULL,  
    idProfessor INTEGER(4) NOT NULL,  
    PRIMARY KEY (idturma),  
    FOREIGN KEY (idProfessor) REFERENCES professor(idProfessor),  
    UNIQUE KEY idturma (idturma)  
)  
  
CREATE TABLE professor (  
    idProfessor INTEGER(4) NOT NULL AUTO_INCREMENT,  
    telefone INTEGER(10) NOT NULL,  
    nome CHAR(80) COLLATE NOT NULL DEFAULT '',  
    PRIMARY KEY (idProfessor),  
    FOREIGN KEY(idTurma) REFERENCES turma(idturma),  
    UNIQUE KEY idProfessor (idProfessor)  
)
```

Referências

- R. Ramakrishnan e J. Gehrke, *Database Management Systems*, 3a Edição, McGraw-Hill, 2003.

Aula 3 – BD 1

Modelo Entidade Relacionamento – Parte 1

Profa. Elaine Faria
UFU

Agradecimentos

Este material é baseado

- No livro R. Ramakrishnan e J. Gehrke, *Database Management Systems*
- Nos slides do prof. Bruno Nassif

- Agradecimentos

- Ao professor Bruno Travençolo que gentilmente cedeu seus slides

Processo do Projeto de um Banco de Dados

- A criação de uma aplicação de banco de dados envolve várias tarefas
 - Projeto do esquema de banco de dados
 - Projeto dos programas que acessam e atualizam os dados
 - Projeto de um esquema de segurança para controlar o acesso aos dados

Processo do Projeto de um Banco de Dados

O projetista de banco de dados interage com os usuários da aplicação para entender as necessidades e representá-las de uma maneira alto nível

Projeto de um Banco de Dados

- Passos

- Análise dos requisitos

- Compreender quais dados serão armazenados no banco de dados
 - Compreender quais aplicações serão construídas usando esses dados
 - Compreender quais operações são mais frequentes

O que os usuários querem do banco de dados?

Projeto de um Banco de Dados

- Passos

- Projeto do banco de dados conceitual

- As informações obtidas no passo de análise são usadas para desenvolver uma descrição de alto nível dos dados
 - Usa-se o modelo ER

Facilitar a discussão entre todas as pessoas envolvidas

Projeto de um Banco de Dados

- Passos
 - Projeto lógico do banco de dados
 - Escolher um SGBD para implementar o banco de dados
 - Converter o projeto conceitual em um projeto lógico

Conversão ER → Relacional

Projeto de um Banco de Dados

- Passos
 - Refinamento do esquema
 - Analisar o conjunto de relações para identificar problemas e refiná-las

Normalização das relações!

Projeto de um Banco de Dados

- Passos

- Projeto físico do banco de dados

- Verificar a carga de trabalho que o banco de dados irá suportar e fazer refinamentos para assegurar os critérios de desempenho
 - Envolve a criação de índices em algumas tabelas ou re-projetar algumas partes do banco de dados

Projeto de um Banco de Dados

- Passos
 - Projeto das aplicações e segurança
 - Identificar as partes do banco de dados que devem estar acessíveis e as que não devem estar acessíveis
 - Garantir que as regras de acesso são executadas

Modelo Entidade-Relacionamento

- O modelo ER
 - Descrever os dados de aplicações do mundo real em termos de objetos (entidades) e seus relacionamentos
 - É largamente utilizado para o desenvolvimento da fase inicial do projeto de BD
 - Fornece conceitos para partir de uma descrição informal dos usuários obter uma descrição mais detalhada

Modelo Entidade-Relacionamento

- O modelo ER
 - É a técnica de modelagem mais difundida e utilizada
 - Foi criada em 1976 por Peter Chen
 - É um modelo de dados conceitual de alto nível
 - Compreensível a usuários
- Diagrama Entidade-Relacionamento(DER)

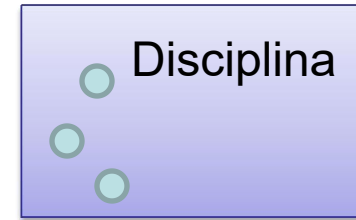
Entidade

- É um objeto do mundo real distinguível de outros objetos
 - Ex: cada pessoa em uma empresa é uma entidade
 - Ex: o gerente do departamento de vendas
- Pode ser um objeto com existência física ou conceitual
- Descrito por propriedades → atributos

Entidade

- Coleção de entidades semelhantes → Conjunto de entidades (Tipo-Entidade)
 - Ex: os empregados de uma empresa são as *entidades* de um *conjunto de entidades* denominado Empregado
- Entidades de um mesmo conjunto de entidades
 - Compartilham atributos
 - É normal referenciar todas as entidades pelo mesmo nome do tipo_entidade. Ex: Empregado

MER - Entidades



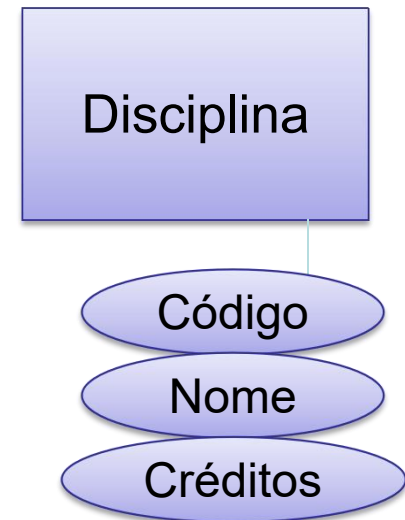
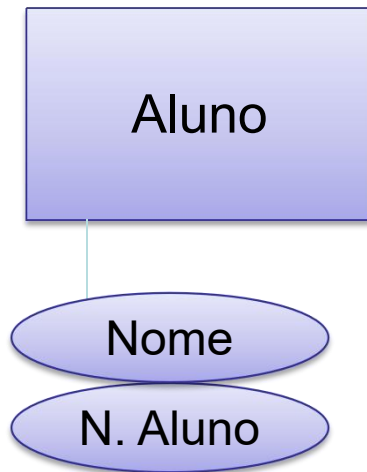
- Utiliza-se um retângulo para representação das entidades no diagrama E-R
- Observe que são *conjunto de entidades* que são consideradas pelo modelo E-R e não as entidades individuais

Atributos

- Usados para descrever um conjunto de entidades ou de relacionamentos
 - Ex: o conjunto de entidades Empregado pode ter os seguintes atributos
 - Nome
 - Matrícula
 - Sexo
 - Idade
 - Endereço

Obs.: todas as entidades em um dado conjunto de entidades têm os mesmos atributos

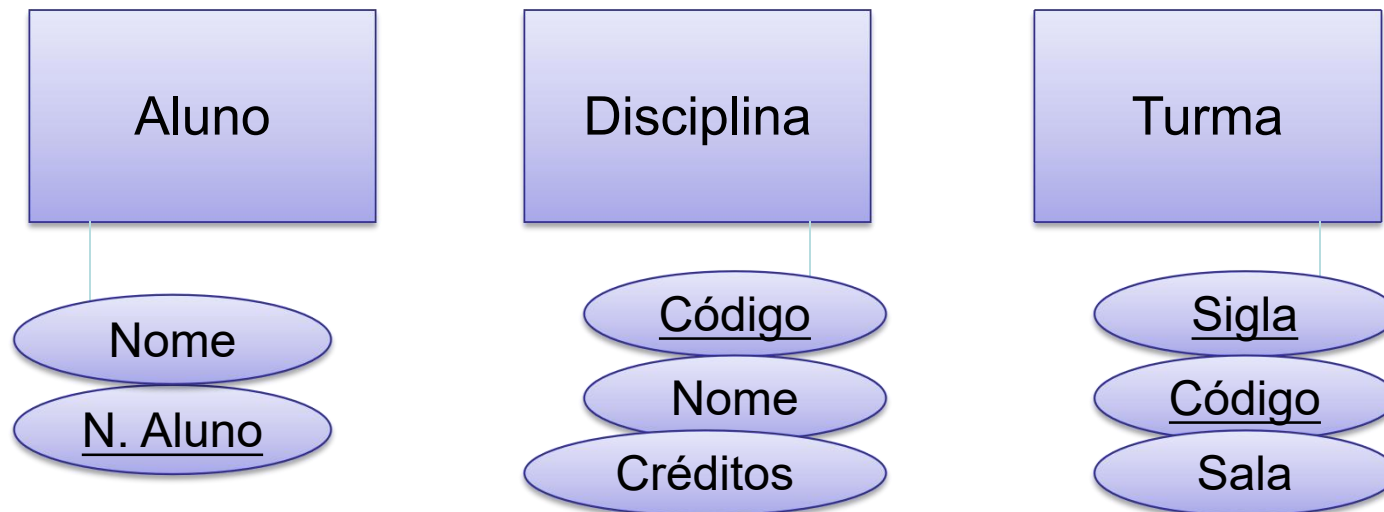
MER - Atributos



- Os atributos nos diagramas E-R são representados por elipses ligadas aos conjunto de entidades
- Um conj. de entidades sem atributos não tem sentido em uma modelagem

MER – Restrições de chaves

- Cada entidade deve ser identificada univocamente* dentro do conjunto de entidades, por meio de um ou mais atributos (chave)
- A **chave** é representada por um sublinhado no atributo correspondente
- Mais de um atributo pode compor a chave



*unívoco – inequívoco, não ambíguo

MER- observações sobre representação por chaves

- Livro Elmasri/Navathe
 - “Uma restrição importante das entidades de um tipo entidade é a chave ou restrição de unicidade em atributos. Um tipo entidade tem, geralmente, um atributo cujos valores são distintos para cada uma das entidades do conjunto de entidade”
 - “Algumas vezes, diversos atributos juntos formam uma chave, significando que a combinação dos valores dos atributos deve ser distinta para cada entidade”
 - “Se um conjunto de atributos [for chave], a forma adequada de representação no modelo ER, que utilizaremos, será a definição de um atributo composto”
 - “Cada atributo chave tem seu nome sublinhado dentro da elipse”

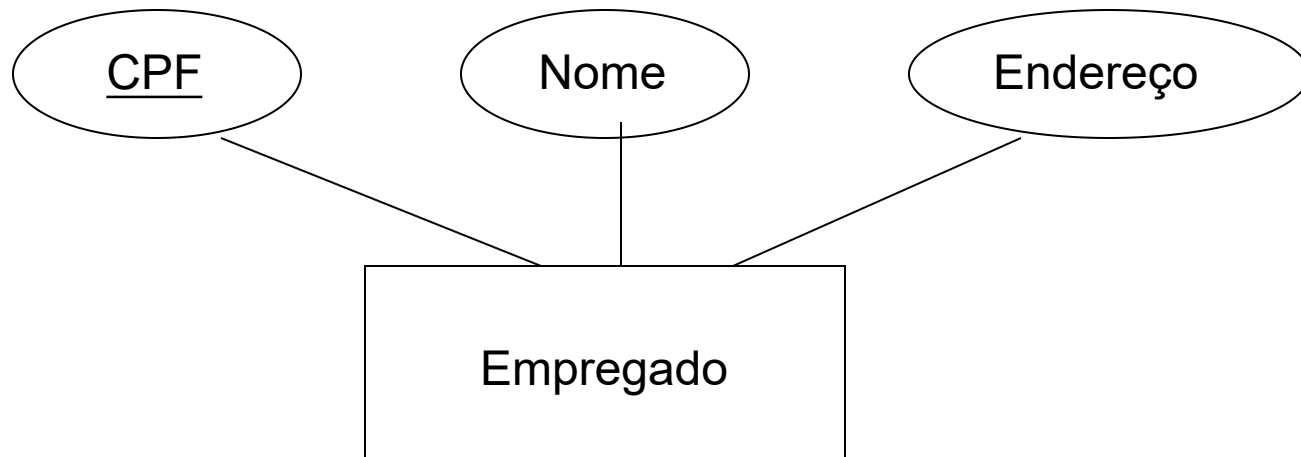
MER- observações sobre representação por chaves

- Livro Elmasri/Navathe
 - “Escolher um dos atributos-chave [da entidade] como chave primária de R”
- Cuidado: chave e chave primária são conceitos distintos

MER- observações sobre representação por chaves

- Definições do Ramakrishnan
 - Uma **chave** é um conjunto mínimo de atributos cujos valores identificam univocamente uma entidade no conjunto de entidades
 - Pode existir mais de uma **chave candidata**
 - Caso exista, uma delas é escolhida como **chave primária**
 - [No modelo ER] cada atributo da **chave primária** é sublinhado.

Representação - Empregado



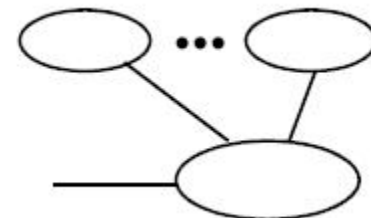
MER - Atributos

- Propriedades dos atributos
- Diversos tipos de *atributos* ocorrem no MER
 - Simples versus compostos
 - Monovalorados versus multivalorados
 - Armazenados versus derivados
- Valores Nulos

Atributos – Classificação

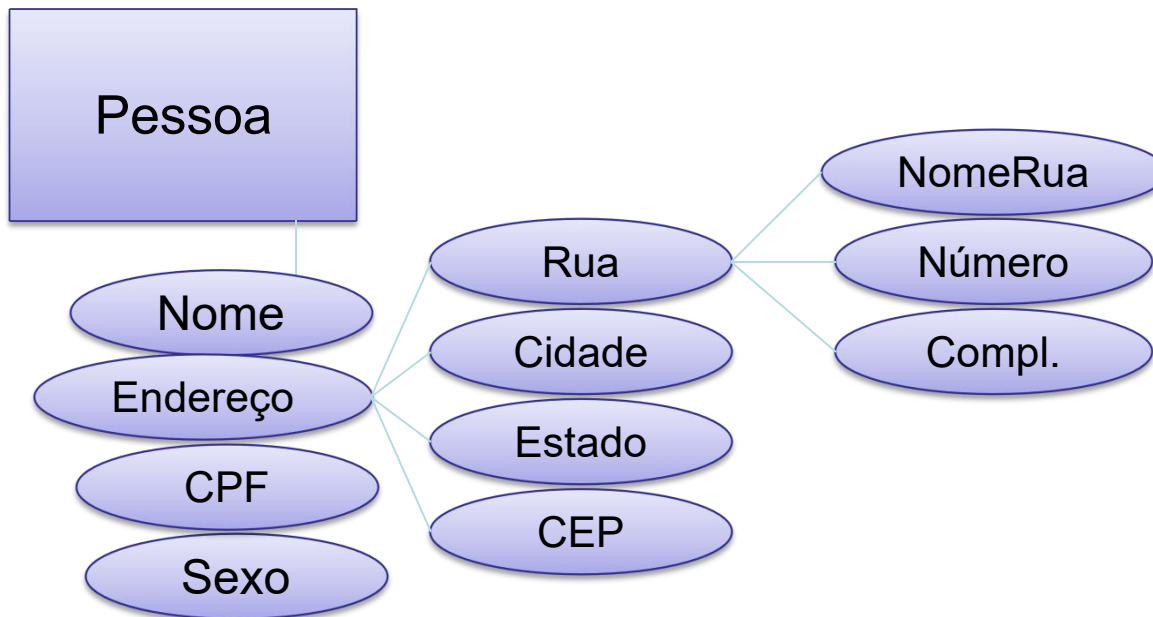
Segundo *Elmasri & Navathe*

- Simples *versus* Compostos
 - atributo simples ou atômico:
 - não pode ser decomposto (dividido) em atributos mais básicos
 - exemplo: sexo {M, F}
 - atributo composto
 - pode ser decomposto em atributos mais básicos
 - exemplo: atributo endereço: *nome_rua*, *nro_casa*, *complemento*, *nome_bairro*, ...



Atributos Simples *versus* Compostos

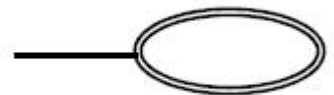
- Atributos compostos podem formar uma hierarquia
 - Rua pode ser dividida em:
 - Nome, Número, Complemento
 - Representação



Atributos – Classificação

Segundo *Elmasri & Navathe*

- Monovalorados *versus* Multivalorados
 - atributo monovalorado
 - possui um único valor para cada entidade
 - exemplo: idade
 - atributo multivalorado
 - possui múltiplos valores para cada entidade
 - exemplo: atributo telefone
 - pode possuir limites inferior/superior com relação à multiplicidade dos valores assumidos
 - exemplo: $nro_min = 0$, $nro_max = 3$



Atributos – Classificação

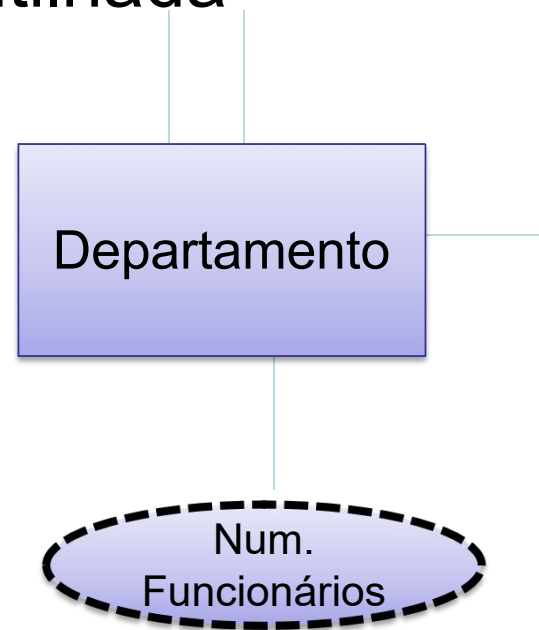
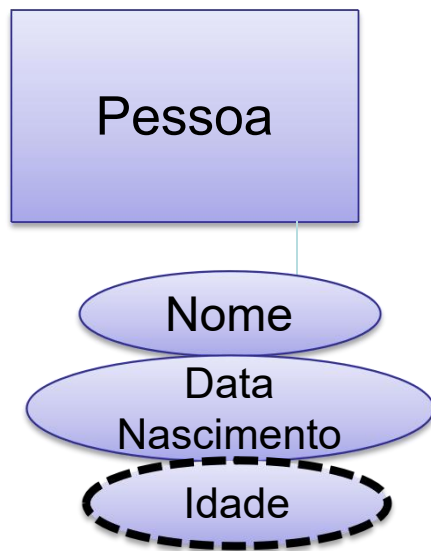
Segundo *Elmasri & Navathe*

- Armazenado *versus* Derivado
 - atributo armazenado
 - está realmente armazenado no BD
 - atributo derivado
 - pode ser determinado através de outros atributos ou através de entidades relacionadas
 - exemplos: $\text{idade} = \text{data_atual} - \text{data_nascimento}$
 $\text{nro_empregados} = \text{“soma de entidades”}$
 - pode ou não ser armazenado no BD



Armazenados ou derivados

- Atributo derivado
 - Representação: elipse pontilhada



Atributos – Classificação

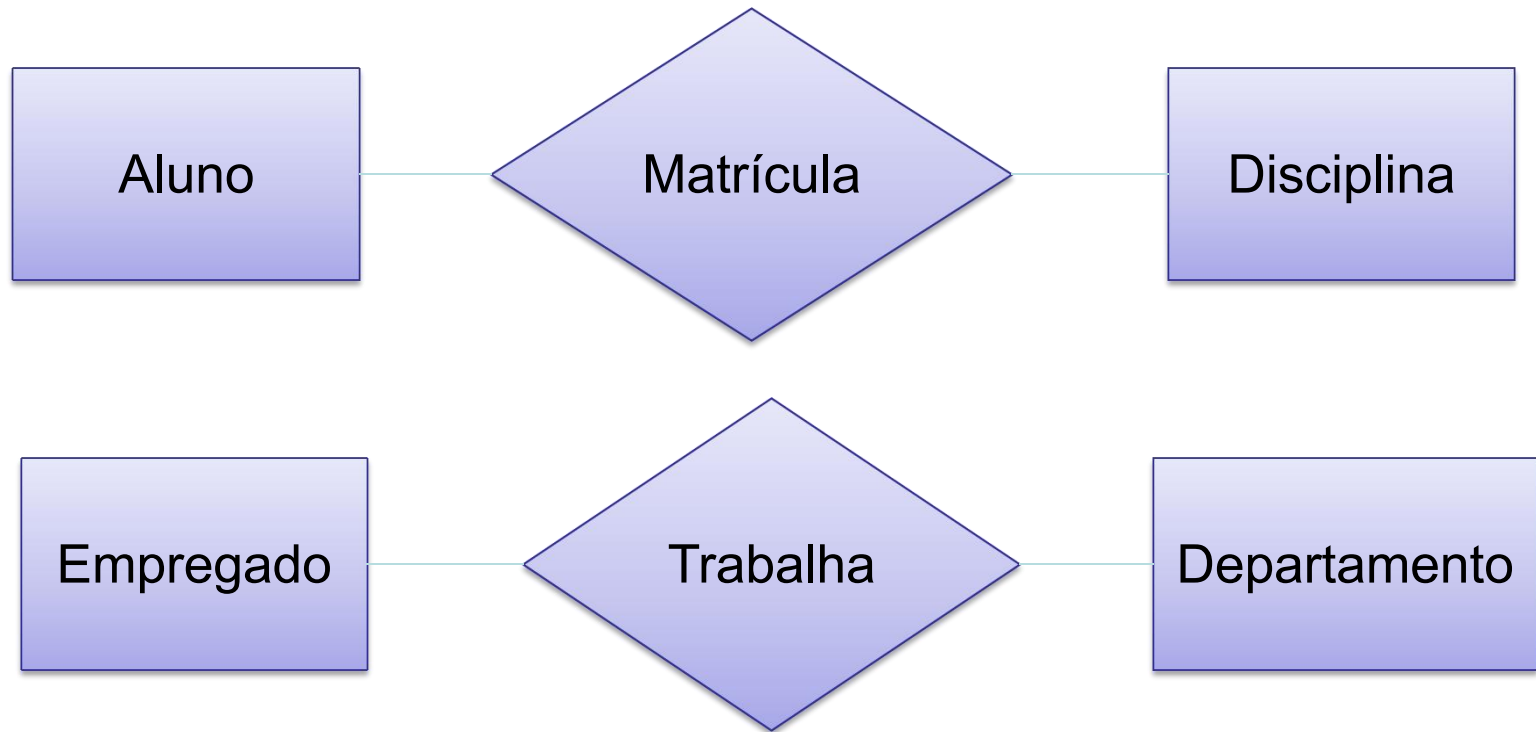
Segundo *Elmasri & Navathe*

- Valores NULL
 - Usado quando uma entidade não tem um valor aplicável para um atributo
 - Usado quando o valor de um atributo para uma entidade não é conhecido
 - Exemplo
 - Atributo Telefone com o valor NULL para a entidade Pessoa indica que não se sabe o telefone da Pessoa

Relacionamento

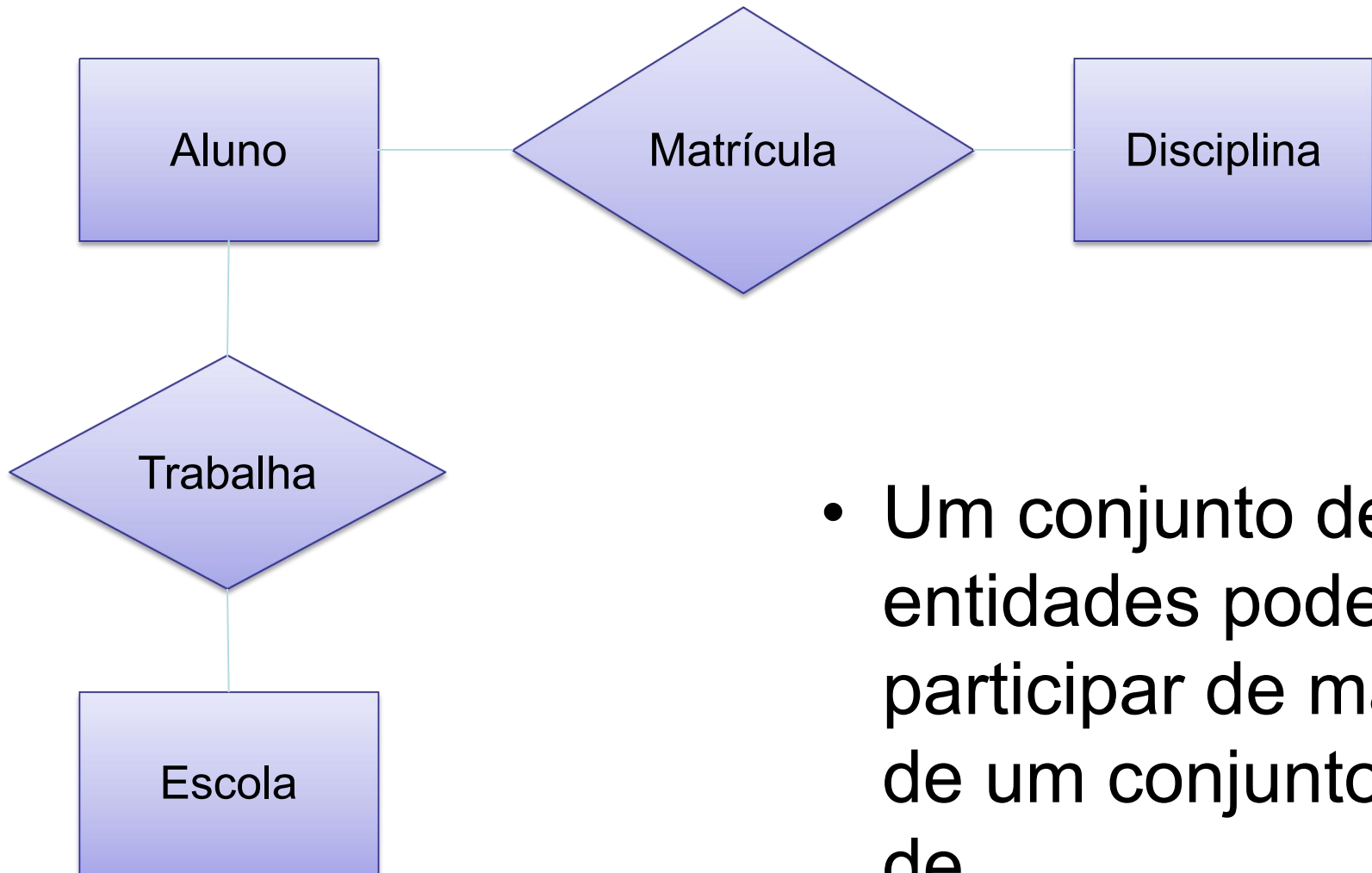
- É uma associação entre duas ou mais entidades
 - Ex: João trabalha no departamento farmacêutico
- Reuni-se um conjunto de relacionamentos similares em um **conjunto de relacionamentos** (Tipo-Relacionamento)
 - Ex: trabalha-em → Empregado trabalha em um Departamento

MER - Relacionamentos



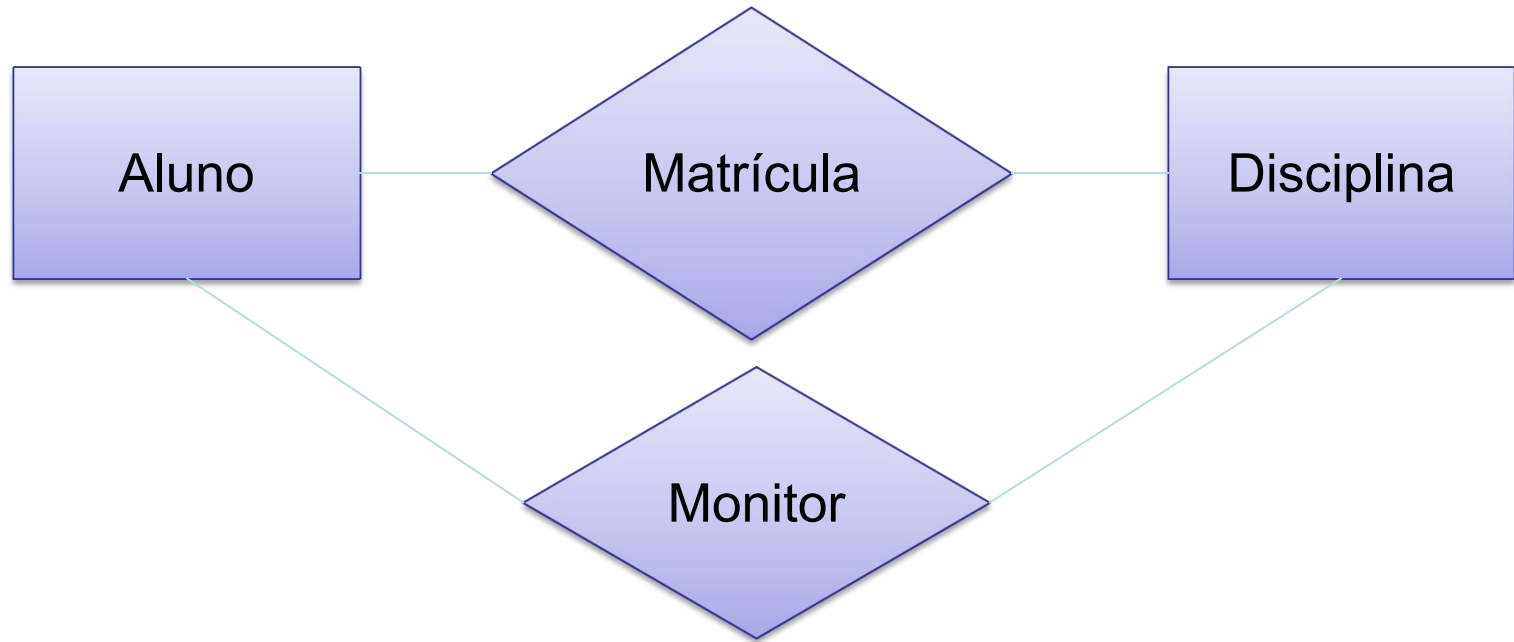
- Um relacionamento é representado por um losango
 - Ex: Um aluno se matricula em uma disciplina.
 - Ex: Um empregado trabalha em um

MER - Relacionamentos



- Um conjunto de entidades pode participar de mais de um conjunto de relacionamentos

MER - Relacionamentos



- Mais de um conjunto de relacionamentos pode envolver os mesmos conjuntos de entidades

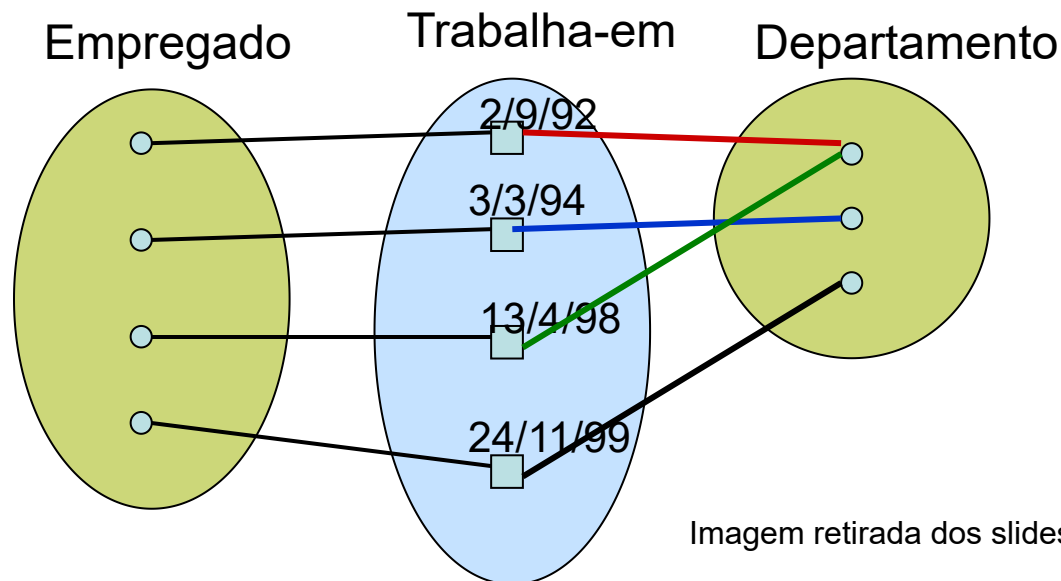
Relacionamento

- Formalmente

Um conjunto de relacionamentos é um conjunto de n-tuplas:

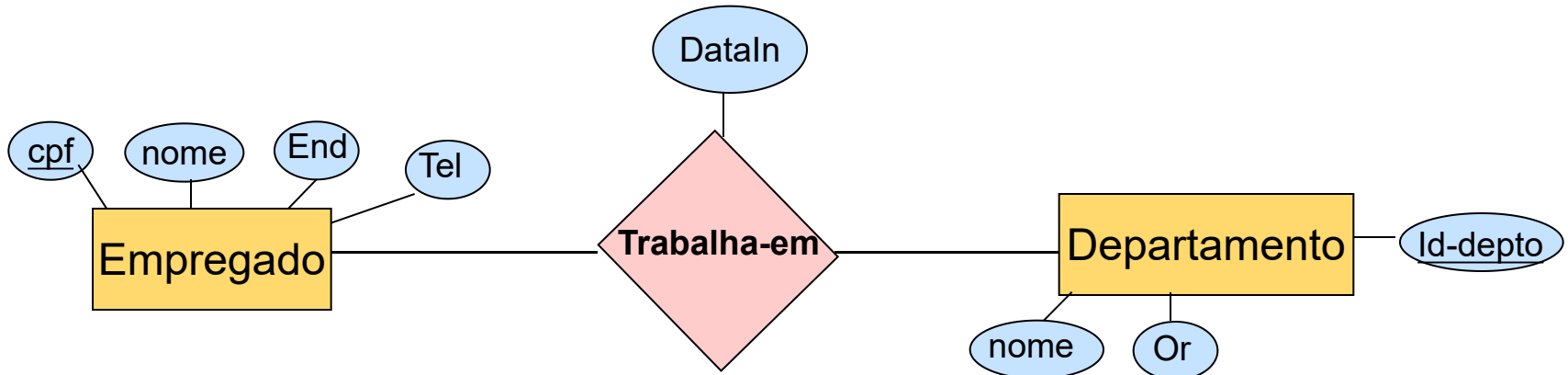
$$\{(e_1, \dots, e_n) | e_1 \in E_1, \dots, e_n \in E_m\}$$

Cada n-tupla denota um relacionamento envolvendo n entidades $e_1, \dots, e_m$ onde cada entidade está em um conjunto de entidades E_i



Relacionamento

- Pode ter um conjunto de atributos descritivos
 - Armazenam informações sobre o relacionamento
 - Ex: João trabalha no departamento farmacêutico desde janeiro de 2000.

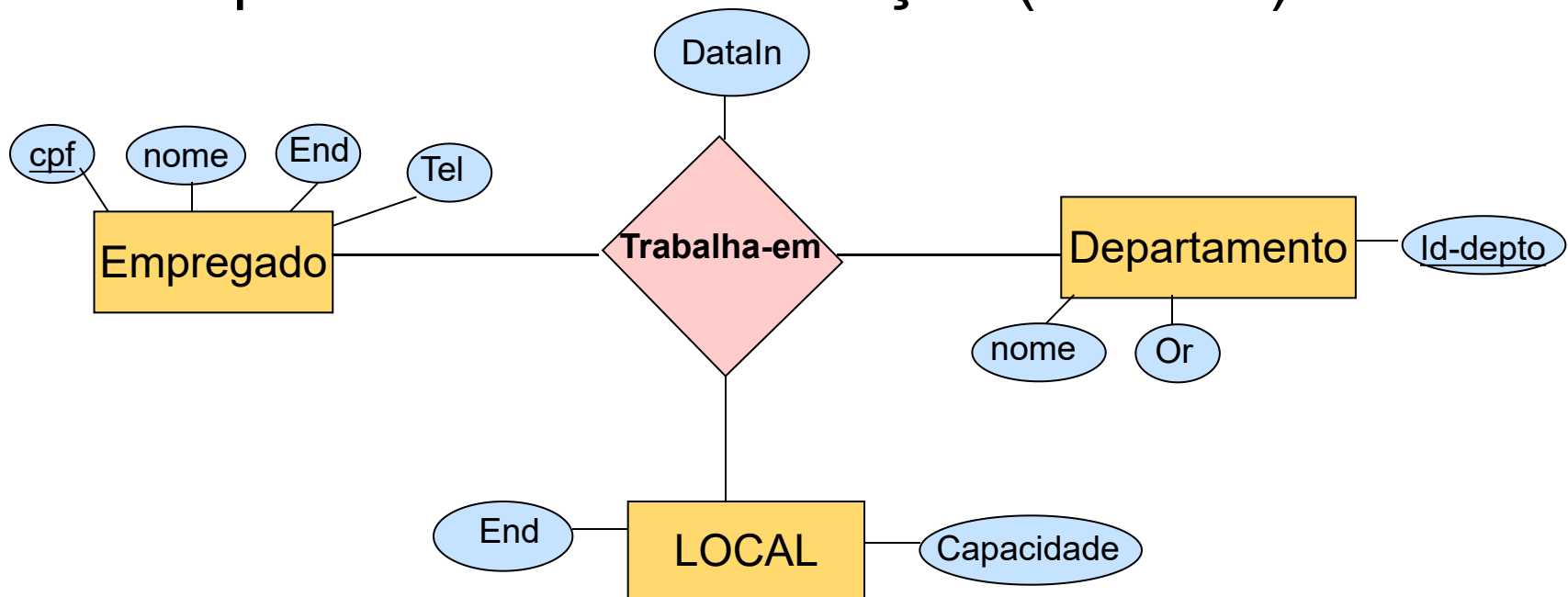


Relacionamento

- Deve ser unicamente identificado pelas entidades participantes
 - Ex: cada relacionamento trabalha_em deve ser identificado pela matrícula do funcionário e código do departamento
- Uma instância de um conjunto de relacionamentos → ‘Retrato’ do conjunto de relacionamentos em dado momento

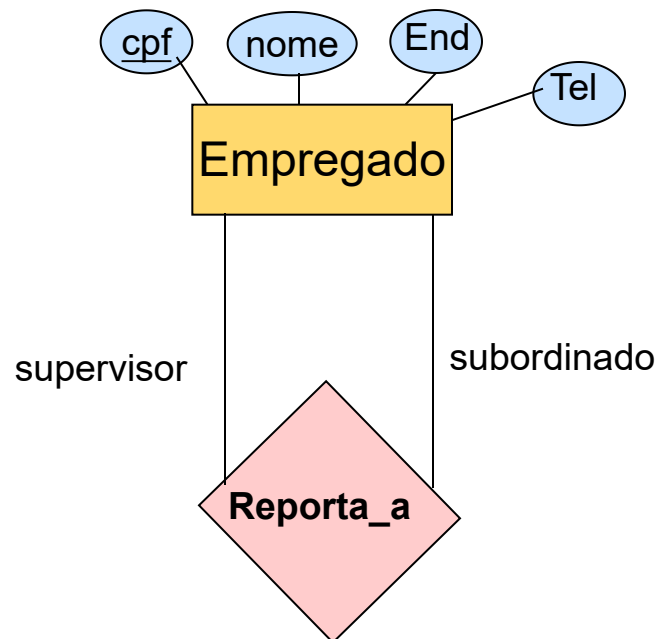
Relacionamento

- Pode envolver 2 ou mais entidades
 - Ex: associação entre empregado, departamento e localização (ternário)



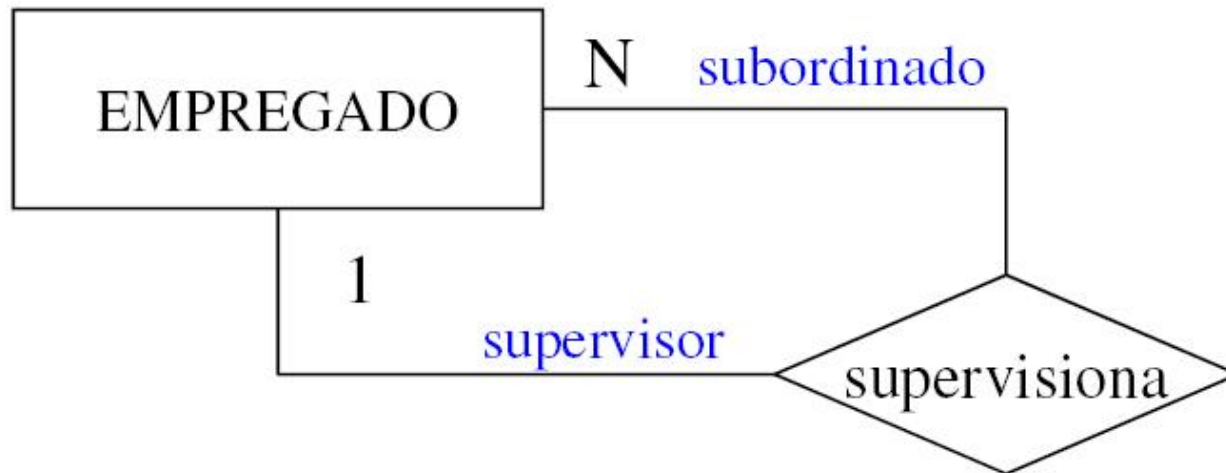
Relacionamento

- Pode envolver duas entidades do mesmo conjunto de entidades → especificar o papel de cada uma



Grau de um conjunto de relacionamentos

- Unário
 - é um relacionamento que envolve um único conjunto de entidades
 - Indicado utilizar nome de papel



Grau de um conjunto de relacionamentos

- Binário
 - É um relacionamento que envolve dois conjuntos de entidades
- Ternário
 - É um relacionamento que envolve três conjuntos de entidades

- Relacionamento Ternário representa informações diferente de 3 conjuntos de relacionamentos binários!
- Ex: Fornecedor x Peça x Projeto

Recurso Adicionais do Modelo ER

- Restrições nos Conjuntos de Relacionamentos
 - Limitam as combinações possíveis de entidades que podem participar dos conjuntos de relacionamentos
 - Exemplos:
 - Um empregado deve trabalhar para algum departamento;
 - Um empregado não precisa ser um gerente
 - Restrições estruturais
 - Cardinalidade
 - Participação

Recurso Adicionais do Modelo ER

- Restrições de Cardinalidade
 - Especifica o número máximo de instâncias de relacionamentos em que uma entidade pode participar
 - Ex: relacionamento Trabalha_para
Departamento:Funcionário
Cardinalidade $\rightarrow$ 1:N
 - Tipos de cardinalidade
 - 1:1, 1:N, N:1 e M:N

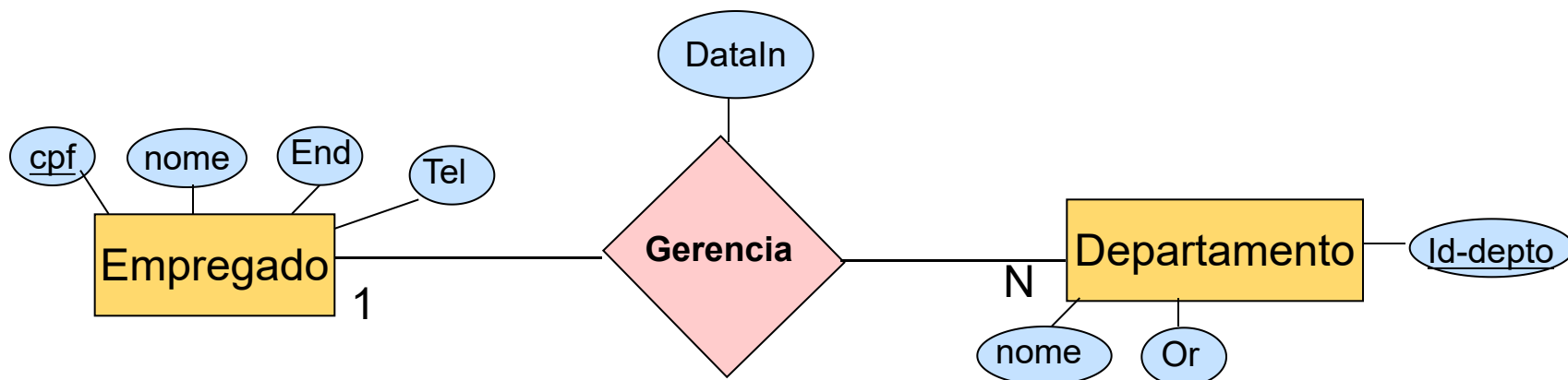
Recurso Adicionais do Modelo ER

Segundo *Elmasri & Navathe*



Recurso Adicionais do Modelo ER

- Cardinalidade

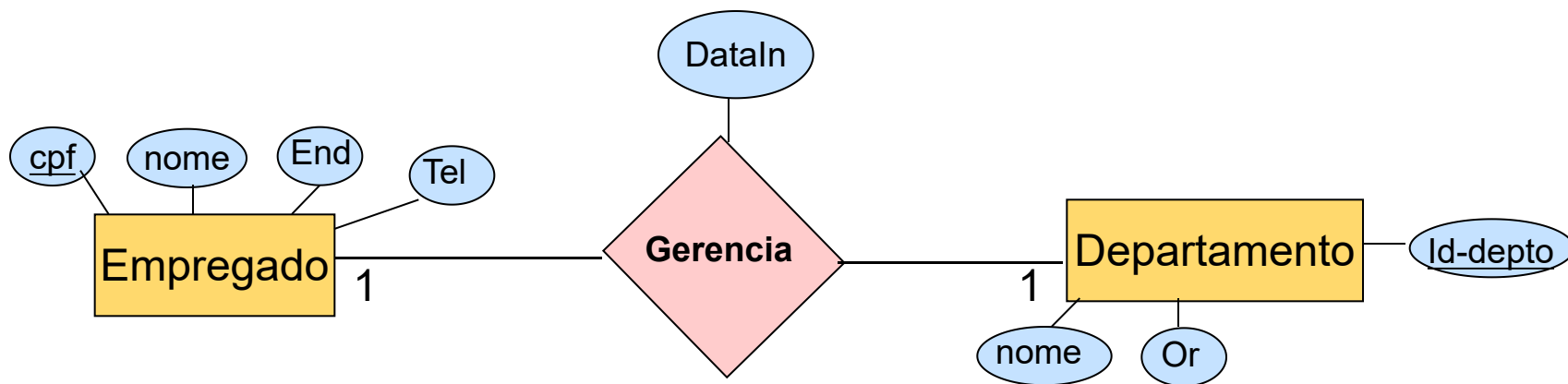


- **Gerência** é considerado **um-para-muitos** indicando que um funcionário pode estar associado a muitos departamentos, enquanto departamento pode estar associado a no máximo um funcionário
- **Trabalha_em** é considerado **muitos_para_muitos**

Recurso Adicionais do Modelo ER

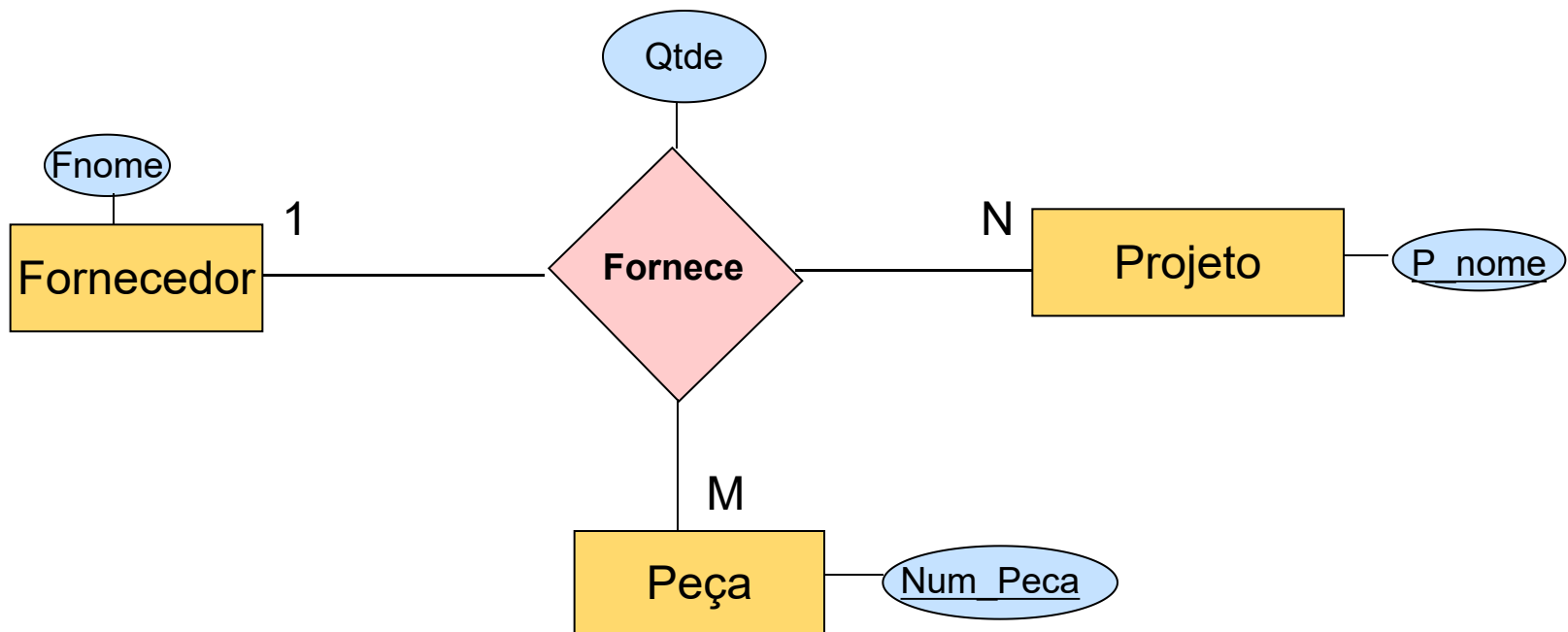
- Cardinalidade

- Ex: Considerando que cada funcionário pode gerenciar no máximo um departamento → **um-para-um**



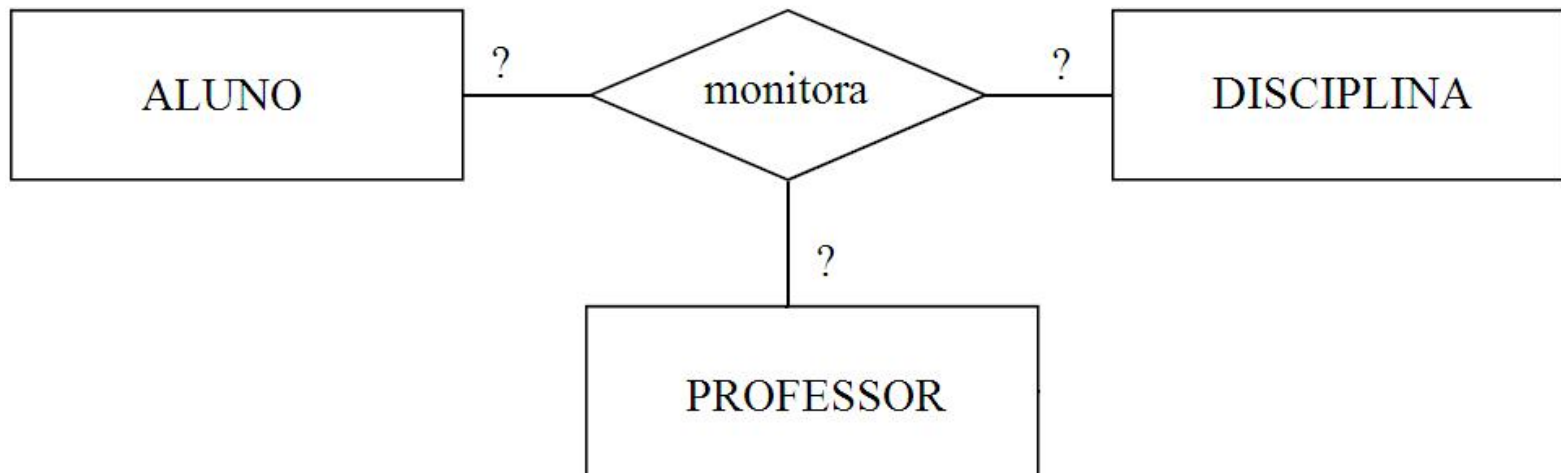
Recurso Adicionais do Modelo ER

- Cardinalidade para Relacionamentos Ternários



Recurso Adicionais do Modelo ER

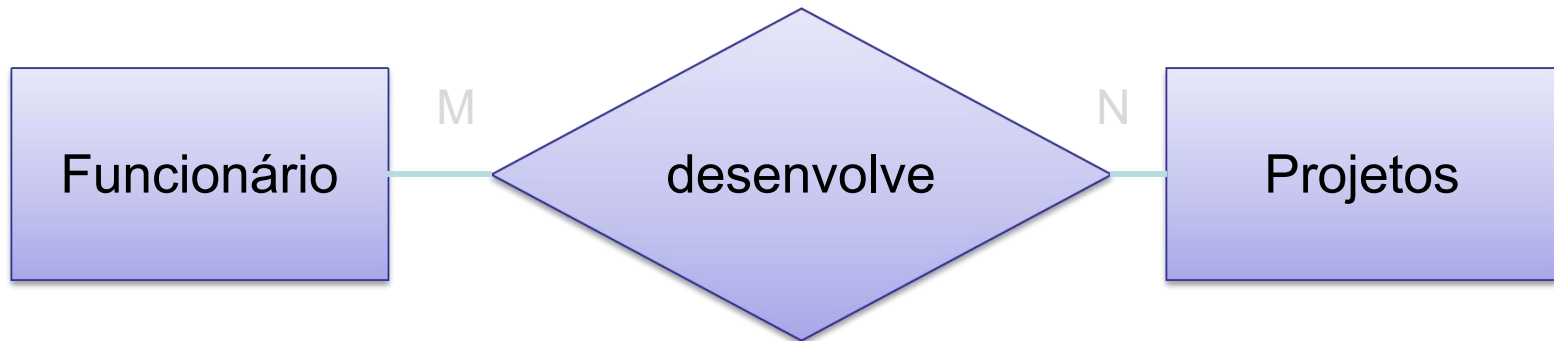
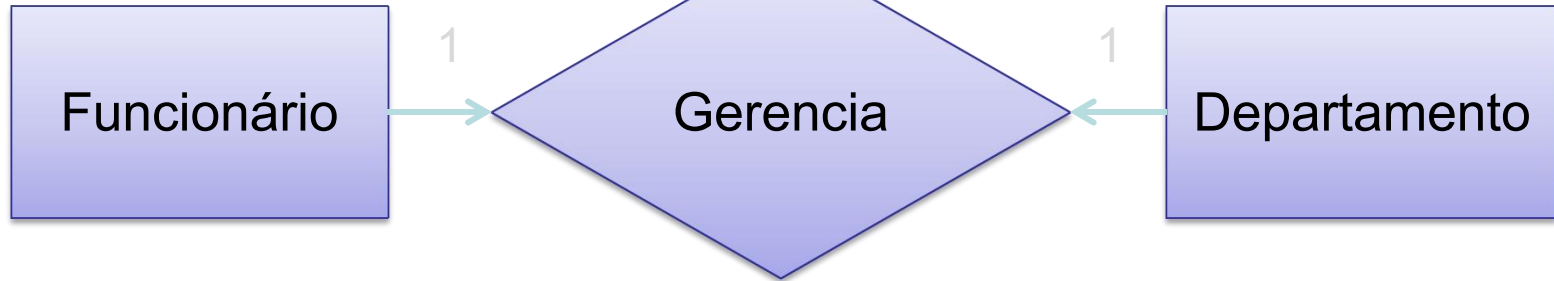
- Ternário



Recurso Adicionais do Modelo ER

- Há também a notação (min,max)
 - Cada entidade está relacionada a pelo menos *min* e no máximo *max* instâncias de relacionamento no conjunto de relacionamentos

Representações – NOTAÇÃO Ramakrishnan



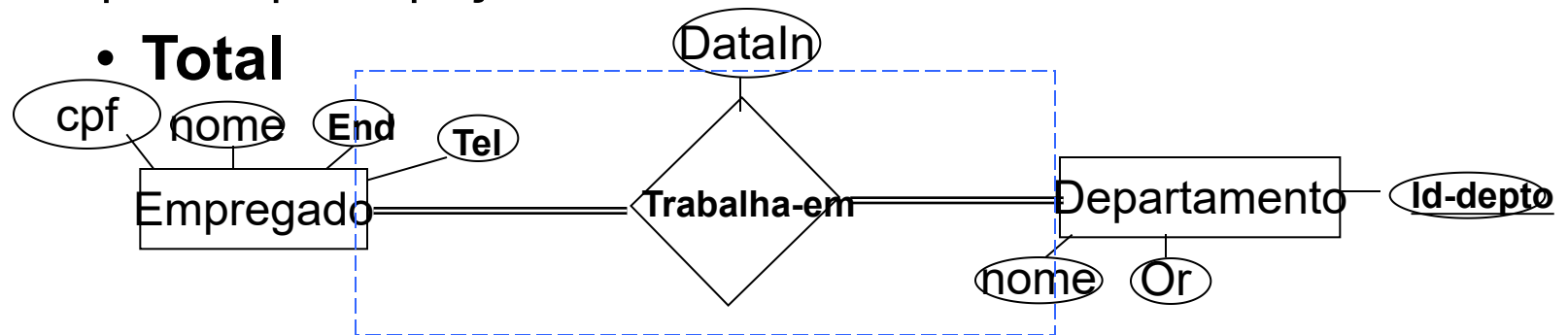
As cardinalidades 1, N, não devem aparecer nesta representação. Elas são mostradas aqui somente para referência

Recurso Adicionais do Modelo ER

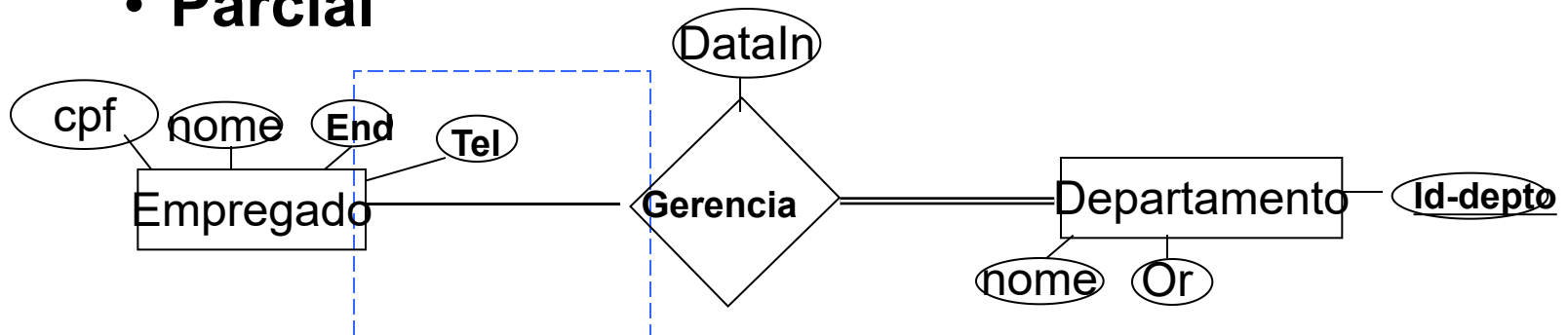
- Restrição de Participação

- Participação do conjunto de entidades no conjunto de relacionamentos
- Tipos de participação

- **Total**



- **Parcial**



Entidade Fraca

- Conjunto de entidades fracas
 - Não possuem atributos chave próprios
 - Identificadas considerando alguns de seus atributos em conjuntos com a chave primária de uma outra entidade → **proprietária identificadora**
 - Devem ser mantidas as restrições
 - O conjunto de entidades proprietárias e o conjunto de entidades fracas devem participar em um conjunto de relacionamentos um-para-muitos
 - Representa dependência de existência
 - um conjunto de entidades fraca deve ter participação total no relacionamento identificador

Entidade Fraca

- Exemplo
 - Dependente e Funcionários
- Chave Parcial
 - Conjunto de atributos de um conjunto de entidades fracas que identificam univocamente uma entidade fraca de uma determinada entidade proprietária
 - Ex: nomedep

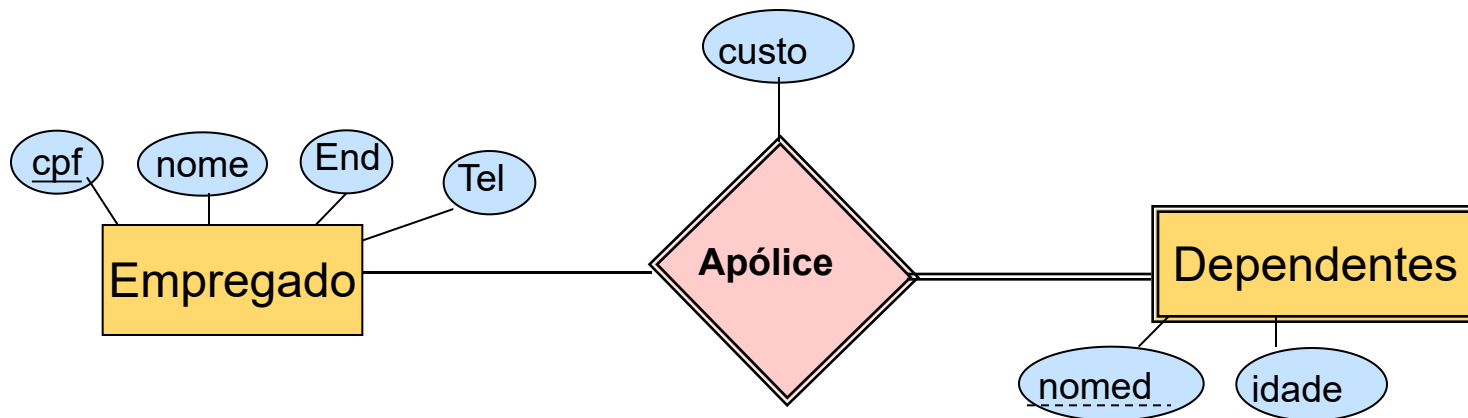
Entidade Fraca

- Entidade forte
 - tem chave primária
- Entidade fraca
 - possui somente uma chave parcial
- Chave primária de uma entidade fraca
 - chave primária da entidade forte correspondente

+

 - chave parcial da entidade fraca

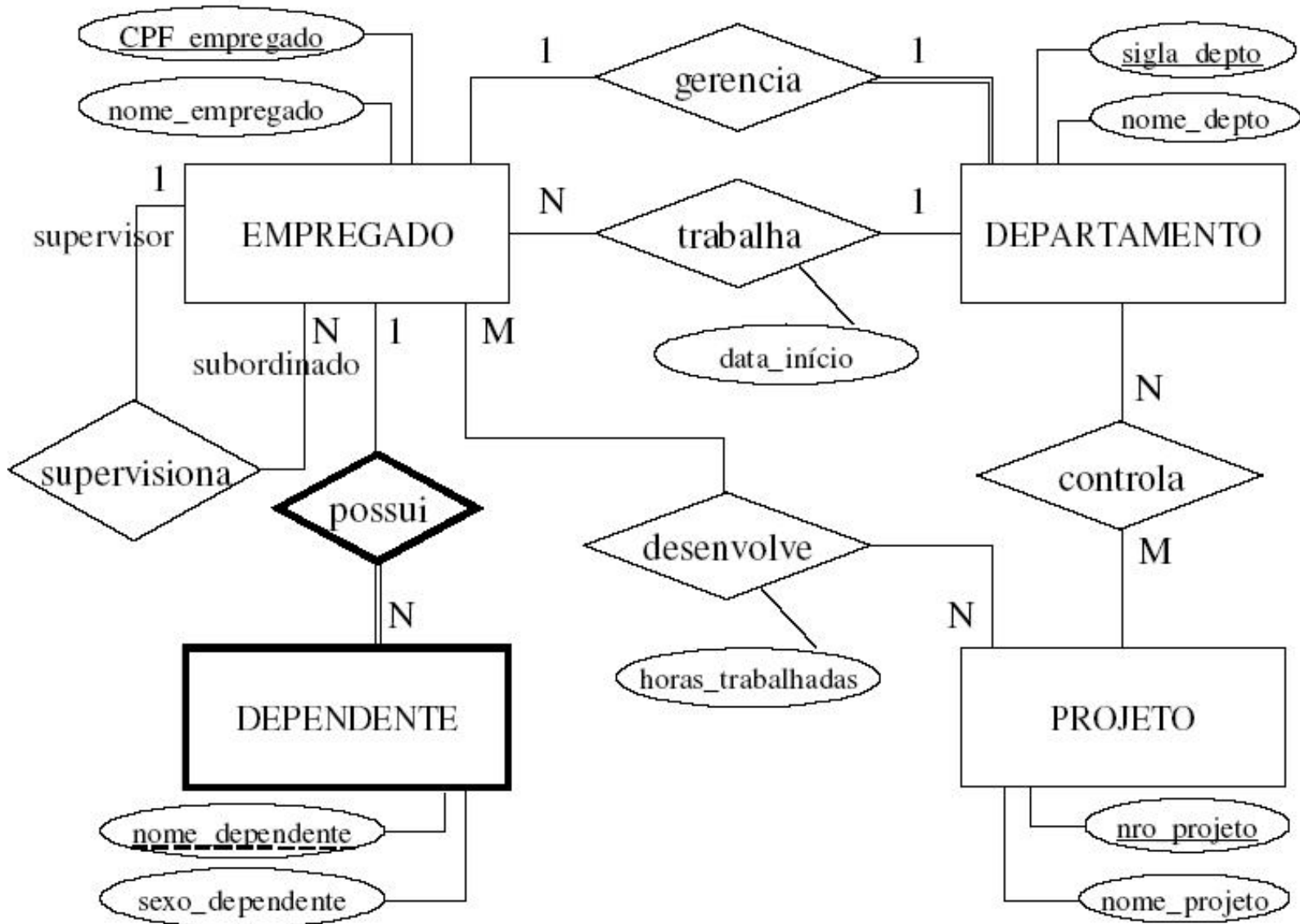
Entidade Fraca



Projeto Conceitual de um BD

- Identificar conjuntos de entidade e atributos
- Identificar chaves primárias
- Identificar conjuntos de relacionamento e seus atributos
 - Determinar o grau do relacionamento
 - Identificar as restrições que se aplicam sobre cada um
 - Cardinalidade
 - Participação
- Identificar conjunto-entidade forte e conjunto-entidade fraca

Exemplo de um DER



Exercício

Referências

- R. Elmasri e S. B. Navathe, Sistema de Banco de Dados, 6ª edição, Pearson, 2011.
- R. Ramakrishnan e J. Gehrke, *Database Management Systems*, 3a Edição, McGraw-Hill, 2003.

Aula 4 – SBD

Modelo Entidade Relacionamento Estendido

Profa. Elaine Faria
UFU

Agradecimentos

Este material é baseado

- No livro R. Ramakrishnan e J. Gehrke, *Database Management Systems*
- Nos slides do prof. Bruno Nassif

- Agradecimentos

- Ao professor Bruno Travençolo que gentilmente cedeu seus slides

Modelo Entidade Relacionamento Estendido

- Características
 - introduz semântica adicional ao modelo ER
 - utilizado na modelagem de aplicações mais complexas
- Conceitos
 - subclasse, superclasse, hierarquia de herança
 - generalização, especialização, e restrições
 - agregação

Subclasse/Superclasse

- Subclasse
 - agrupamento das entidades de um subgrupo do tipo-entidade
- Exemplo
 - superclasse: tipo-entidade empregado
 - subclasses: secretário, engenheiro, técnico

cada entidade que é membro de qualquer uma das subclasses também é um empregado

Herança

- de atributos
 - atributos da superclasse são herdados pelas subclasses
- de relacionamentos
 - instâncias de relacionamento da superclasse são herdados pelas entidades das subclasses
- Observação
 - qualquer entidade membro de uma subclasse deve ser também membro da superclasse
 - qualquer entidade membro da superclasse pode ser opcionalmente incluída como membro de qualquer número de subclasses

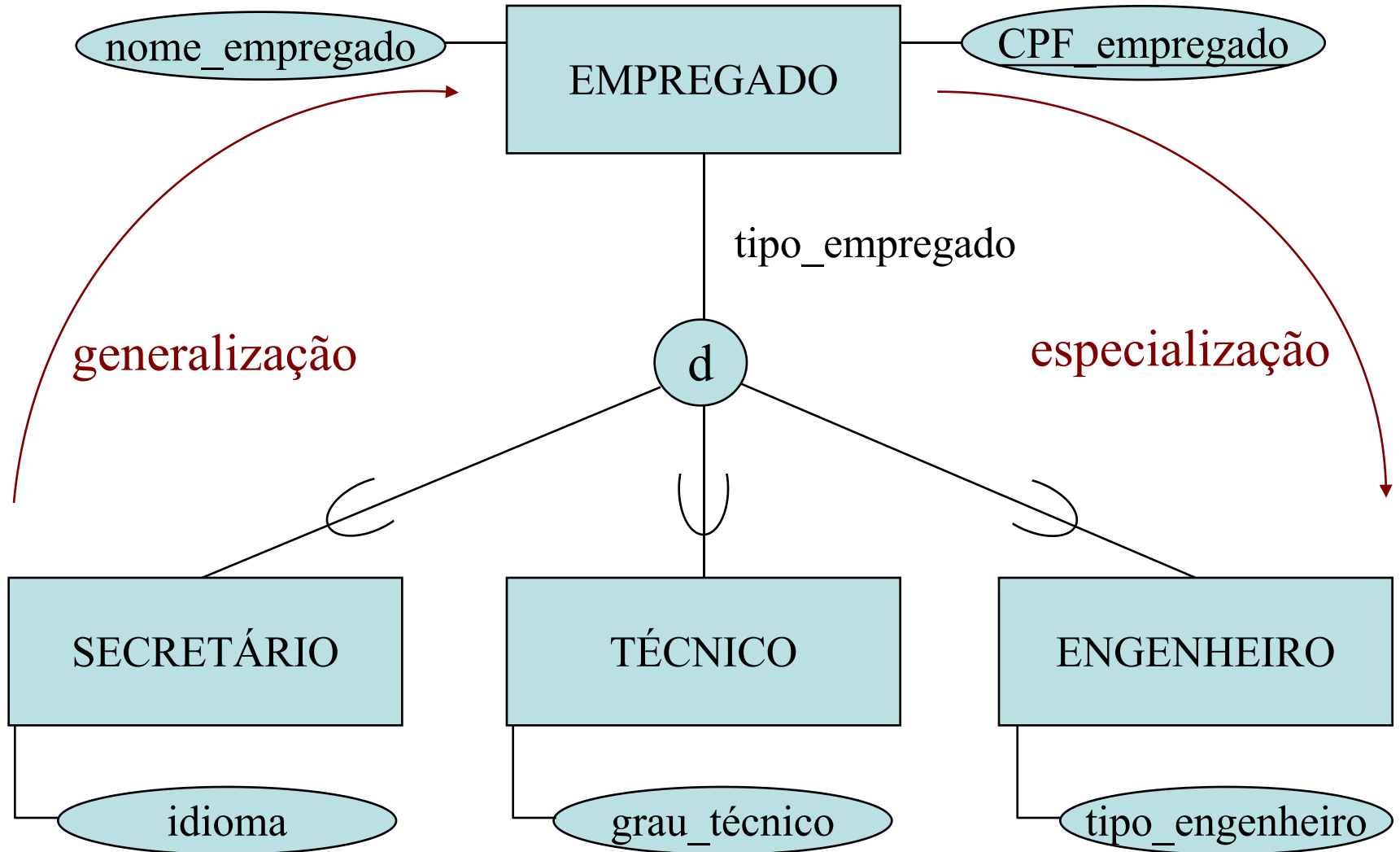
Generalização/Especialização

- Especialização
 - resultado da separação de um tipo-entidade de nível mais alto (superclasse), formando vários tipos-entidade de nível mais baixo (subclasse)
 - passos:
 - define-se um conjunto de subclasses de um tipo-entidade
 - associa-se atributos adicionais específicos às subclasses
 - estabelece-se tipos-relacionamento adicionais específicos às subclasses, caso necessário

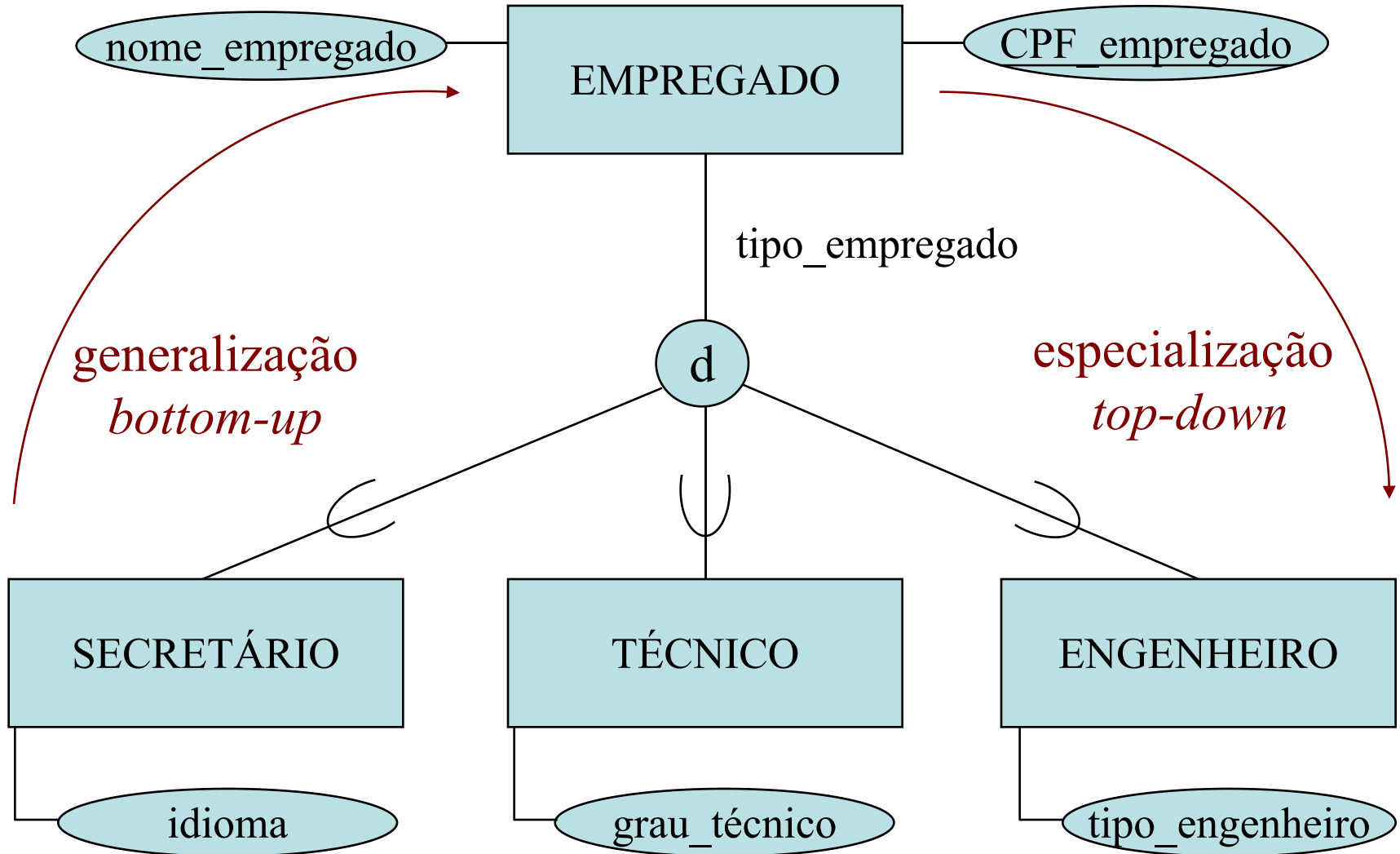
Generalização/Especialização

- Generalização
 - resultado da união de dois ou mais tipos-entidade de nível mais baixo (subclasse), produzindo um tipo-entidade de nível mais alto (superclasse)
 - é uma abstração de um conjunto de entidades
 - passos:
 - suprime-se as diferenças entre os tipos-entidade
 - identifica-se os atributos em comum
 - generaliza-os em uma superclasse

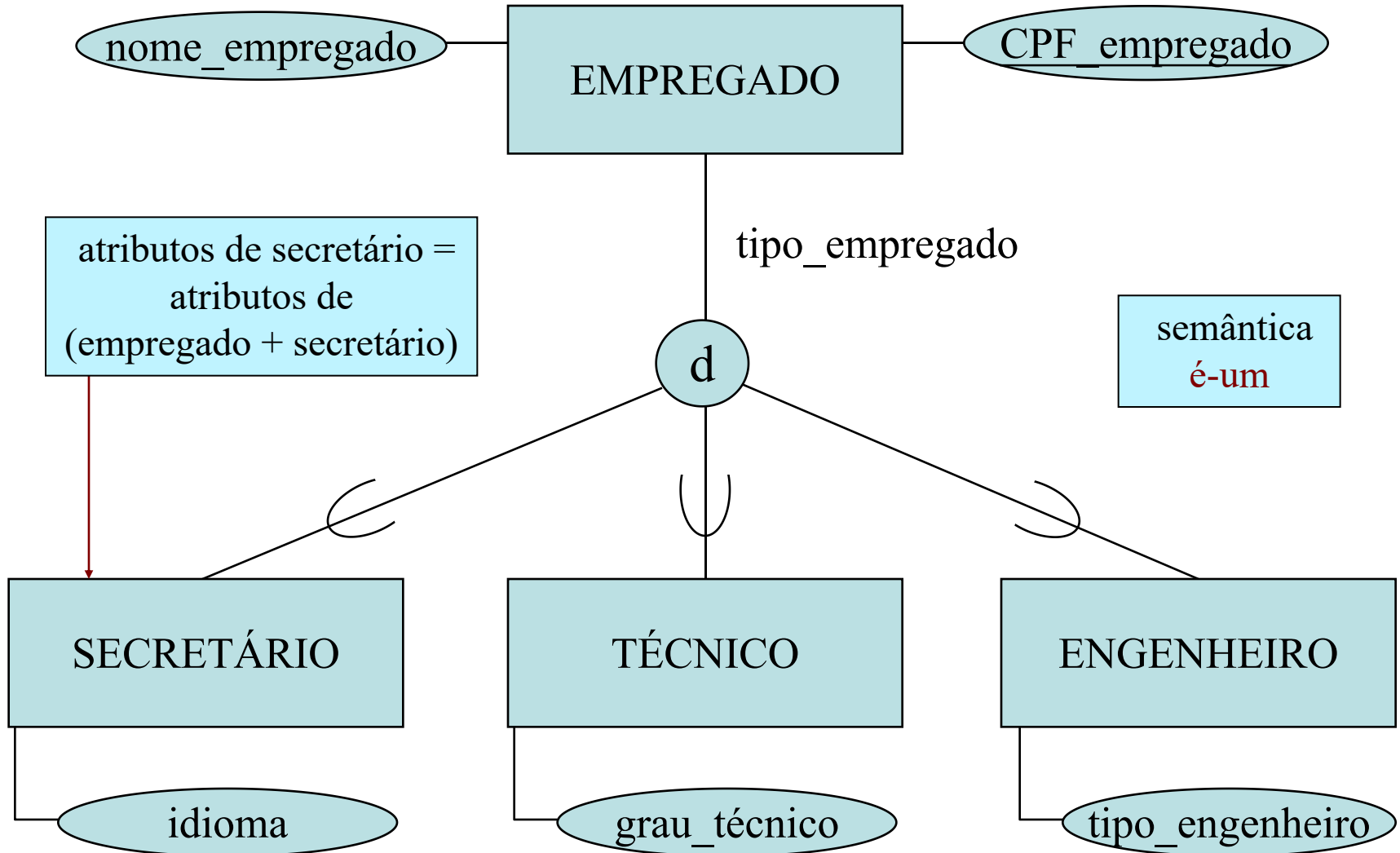
Representação



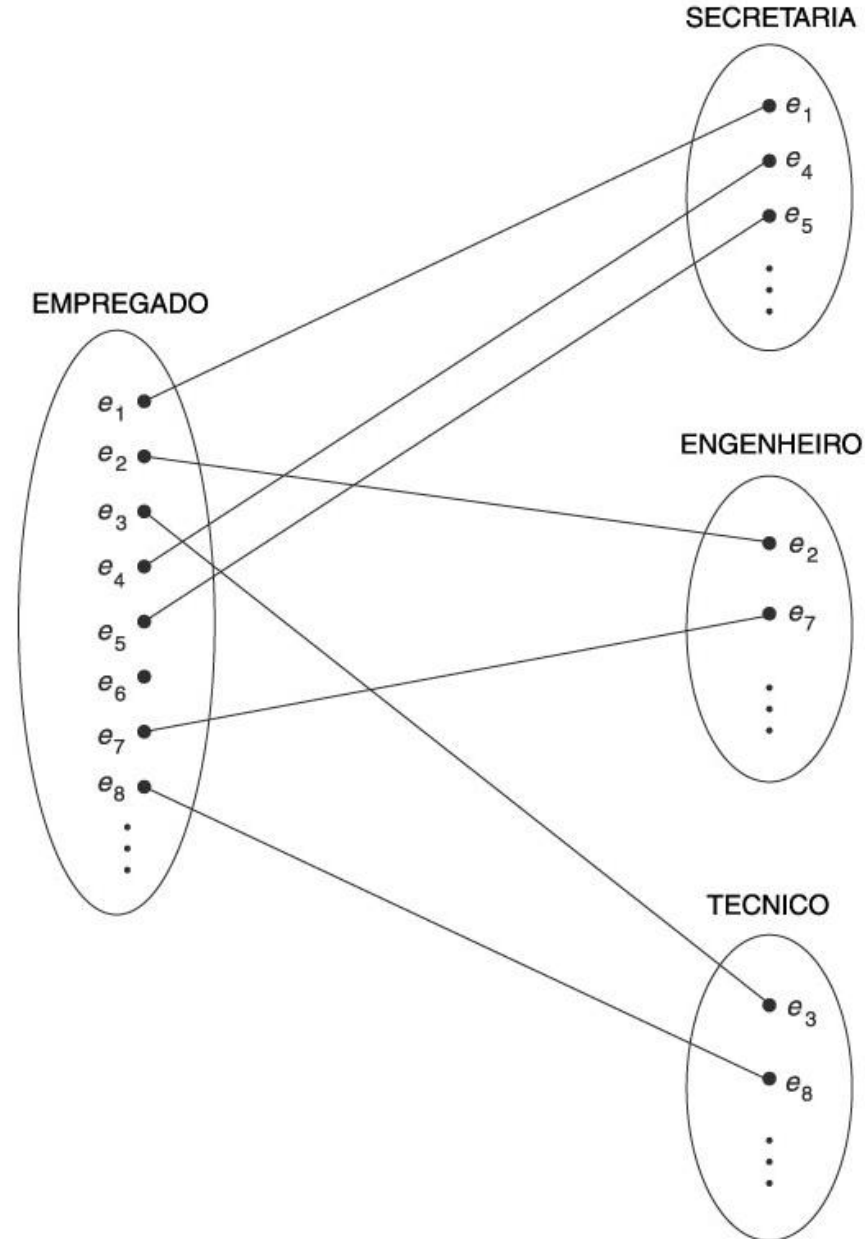
Representação



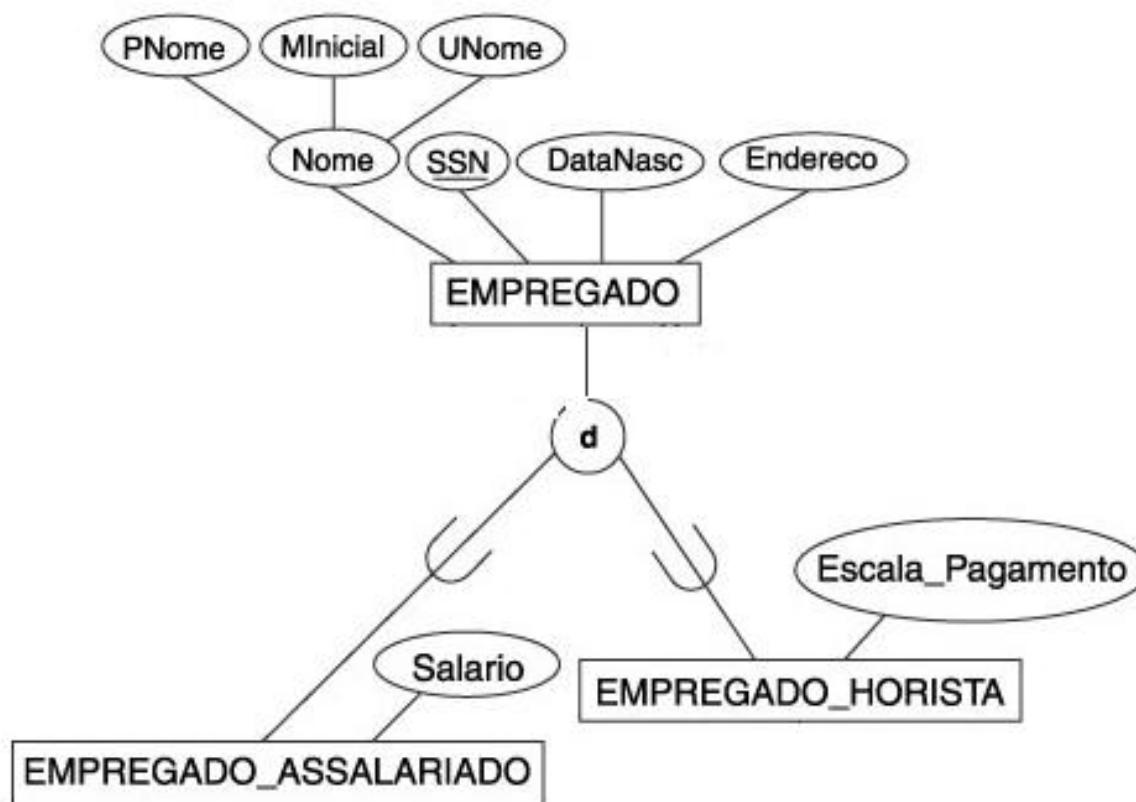
Representação



Hierarquia de Classe



Hierarquia de Classe - Exemplo



Hierarquia de Classe

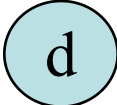
- Observações
 - Conjuntos de entidades mais específicos participam de todos os conjuntos de relacionamentos definidos para o conjunto de entidades mais genérico

Chaves dos Tipos-Entidade

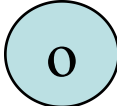
- Restrição de chave do ME-R: todos os tipos-entidade devem ter uma chave única
 - Restrição relaxada para o MER-X
 - subclasses não precisam ter chave explicitamente definida

Restrição de Disjunção

- Subclasses mutuamente exclusivas
 - uma entidade de uma superclasse deve ser membro, quando muito, de apenas uma subclasse

– representação:  ← 

- Subclasses que se sobrepõem
 - uma entidade de uma superclasse pode ser membro de mais do que uma subclasse

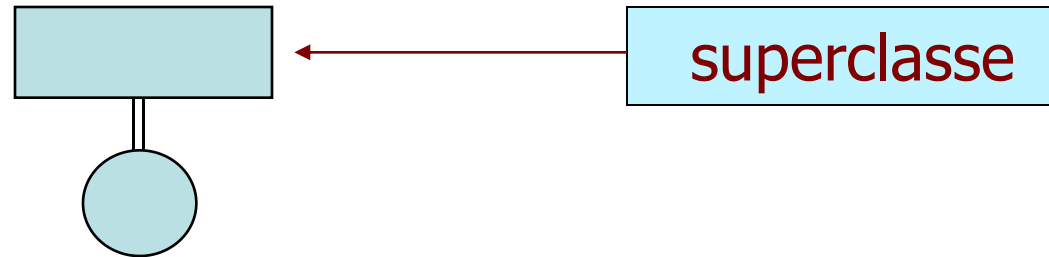
– representação:  ← 

Restrição de Completude

- Total

- cada entidade de uma superclasse deve ser membro de alguma subclasse na especialização

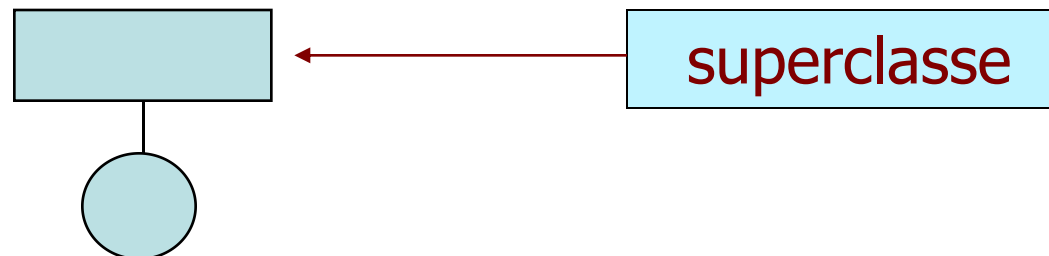
- representação:



- Parcial

- uma entidade de uma superclasse pode não pertencer a qualquer uma das subclasses

- representação:

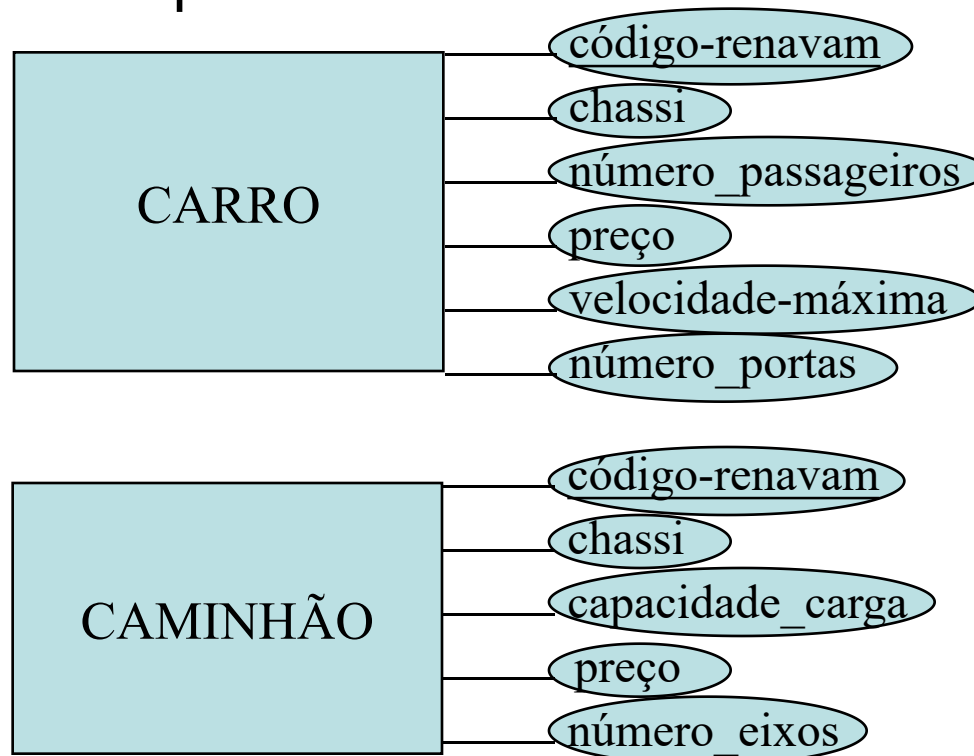


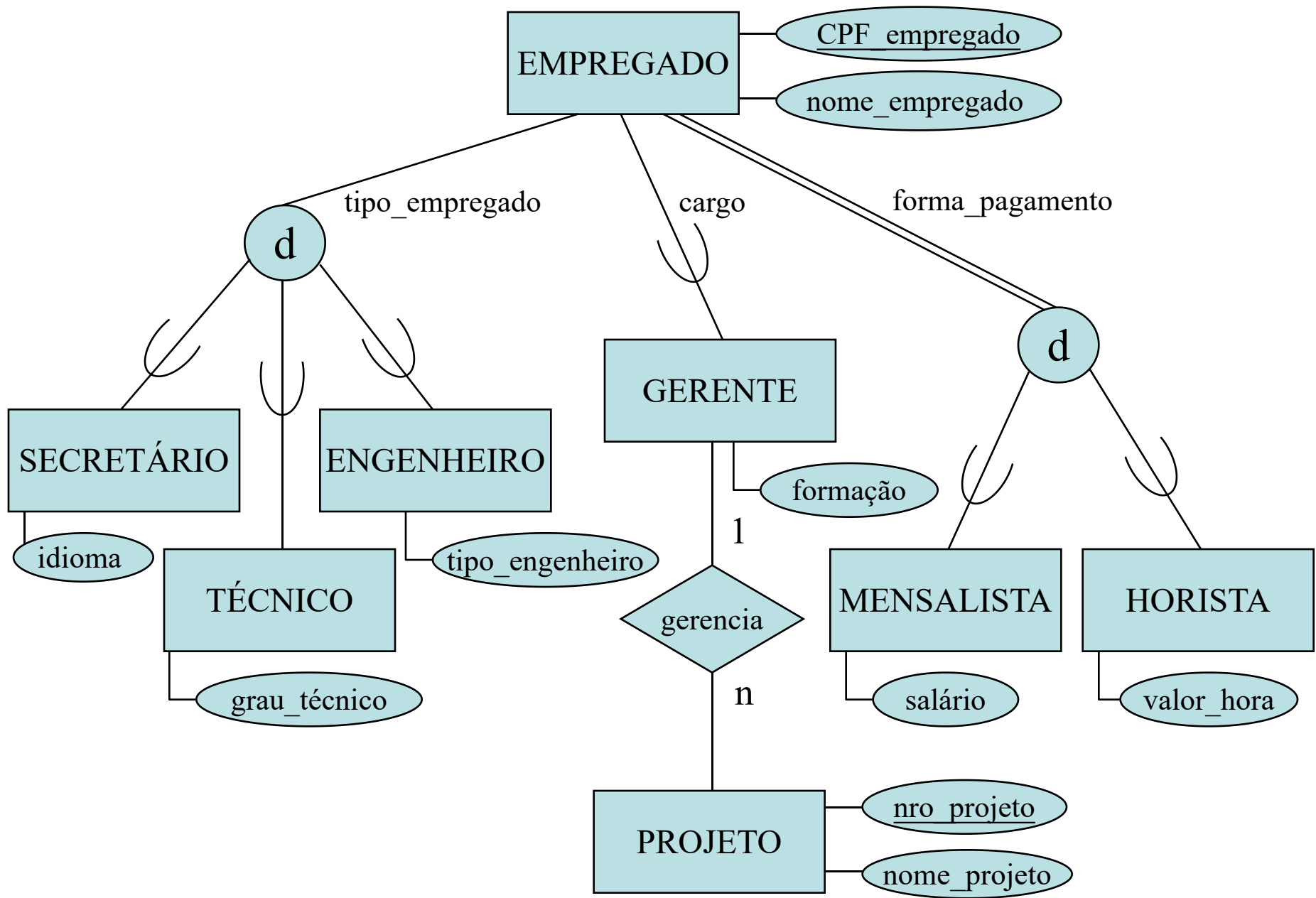
Observações

- Restrições de disjunção e de completude são independentes
 - possibilidades de hierarquias
 - total disjunta
 - parcial disjunta
 - total com sobreposição
 - parcial com sobreposição

Exercício

- Modele uma hierarquia de generalização/especialização para os tipos-entidade *carro* e *caminhão*. Defina as restrições de disjunção e de completude.



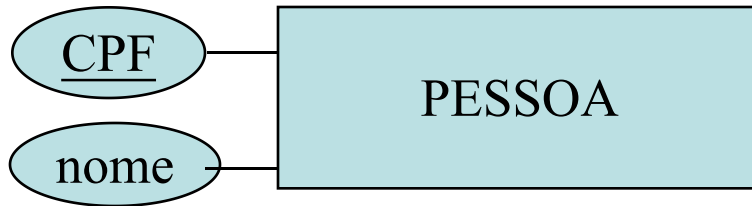


Generalização/Especialização

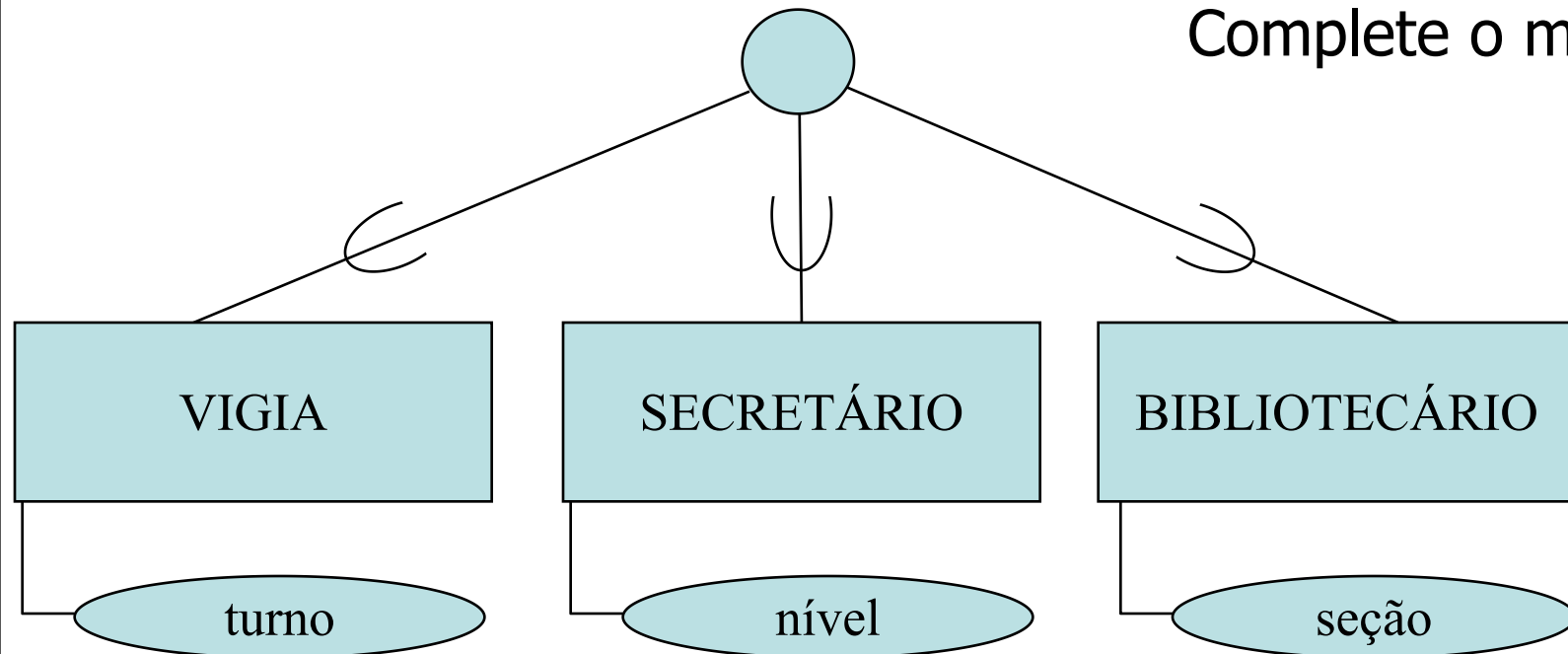
- Uma subclasse pode possuir outras subclases especificadas a partir dela
- Herança simples
 - cada subclasse participa como subclasse em **apenas um** relacionamento superclasse/subclasse
- Herança múltipla
 - cada subclasse pode participar como uma subclasse em **mais do que um** relacionamento superclasse/subclasse

Exercício

Além de vigia, secretário e bibliotecário existem outras funções. Um funcionário pode acumular mais de uma função. Complete o modelo!

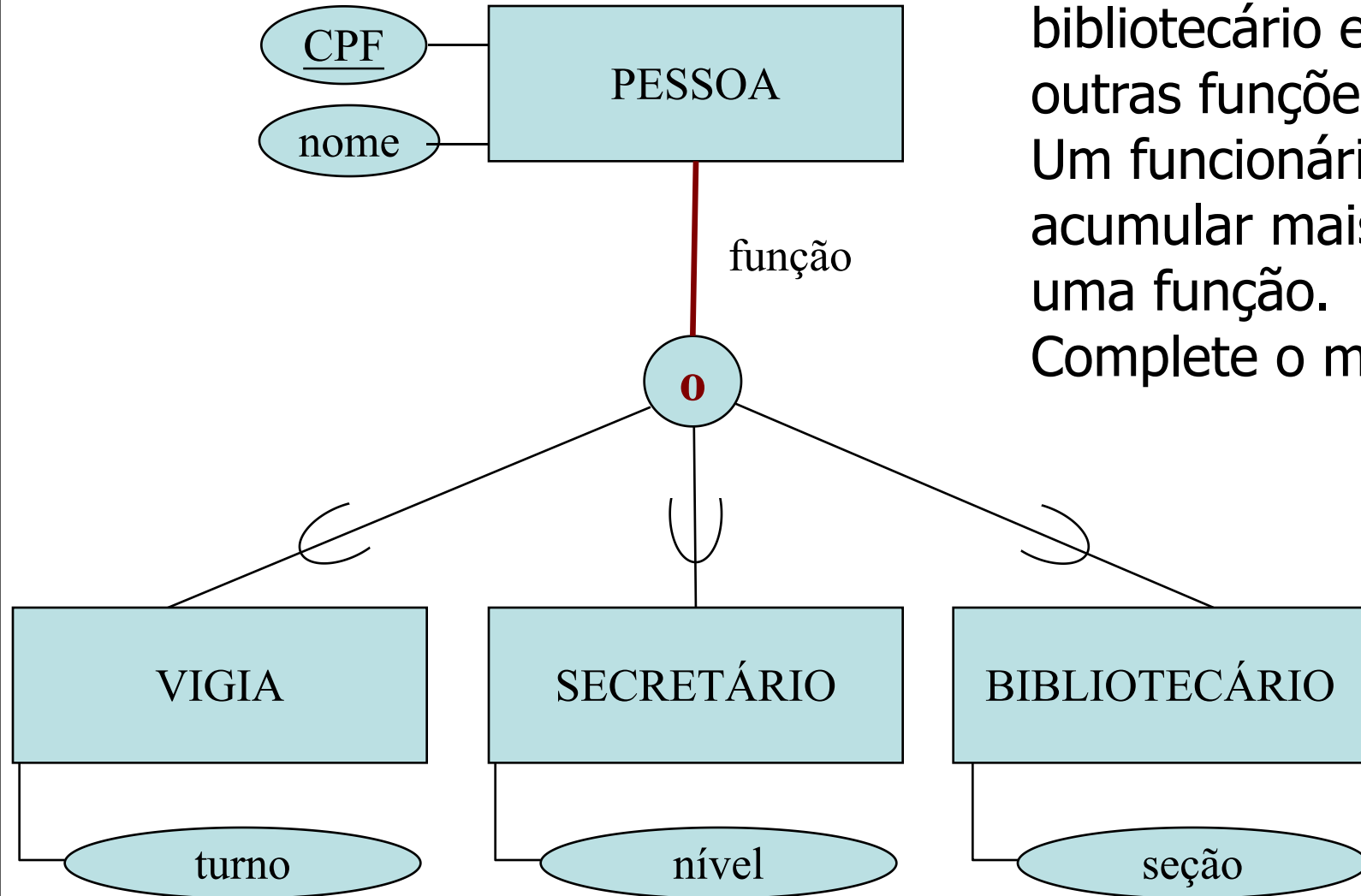


função



Exercício

Além de vigia, secretário e bibliotecário existem outras funções. Um funcionário pode acumular mais de uma função. Complete o modelo!

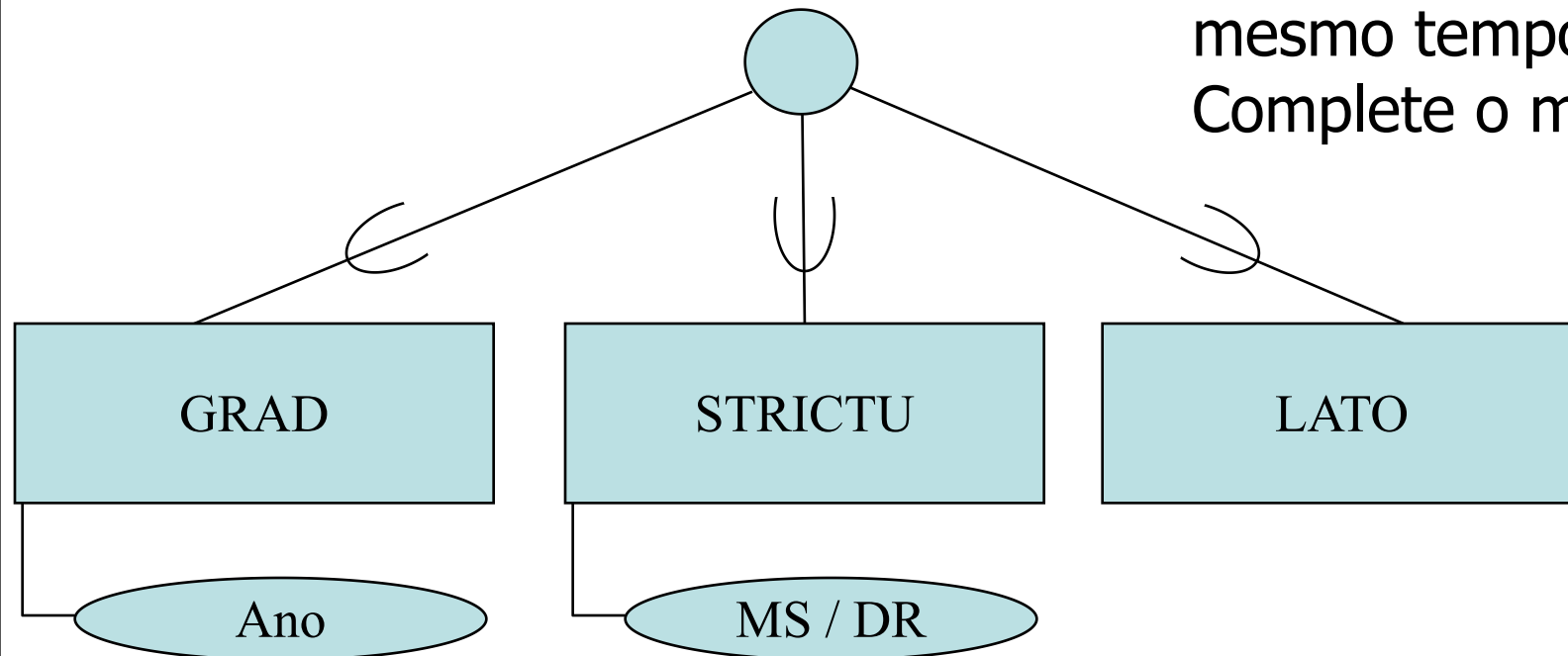


Exercício

Somente existem alunos de graduação, strictu (mestrado ou doutorado) e lato (especialização). Um aluno pode cursar mais de um curso ao mesmo tempo. Complete o modelo!

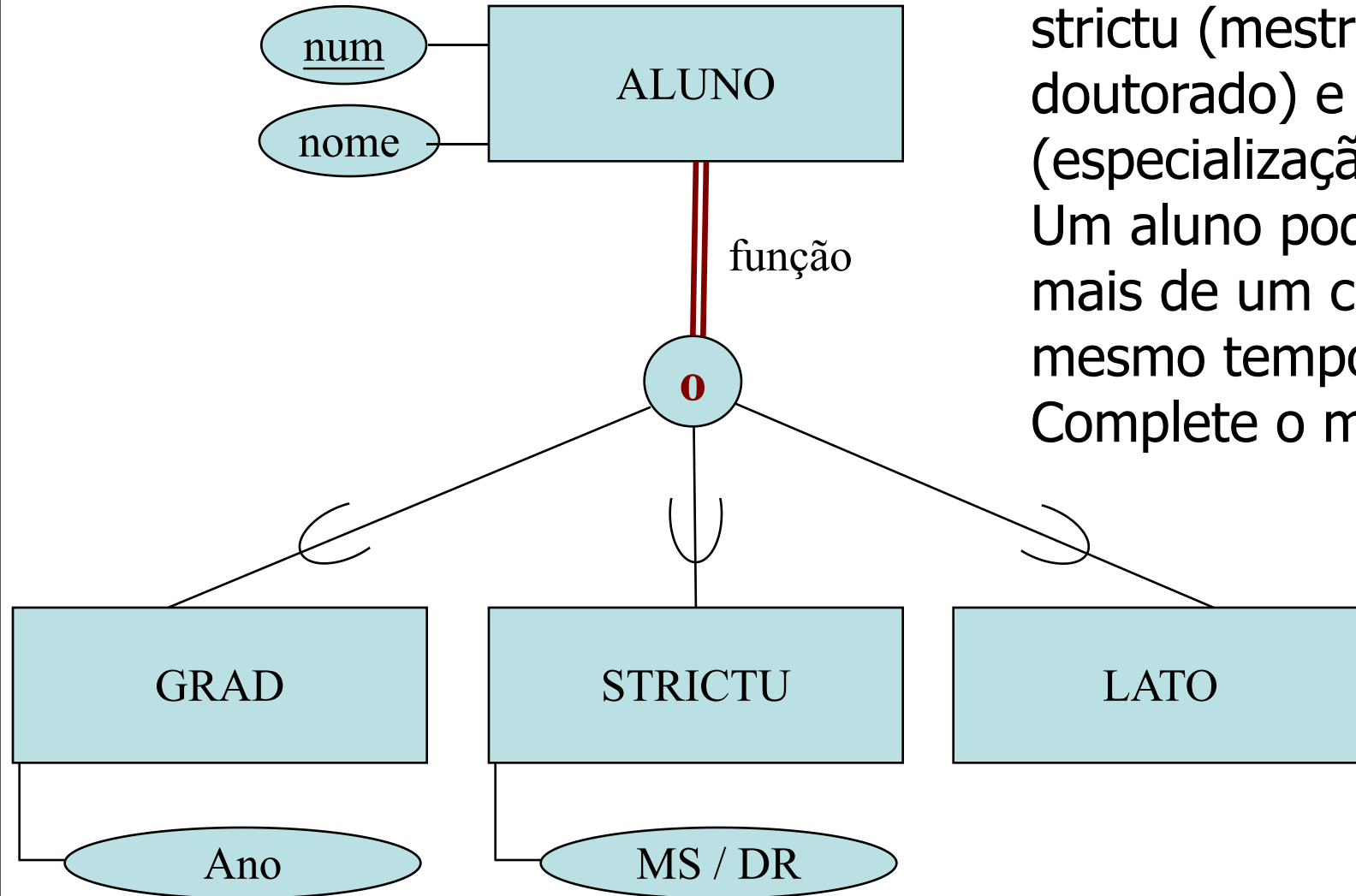


função



Exercício

Somente existem alunos de graduação, strictu (mestrado ou doutorado) e lato (especialização). Um aluno pode cursar mais de um curso ao mesmo tempo. Complete o modelo!

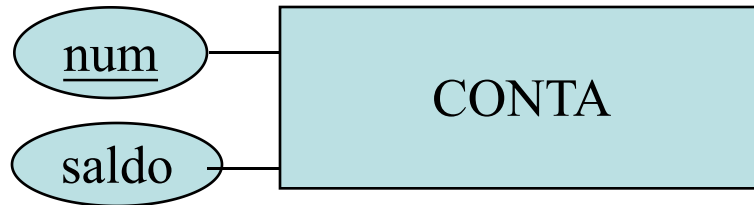


Exercício

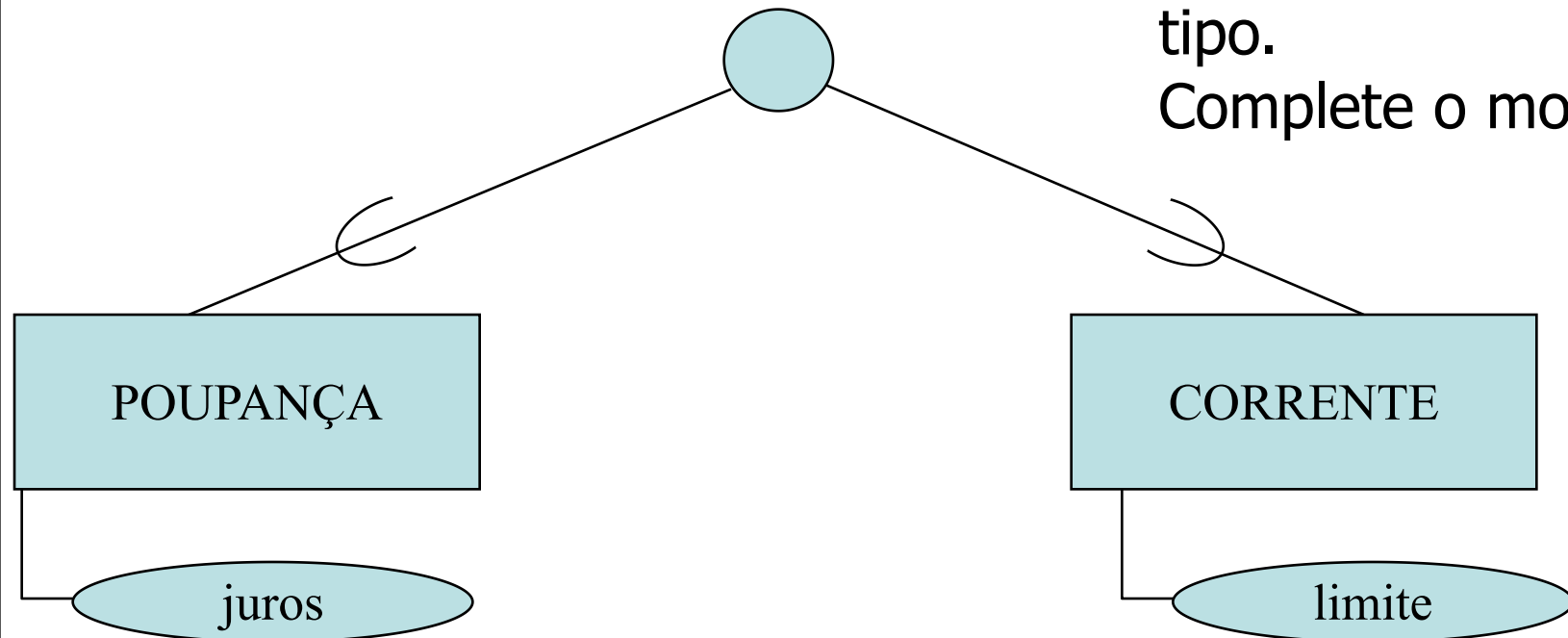
Existem contas que não são nem de poupança nem corrente.

Um conta não pode ser classificada como sendo mais de um tipo.

Complete o modelo!



tipo

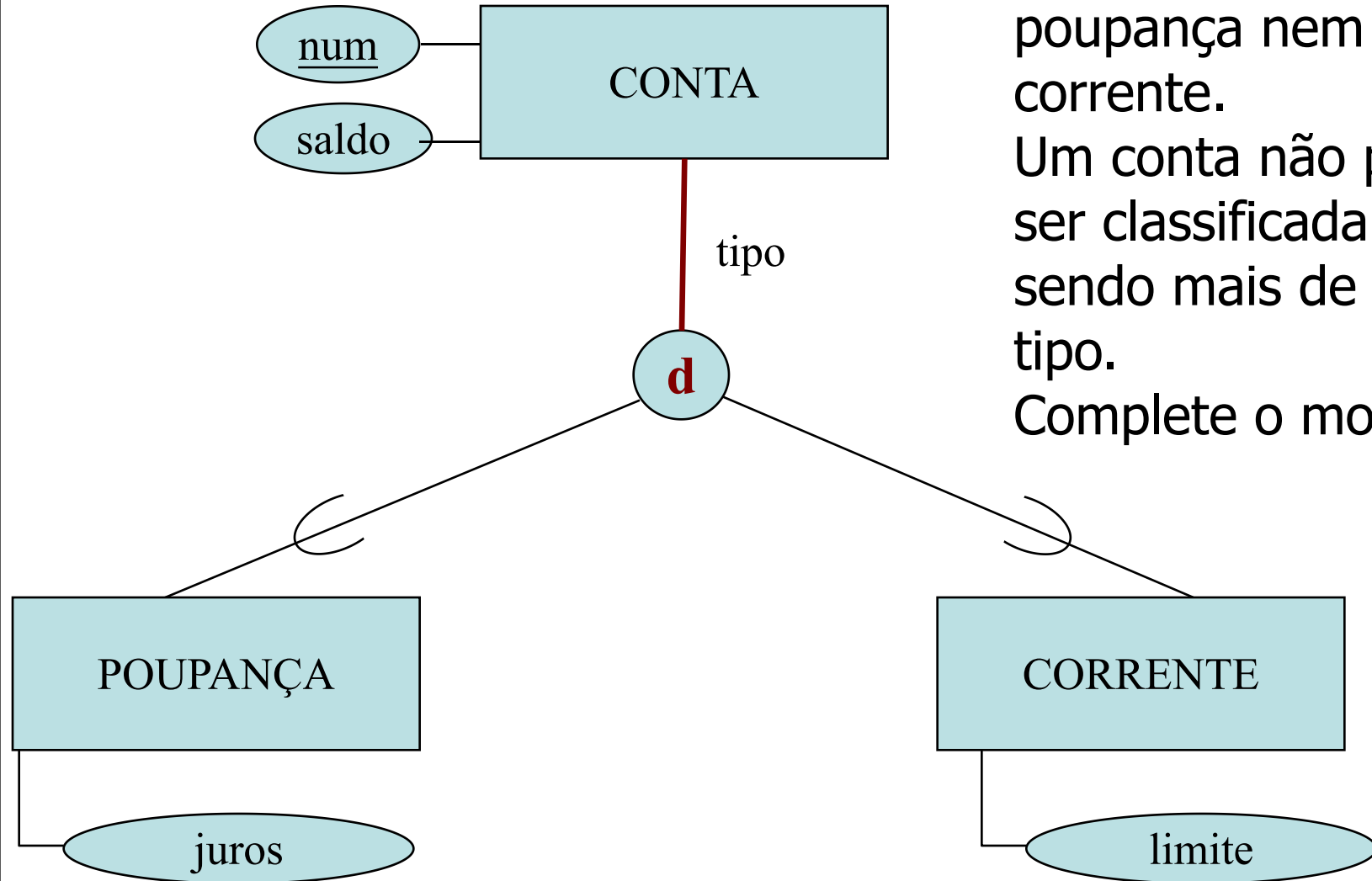


Exercício

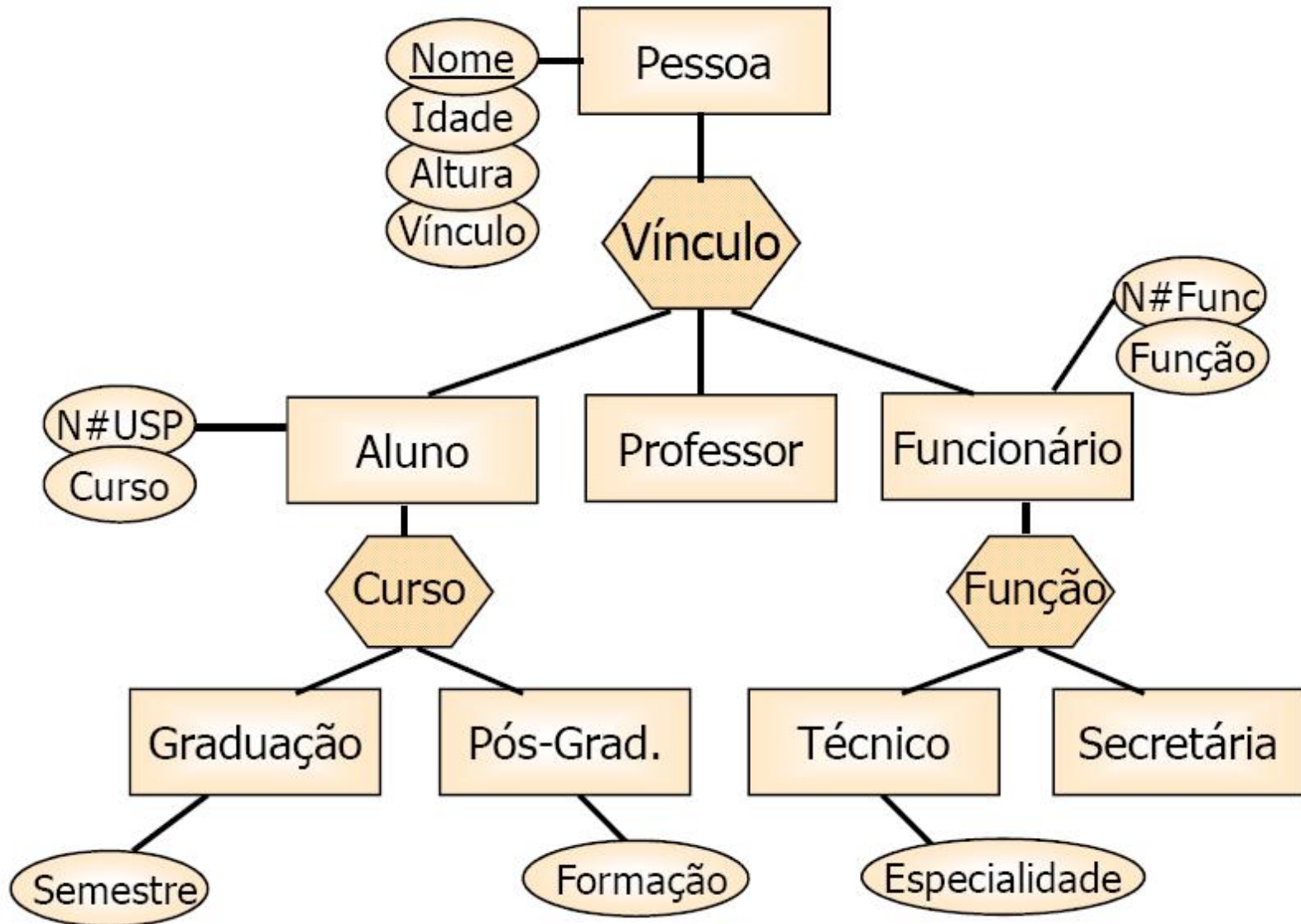
Existem contas que não são nem de poupança nem corrente.

Um conta não pode ser classificada como sendo mais de um tipo.

Complete o modelo!



Hierarquia de Classe – Múltiplos Níveis



Agregação

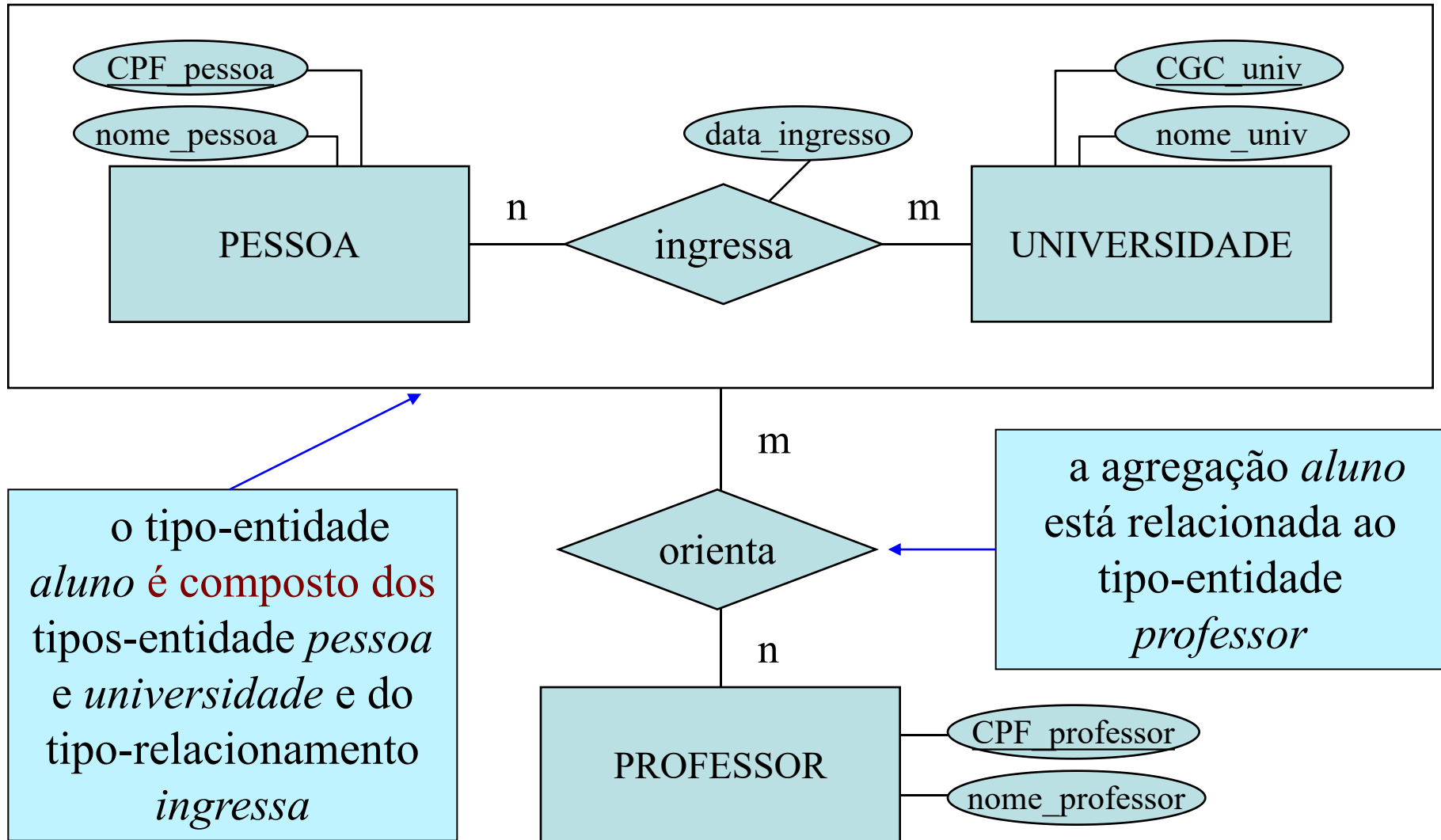
- É um conceito para a construção de objetos compostos a partir de seus objetos componentes
 - Idéia: elementos de modelagem podem associar-se, formando outros elementos que representam essa associação
- Pode assumir diversas formas:
 - Agregando Tipos-Entidade e Tipos-Relacionamentos
 - combinar entidades que estão relacionadas por uma instância de relacionamento em uma entidade agregada de alto nível

Agregação

- Tipos-entidades agregados são representados como tipos-entidades comuns
- Engloba
 - dois tipos-entidades e um tipo-relacionamento

Representação

ALUNO

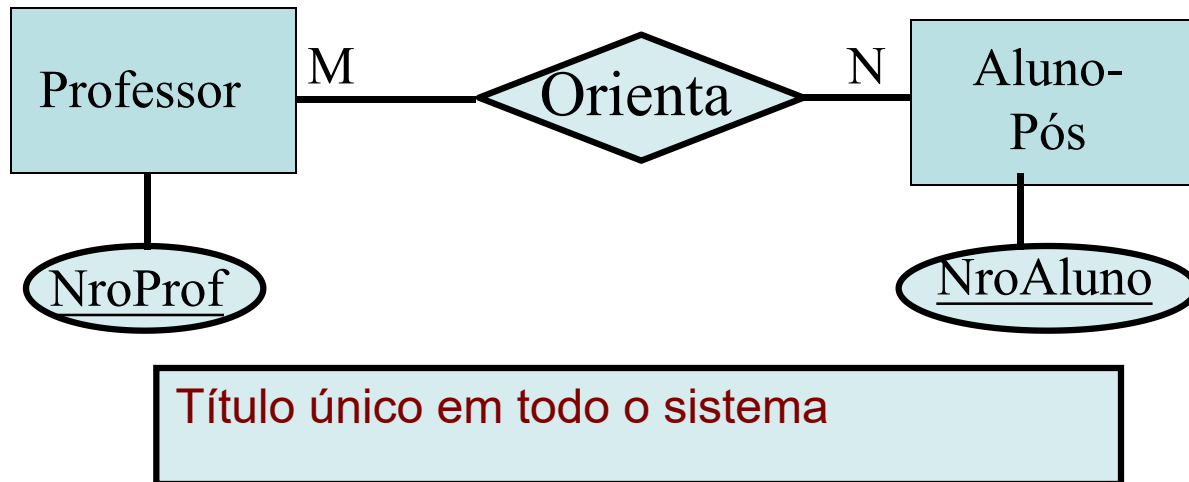


Agregação

- Situações que indicam a necessidade de agregação:
 - 1) Quando é necessário identificar cada relacionamento (o relacionamento tem chave)
 - 2) Quando é necessário mais de um relacionamento envolvendo as mesmas entidades
 - 3) Quando existe a necessidade de associar dois relacionamentos

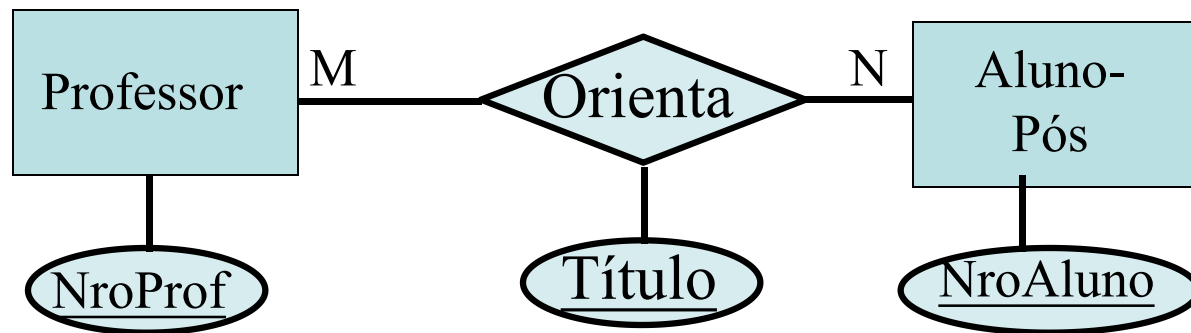
Agregação

- **1o. Caso:** O tipo relacionamento tem um identificador próprio:
 - Nesse caso, embora seja possível identificar a entidade agregação por um identificador próprio, ela também pode ser identificada pelo relacionamento entre as entidades que participam do relacionamento:



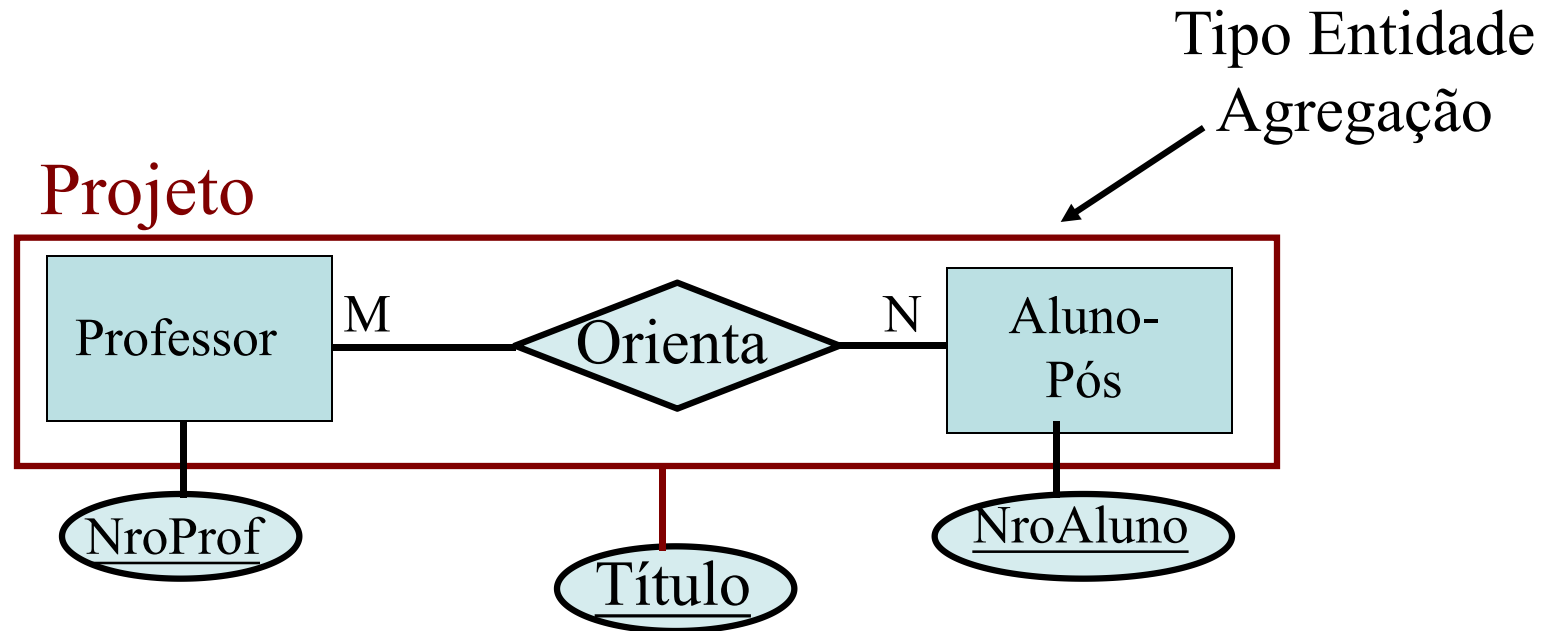
Agregação

- **1o. Caso:** O tipo relacionamento tem um identificador próprio:
 - Nesse caso, embora seja possível identificar a entidade agregação por um identificador próprio, ela também pode ser identificada pelo relacionamento entre as entidades que participam do relacionamento:



Um Tipo-Relacionamento não pode ter atributo identificador!

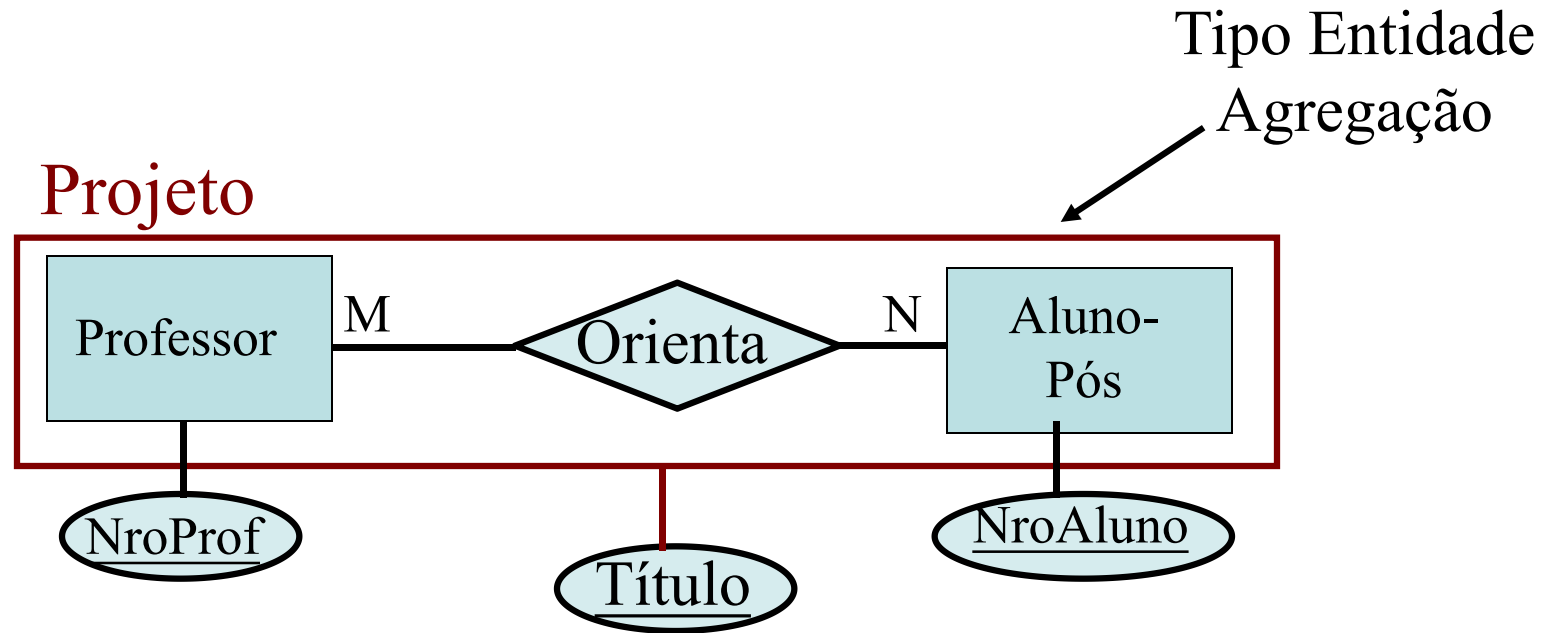
Agregação



Chave:

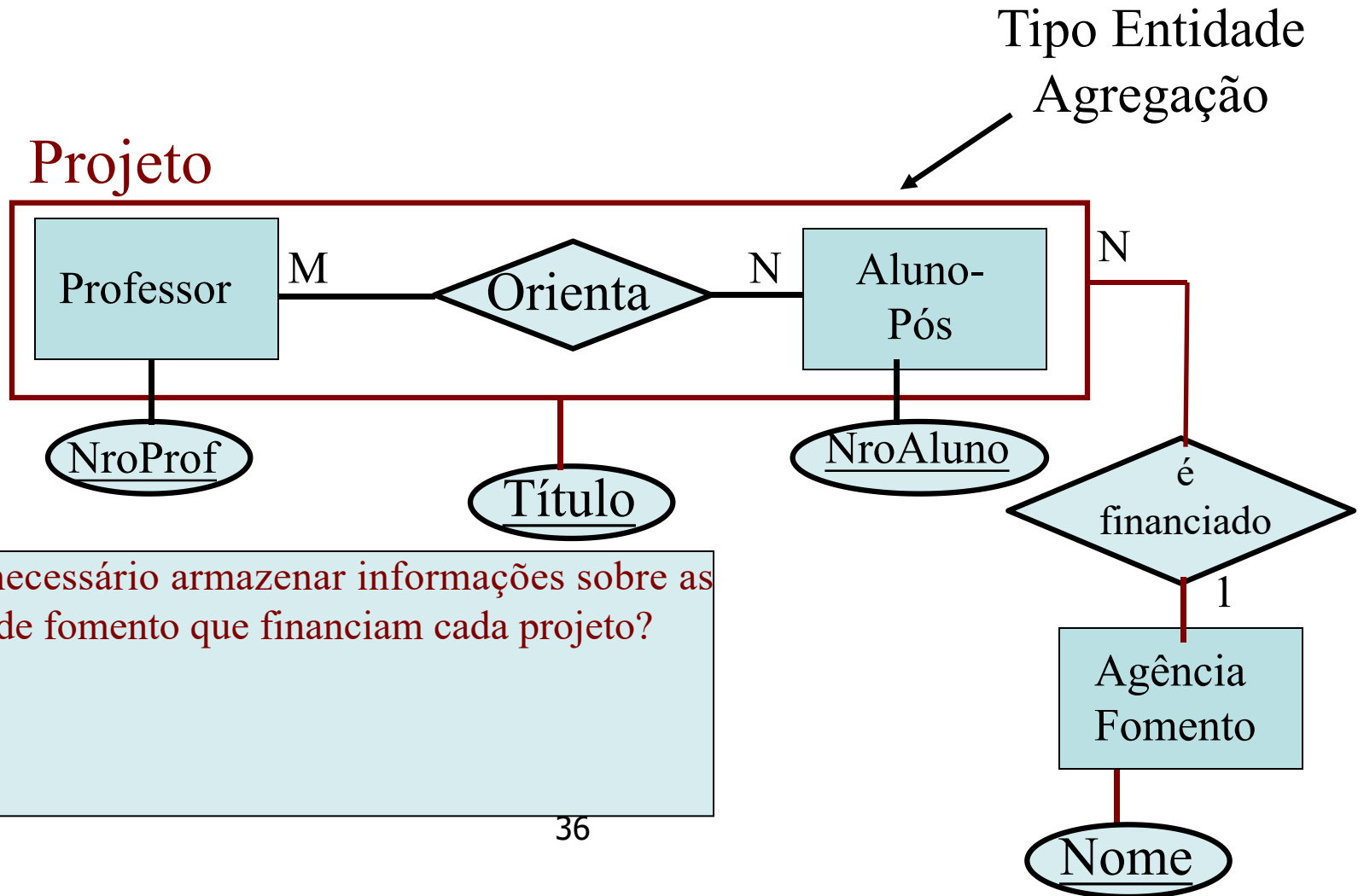
- NroProfessor + NroAluno
- ou
- Título

Agregação



E se for necessário armazenar informações sobre as agências de fomento que financiam cada projeto?

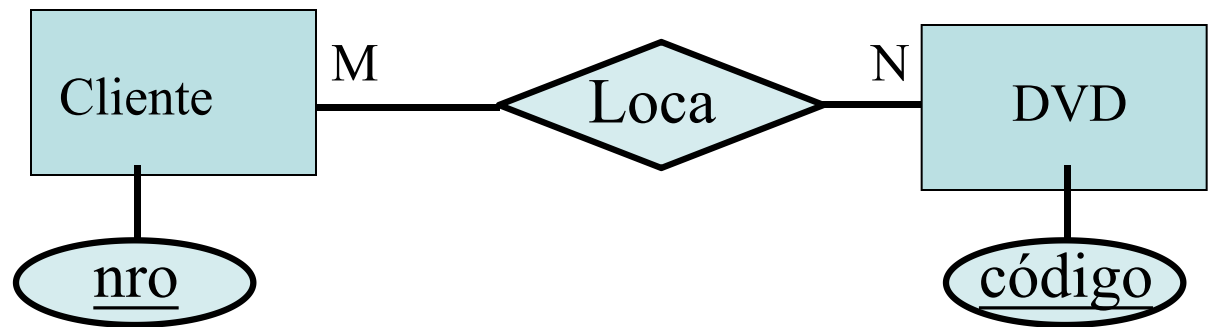
Agregação



E se for necessário armazenar informações sobre as agências de fomento que financiam cada projeto?

Agregação

- **2o. Caso:** Pode haver mais de um relacionamento envolvendo as mesmas entidades
 - Como identificar cada locação?

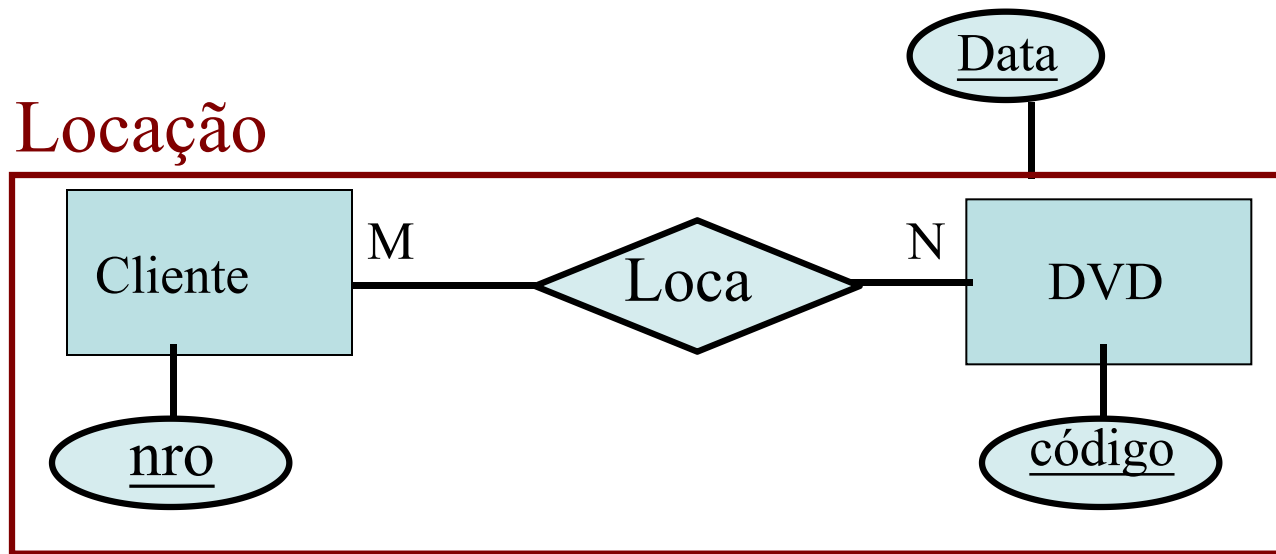


Aggregação

- **2o. Caso:** Pode haver mais de uma instância de relacionamento envolvendo as mesmas entidades
 - Como identificar cada locação?

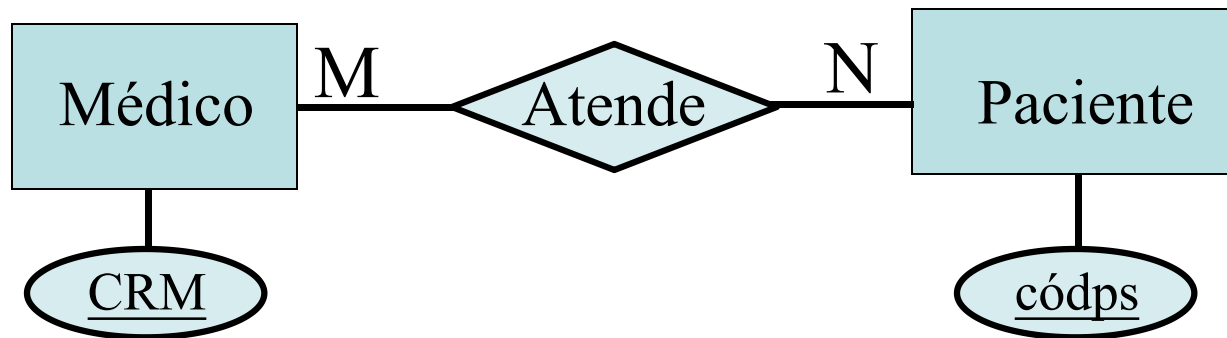
Chave:

- nro +
- código +
- data



Agregação

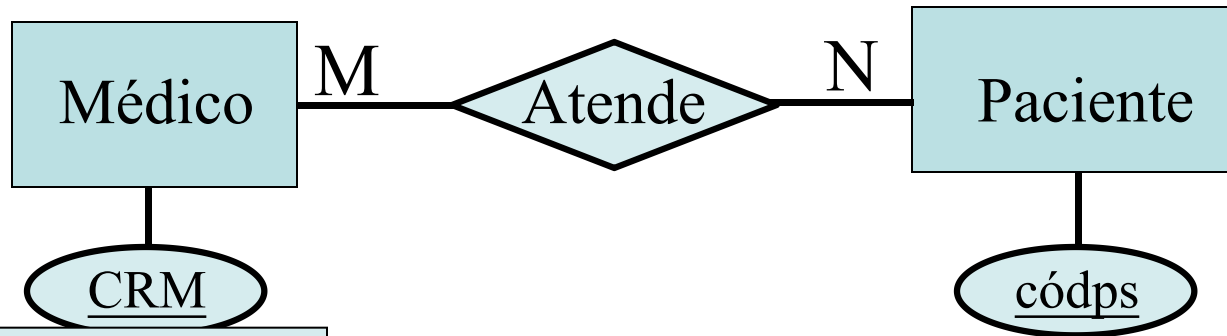
- **3o. Caso:** Pode haver a necessidade de associar dois tipos relacionamentos



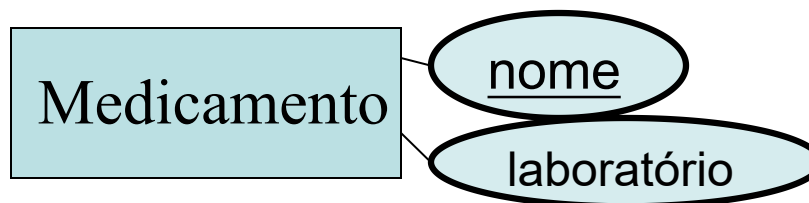
- Suponha que seja necessário alterar esse modelo do seguinte modo: é necessário saber que medicamentos existem e que medicamentos foram prescritos em cada consulta.

Agregação

- **3o. Caso:** Pode haver a necessidade de associar dois tipos relacionamentos



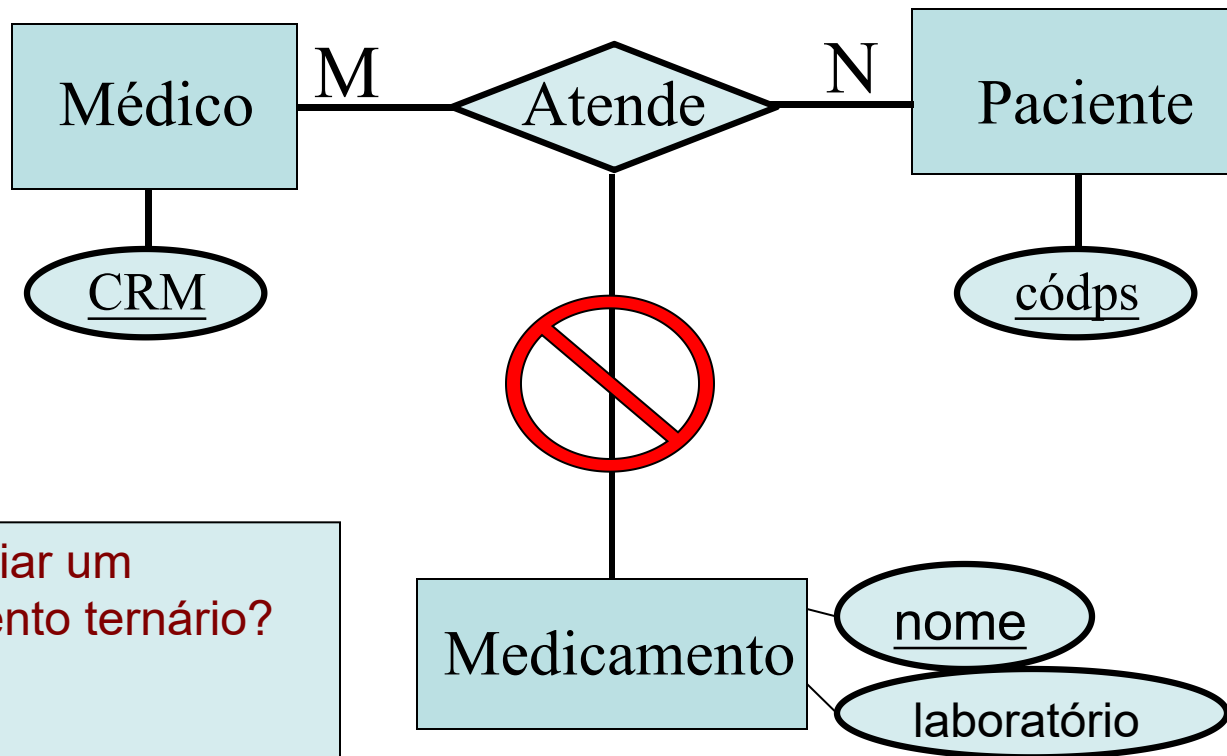
• A questão agora é: com que tipo entidade existente o novo tipo entidade deve ser associado?



Agregação

- Relacionamento Ternário?
- uma instância do relacionamento teria 1 médico atendendo 1 paciente e receitando 1 medicamento

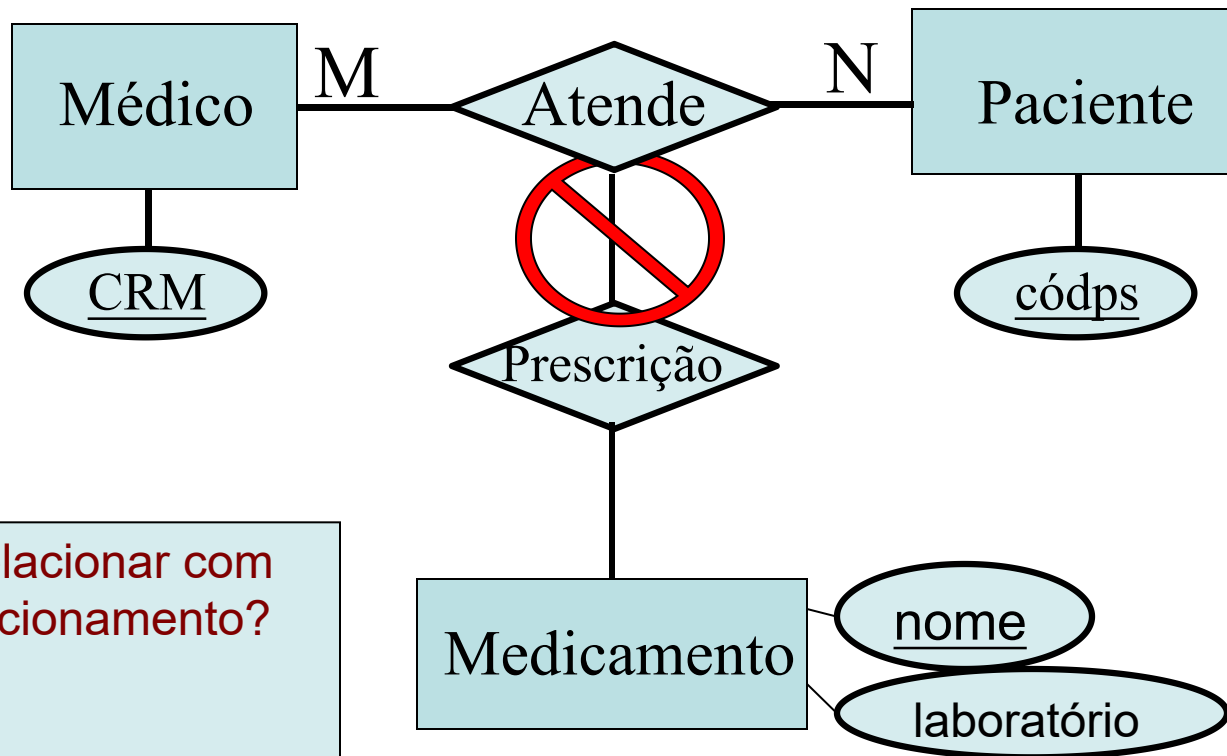
- **3o. Caso:** Pode haver a necessidade de associar dois tipos relacionamentos



- Podemos criar um relacionamento ternário?

Aggregação

- **3o. Caso:** Pode haver a necessidade de associar dois tipos relacionamentos

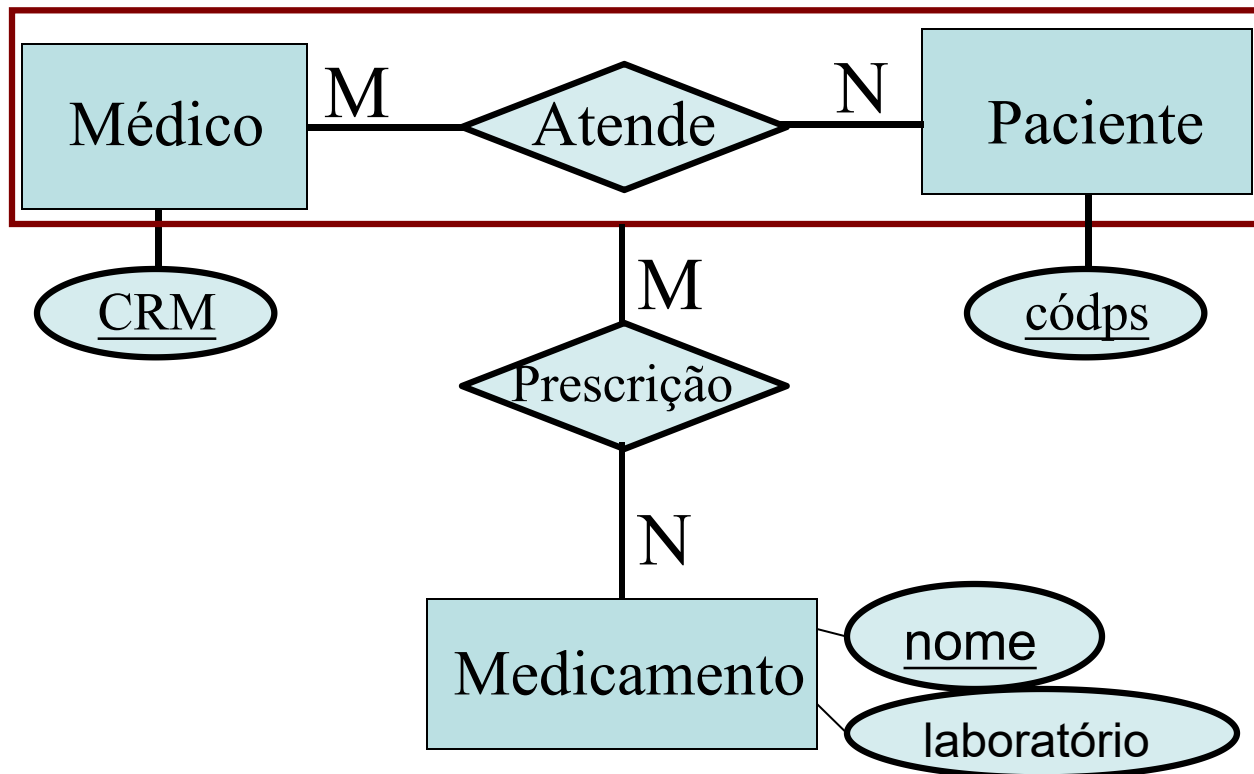


• Podemos relacionar com um tipo-relacionamento?

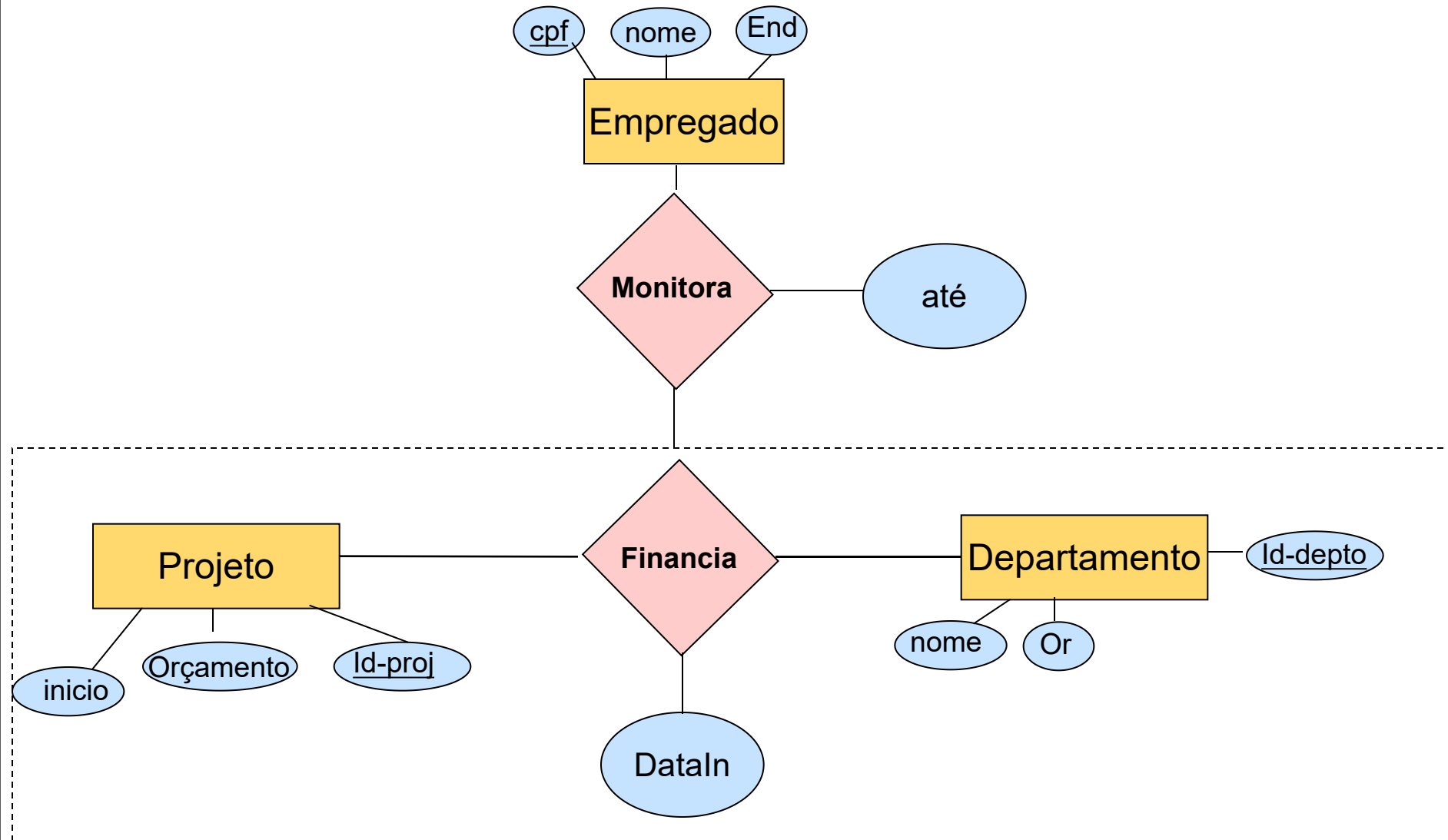
Aggregação

- **3o. Caso:** Pode haver a necessidade de associar dois tipos relacionamentos

Consulta



Agregação - Exemplos



Agregação

- Quando usar?
 - Quando é preciso expressar um relacionamento entre relacionamentos
 - No exemplo Financia por que não usar um relacionamento ternário?
 - Há dois relacionamentos distintos financia e monitora, cada um com seus próprios atributos

Projeto Conceitual de um BD

- Classificar conjuntos de entidades e atributos
 - Conjuntos de entidade possuem informações descritivas, atributos não
 - Atributos devem ser mantidos de forma atômica
 - Atributos devem ser relacionados às entidades que eles descrevem
- Identificar chaves primárias

Projeto Conceitual de um BD

- Identificar conjuntos de relacionamentos e seus atributos
 - Determinar o grau dos conjuntos de relacionamentos
 - Identificar as restrições que se aplicam sobre cada conjunto de relacionamento
 - Chave
 - Participação
- Identificar conjunto de entidades forte e conjunto de entidades fraca

Projeto Conceitual de um BD

- Modelar hierarquias de generalização
 - Identificar atributos e relacionamentos comuns
 - Determinar as restrições de sobreposição e cobertura
 - Modelar agregações
- Bom conhecimento do domínio do problema, que está sendo modelado

Projeto conceitual com o modelo ER - Escolhas!

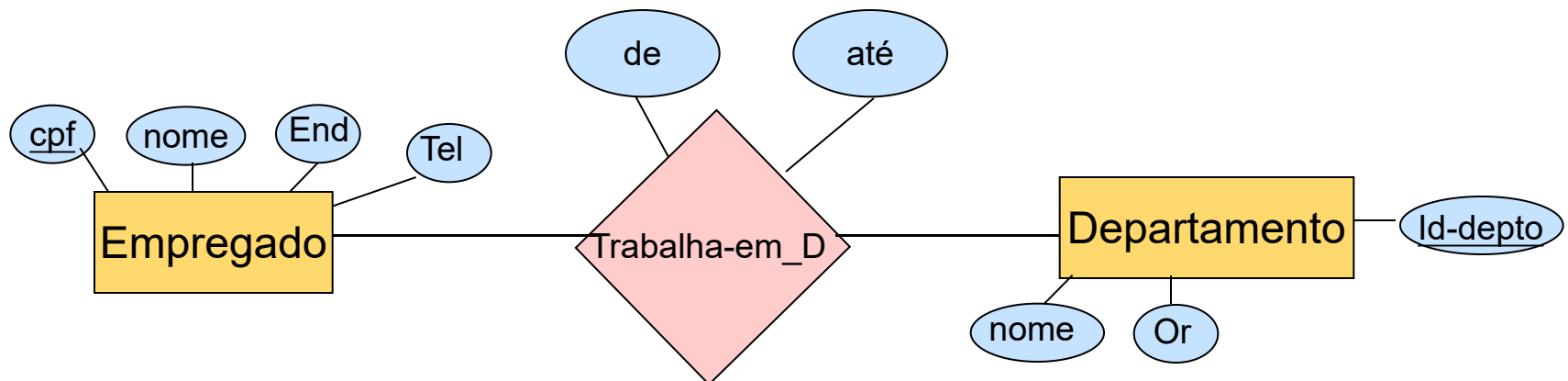
- Um conceito deve ser modelado como uma entidade ou um atributo?
- Um conceito deve ser modelado com uma entidade ou um relacionamento?
- Quais são os conjuntos de relacionamentos e seus conjuntos de entidades participantes? Devemos usar relacionamentos binários ou ternários?
- Devemos usar agregação?

Entidade versus Atributo

- Ex 1: Adição de informações de endereço ao conjunto de entidades Funcionários
 - Opção 1: usar um atributo Endereço
 - Adequada se precisarmos registrar apenas um endereço por funcionário
 - Opção 2: conjunto de entidades Endereços e registrar as associações entre funcionários e endereços por meio de um relacionamento
 - Adequada quando se deve registrar mais de um endereço para cada funcionários
 - Quando se deseja capturar a estrutura de um endereço no diagrama ER. Ex: Rua, Nro, Cidade, Estado, País e CEP

Entidade versus Atributo

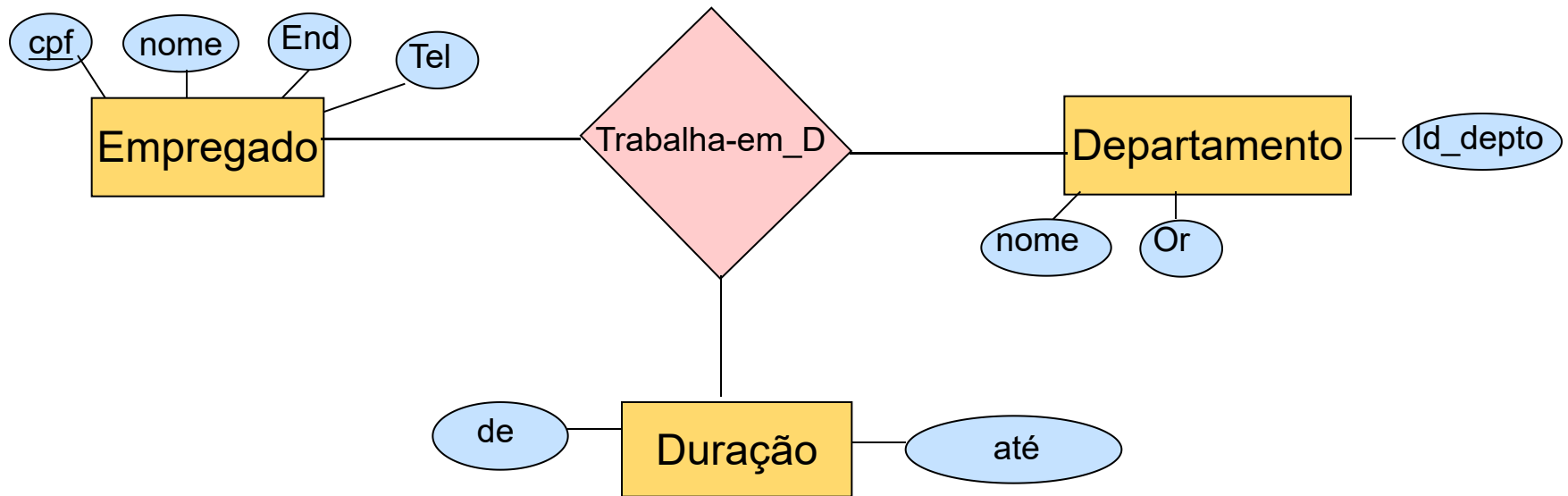
- Ex 2: conjunto de relacionamentos
Trabalha_em_D
 - Possui os atributos *de* e *até* ao invés de *desde* (usado em Trabalha_em)
 - Registra o intervalo durante o qual um empregado trabalha para um departamento



Entidade versus Atributo

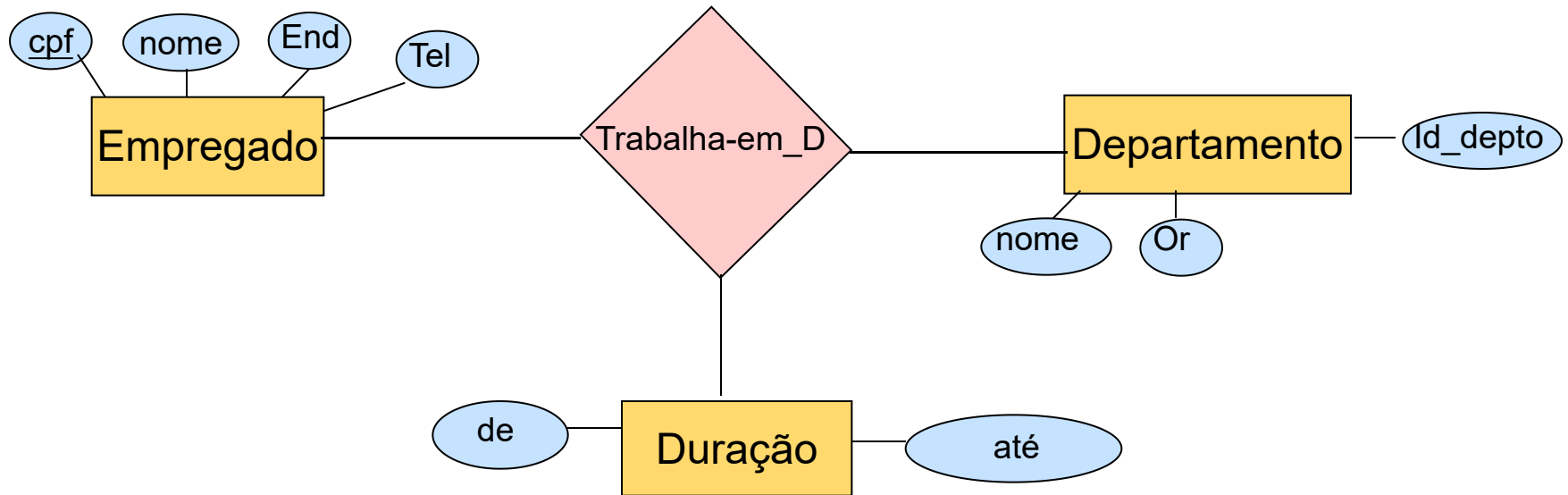
- Ex 2: Suponha que seja possível que um empregado trabalhe em determinado departamento por mais do que um período
 - Essa possibilidade não é permitida pela semântica do diagrama ER → um relacionamento é identificado univocamente pelas entidades participantes
 - Sugestão: introduzir um conjunto de entidades Duração com atributos *de* e *até*

Entidade versus Atributo



Como poderia ser uma solução melhor para o problema?

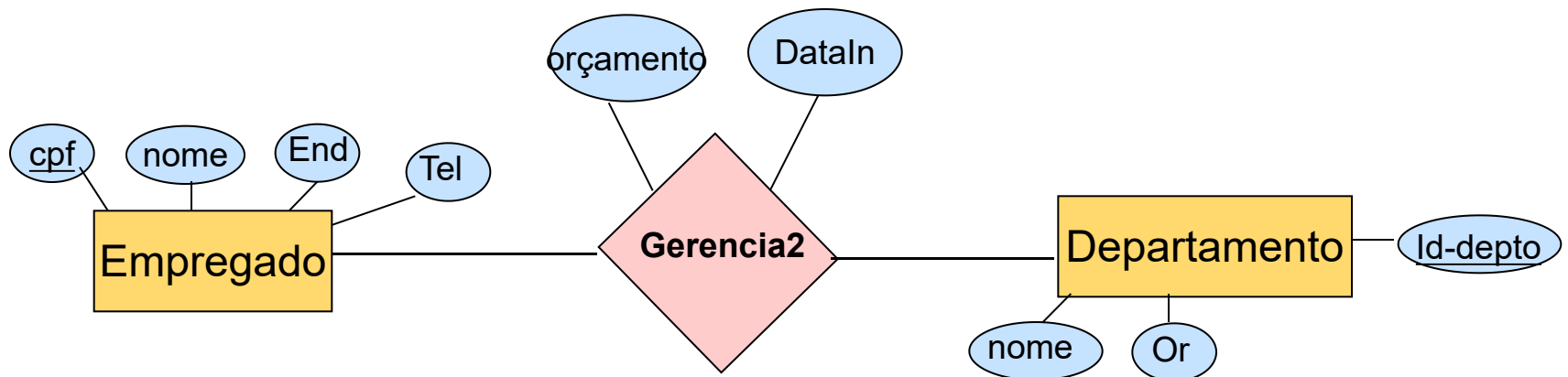
Entidade versus Atributo



Como poderia ser uma solução melhor para o problema? **Agregação**

Entidade versus Relacionamento

- Ex 3: considere o conjunto de relacionamentos Gerencia2 no qual cada gerente de departamento possua um orçamento arbitrário (orçamento)
 - Gerente recebe um orçamento arbitrário separado para cada departamento que ele gerencia

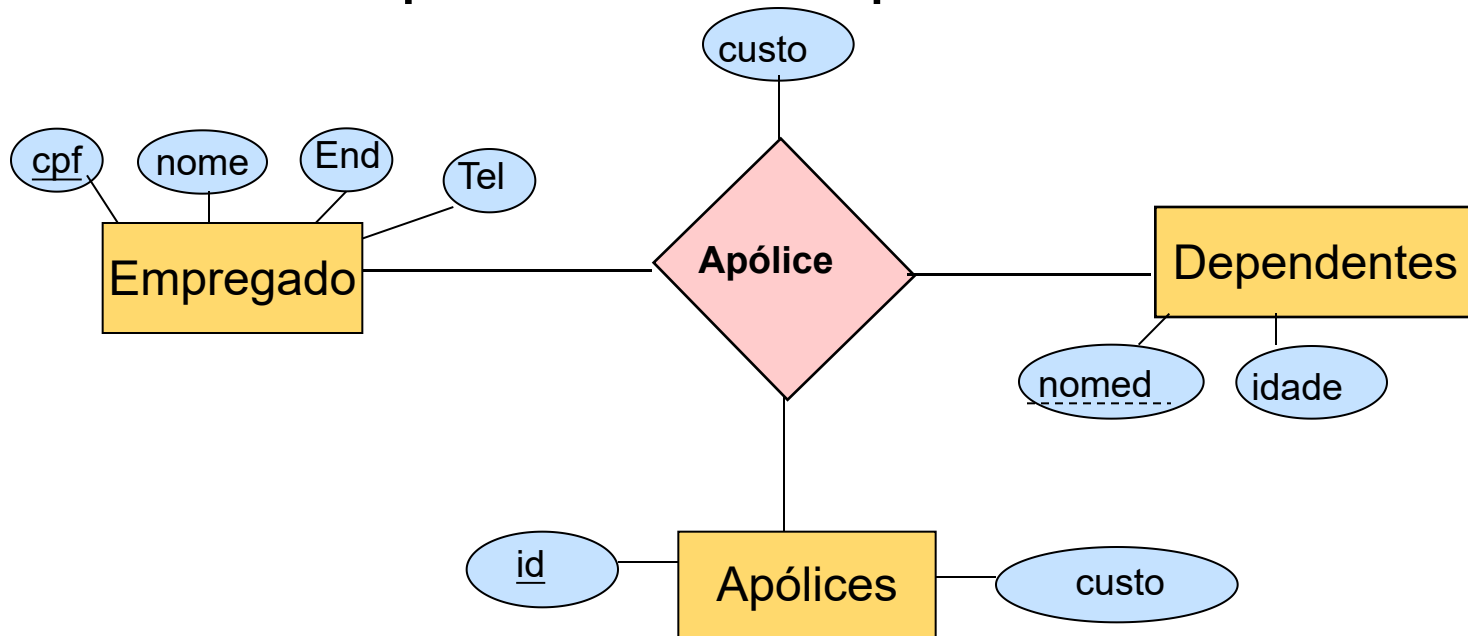


Entidade versus Relacionamento

- Ex 3: E se o orçamento arbitrário for uma soma de todos os departamentos gerenciados por esse funcionário?
 - Atributo Orçamento no relacionamento Gerencia2 → armazenamento redundante
 - Solução: introduzir um novo conjunto de entidades chamado Gerentes → hierarquia - É-UM em funcionário
 - Gerente terá o atributo Orçamento

Relacionamento Binário versus Ternário

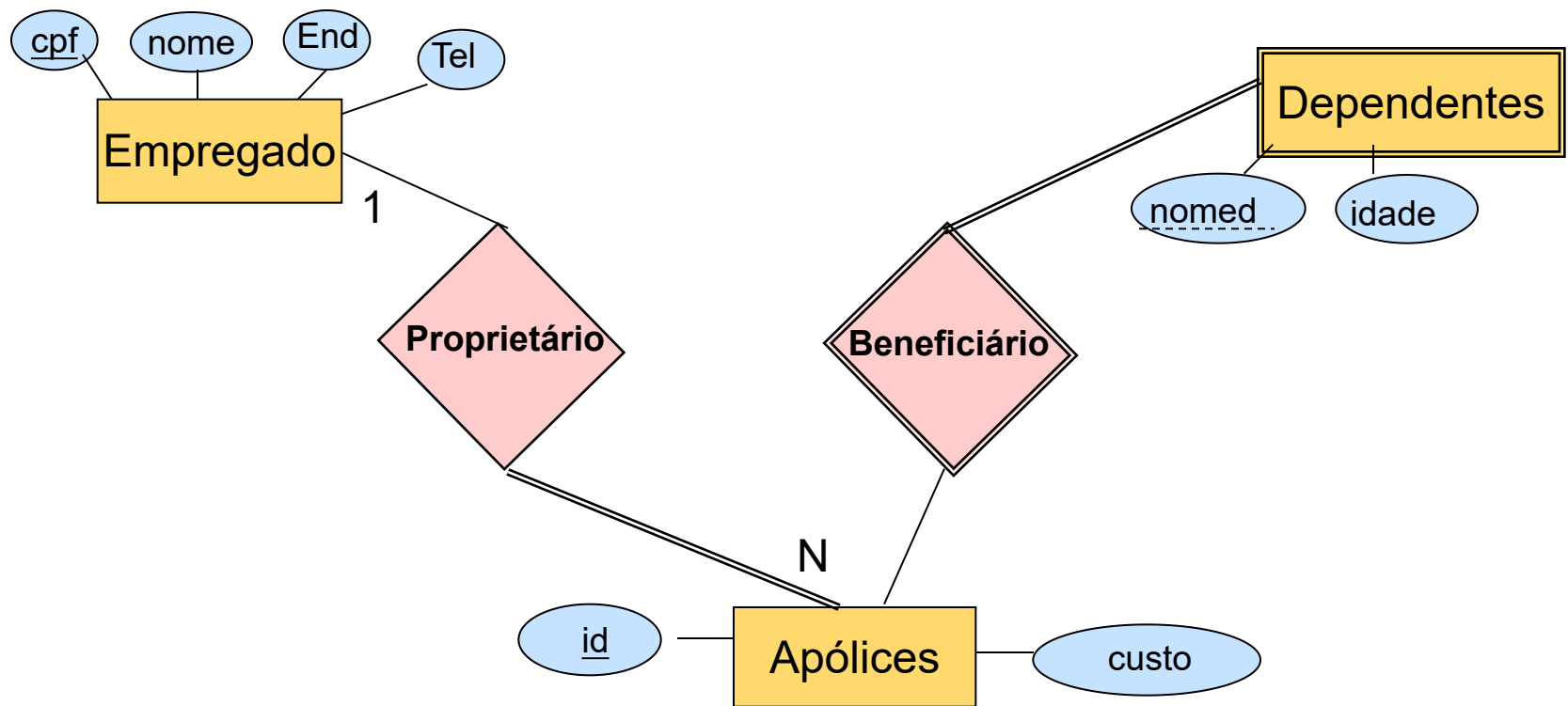
- Ex: um funcionário pode possuir várias apólices de seguro, cada uma pode ser pertencente a diversos funcionários e cada dependente pode ser coberto por diversas apólices



Relacionamento Binário versus Ternário

- Ex: continuação
 - Requisitos adicionais
 - R1: Uma apólice não pode pertencer em conjunto a dois ou mais funcionários
 - R2: Cada apólice deve pertencer a algum funcionário
 - R3: Dependentes é um conjunto de entidades fracas e cada entidade dependente é univocamente identificada pelo nome em conjunto com id-apólice
 - Ações
 - R1: Restrição de chave em Apólices → Prob: uma apólice só cobre um dependente
 - R2: Restrição de participação total em Apólices → Prob: cada apólice cobre no mínimo um dependente
 - R3: relacionamento binário

Relacionamento Binário versus Ternário



Relacionamento Binário versus Ternário

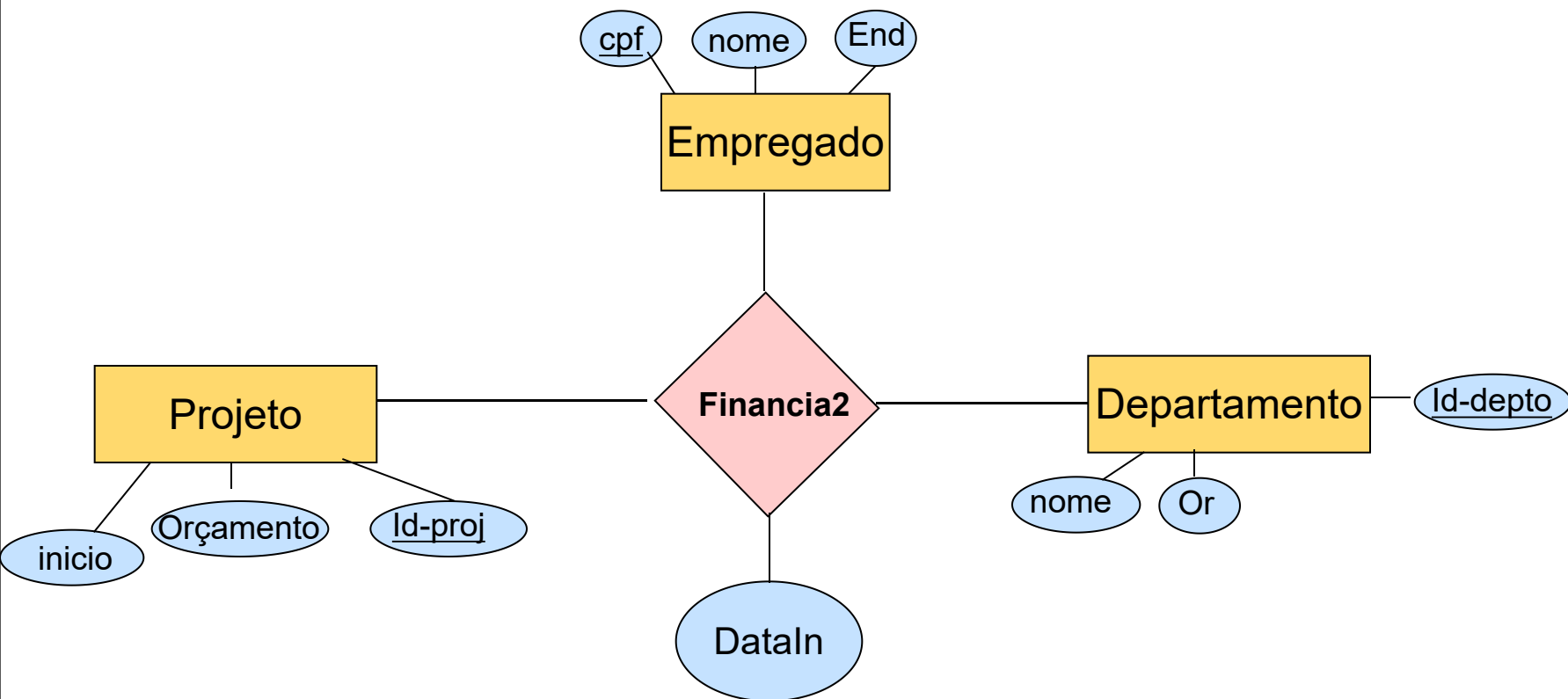
- Ex: relacionamento típico ternário
 - Suprimentos, Fornecedores e Departamentos
 - Conjunto de relacionamentos Contratos
 - Atributo descritivo: quantidades
 - Um contrato: Um fornecedor fornecerá um suprimento a um departamento

Relacionamentos binários não são adequados para esse caso

Agregação versus Relacionamentos Ternários

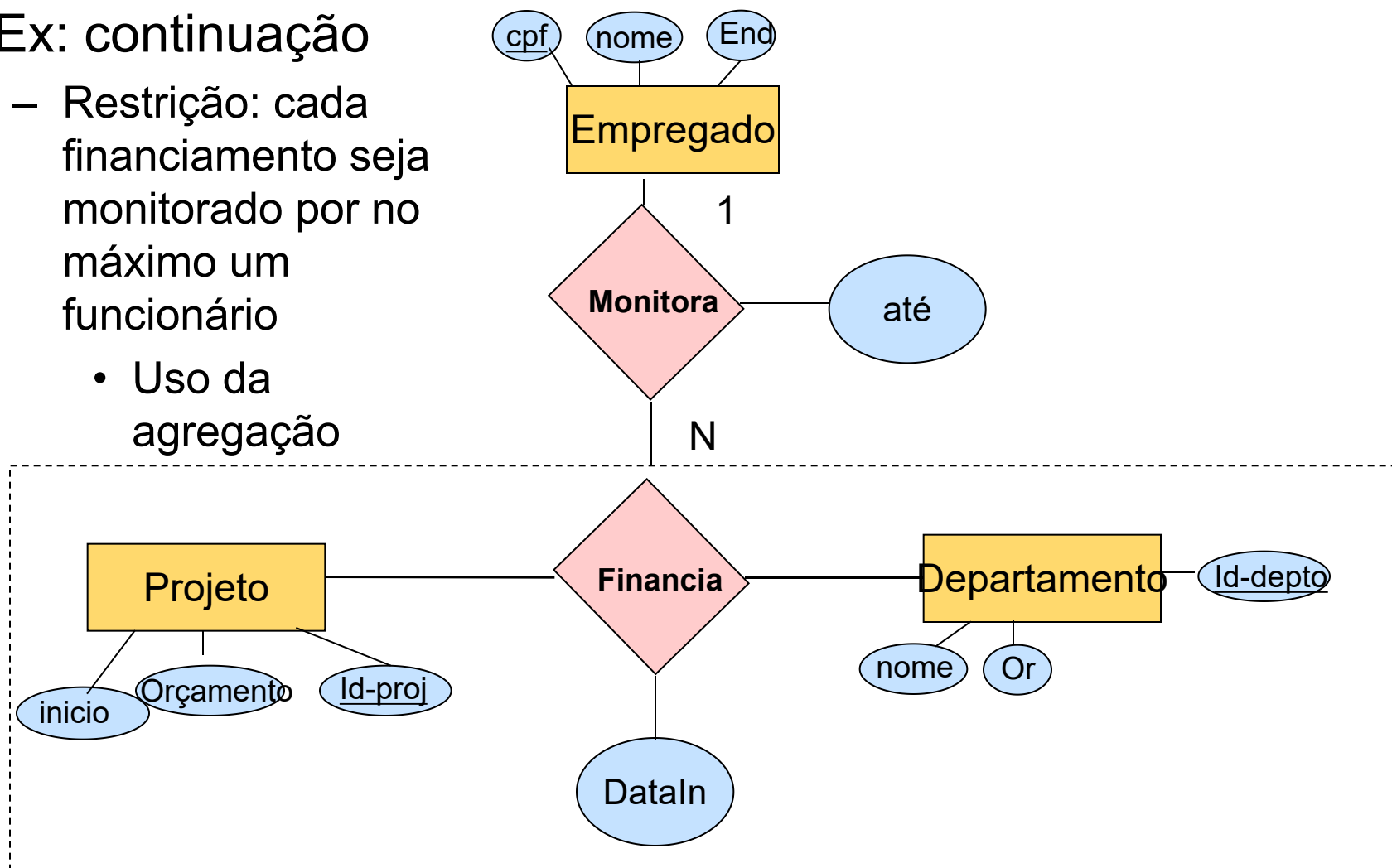
- Ex: Um projeto pode ser financiado por qualquer número de departamentos, um departamento pode financiar um ou mais projetos e cada financiamento é monitorado por um ou mais funcionários

Agregação versus Relacionamentos Ternários



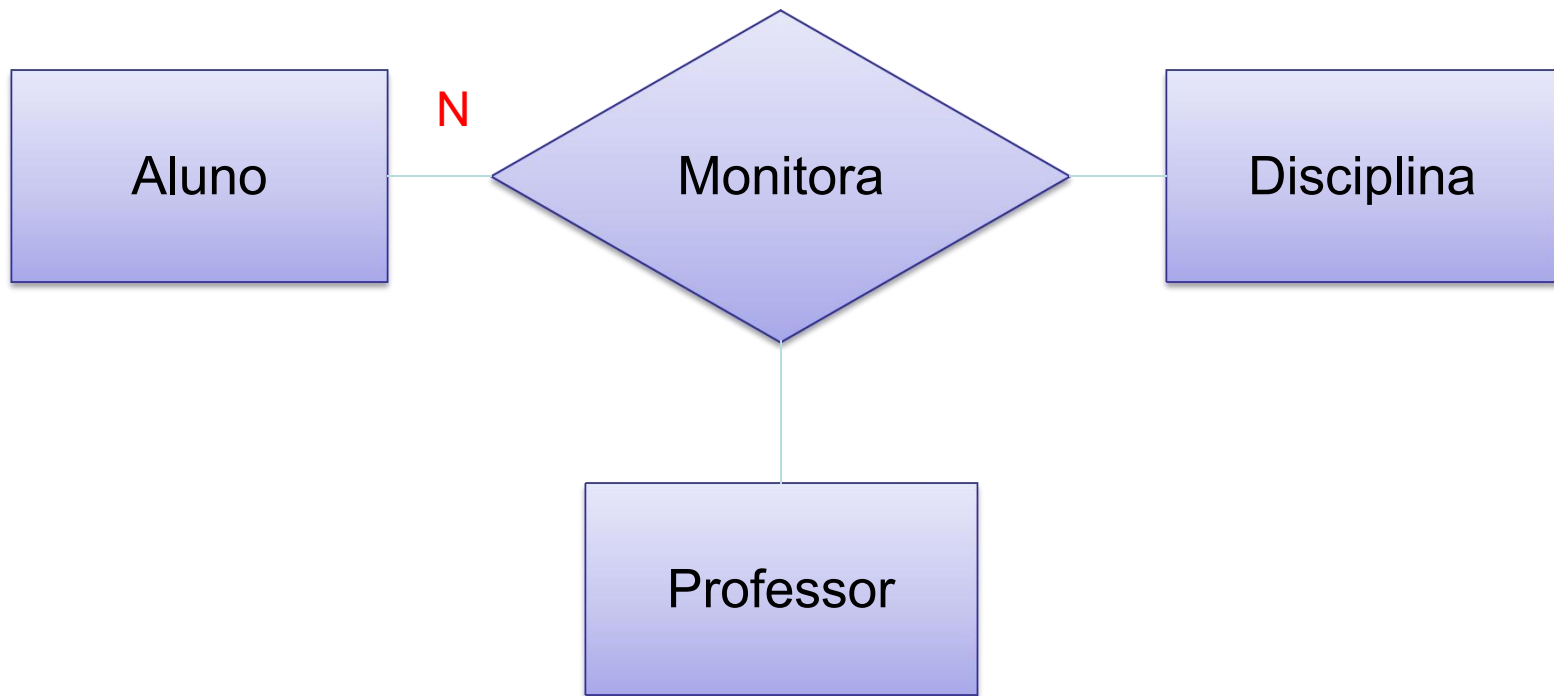
Agregação versus Relacionamentos Ternários

- Ex: continuação
 - Restrição: cada financiamento seja monitorado por no máximo um funcionário
 - Uso da agregação



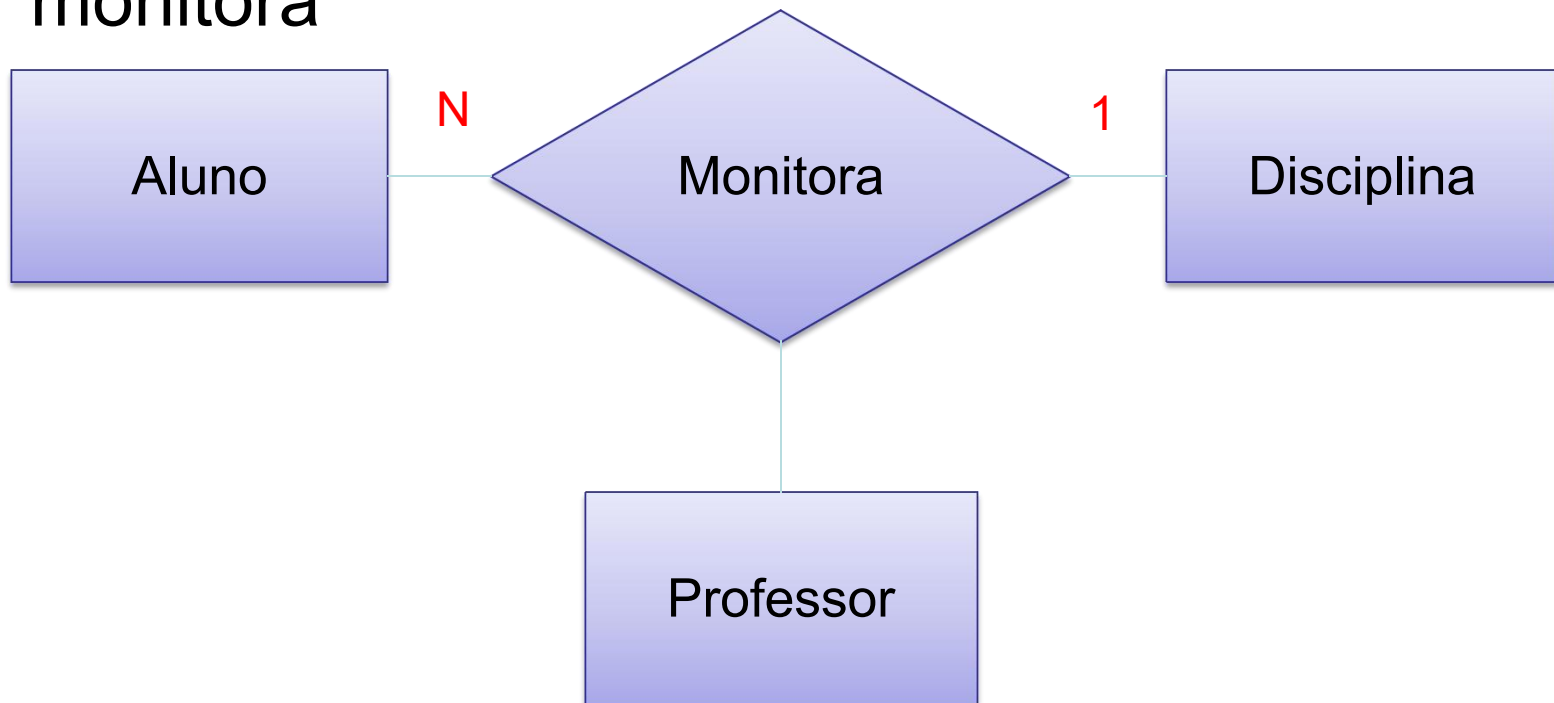
Solução exercício

- Dado um Professor e uma Disciplina, pode existir **mais de um** aluno monitor que a monitora



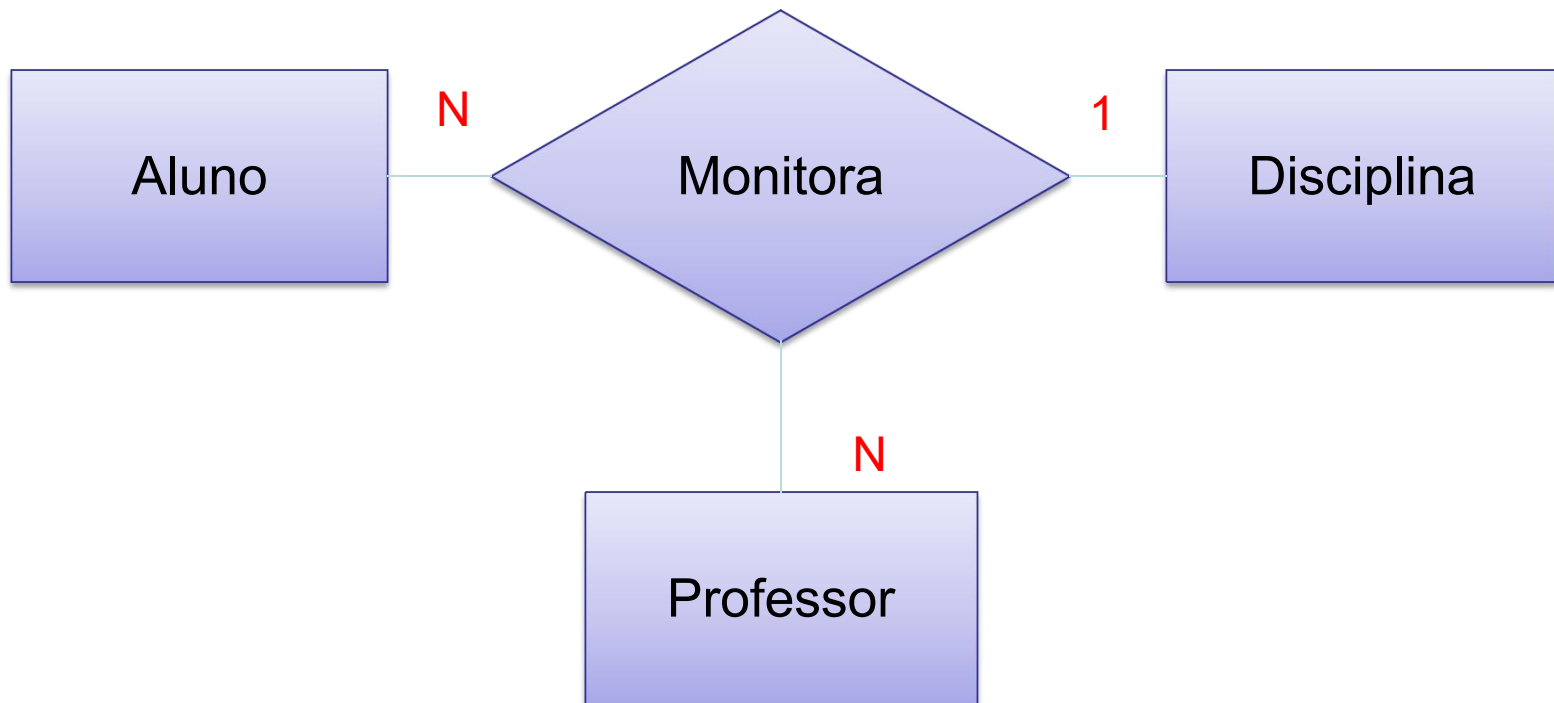
Solução exercício

- Dado um Professor e um Aluno monitor, existe **no máximo uma** disciplina que esse aluno monitora



Solução exercício

- Dado uma Disciplina e um Aluno monitor, **mais de um** professor pode ser responsável



Exercício 1

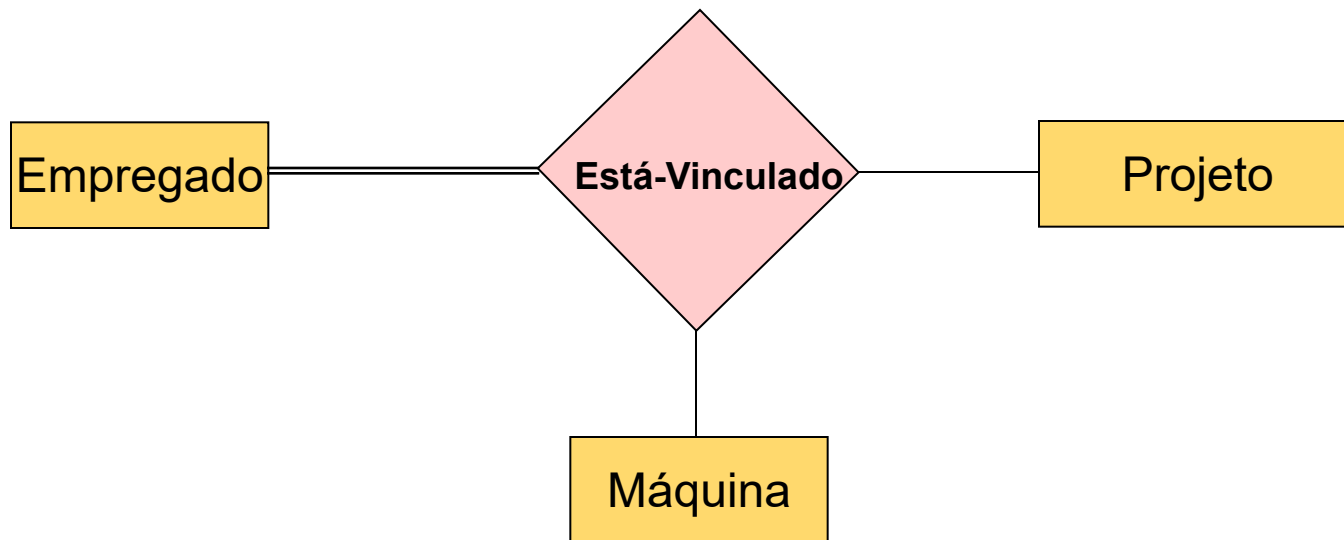
Em uma companhia têm-se **Empregados** trabalhando em **Projetos** e utilizando **Máquinas** nestes projetos.

Fazer o DER para cada uma das situações dadas a seguir:

- **S1:** Todo empregado trabalha em algum projeto e utiliza alguma máquina nos projetos em que trabalha.
- **S2:** Nem todo empregado trabalha em projetos, mas aqueles que trabalham em projetos utilizam necessariamente alguma máquina nestes projetos
- **S3:** Todo empregado que trabalha em projetos utiliza uma única máquina nos projetos em que trabalha
- **S4:** Nem todo empregado trabalhando em projetos utiliza máquinas nestes projetos.
- **S5:** Nem todo empregado que faz uso de máquinas está trabalhando em algum projeto. Aqueles que trabalham em projetos podem fazer uso de máquinas ou não.

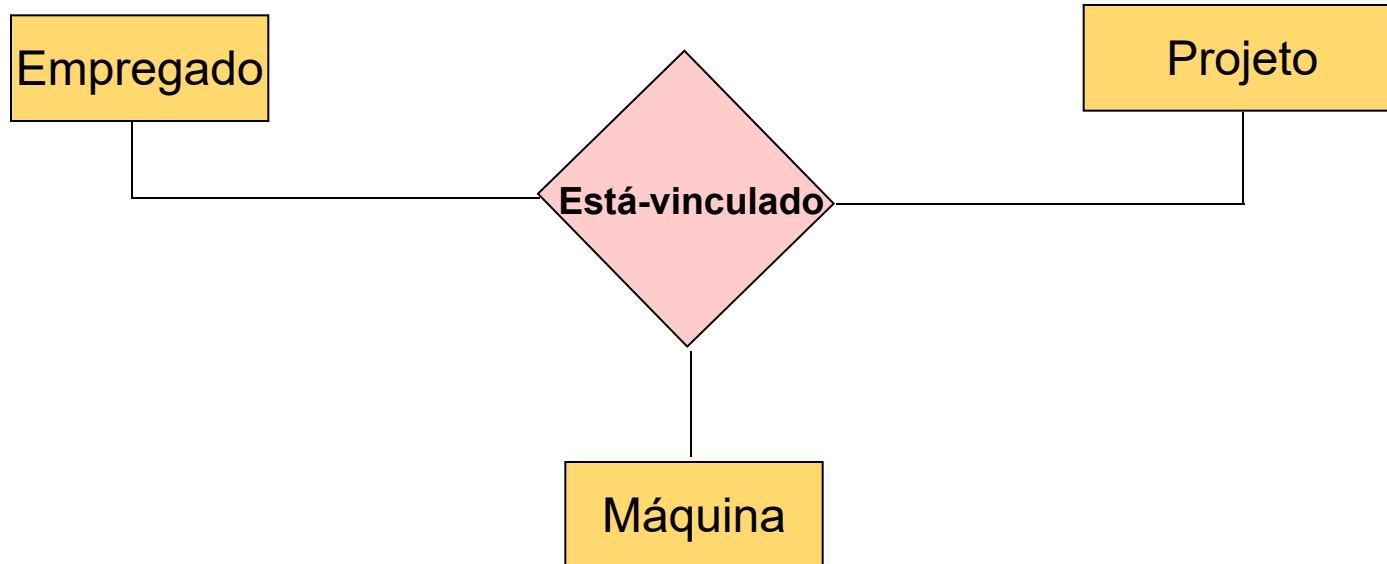
Situação 1

- Todo empregado trabalha em algum projeto e utiliza alguma máquina nos projetos em que trabalha.



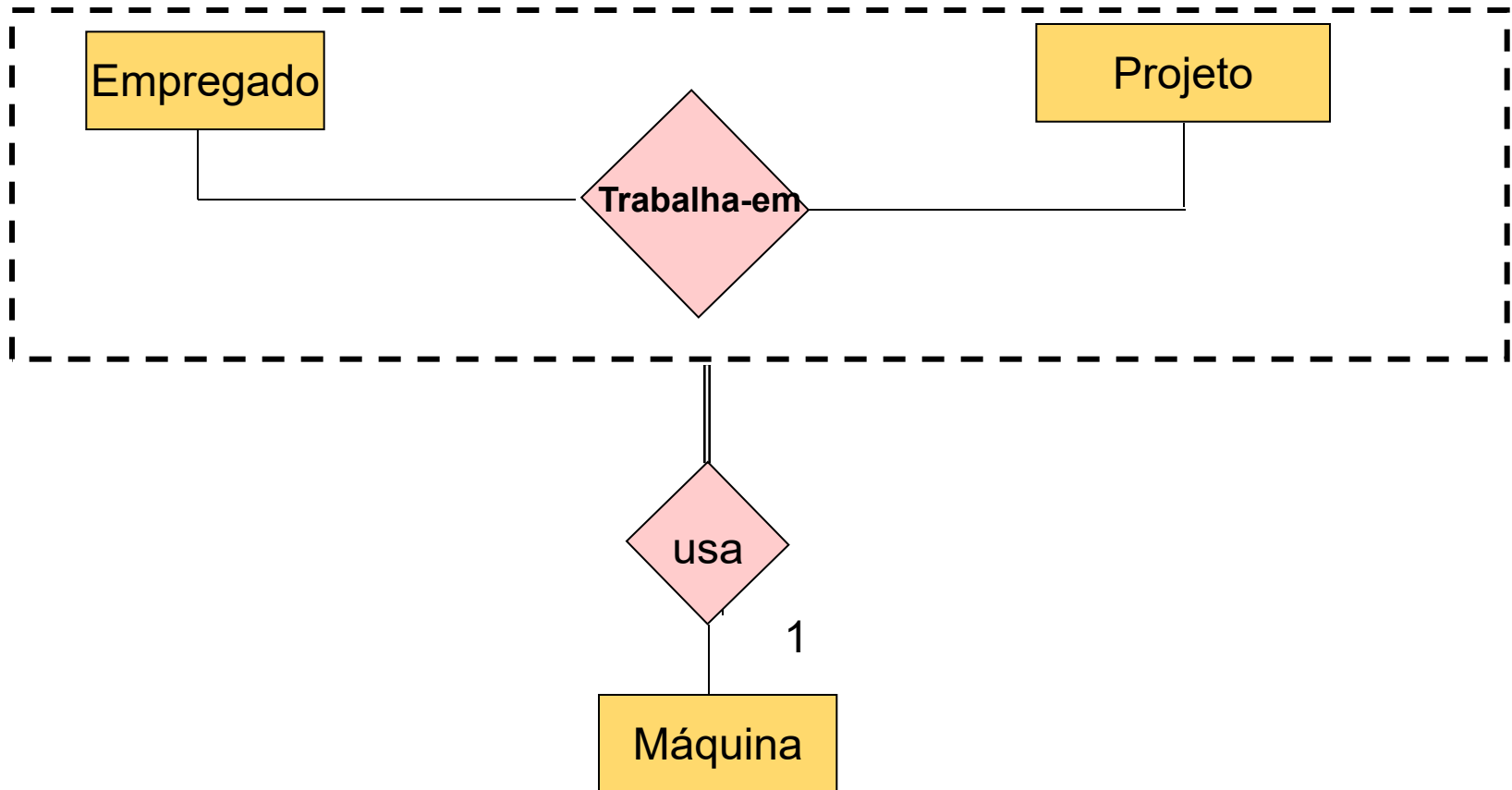
Situação 2

- Nem todo empregado trabalha em projetos, mas aqueles que trabalham em projetos utilizam necessariamente alguma máquina nestes projetos



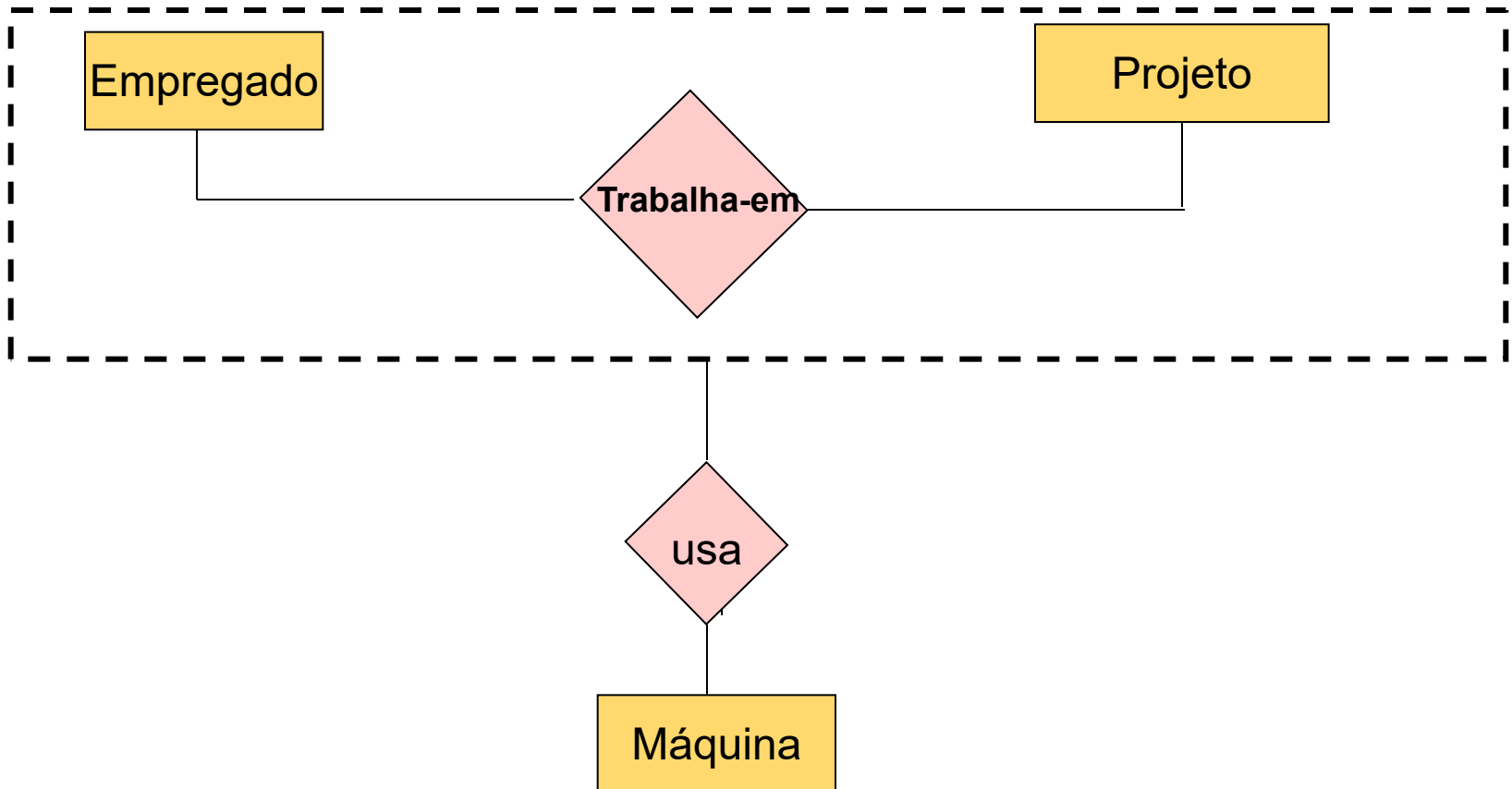
Situação 3

- Todo empregado que trabalha em projetos utiliza **uma única máquina** nos projetos em que trabalha



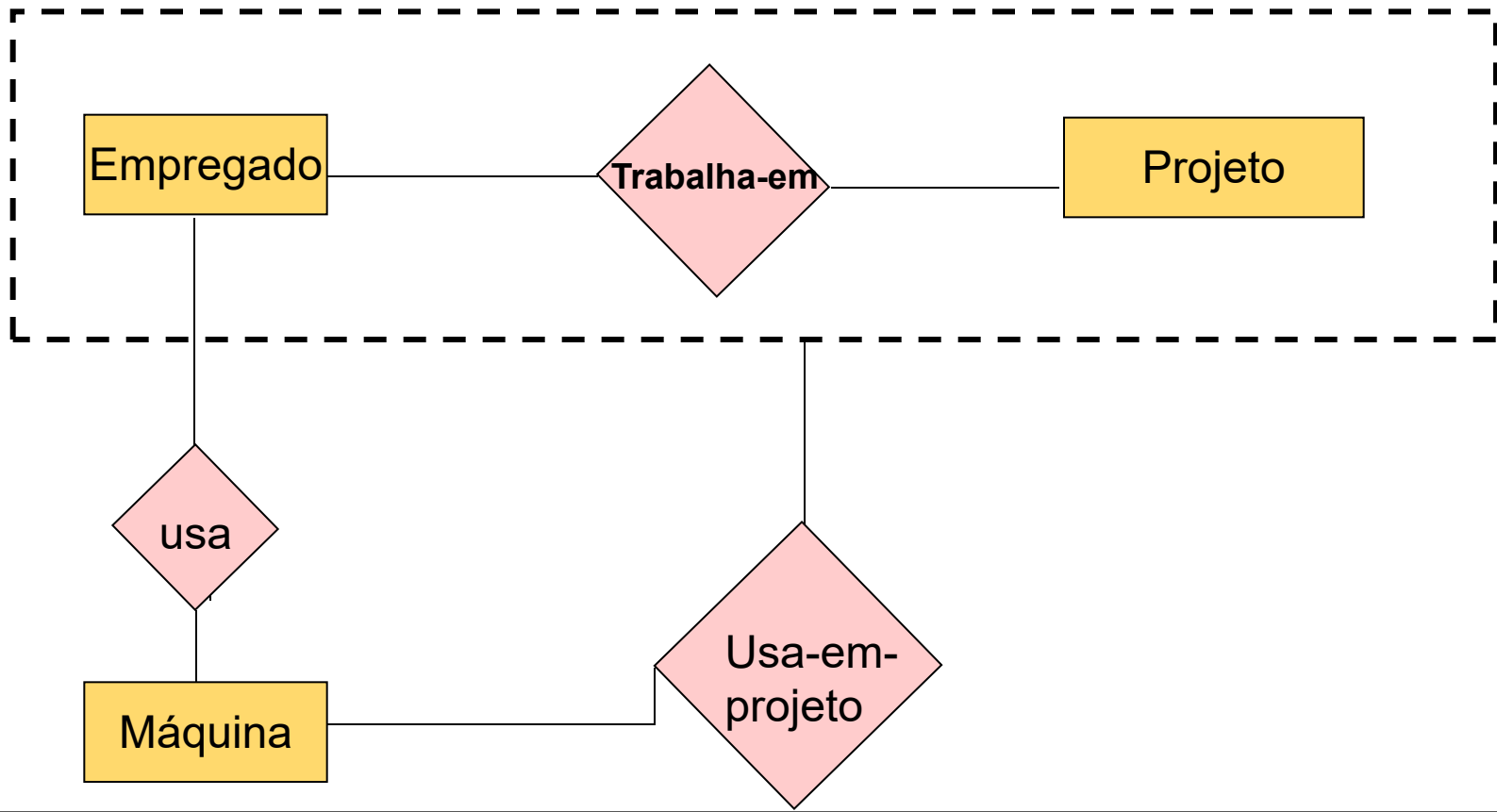
Situação 4

- Nem todo empregado trabalhando em projetos utiliza máquinas nestes projetos.



Situação 5

- Nem todo empregado que faz uso de máquinas está trabalhando em algum projeto. Aqueles que trabalham em projetos podem fazer uso de máquinas ou não.



Referências

- R. Elmasri e S. B. Navathe, Sistema de Banco de Dados, 6ª edição, Pearson, 2011.
- R. Ramakrishnan e J. Gehrke, *Database Management Systems*, 3a Edição, McGraw-Hill, 2003.
- Slides da Profa. Sandra de Amo – FACOM - UFU
- Slides do Prof. Eduardo R. Hruschka – ICMC - USP

Aula 5 – SBD

Modelo Relacional

Profa. Elaine Faria
UFU

Agradecimentos

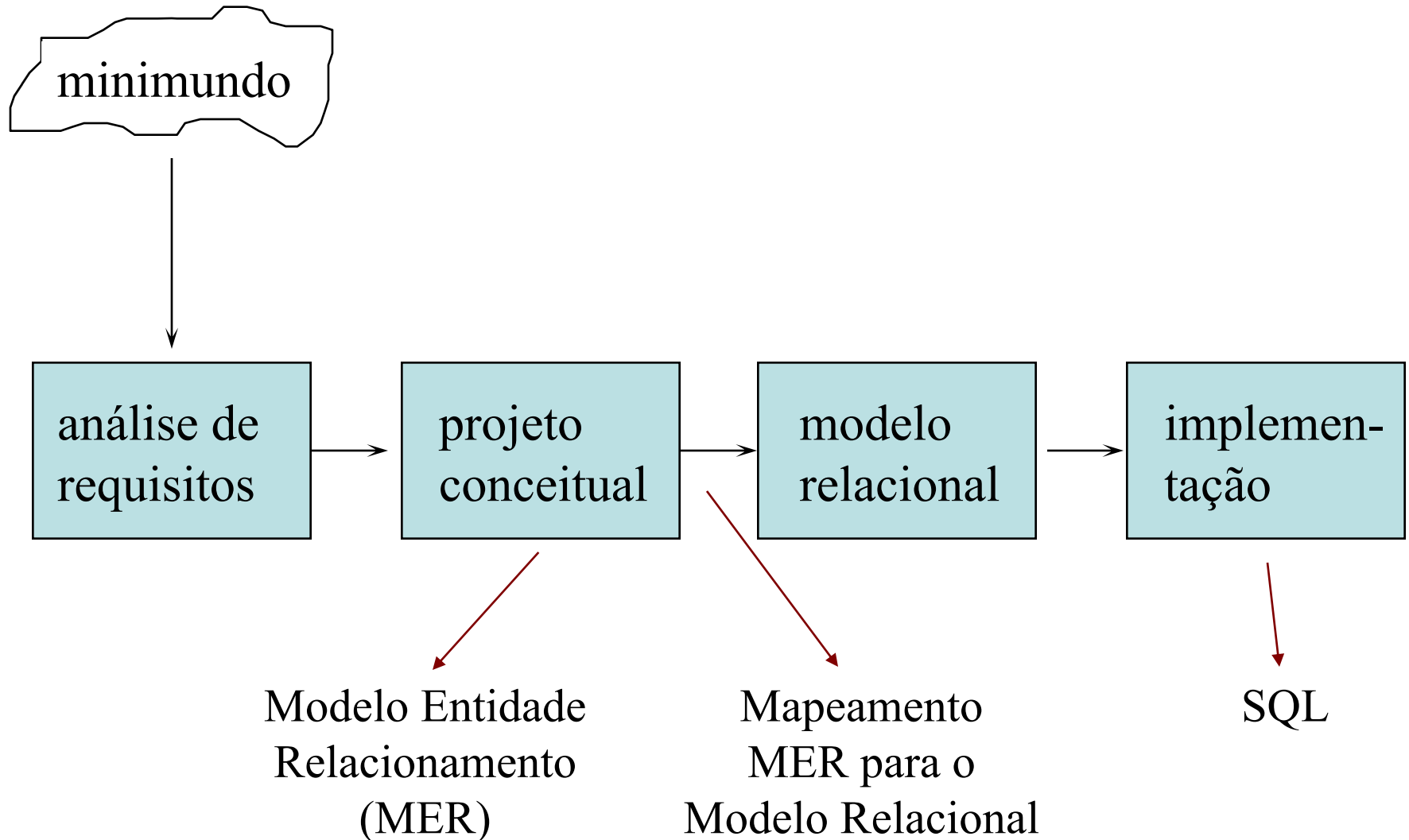
Este material é baseado

- No livro R. Ramakrishnan e J. Gehrke, *Database Management Systems*
- Nos slides do prof. Bruno Nassif

- Agradecimentos

- Ao professor Bruno Travençolo que gentilmente cedeu seus slides

Modelo de Dados e o Projeto de BD



Introdução

- Modelo Relacional
 - Proposto por Codd em 1970
 - Revolucionou a área de banco de dados
 - É o modelo dominante → base para os SGBDs líderes de mercado
 - DB2, Oracle, Sybase, Access, SQLServer e FoxBase
 - Simples e elegante
 - Embasado na teoria de conjuntos

Introdução

- Banco de Dados do Modelo Relacional
 - É uma coleção de uma ou mais relações
 - Cada relação é uma tabela com linhas (tupla, registro) e colunas (atributos, campos) - bi-dimensional
 - Possibilita o uso de linguagens de alto nível para consultar os dados
- Vantagens do Modelo Relacional
 - Representação de dados simples e facilidade para expressar consultas complexas

Tabela Bi-Dimensional

- Características
 - cada coluna tem um nome distinto e representa um **atributo**
 - cada atributo possui um **domínio** de valores
 - cada domínio possui **VALORES ATÔMICOS**
 - por atômico entendemos que o valor é indivisível no domínio
 - todos os valores de uma coluna são valores do **mesmo** atributo

Tabela Bi-Dimensional

- Características
 - cada **linha** da tabela representa o relacionamento entre um conjunto de valores
 - cada linha é distinta e representa uma **tupla**
 - uma **n-tupla** representa uma tupla que possui **n** valores
 - grau da relação: número **n** de atributos de sua relação esquema

Tabela Bi-Dimensional

- Características
 - Cada coluna tem um nome distinto
 - O valor *null* deve ser utilizado quando um atributo não possui valor ou seu valor não é conhecido
 - A ordem das colunas/linhas é irrelevante

Exemplo

- Esquema de uma relação

Alunos (**id_aluno**: `string`, **nome**:`string`,
login: `string`, **idade**: `integer`, **média**:
`real`)

→ Conjunto de valores associados ao domínio `string`: todas as strings de caracteres

Exemplo

Campos (ou atributos ou
colunas)

Nome dos campos

id_aluno	nome	login	idade	média
-----------------	-------------	--------------	--------------	--------------

50000	João	joao@ufu	19	3.3
53666	Maria	maria@ufu	18	3.4
53688	Pedro	pedro@comp	18	3.2
53650	Pedro	pedro@mat	19	3.8
53831	Ana	ana@feelt	11	1,8
53832	Tony	tony@mec	12	2.0

Tuplas
(registros,
linhas)

Definições Formais

- Relação esquema R:
 - utilizada para descrever uma relação
 - denotada por $R(A_1, A_2, \dots, A_n)$
 - formada por
 - um nome de relação R
 - uma lista de atributos $A_1, A_2, \dots, A_n$
 - para cada atributo A_i ($1 \leq i \leq n$)
 - $\text{dom}(A_i)$: domínio de A_i
 - domínio: conjunto de valores atômicos
 - caracteriza a **intenção** do BD

Esquema de relação

- Nível (aridade, grau) de uma relação
 - Número de campos (atributos)
- Cardinalidade de uma instância da relação
 - Número de tuplas
- Banco de dados relacional
 - Coleção de relações com nomes distintos
- Esquema do banco de dados relacional
 - Coleção de esquemas para as relações no banco de dados
- Instância de um banco de dados relacional
 - Coleção de instâncias das relações (uma por esquema)

Definições Formais

- Exemplos de domínios para
 - Aluno (nmat, nome, telefone, celular, idade)
 - Números de telefone
 - Nomes de aluno
 - Idade
 - Um método comum para especificar o domínio compreende
 - Definição lógica
 - Definição do tipo de dado ou formato

Definições Formais

- Aluno (nmat, nome, telefone, celular, idade)
 - Definição lógica
 1. **Números de matrícula:** conjunto de dígitos válidos para matrícula de alunos
 2. **Números de telefone:** conjunto de números de telefone válido no Brasil
 3. **Nomes de aluno:** conjunto de todos os nomes possíveis para pessoas
 4. **Idade:** conjunto de idades possíveis para alunos
 - Definição do tipo de dado ou formato
 1. **Números de matrícula:** inteiro com 8 dígitos
 2. **Números de telefone:** inteiro com 10 dígitos
 3. **Nomes de aluno:** string de 60 caracteres
 4. **Idade:** inteiro entre 0 e 127

Definições Formais

- Exemplos de domínios para
 - Aluno (nmat, nome, telefone, celular, idade)
 - $\text{dom}(\text{nmat}) = \text{Números de matrícula}$
 - $\text{dom}(\text{nome}) = \text{Nomes de aluno}$
 - $\text{dom}(\text{telefone}) = \text{Números de telefone}$
 - $\text{dom}(\text{celular}) = \text{Números de telefone}$
 - $\text{dom}(\text{idade}) = \text{Idade}$

Definições Formais

- Relação r da relação esquema $R(A_1, A_2, \dots, A_n)$
 - representa a instância da relação
 - denotada por $r(R)$
 - formada por um conjunto de n -tuplas
 $r = \{t_1, t_2, \dots, t_m\}$
 - cada n -tupla t é uma lista de n valores
 $t = \langle v_1, v_2, \dots, v_n \rangle$
 - v_i ($1 \leq i \leq n$) é um elemento de $\text{dom}(A_i)$ ou um valor nulo (i.e., null)
 - caracteriza a **extensão** do BD

Definições Formais

◆ Exemplo de possível relação do esquema

◆ Aluno (nmat, nome, telefone, celular, idade)

$r(\text{Aluno}) = \{ \langle 22222222 \rangle, \text{Júlia}, 1134343434, 1126262626, 21 \rangle, \\ \langle 11111111 \rangle, \text{Pedro}, 1965656565, 1977777777, 18 \rangle, \\ \langle 99999999 \rangle, \text{Cecília}, 1144443333, 1165658888, 23 \rangle \}$

Características das relações

- Ordenação de tuplas em uma relação (nível abstrato)
 - matematicamente, não há ordem entre os elementos de um conjunto
 - na implementação de um SGBDR existe uma ordem física de armazenamento das tuplas na memória externa
 - determina uma ordem na recuperação das informações
- Ordenação de tuplas em uma relação (nível lógico)
 - muitas ordens lógicas podem ser especificadas para uma relação
 - relação ALUNO pode ser ordenada pelos atributos NOME, DATANASCIMENTO, CPF, etc.

Características das relações

- Ordenação de valores dentro de uma tupla
 - uma tupla é uma lista de n valores dispostos em uma ordem determinada de acordo com a disposição dos atributos no esquema da relação
- Valores nas tuplas
 - são atômicos e monovalorados
 - relações não permitem atributos multivalorados
 - o valor **null** deve ser utilizado quando um atributo não possui valor ou seu valor não é conhecido

Restrições sobre uma Relação

Chaves e Superchaves

- Uma superchave de uma relação R é um conjunto de atributos S contido em R
 - no qual não haverá duas tuplas t_1 e t_2 cujo $t_1[S] = t_2[S]$

Restrições sobre uma Relação

Chaves e Superchaves

- Uma chave K é uma superchave com a propriedade adicional de que a remoção de qualquer atributo da chave fará com que K não identifique mais unicamente cada tupla da relação
 - a diferença é que uma chave tem que ser mínima

Restrições sobre uma Relação

Chaves e Superchaves

Aluno (nmat, nome, telefone, celular, idade)

- Exemplo:
 - {nmat} é uma chave de aluno
 - Superchaves
 - {nmat, nome}
 - {nmat, nome, telefone}
 - {nmat, nome, telefone, celular}
 - {nmat, nome, telefone, celular, idade}
 - ...

Restrições sobre uma Relação

Chaves e Superchaves

- Chave candidata:
 - se um esquema de relação tiver mais de uma chave, cada uma delas é chamada chave candidata
 - uma delas é arbitrariamente designada para ser chave primária
- Um atributo de um esquema de relação R é chamado atributo primário se for membro de alguma chave candidata

Restrições sobre uma Relação

Chaves e Superchaves

Aluno (nmat, nome, telefone, celular, idade)

- Exemplo:
 - {nmat} é a única chave candidata de aluno, portanto também é a chave primária

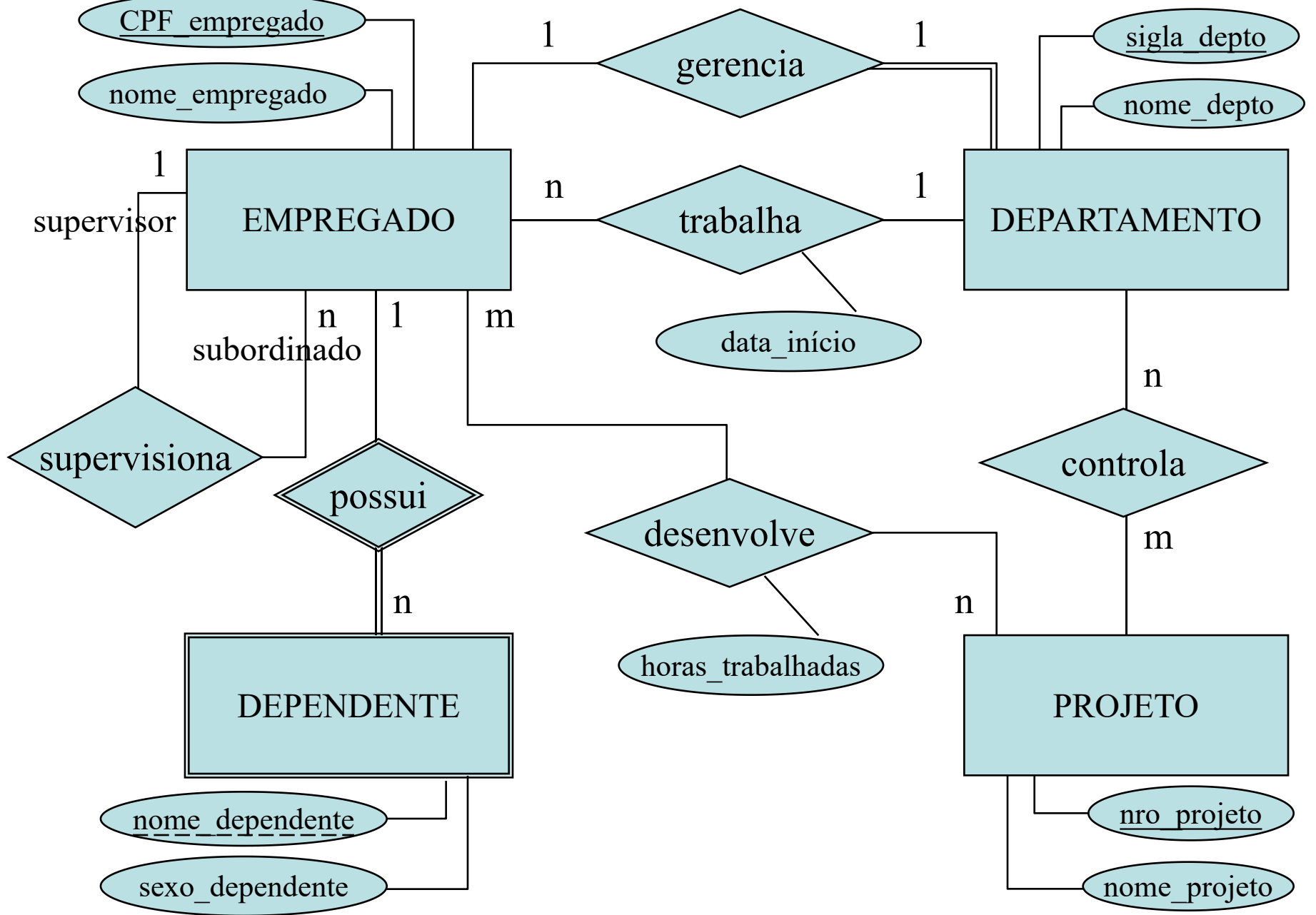
Restrições sobre uma Relação

Chave Primária

- Resumindo:
 - chave primária para um esquema de relação R satisfaz duas restrições
 - duas tuplas distintas não podem ter valores idênticos para os atributos da chave
 - ela é uma superchave mínima

Definições

- Esquema de banco de dados S
 - conjunto de relações esquema
 $S = \{R_1, R_2, \dots, R_m\}$
 - conjunto de restrições de integridade IC
- Estado do banco de dados DB
 - conjunto de estados da relação
 $DB = \{r_1, r_2, \dots, r_m\},$
onde cada r_i é um estado de R_i
 - os estados de r_i devem satisfazer às restrições de integridade especificadas em IC



Esquema do BD Relacional

empregado (CPF_empregado, nome_empregado,
cod_supervisor, sigla_depto, data_início)

dependente (CPF_empregado, nome_dependente,
sexo_dependente)

departamento (sigla_depto, nome_depto,
CPF_empregado)

projeto (nro_projeto, nome_projeto)

controla (sigla_depto, nro_projeto)

desenvolve (CPF_empregado, nro_projeto,
horas_trabalhadas)

Restrições entre duas Relações

- Integridade referencial
 - mantém a consistência entre as tuplas nas duas relações
 - declara que uma tupla em uma relação, a qual faz referência a uma outra relação, deve se referir a uma tupla existente nessa segunda relação
 - definida entre a **chave estrangeira** (do inglês *foreign key* - FK) de uma relação esquema R_1 e a **chave primária** (do inglês *primary key* - PK) de uma relação esquema R_2

Restrições entre duas Relações

- FK de R_1 é **chave estrangeira** de R_1 , que faz referência à PK de R_2 , se:
 - os atributos de FK têm os mesmos domínios que os atributos de PK
 - um valor de FK em uma tupla t_1 do estado corrente de $r_1(R_1)$
 - ocorre como um valor de PK para alguma tupla t_2 no estado corrente $r_2(R_2)$ ou
 - tem o valor null

Integridade Referencial

empregado (CPF_empregado, nome_empregado,
cod_supervisor, sigla_depto, data_início)

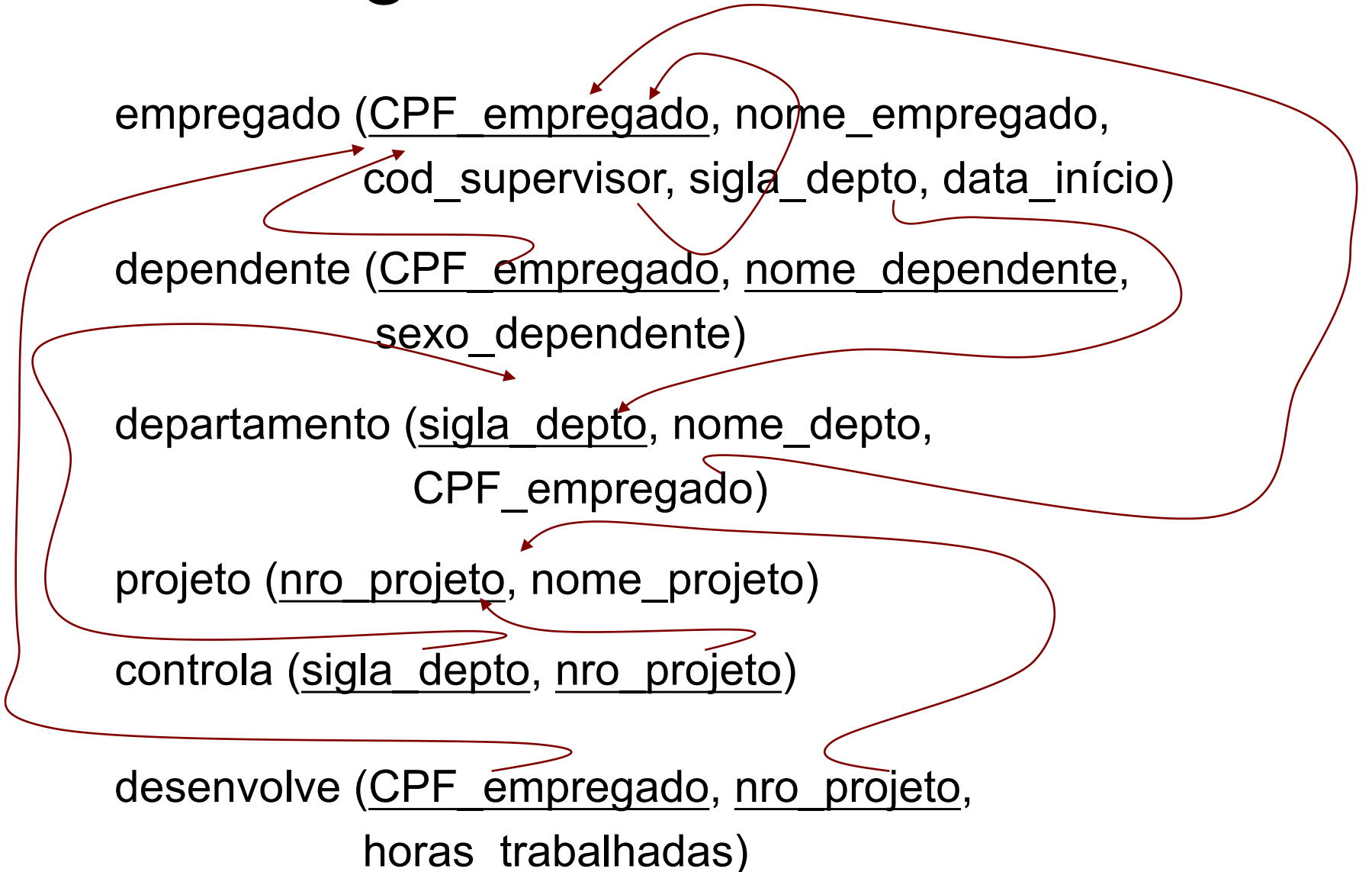
dependente (CPF_empregado, nome_dependente,
sexo_dependente)

departamento (sigla_depto, nome_depto,
CPF_empregado)

projeto (nro_projeto, nome_projeto)

controla (sigla_depto, nro_projeto)

desenvolve (CPF_empregado, nro_projeto,
horas_trabalhadas)



Restrições *versus* Operações

- Operações de modificação
 - insert → inserção
 - delete → remoção
 - update → atualização
- Quando estas operações são aplicadas, as restrições de integridade especificadas no esquema do banco de dados relacional **não** devem ser violadas

Operação Insert

- Característica
 - fornece uma lista de valores de atributos para uma nova tupla t , que é inserida em uma relação R
- Pode violar as seguintes restrições
 - domínio
 - unicidade de chave
 - integridade de entidade
 - (chave primária *null*)
 - integridade referencial

Solução:

- rejeitar a inserção
- enviar mensagem de erro ao usuário

Operação Delete

- Característica
 - remove uma ou mais tuplas
- Pode violar a integridade referencial
 - quando as tuplas removidas forem referidas por chaves estrangeiras de outras tuplas
- Soluções
 - rejeitar a remoção
 - remover em cascata
 - modificar valores dos atributos de referência

Operação Update

- Característica
 - altera valores de alguns atributos em tuplas
- Pode violar as seguintes restrições
 - domínio
 - unicidade de chave (se atributo é PK)
 - integridade de entidade (se atributo é PK)
 - integridade referencial (se atributo é FK)
- Soluções
 - idem anteriores (para insert e delete)

Exercício

- A administradora trabalha com administração de condomínios e de aluguéis
- Uma entrevista com o gerente resultou nas seguintes informações:
 - São administrados condomínios formados por unidades condominiais. Para cada condomínio a administradora precisa saber seu nome (que é único) e endereço (nome da rua, número, bairro e cep). Para cada unidade condominal seu número único e o andar
 - Cada unidade condominal é de propriedade de uma ou mais pessoas. Uma pessoa pode possuir diversas unidades
 - Cada unidade pode estar alugada para no máximo uma pessoa. Uma pessoa pode alugar diversas unidades. Uma data determina quando a pessoa alugou uma unidade condominal
 - Para cada pessoa são armazenadas informações como CPF, nome e telefones (residencial, comercial e celular)

1. Definir os domínios (definição lógica, tipo de dado e formato) necessários para a criação dos esquemas de relação para essa especificação
 - ex: nomes de pessoas: conjunto de todos os nomes possíveis para pessoas – strings de 60 caracteresD
2. Definir os esquemas de relação para a especificação
3. Instancie relações a partir dos esquemas criados exemplificando situações em que não são satisfeitas as seguintes restrições de integridade:
 - unicidade de chave, integridade de entidade e integridade referencial

Referências

- R. Elmasri e S. B. Navathe, Sistema de Banco de Dados, 6ª edição, Pearson, 2011.
- R. Ramakrishnan e J. Gehrke, *Database Management Systems*, 3a Edição, McGraw-Hill, 2003.

Aula 6 – SBD

ER para Relacional

Profa. Elaine Faria
UFU

Agradecimentos

Este material é baseado

- No livro R. Ramakrishnan e J. Gehrke, *Database Management Systems*
- Nos slides do prof. Bruno Nassif

- Agradecimentos

- Ao professor Bruno Travençolo que gentilmente cedeu seus slides

Introdução

- Modelo ER
 - Conveniente para representar um projeto de BD inicial de alto nível
- Modelo Relacional
 - Pode ser usado para suportar a implementação de aplicações
 - É necessário que exista um SGBD que se apóie no modelo relacional: um SGBDR
- Dado um DER é adotada uma estratégia padrão para gerar um esquema de BD relacional

Introdução

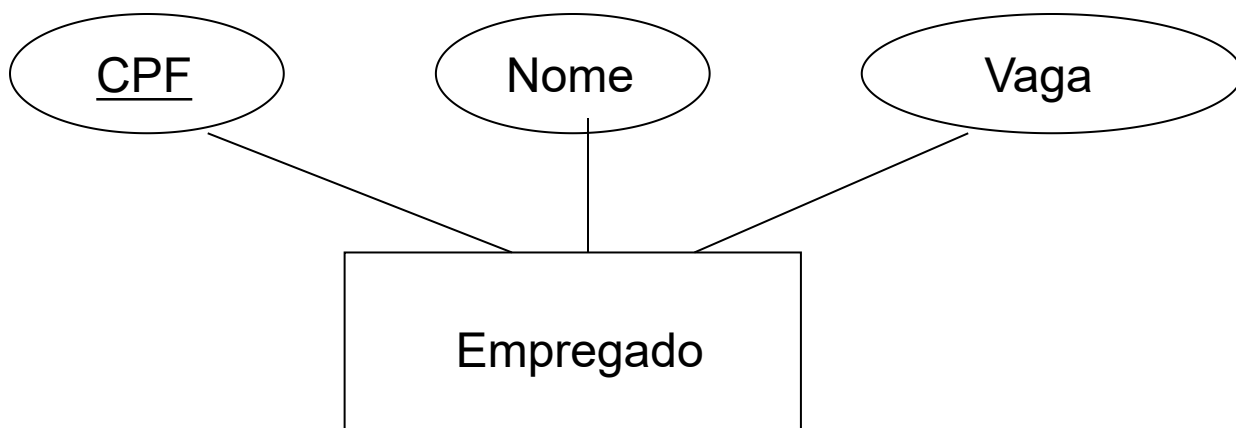
Como transformar um diagrama ER para um esquema de Banco de Dados Relacional?

Passo 1

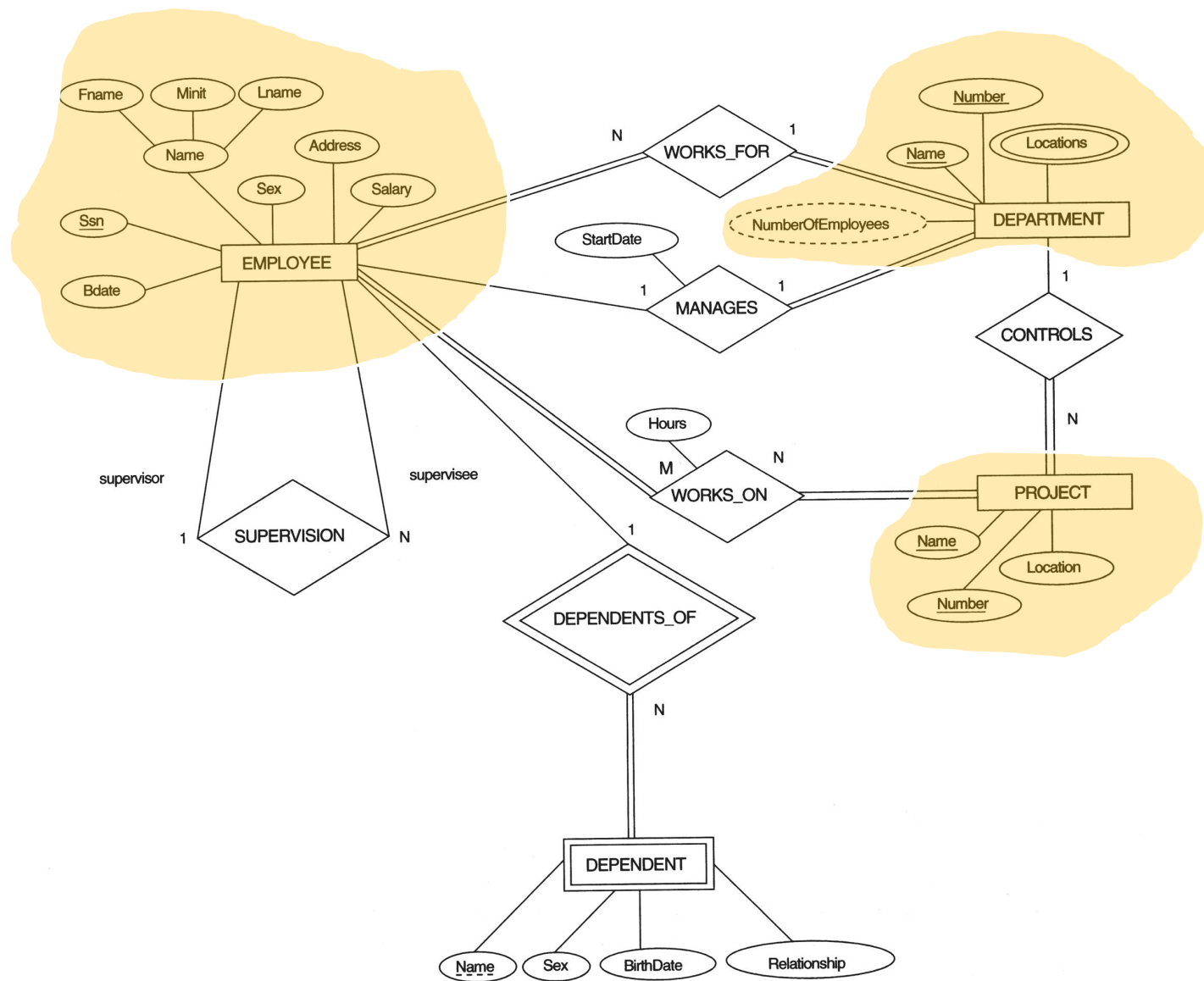
- Mapeamento dos Tipos Entidade Regulares

- Para cada entidade regular (forte) criar uma relação que inclua todos os atributos simples da entidade
- Incluir somente os atributos simples dos atributos compostos
 - Poderia incluir somente o atributo composto
- Escolher um dos atributos chave da entidade para ser a chave primária da relação
- O atributo derivado – pode ou não ser mapeado. Tomar a decisão com base na complexidade de seu cálculo e no número de possíveis consultas que serão feitas para este atributo

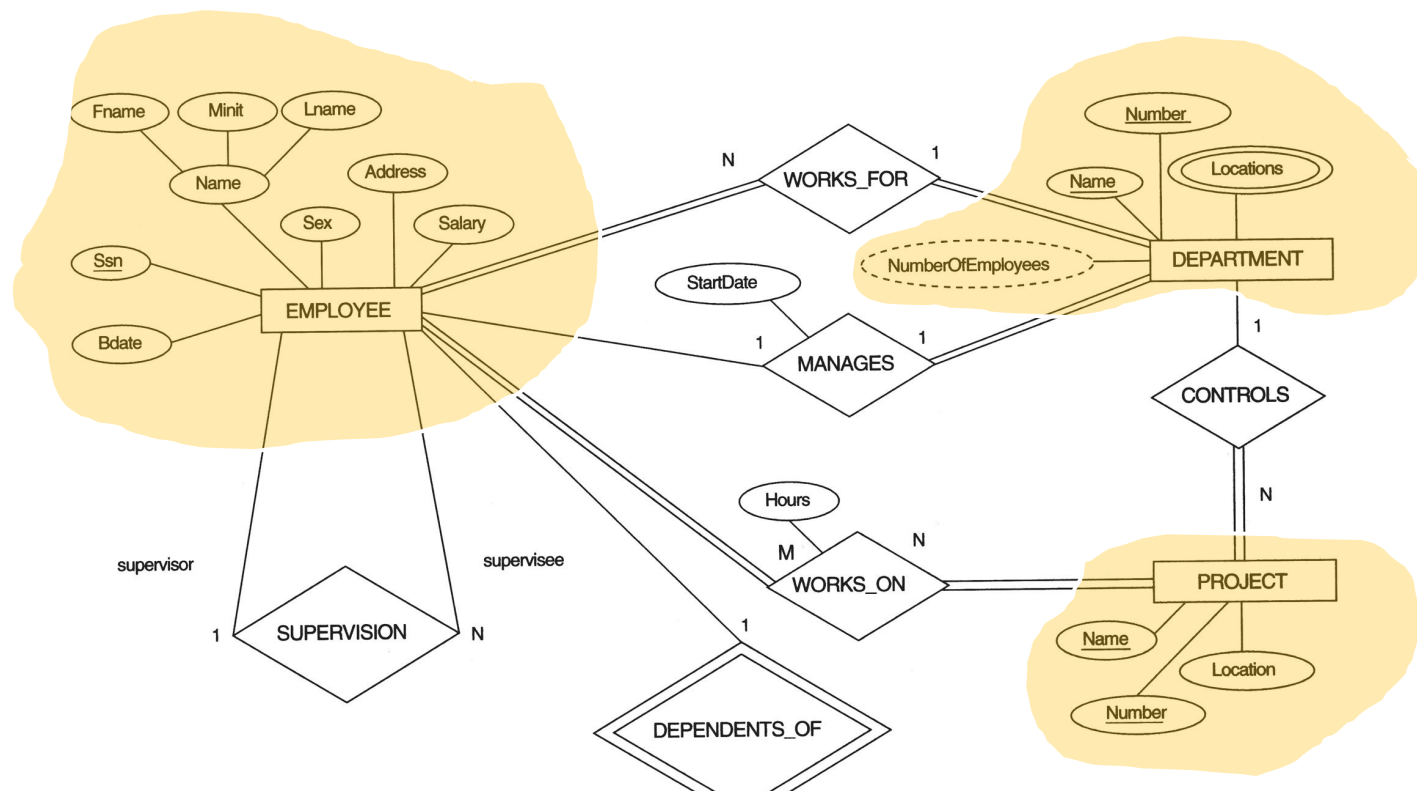
Mapeamento dos Tipos Entidade Regulares



<u>CPF</u>	Nome	Vaga
111.111.111-11	João	48
222.222.222.22	Maria	33
333.333.333-33	Pedro	22



Esquema disponível no livro do Elmasri – ©



EMPREGADO

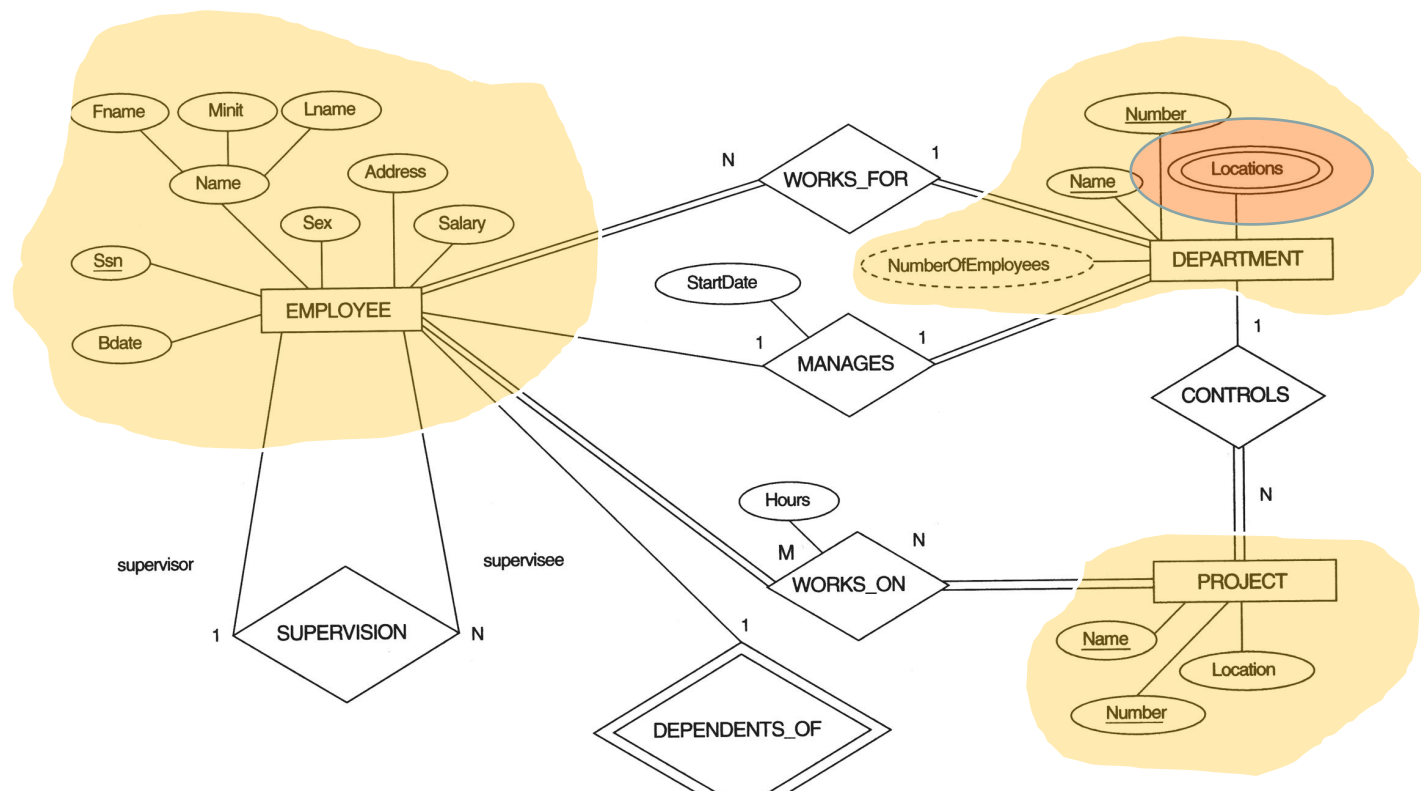
= {PNOME, MINICIAL, UNOME, SSN, DATANASC, ENDERECO, SEXO, SALÁRIO}

DEPARTAMENTO

= {DNOME, DNUMERO} // Poderia ter escolhido DNOME como chave primária

PROJETO

= {PJNOME, PNUMERO, PLOCALIZACAO} // Poderia ter escolhido PJNOME como chave primária



EMPREGADO

= {PNOME, MINICIAL, UNOME, SSN, DATANASC, ENDERECO, SEXO, SALÁRIO}

DEPARTAMENTO

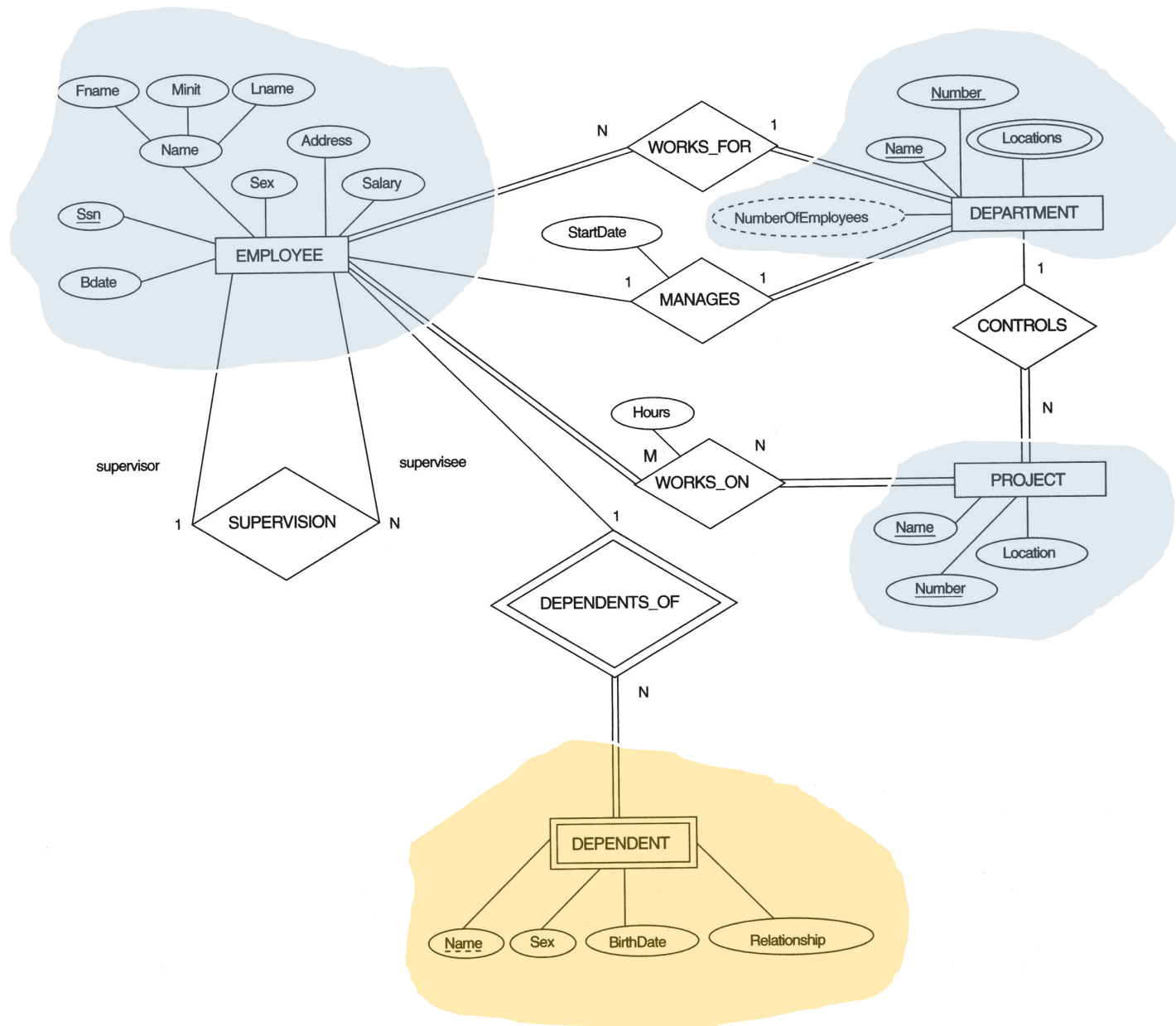
= {DNOME, DNUMERO} // Poderia ter escolhido DNOME como chave

PROJETO

= {PJNOME, PNUMERO, PLOCALIZACAO} // Poderia ter escolhido PJNOME como chave

Passo 2

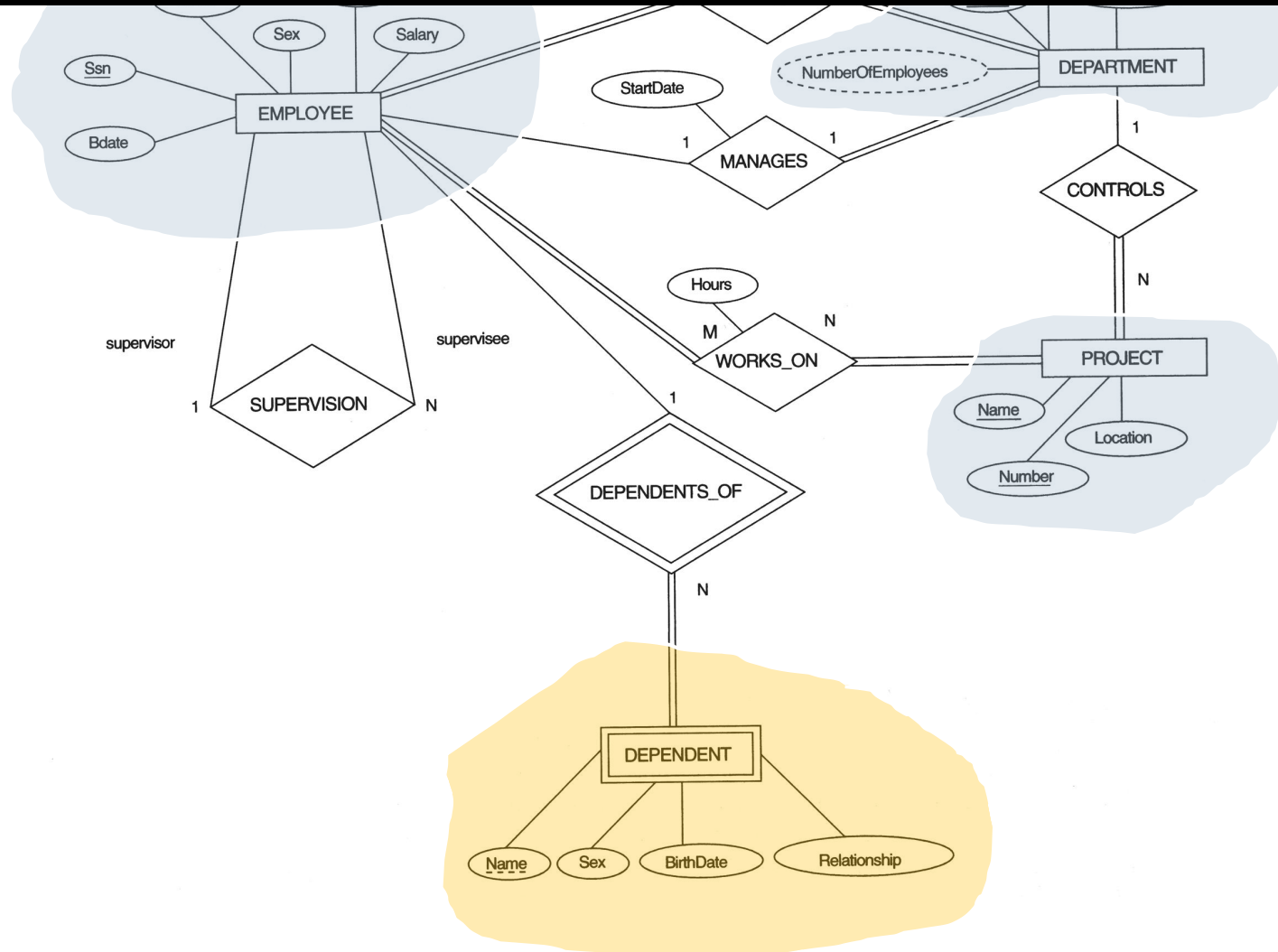
- Mapeamento dos tipos Entidades Fracas
 - Criar uma relação com todos os atributos simples (e os componentes simples dos atributos compostos)
 - Inserir como chave estrangeira os atributos que são chave primária das entidades fortes indentificadoras com quem a entidade fraca se relaciona
 - A chave primária é composta pelas chaves primárias das entidades fortes identificadoras relacionadas em conjunto com a chave parcial da entidade fraca



DEPENDENTE

= {ESSN, NOME_DEPENDENTE, SEXO, DATANASC, PARENTESCO}

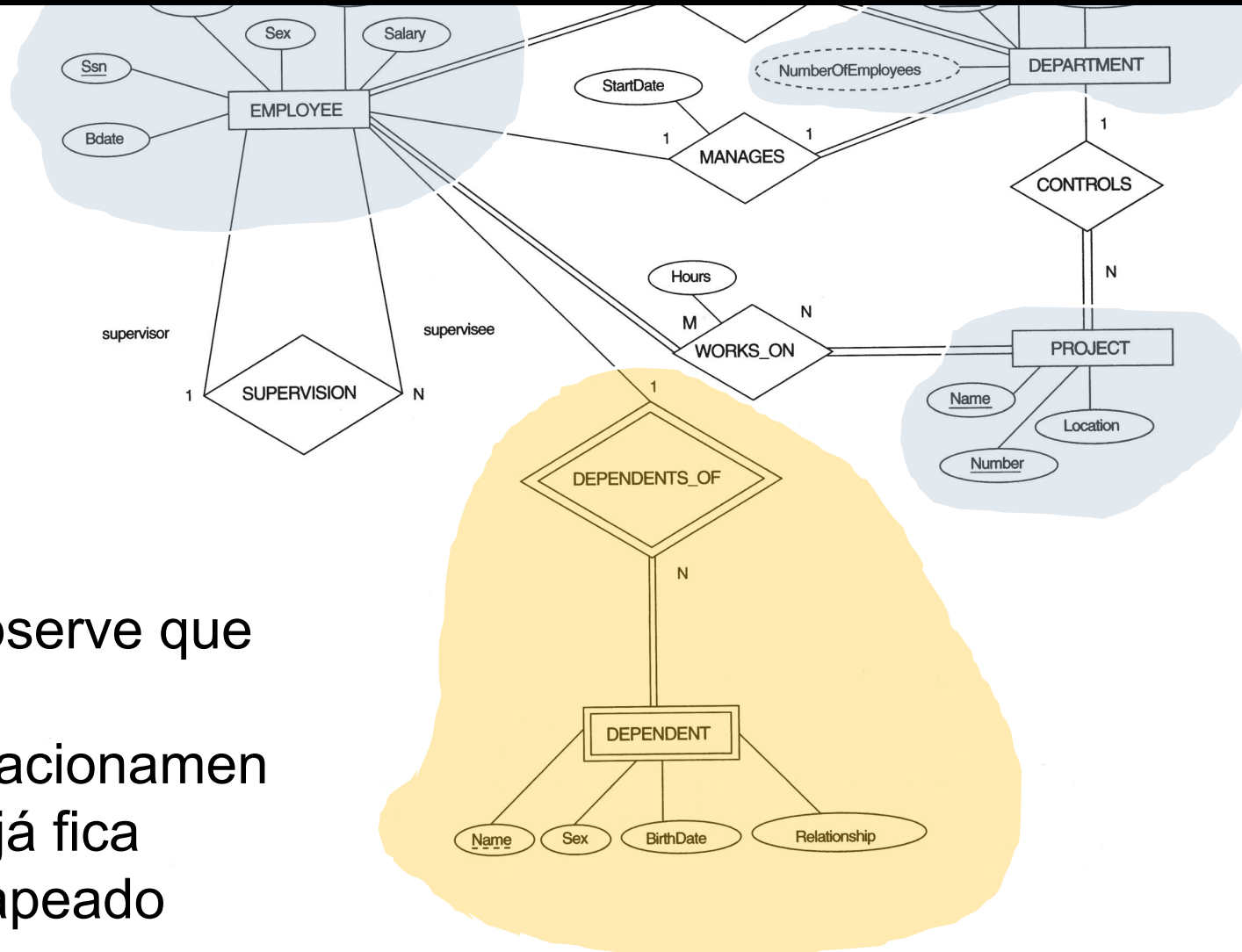
// ESSN é chave estrangeira para o atributo SSN da relação Empregado



DEPENDENTE

= {ESSN, NOME_DEPENDENTE, SEXO, DATANASC, PARENTESCO}

// ESSN é chave estrangeira para o atributo SSN da relação Empregado



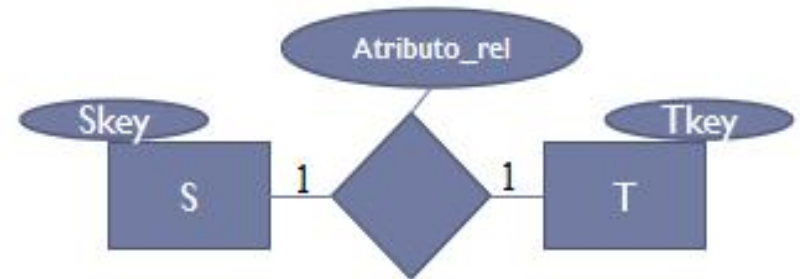
Observe que
o
relacionamen
to já fica
mapeado

Passo 3

- Mapeamento dos relacionamentos binários 1:1
 - Identificar as entidades que participam do relacionamento
 - Três opções
 - Escolha da chave estrangeira
 - Opção da relação unificada – junta as duas tabelas (consultar Elmasri)
 - Opção referência cruzada ou relação de relacionamento – nova tabela com um dos campos chave e o outro unique (consultar Elmasri)

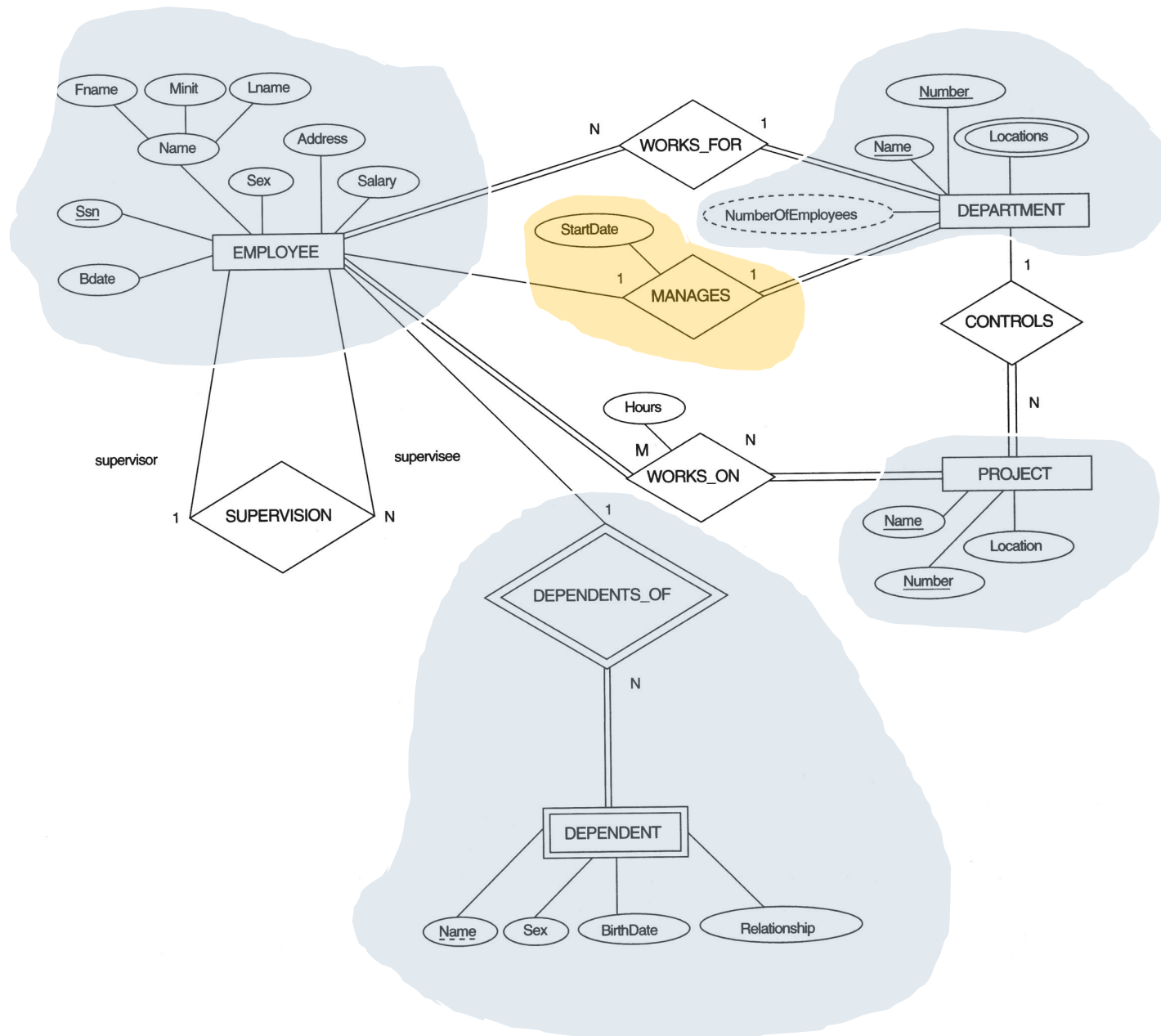
Passo 3

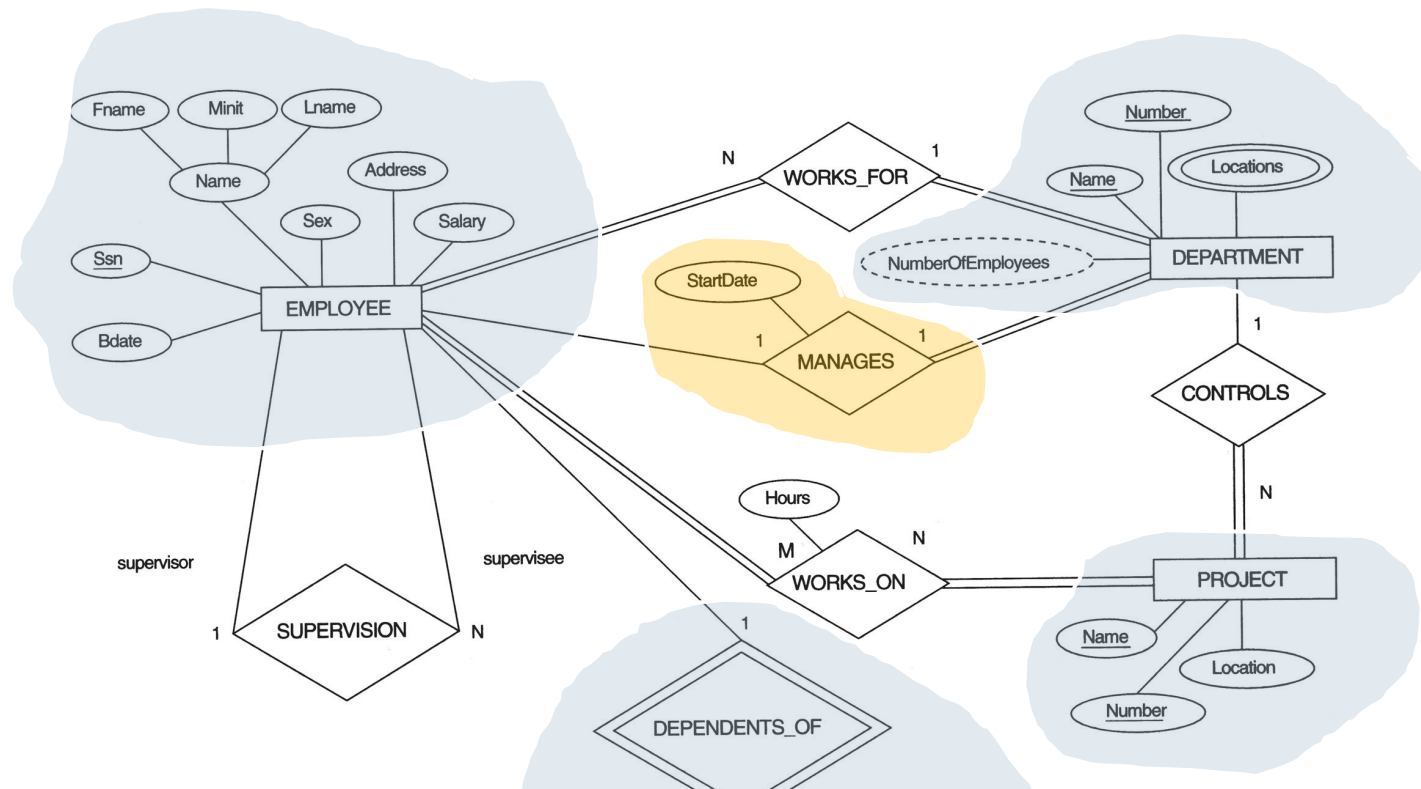
- Escolha da chave estrangeira
 - Vamos chamar as duas relações que participam do relacionamento de S e T
 - Escolha uma das relações (por exemplo, S) e inserir nela, como chave estrangeira, a chave primária da outra relação (no caso, T)
 - Incluir os atributos simples do relacionamento como atributos de S (e também os componentes simples dos atributos compostos)



$S = \{\underline{SKEY}, TKEY, S_ATRIBUTO1, SATRIBUTO2, \dots, ATRIBUTO_REL\}$

↙





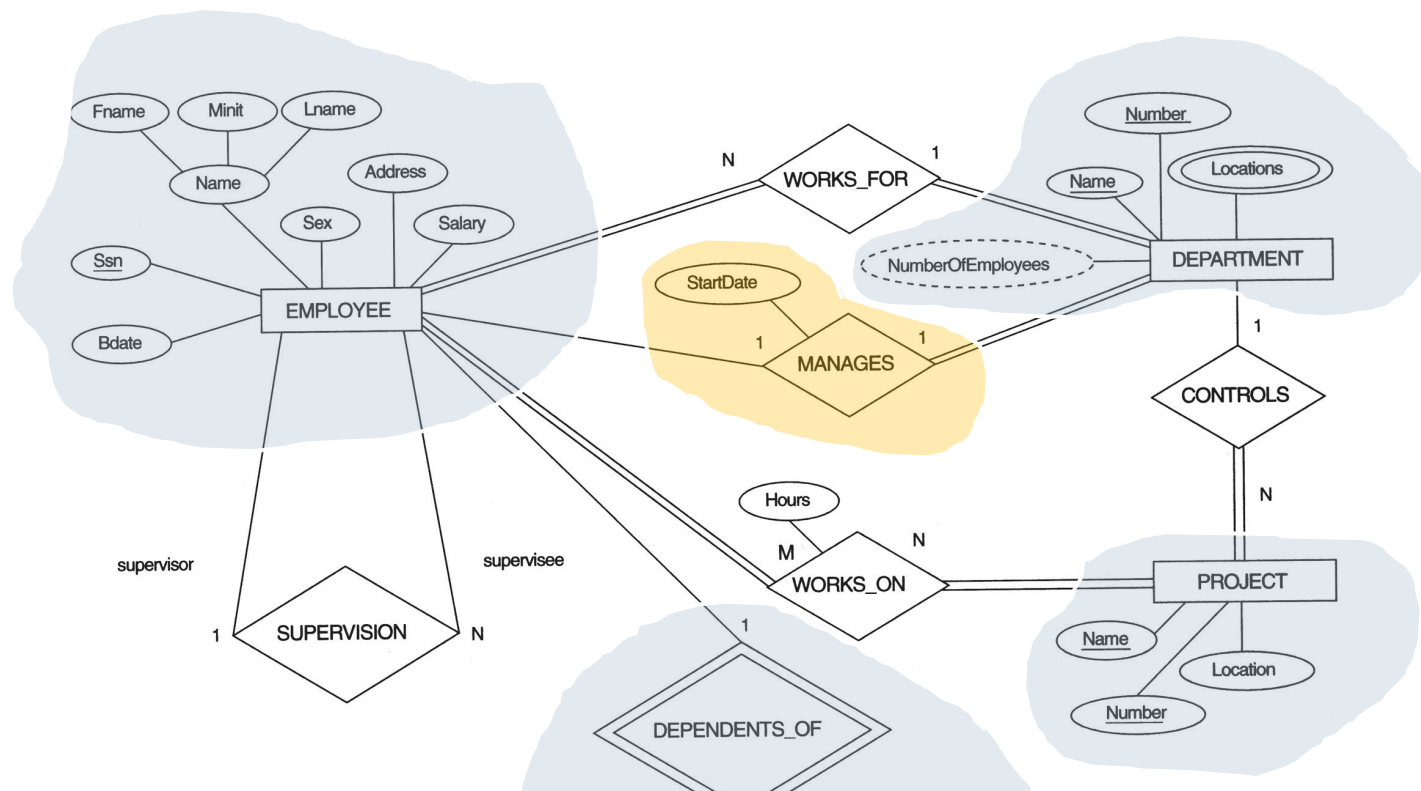
EMPREGADO

= {PNOME, MINICIAL, UNOME, SSN, DATANASC, ENDERECO, SEXO, SALÁRIO}

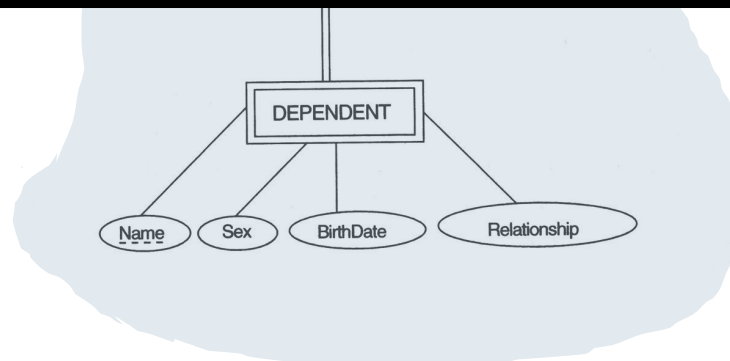
DEPARTAMENTO

= {DNOME, DNUMERO, GERSSN, GERDATAINICIO}



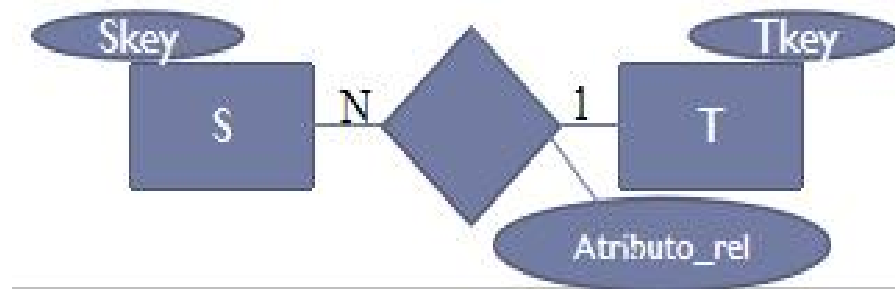


Por que neste caso é melhor colocar o atributo chave de empregado em departamento e não o contrário?



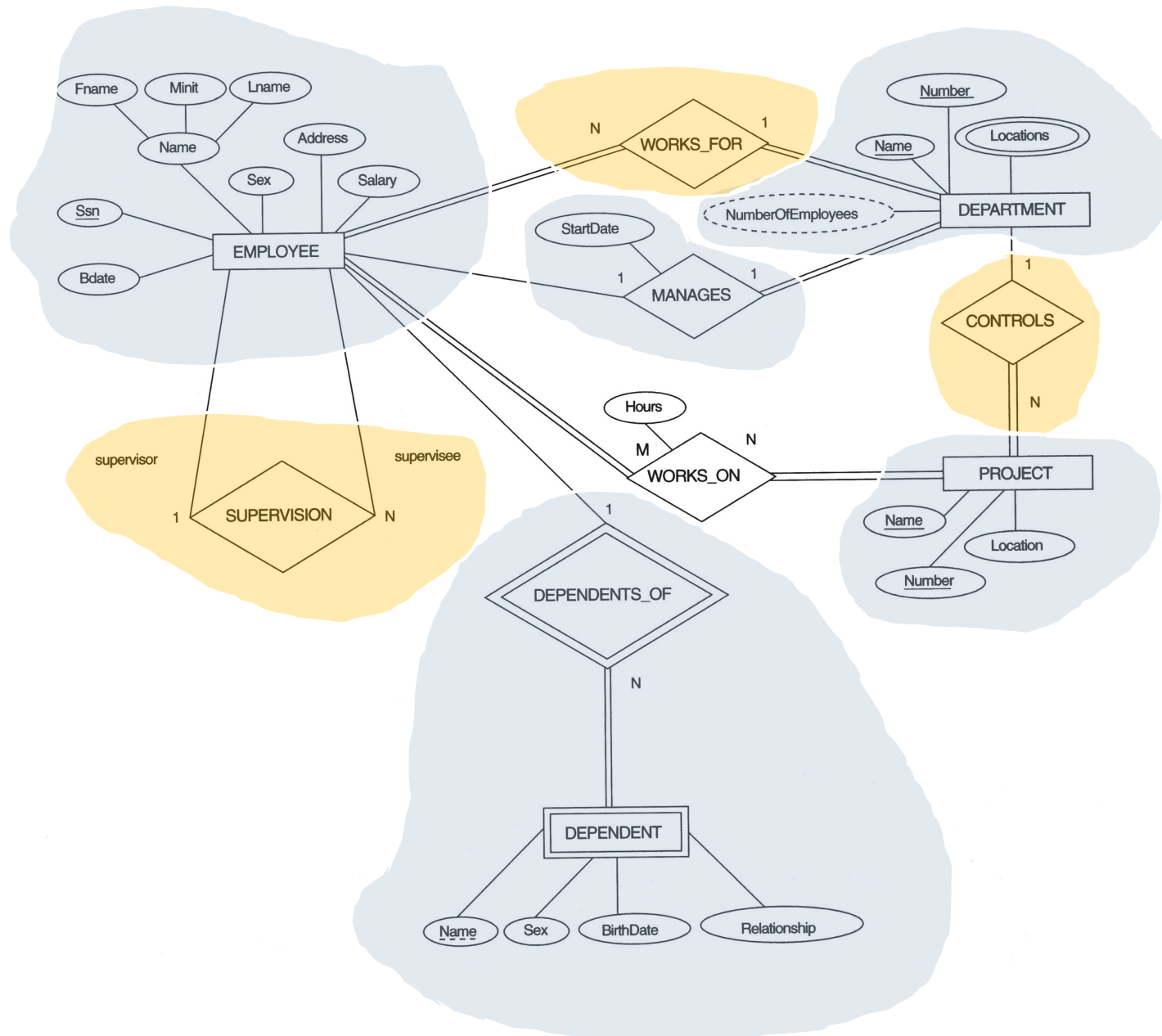
Passo 4

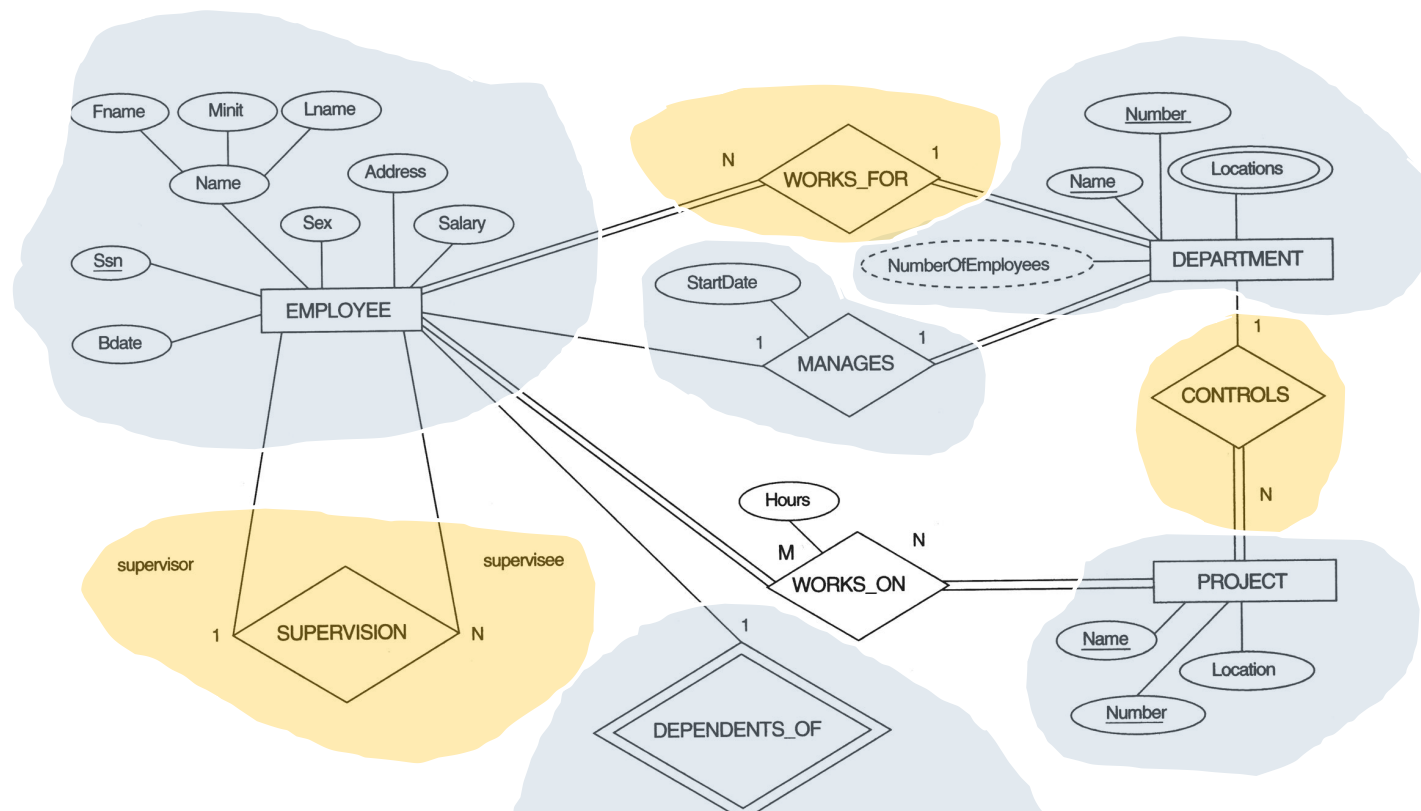
- Mapeamento dos relacionamentos binários 1:N
 - Considere S e T novamente. Suponha que S é a relação do lado N da cardinalidade
 - Inserir em S a chave primária da relação T (vai ser chave estrangeira)
 - Isso é feito pois cada entidade de S (lado N) está relacionado a, **no máximo**, uma entidade de T (lado 1)
 - O mapeamento de atributos vai para a relação S também



$S = \{\underline{SKEY}, TKEY, S_ATRIBUTO1, SATRIBUTO2, \dots, ATRIBUTO_REL\}$







EMPREGADO

= {PNOME, MINICIAL, UNOME, SSN, DATANASC, ENDERECO, SEXO, SALÁRIO, **SUPERSSN**, **DNO**}

DEPARTAMENTO

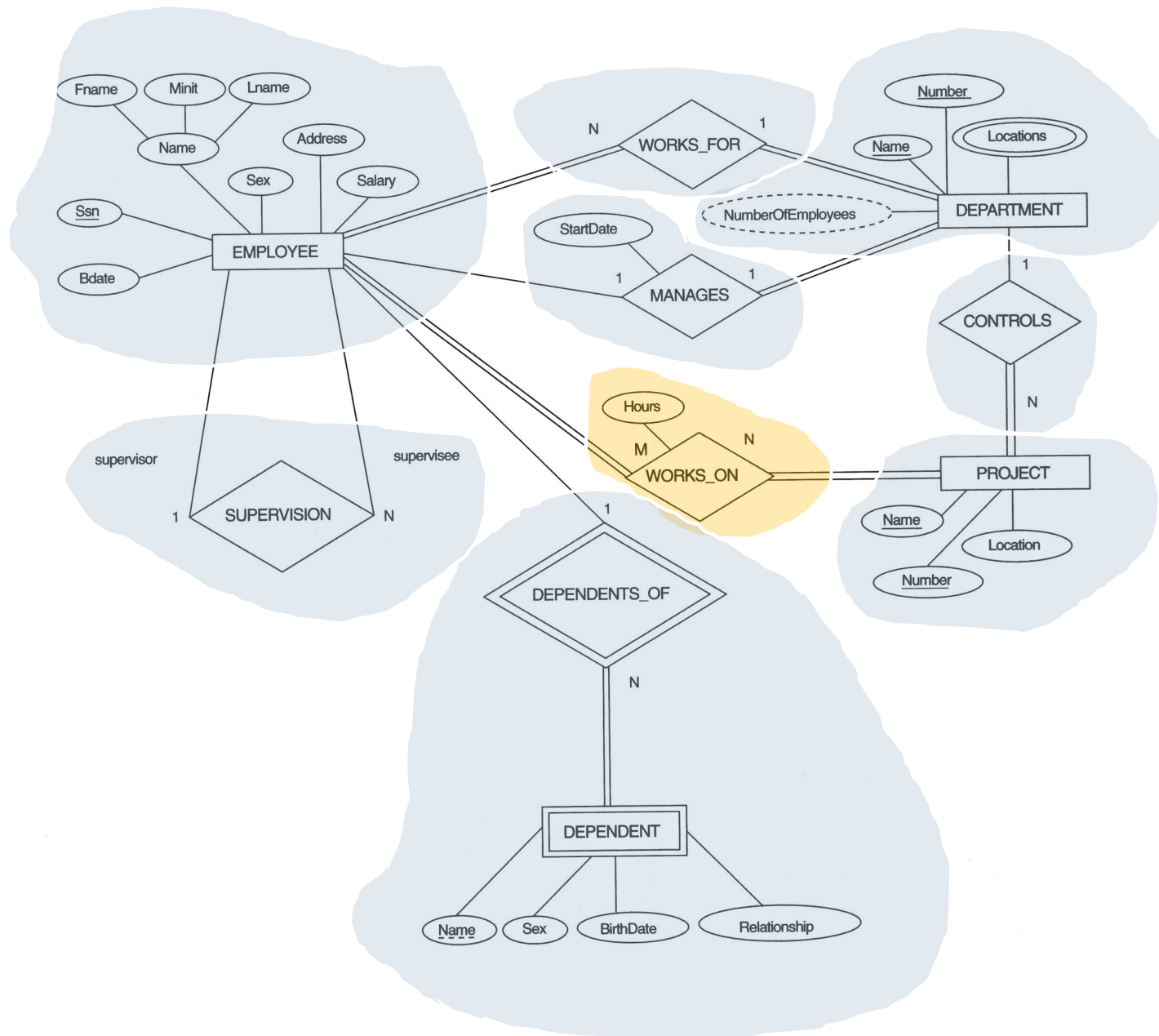
= {DNOME, DNUMERO, GERSSN, GERDATAINICIO}

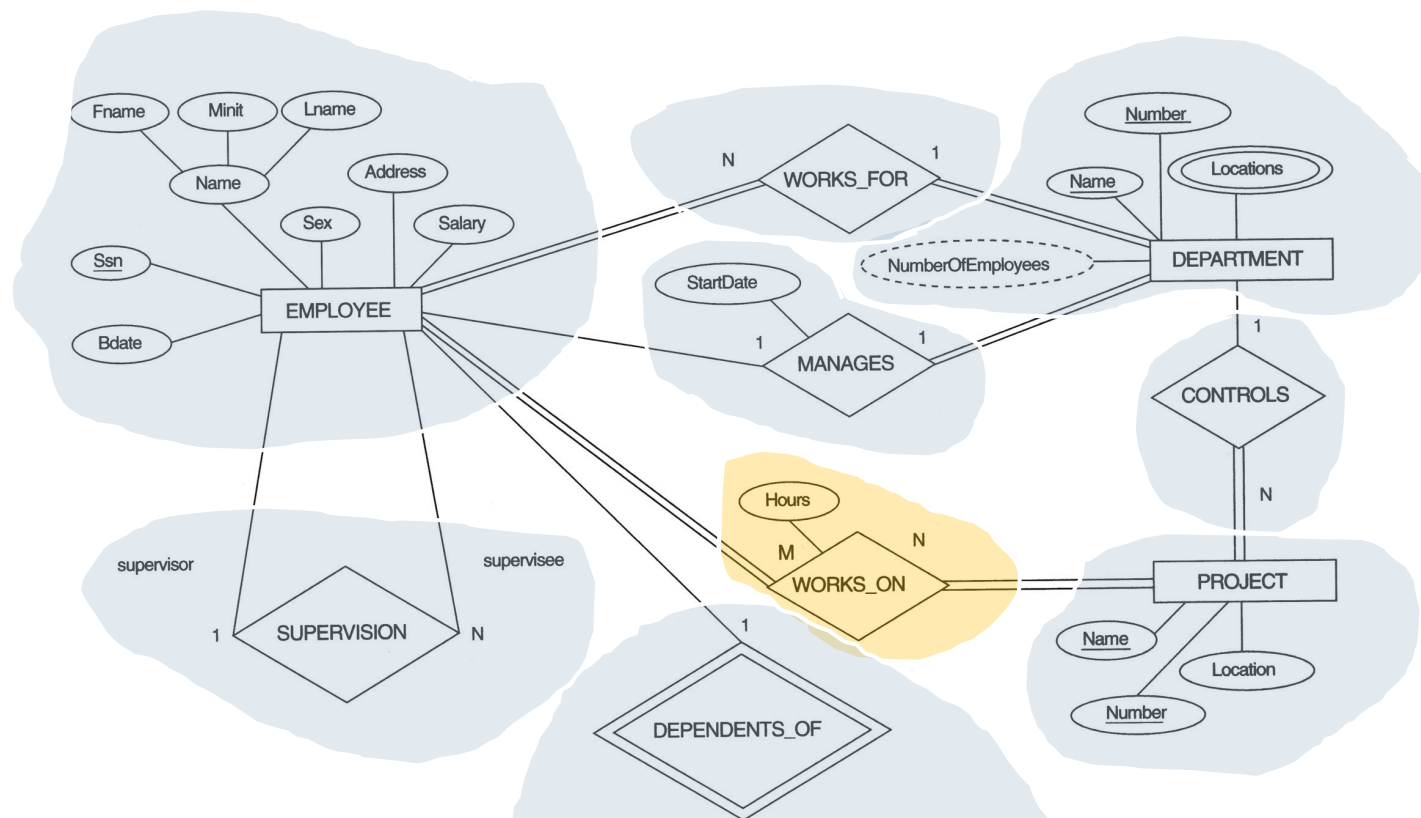
PROJETO

= {PJNOME, PNUMERO, PLOCALIZACAO, **DNUM**}

Passo 5

- Mapeamento dos tipos de relacionamento binário N:M
 - Deve ser criada uma nova relação (.. uma nova tabela)
 - Inserir, como chave estrangeira, as chaves primárias das entidades envolvidas no relacionamento
 - A chave primária da nova relação é composta pela concatenação dessas chaves estrangeiras
 - Mapear os atributos simples do relacionamento nessa nova relação (atributos simples pertencentes aos atributos compostos)





EMPREGADO

= {PNOME, MINICIAL, UNOME, SSN, DATANASC, ENDERECO, SEXO, SALÁRIO, SUPERSSN, DNO}

PROJETO

= {PJNOME, PNUMERO, PLOCALIZACAO, DNUM}

TRABALHA_EM

= {HORAS, ESSN, PNO}

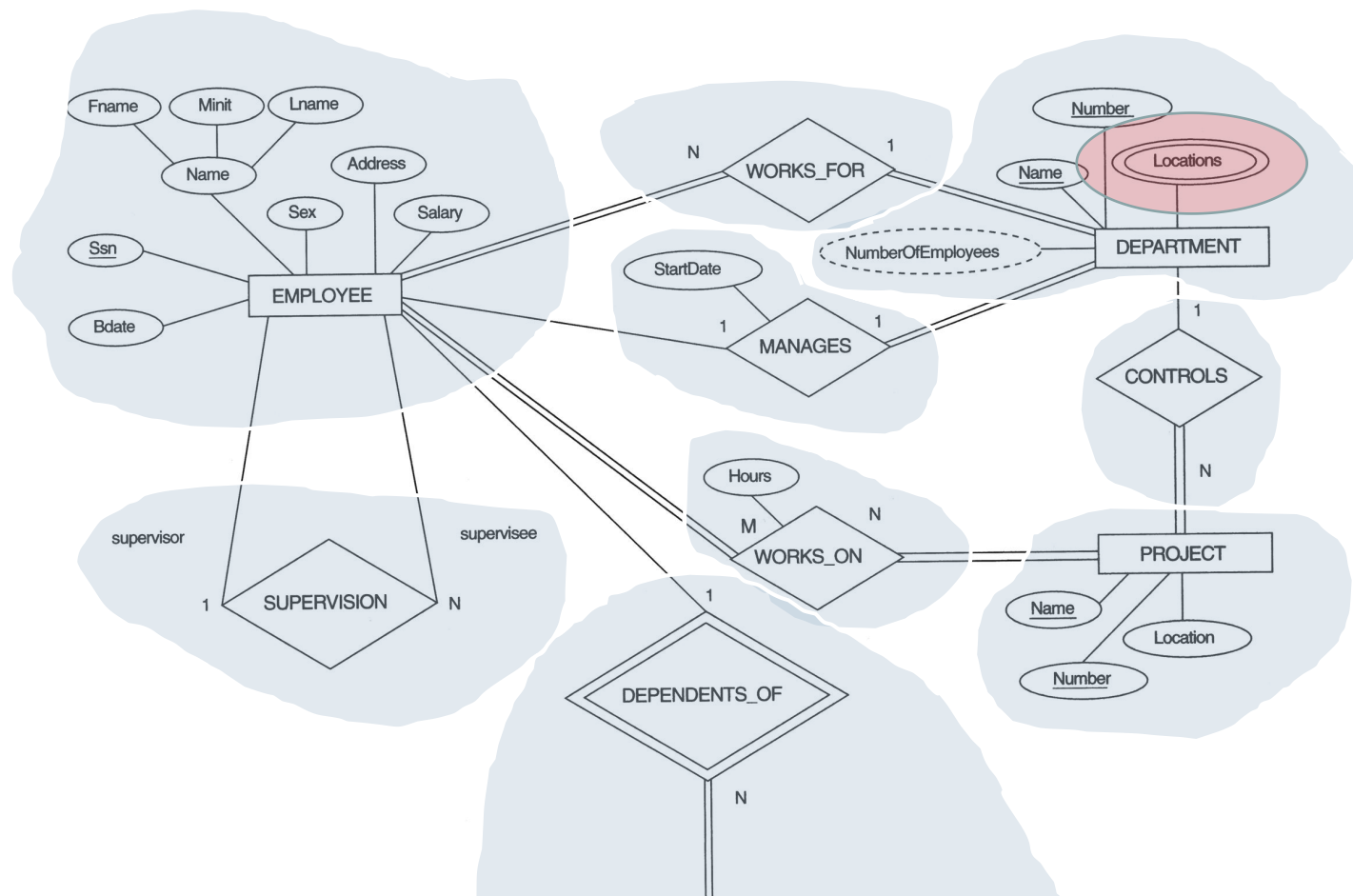
Restrição de Participação Total em Relacionamento N:M

- Como garantir a participação Total de Departamentos em Trabalha_em?

Uso de Asserções em SQL (programas específicos)

Passo 6

- Mapeamento de Atributos multivalorados
 - Criar uma nova relação para cada atributo e mapeá-lo.
 - Incluir, como chave estrangeira, a chave primária (*Ch*) da relação que representa a entidade que contém o atributo multivalorado
 - Se o atributo multivalorado for composto, mapear seus atributos simples
 - A chave primária é a combinação da chave primária *Ch* e o atributo multivalorado
 - Se o atributo for composto, deve-se escolher qual fará parte da chave primária.



DEPARTAMENTO

= {DNOME, DNUMERO, GERSSN, GERDATAINICIO}

DEPT_LOCALIZACOES

= {DNUMERO, DLOCALIZACAO}

EMPLOYEE

FNAME	MINIT	LNAME	<u>SSN</u>	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
-------	-------	-------	------------	-------	---------	-----	--------	----------	-----

DEPARTMENT

DNAME	<u>DNUMBER</u>	MGRSSN	MGRSTARTDATE
-------	----------------	--------	--------------

DEPT_LOCATIONS

<u>DNUMBER</u>	<u>DLOCATION</u>
----------------	------------------

PROJECT

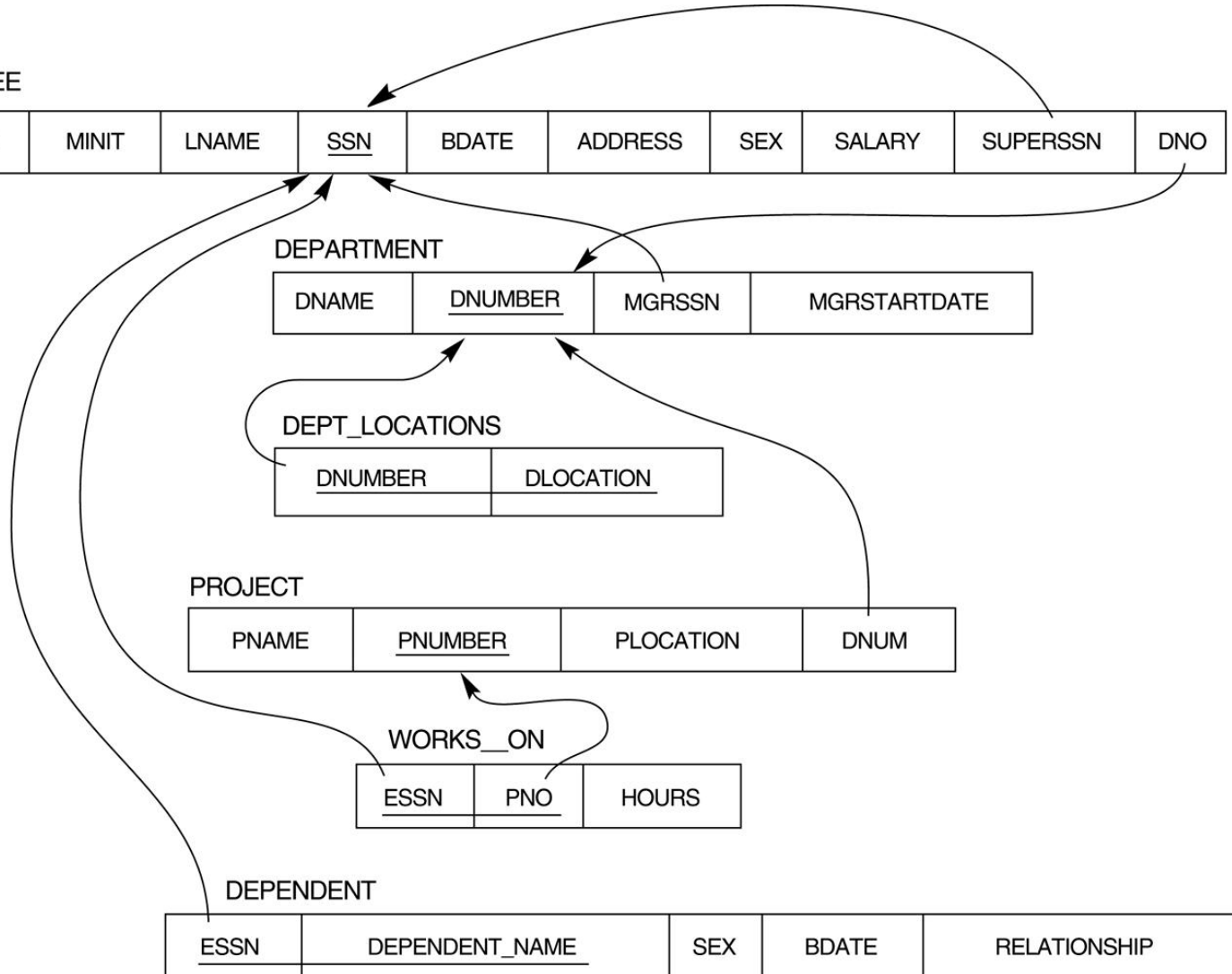
PNAME	<u>PNUMBER</u>	PLOCATION	DNUM
-------	----------------	-----------	------

WORKS_ON

<u>ESSN</u>	<u>PNO</u>	HOURS
-------------	------------	-------

DEPENDENT

<u>ESSN</u>	<u>DEPENDENT_NAME</u>	SEX	BDATE	RELATIONSHIP
-------------	-----------------------	-----	-------	--------------



Passo 7

- Mapear todos os tipos-relacionamento de grau > 2
 - Seguir a mesma ideia do mapeamento binário

Mapeamento ME-R → MRel

Os 7 passos do procedimento

1. Mapear todos os tipos-entidade forte
2. Mapear todos os tipos-entidade fraca
3. Mapear todos os tipos-relacionamento 1:1
4. Mapear todos os tipos-relacionamento 1:n
5. Mapear todos os tipos-relacionamento n:m
6. Mapear todos os atributos multivalorados
7. Mapear todos os tipos-relacionamento de grau > 2

Mapeamento MER-X → MRel

Os 9 passos do procedimento

1. Mapear todos os tipos-entidade forte **que não são especializações**
2. Mapear todos os tipos-entidade fraca **que não são especializações**
3. Mapear todos os tipos-relacionamento 1:1
4. Mapear todos os tipos-relacionamento 1:n
5. Mapear todos os tipos-relacionamento n:m
6. Mapear todos os atributos multivalorados
7. Mapear todos os tipos-relacionamento de grau > 2
8. **Mapear todas as ocorrências de abstração de generalização/especialização**
9. **Mapear as ocorrências de agregação**

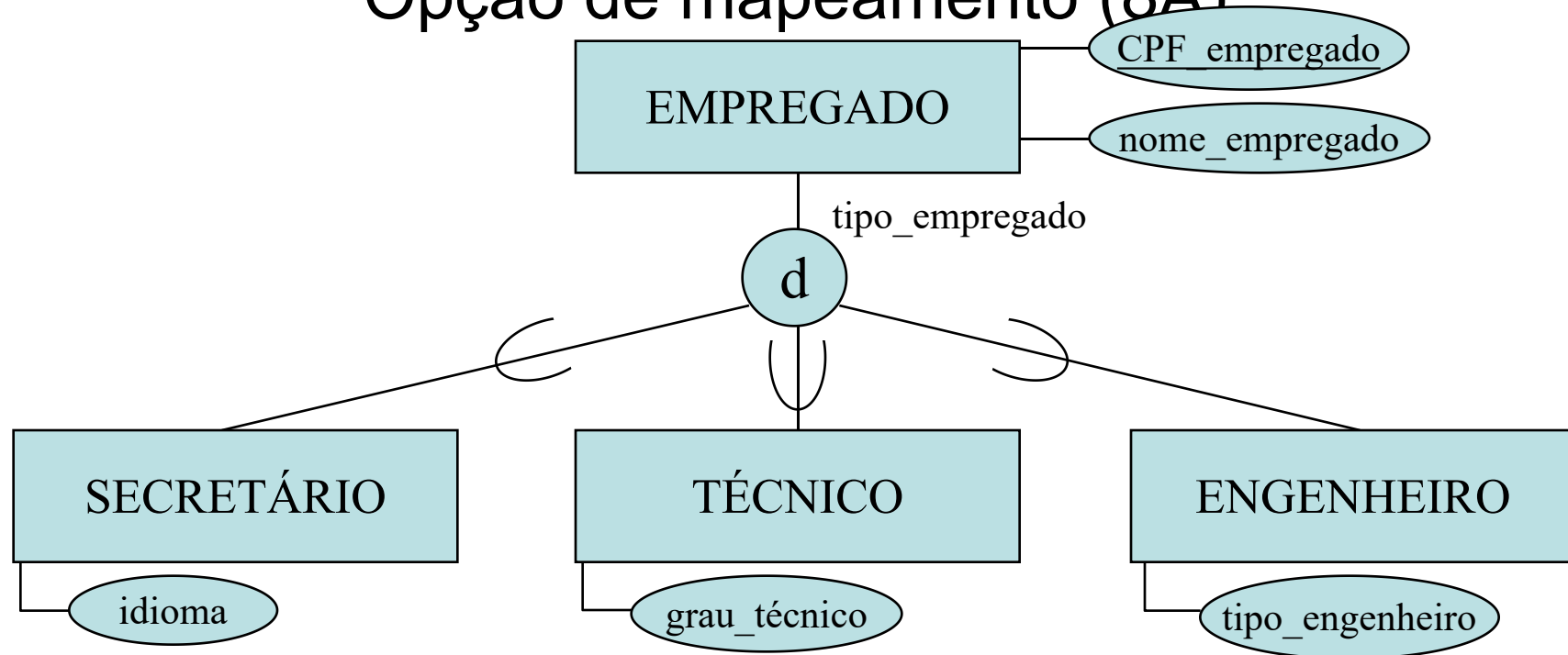
Generalização/Especialização

Opção de mapeamento (8A)

- Modelo entidade relacionamento
 - E_1 : superclasse
 - $E_2, \dots, E_n$: subclasses de E_1
- Modelo relacional
 - a tabela de E_1 possuirá:
 - os atributos de E_1
 - um atributo discriminador, caso necessário
 - as tabelas de E_2 a E_n possuirão:
 - os seus atributos específicos
 - a chave primária de E_1
- Chave primária das subclasses
 - chave primária de E_1

Generalização/Especialização

Opção de mapeamento (8A)



empregado (CPF empregado, nome_empregado, **tipo_empregado**)

secretário (CPF empregado, idioma)

técnico (CPF empregado, grau técnico)

engenheiro (CPF empregado, tipo engenheiro)

opcional

Generalização/Especialização

Opção de mapeamento (8A)

- Essa opção funciona para qualquer especialização
 - Total ou Parcial
 - Disjuntas ou Sobrepostas
- Interessante quando
 - existem poucas subclasses, cada uma com diversos atributos específicos
 - uma consulta tipicamente se concentra em uma ou poucas subclasses de cada vez

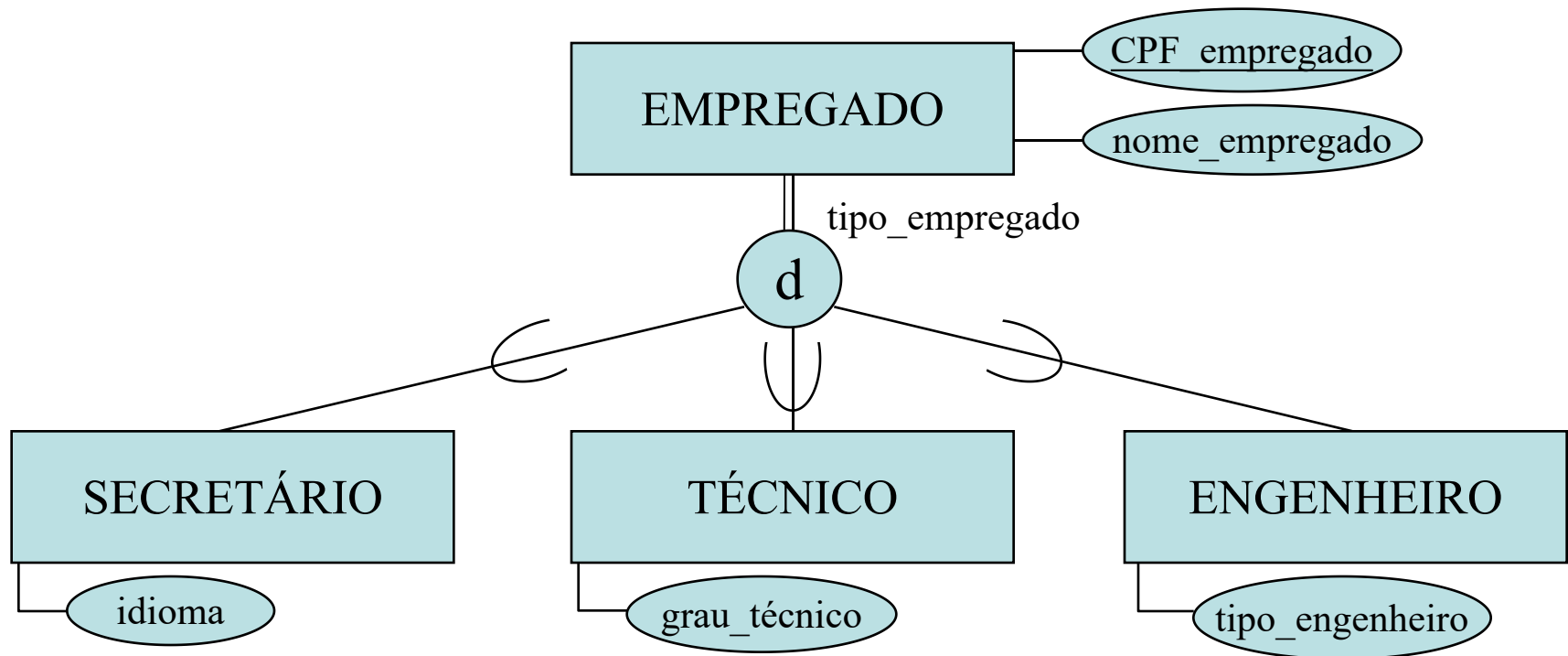
Generalização/Especialização

Opção de mapeamento (8B)

- Modelo entidade relacionamento
 - E_1 : superclasse
 - $E_2, \dots, E_n$: subclasses de E_1
- Modelo relacional
 - as tabelas de E_2 a E_n possuirão:
 - os seus atributos específicos
 - os atributos de E_1
 - a chave primária de E_1
- Chave primária das subclasses
 - chave primária de E_1

Generalização/Especialização

Opção de mapeamento (8B)



secretário (CPF empregado, nome empregado, idioma)
técnico (CPF empregado, nome empregado, grau técnico)
engenheiro (CPF empregado, nome empregado, tipo engenheiro)

Generalização/Especialização

Opção de mapeamento (8B)

- Essa opção funciona
 - apenas para participação total
 - é mais adequada para disjunção, mas suporta sobreposição
- Interessante quando
 - é frequente o acesso a cada entidade em sua totalidade, incluindo-se seus dados genéricos e específicos
 - esta alternativa, comparada com as alternativas que mantêm uma relação para a superclasse, permite evitar uma operação de junção na consulta

Generalização/Especialização

Opção de mapeamento (8B)

- Observação importante
 - esta alternativa não é indicada quando
 - houver necessidade frequente de acessar informações envolvendo todas as entidades genéricas
 - houver a possibilidade de existirem especializações não previstas à priori

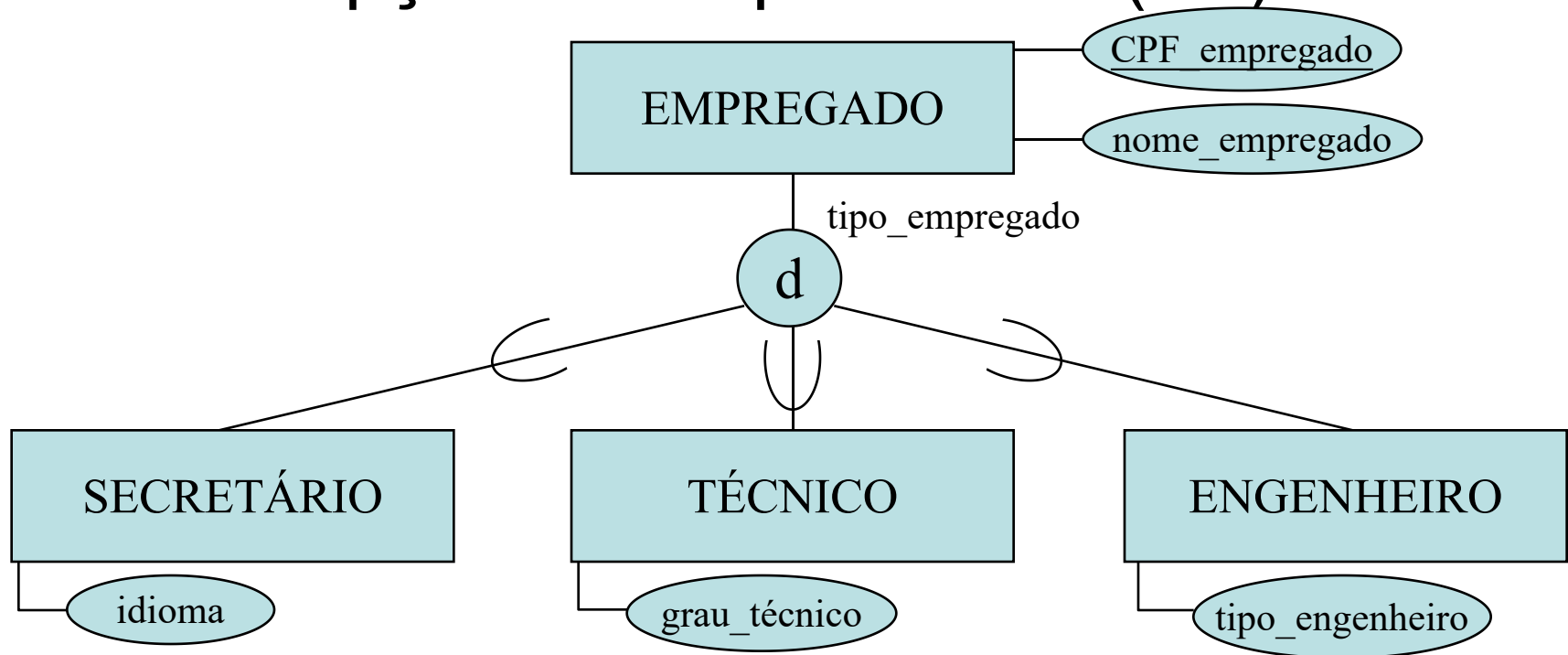
Generalização/Especialização

Opção de mapeamento (8C)

- Modelo entidade relacionamento
 - E_1 : superclasse
 - $E_2, \dots, E_n$: subclasses de E_1
- Modelo relacional
 - a tabela de E_1 possuirá:
 - os atributos de E_1
 - os atributos de $E_2, \dots, E_n$
 - o atributo discriminador, caso necessário

Generalização/Especialização

Opção de mapeamento (8C)



empregado (CPF_empregado, nome_empregado, **tipo_empregado**, idioma, grau_técnico, tipo_engenheiro)

opcional

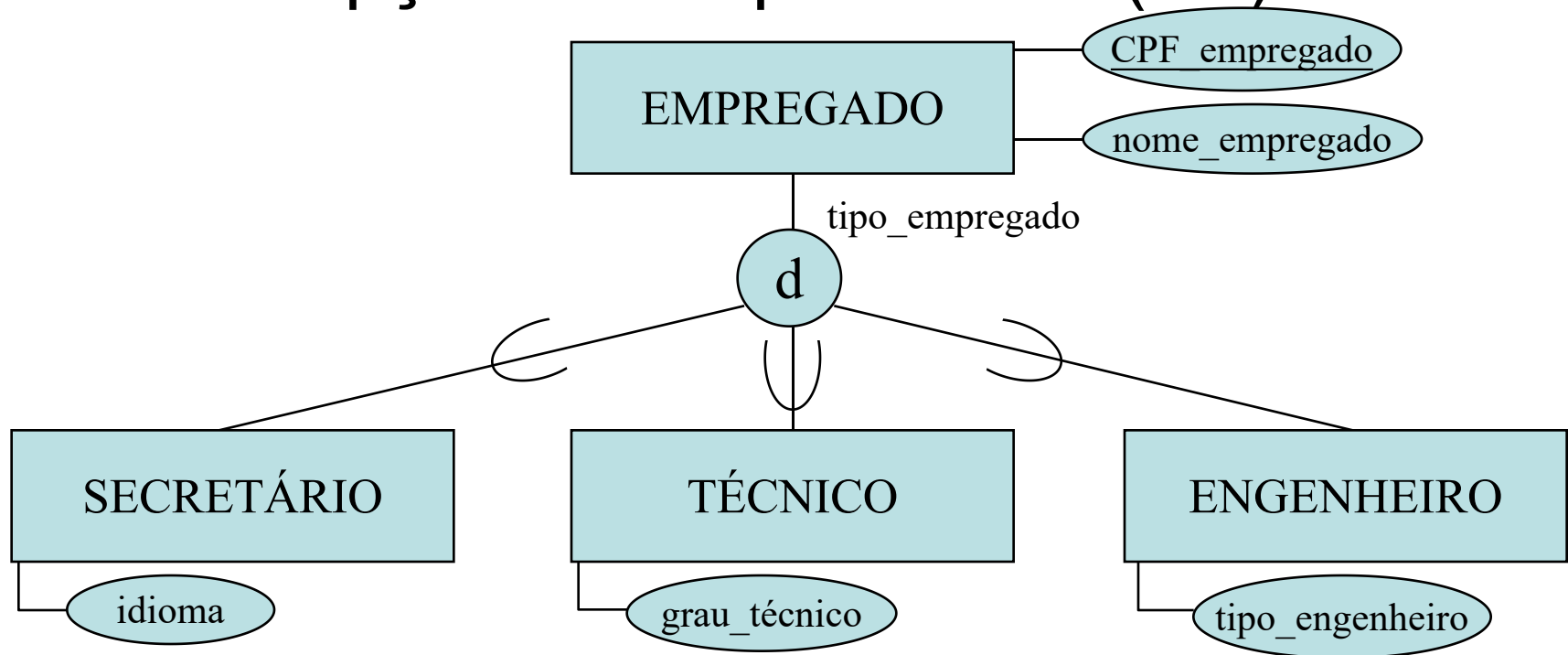
Generalização/Especialização

Opção de mapeamento (8D)

- Modelo entidade relacionamento
 - E_1 : superclasse
 - $E_2, \dots, E_n$: subclasses de E_1
- Modelo relacional
 - a tabela de E_1 possuirá:
 - os atributos de E_1
 - os atributos de $E_2, \dots, E_n$
 - vários atributos discriminadores de valores booleanos, cada um referente à uma subclasse

Generalização/Especialização

Opção de mapeamento (8D)



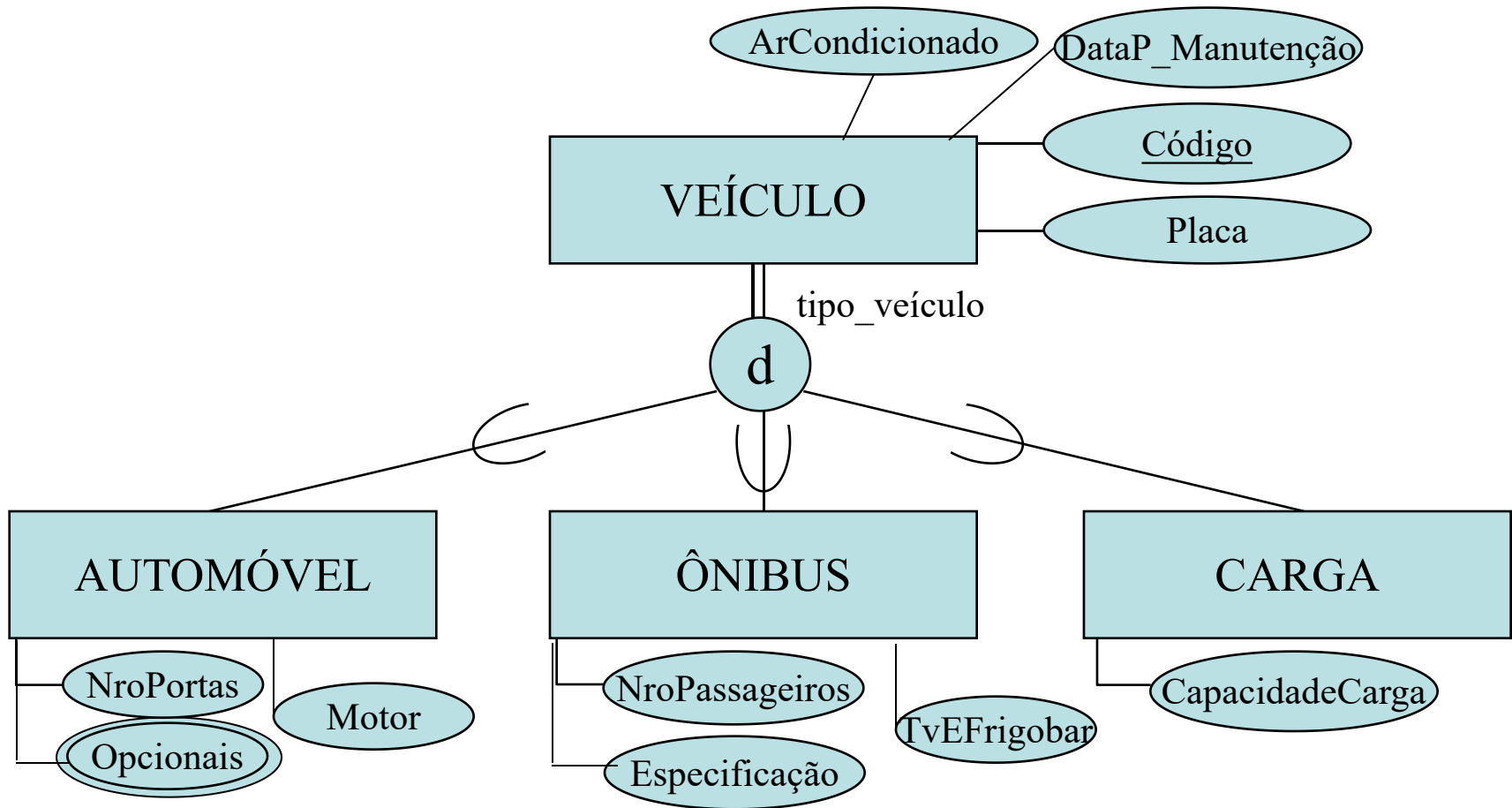
empregado (CPF_empregado, nome_empregado,
tipo_empS, idioma, tipo_empT, grau_técnico,
tipo_empE, tipo_engenheiro)

Generalização/Especialização

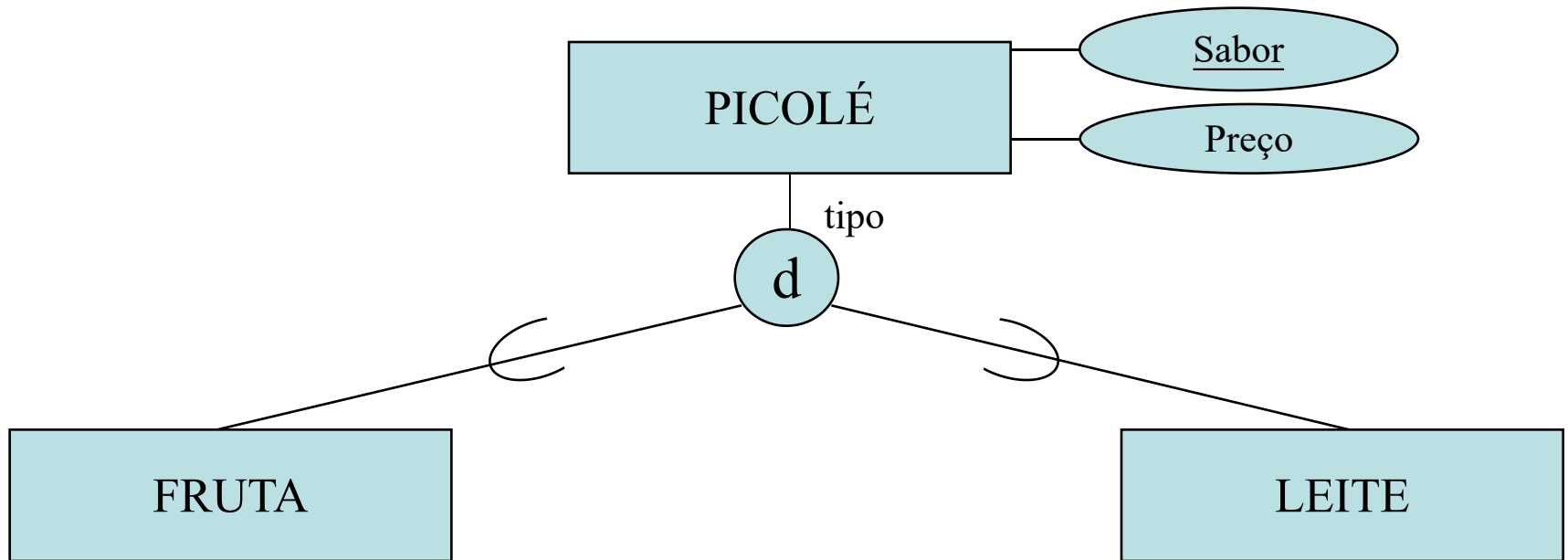
Opção de mapeamento (8C e 8D)

- Interessantes quando
 - existem poucos atributos específicos nas subclasses
 - houver a possibilidade de existirem especializações (sem atributos específicos) não previstas à priori

Exercício



Exercício



Agregação

- Para mapear ocorrências de agregação
 - considerar cada um dos casos de como o tipo-entidade resultante da agregação é identificado
 - levar em consideração as chaves dos tipos-entidade componentes, o tipo-relacionamento gerador, os atributos do tipo-relacionamento gerador, o tipo-entidade agregação, e os atributos do tipo-entidade agregação

Agregação

Opção 1

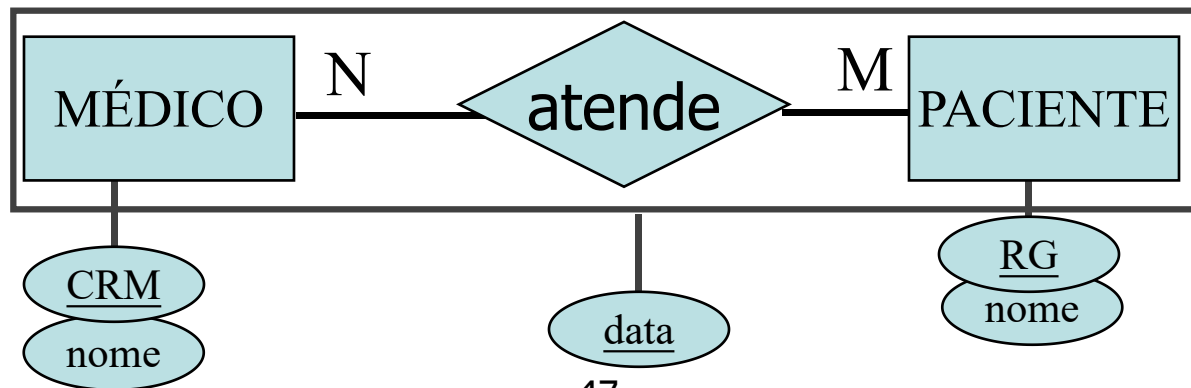
Paciente = {RG, Nome}

Médico = {CRM, Nome}

Consulta = {RG, CRM, Data}

- Deve ser usada
 - quando o tipo-entidade agregação é identificado por atributo próprio + chaves dos tipos-entidade que participam do tipo-relacionamento gerador
 - quando uma mesma instância do tipo-relacionamento gerador resulta em mais de uma entidade agregada

CONSULTA



Agregação

Opção 2

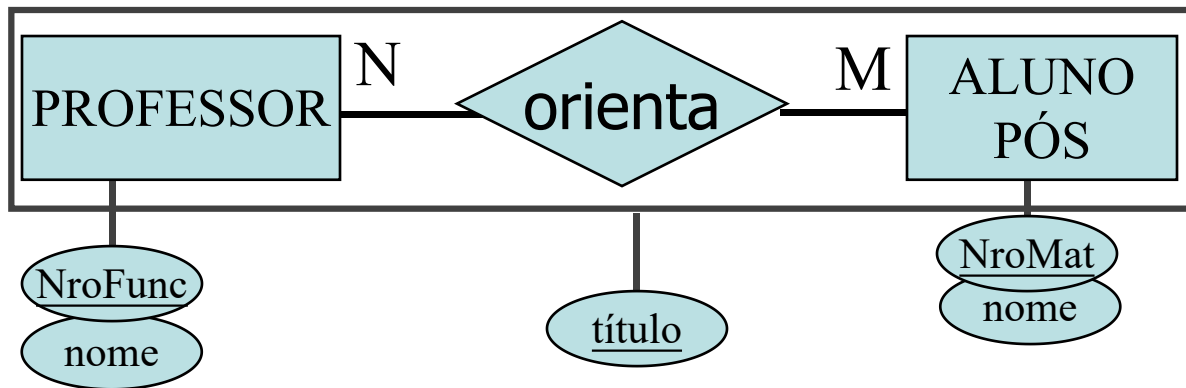
Professor = {NroFunc, Nome}

AlunoPos = {NroMat, Nome}

Projeto = {Orientador, Aluno, Titulo}

- Deve ser usada
 - quando o tipo-entidade agregação é identificado por um de seus atributos
 - em geral o atributo identificador da agregação era identificador do tipo-relacionamento gerador

PROJETO

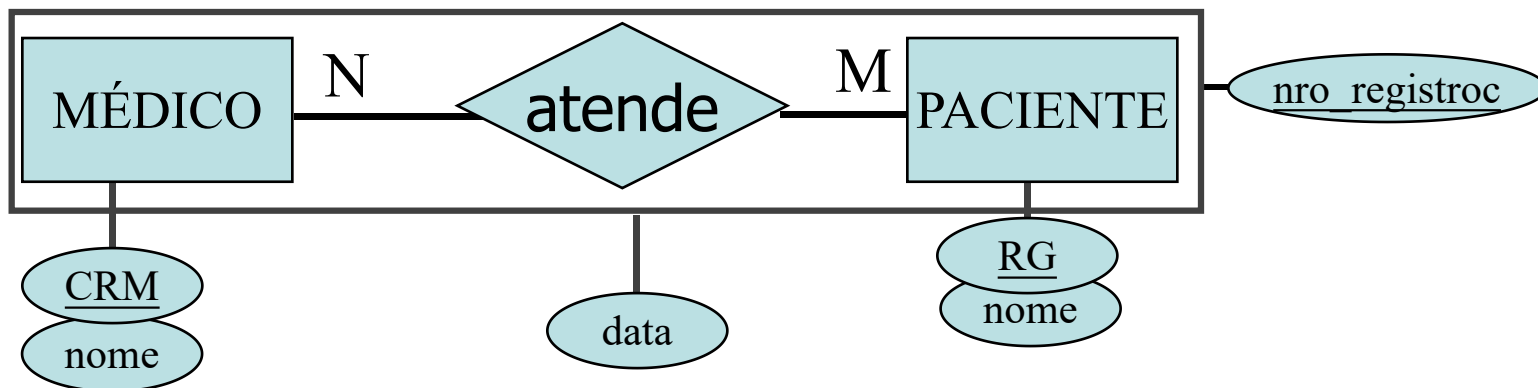


Agregação

Opção de mapeamento 3

- Mistura das opções 1 e 2

CONSULTA



Paciente = {RG, Nome}

Médico = {CRM, Nome}

Consulta = {RGPa, CRMMe, Data, NroRegistroC}

Agregação

Analisar os atributos
do tipo-relacionamento
gerador

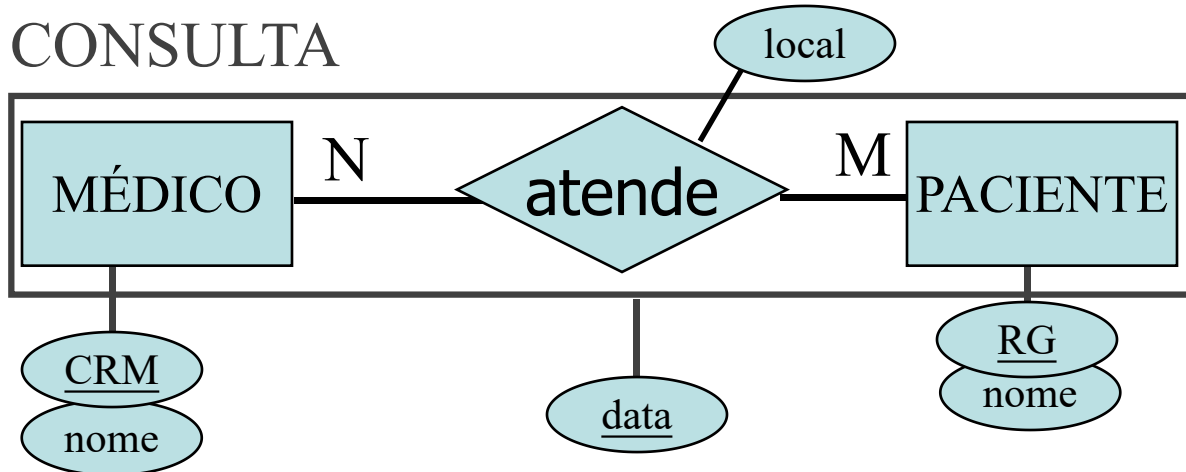
Paciente = {RG, Nome}

Médico = {CRM, Nome}

Consulta = {RGP_a, CRM_{Me}, Data, Local}

- ◆ Sempre que uma instância do tipo-relacionamento gerador puder resultar em mais de uma entidade agregada

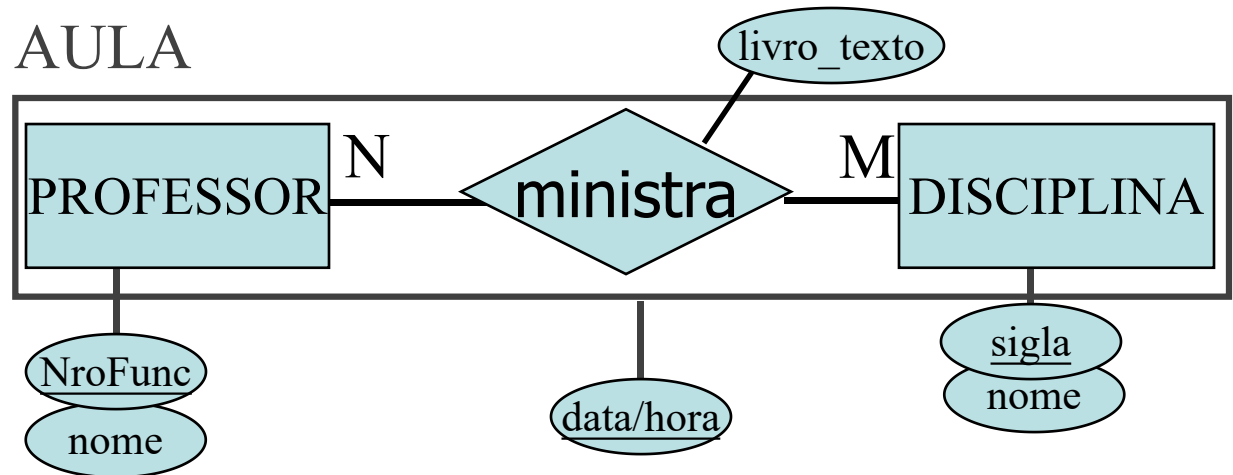
- Quando puderem ser repassados para a agregação → não mapear o relacionamento



Agregação

Analisar os atributos do tipo-relacionamento gerador

- ◆ Sempre que uma instância do tipo-relacionamento gerador puder resultar em mais de uma entidade agregada
 - Quando não puderem ser repassados para a agregação → mapear o relacionamento



Professor = {NroFunc, Nome}

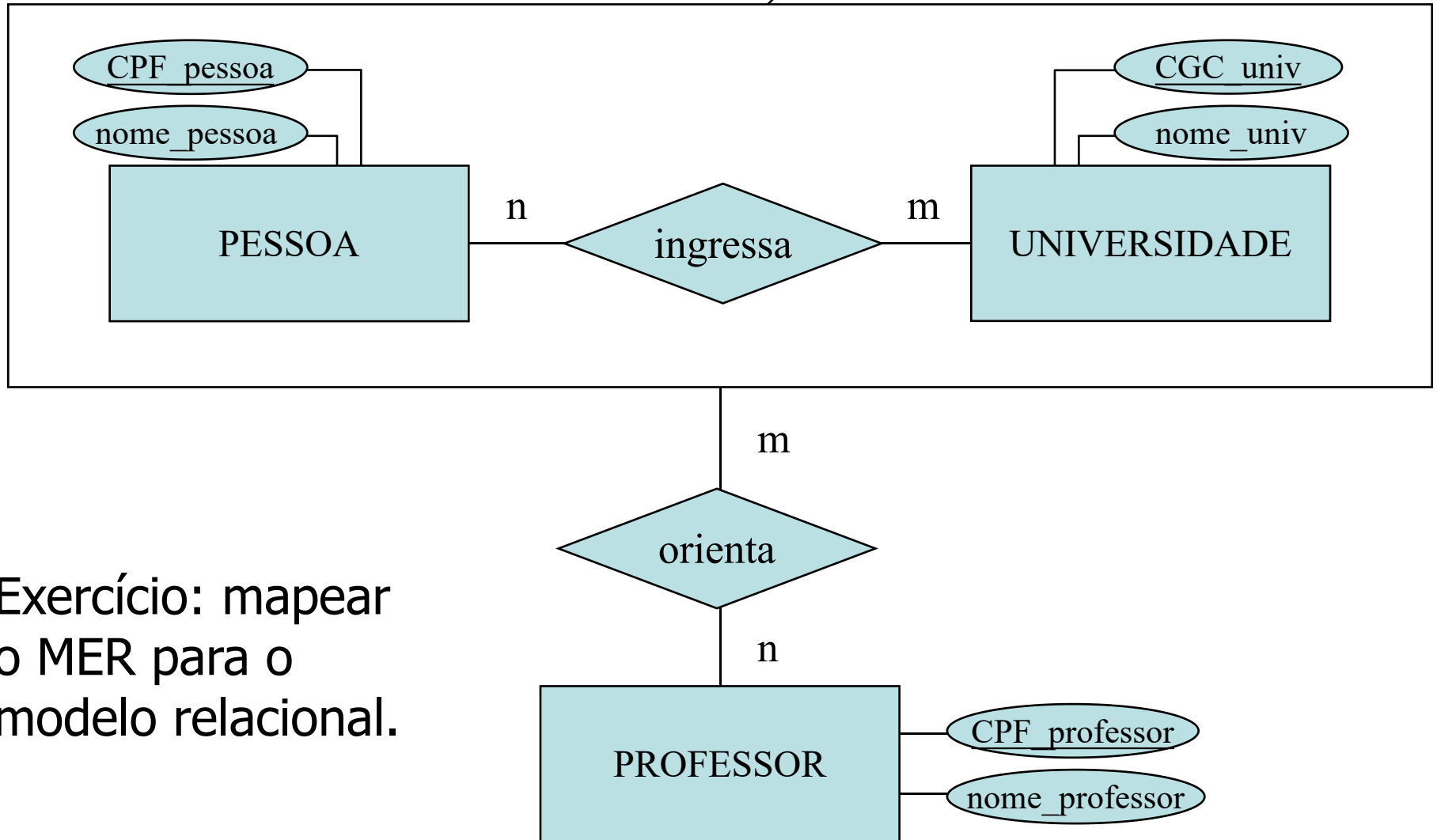
Disciplina = {Sigla, Nome}

Ministra = {Professor, Disciplina, LivroTexto}

Aula = {Professor, Disciplina, DataHora}

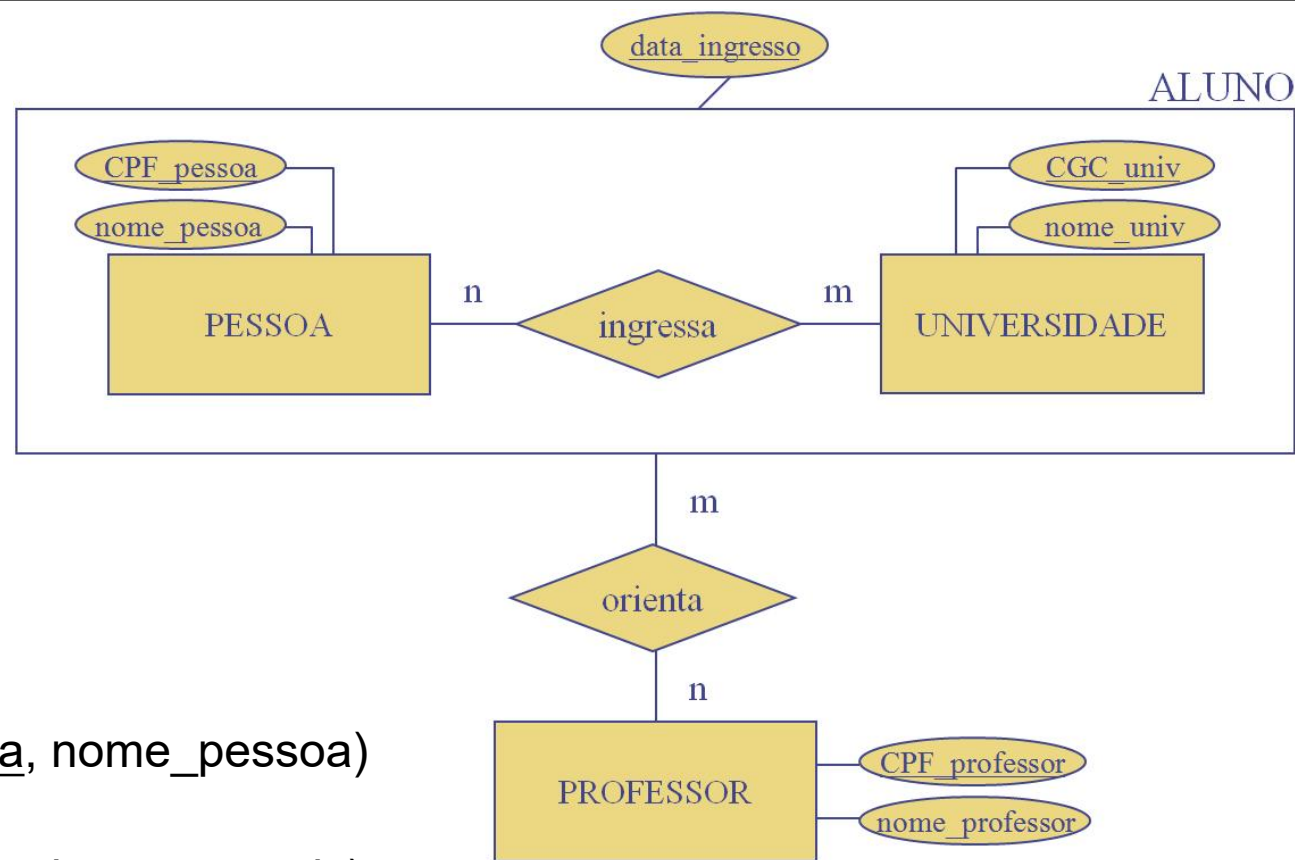
Exercício 1

ALUNO



Exercício: mapear
o MER para o
modelo relacional.

Exercício 1



- Mapeamento

pessoa (CPF_pessoa, nome_pessoa)

universidade (CGC_univ, nome_univ)

aluno (CPF_pessoa, CGC_univ, data_ingresso)

professor (CPF_professor, nome_professor)

orienta (CPF_professor, CGC_univ, CPF_pessoa, data_ingresso)

Referências

- R. Elmasri e S. B. Navathe, Sistema de Banco de Dados, 6ª edição, Pearson, 2011.
- R. Ramakrishnan e J. Gehrke, *Database Management Systems*, 3a Edição, McGraw-Hill, 2003.
- Slides cedidos pelo Prof. Humberto Luiz Razente

Aula 7 – SBD

Postgres

SQL: DDL

Profa. Elaine Faria

UFU

Agradecimentos

- Agradecimentos
 - Ao professor Bruno Travençolo que gentilmente cedeu seus slides

PostgreSQL

- É um SGBD que incorpora o modelo relacional para o banco de dados e suporta a linguagem SQL como linguagem de consulta padrão.
- Executa em diversas plataformas
 - Unix-like: FreeBSD; Linux; Mac OS X
 - Windows
- É um software livre com código-fonte aberto.

Histórico do PostgreSQL

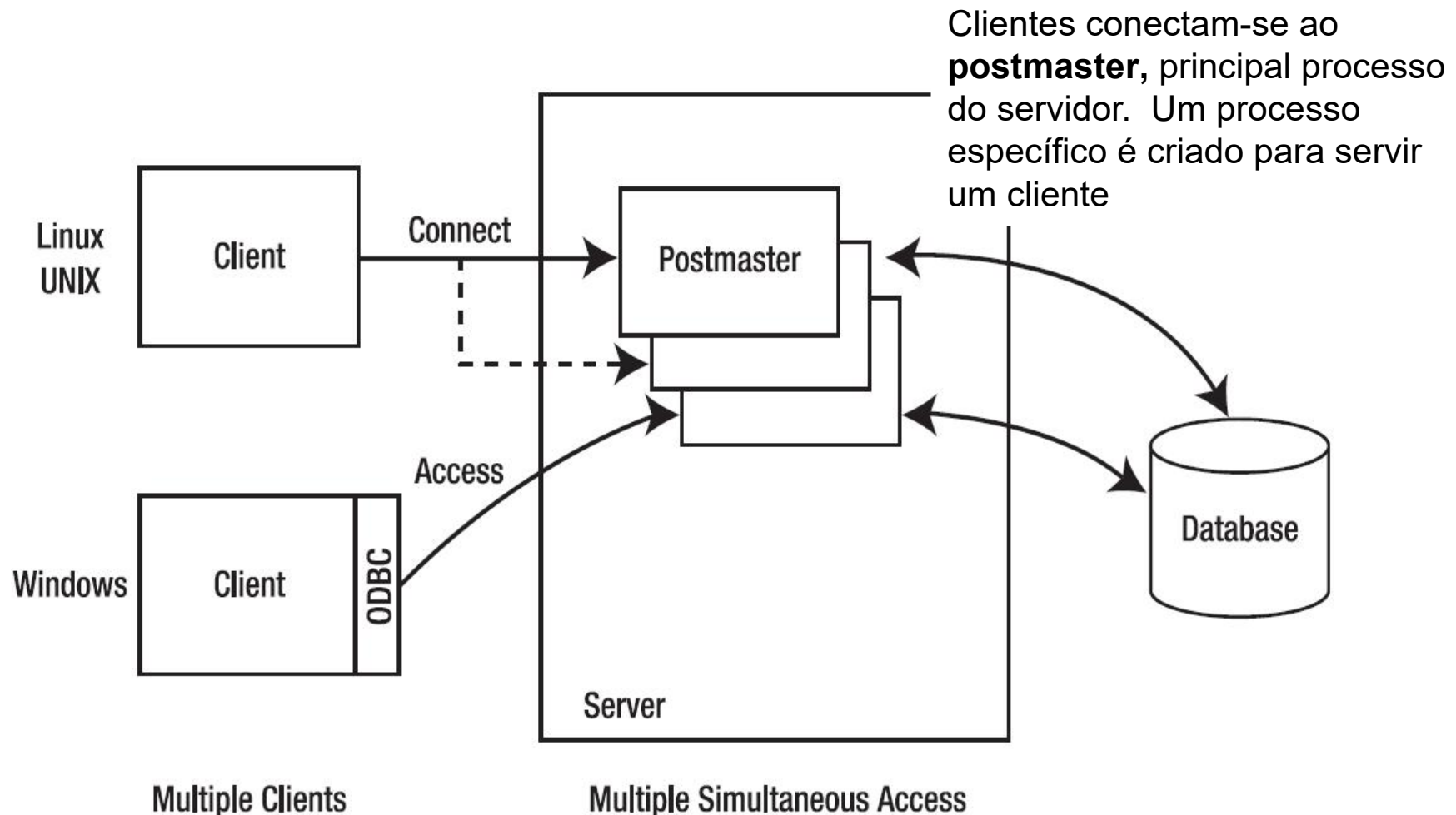
- Banco de dados relacional chamado Ingres foi desenvolvido entre 1977-1985 na Universidade da Califórnia em Berkley,
- Seu código foi usado para a construção do primeiro SGBD comercial
 - Hoje o Ingres é um software livre
- Em 1986 um novo projeto na UCB foi iniciado: o Postgres, que utilizou várias idéias do Ingres.
- Em 1994 o Postgres passou receber suporte à **SQL**, mudando assim sua denominação para Postgres95
- Em 1996 recebeu uma nova denominação: PostgreSQL e foi aberto à desenvolvedores na rede.
- Em 2005: *PostgreSQL 8.0* com facilidades de instalação em Windows

Arquitetura do PostgreSQL

- Assim como vários SGBDs comerciais, o PostgreSQL pode ser usado em um ambiente **Cliente/Servidor**
- Existe um processo servidor que é executado em uma única máquina
- Os aplicativos que necessitam acessar os dados devem fazer isso via o processo servidor
- Os clientes não conseguem acessar o dado diretamente, mesmo quando estão rodando no mesmo computador que o servidor

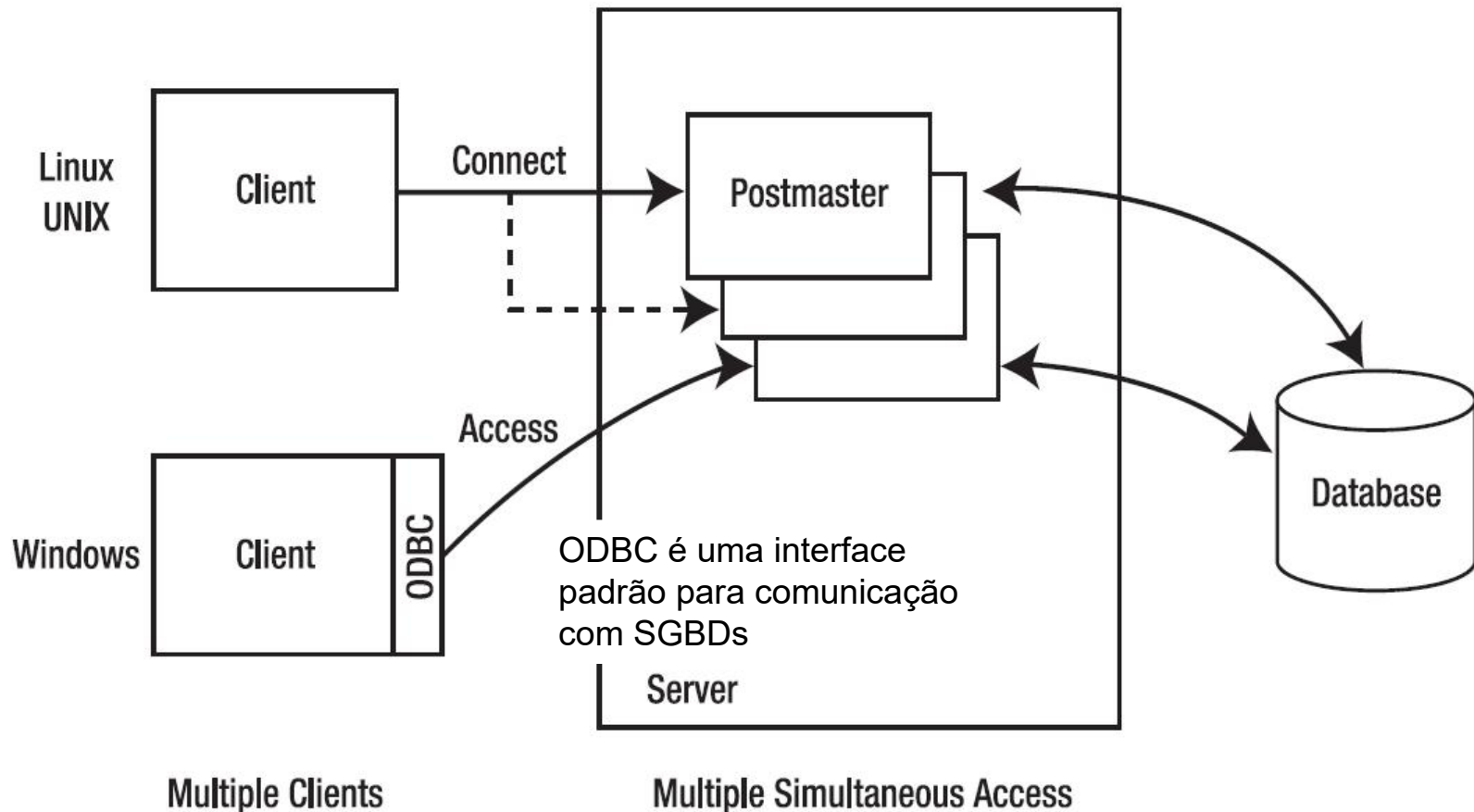
Arquitetura Cliente/Servidor

- É possível que o cliente rode Windows e o servidor seja Unix
- Acesso simultâneo/ Necessária rede TCP/IP



Arquitetura Cliente/Servidor

- ODBC – *Open Database connectivity* permite que vários aplicativos usem o BD



Acessando os dados em PostgreSQL

- Aplicativo em linha de comando: **psql**
 - É possível executar consultas SQL interativamente
- Aplicação gráfica: **pgAdmin**
 - Utilizaremos o pgAdmin 4 neste curso
- Servidor web: apache rodando **phpPgAdmin**
- Criação de aplicativos clientes utilizando bibliotecas de acesso ao BD
- Acesso indireto via ODBC ou JDBC (Java Database Connectivity)

Conectando-se ao servidor

Novo Registro de Servidor

Propriedades

Nome: galileo

Máquina:

Porta: 5432

SSL:

Manutenção do BD:

Nome de usuário:

Senha:

Armazenar senha: ☒

Restaurar ambiente?: ☒

restrição de BD:

Serviço:

Conectar agora: ☒

Colour:

Ajuda OK Cancelar

Especifique o nome de usuário

No laboratório

Host/Maquina: localhost

Username/Nome do usuário: postgres

Password/Senha: root

Port/Porta: 5432

DB/BD: postgres

Em sua casa:

Host/Maquina: localhost

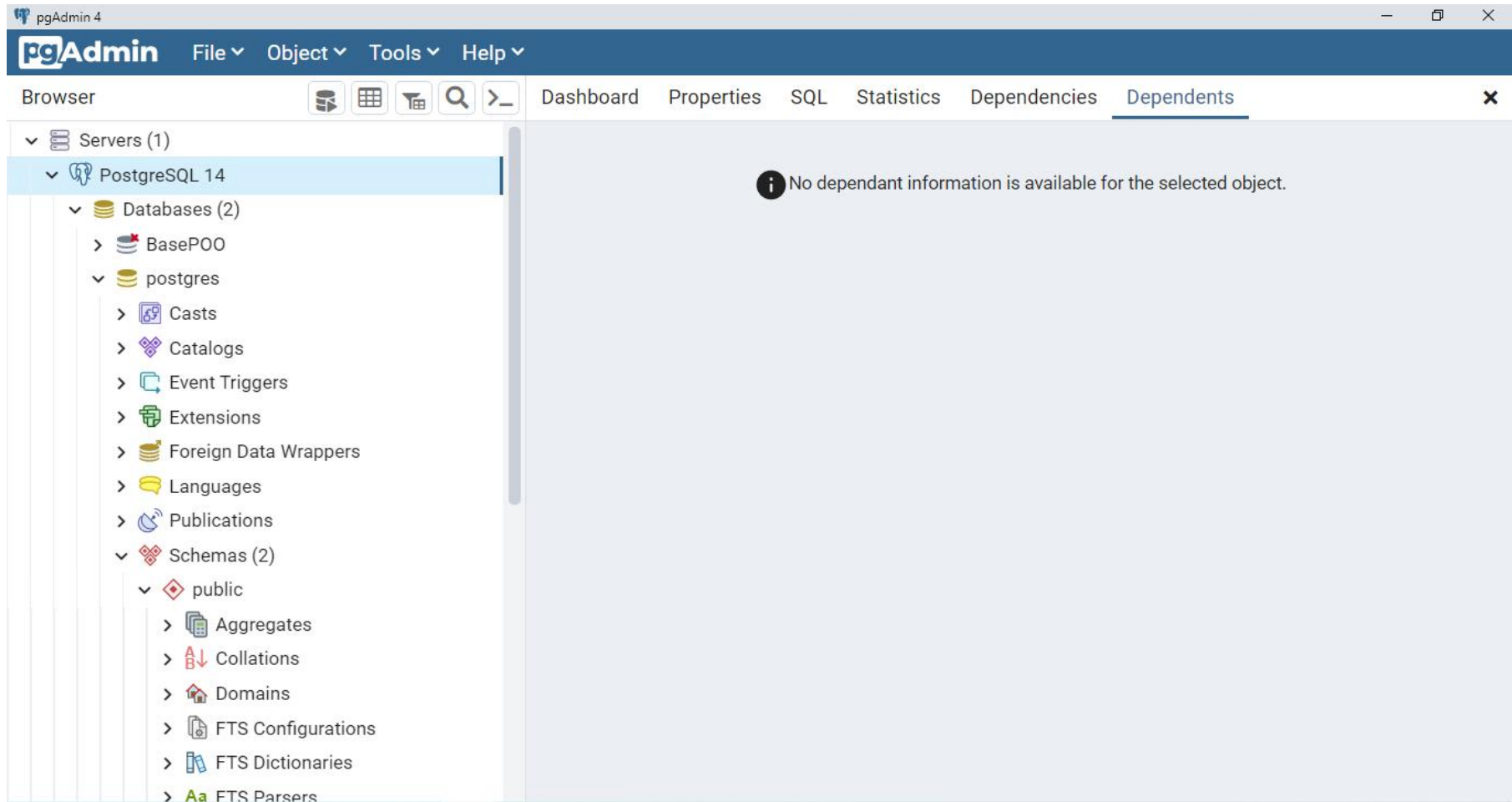
Username/Nome do usuário: postgres

Password/Senha: a que vc definiu na instalação

Port/Porta: 5432

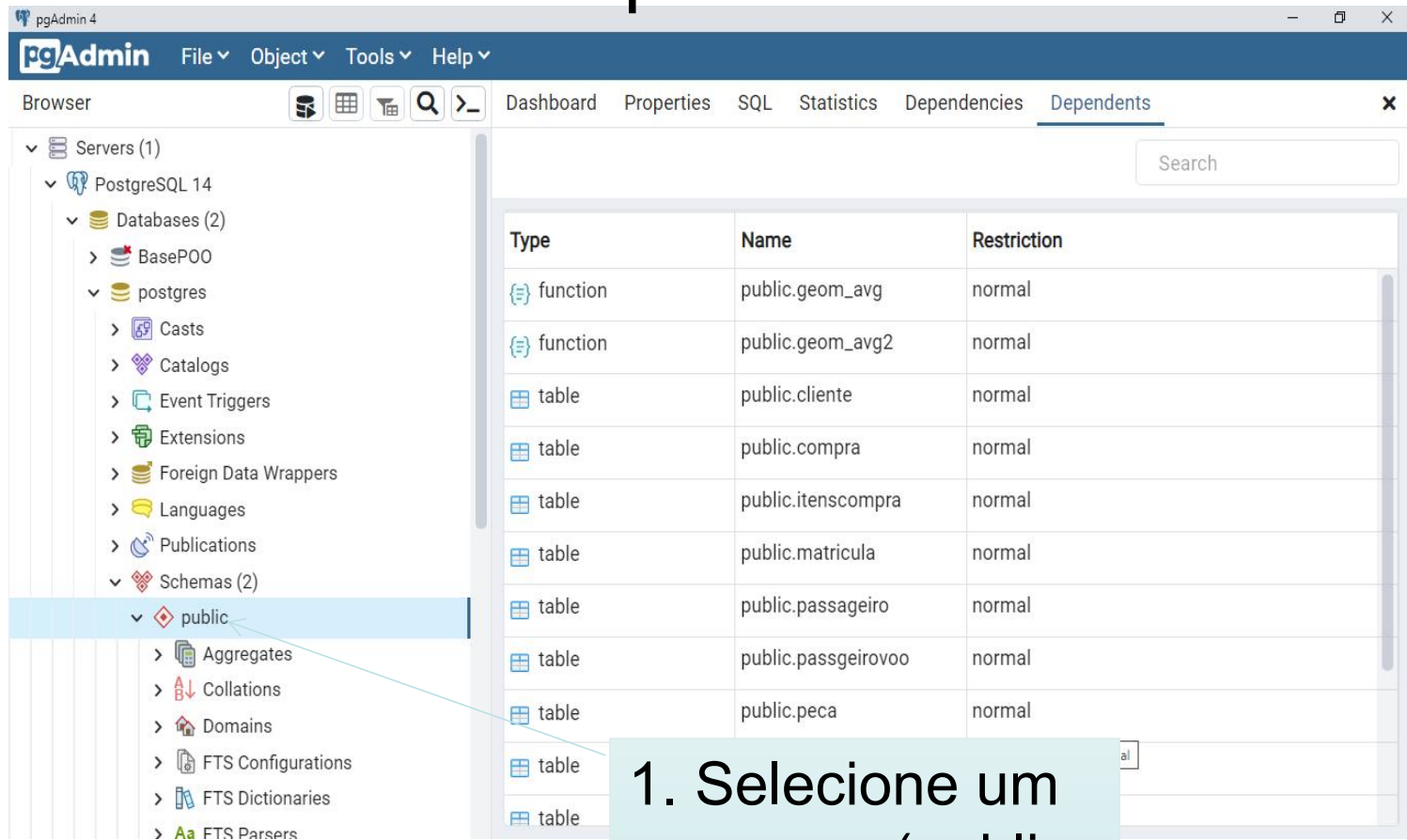
DB/BD: postgres

Crie a base de dados de seu grupo



Utilizando o pgAdmin III

- Seleccione um esquema



1. Seleccione um esquema (public ou outro)

Esquemas

- É representado por uma coleção de vários objetos de um ou mais usuário de banco de dados como exemplo: tabelas, seqüências, índices, etc.
- São associados a um banco de dados na razão de vários esquemas para um BD.
- É uma ponte de associação entre o usuário (ou grupo de usuários) e um (ou mais) objeto(s) de banco

Não há nada de especial sobre o esquema *public* → ele existe por *default* e pode ser *dropped*

Apagando os esquemas

-- criando esquemas

```
create schema universidade;  
create schema farmacia;  
create schema oficina;
```

-- usando os esquemas

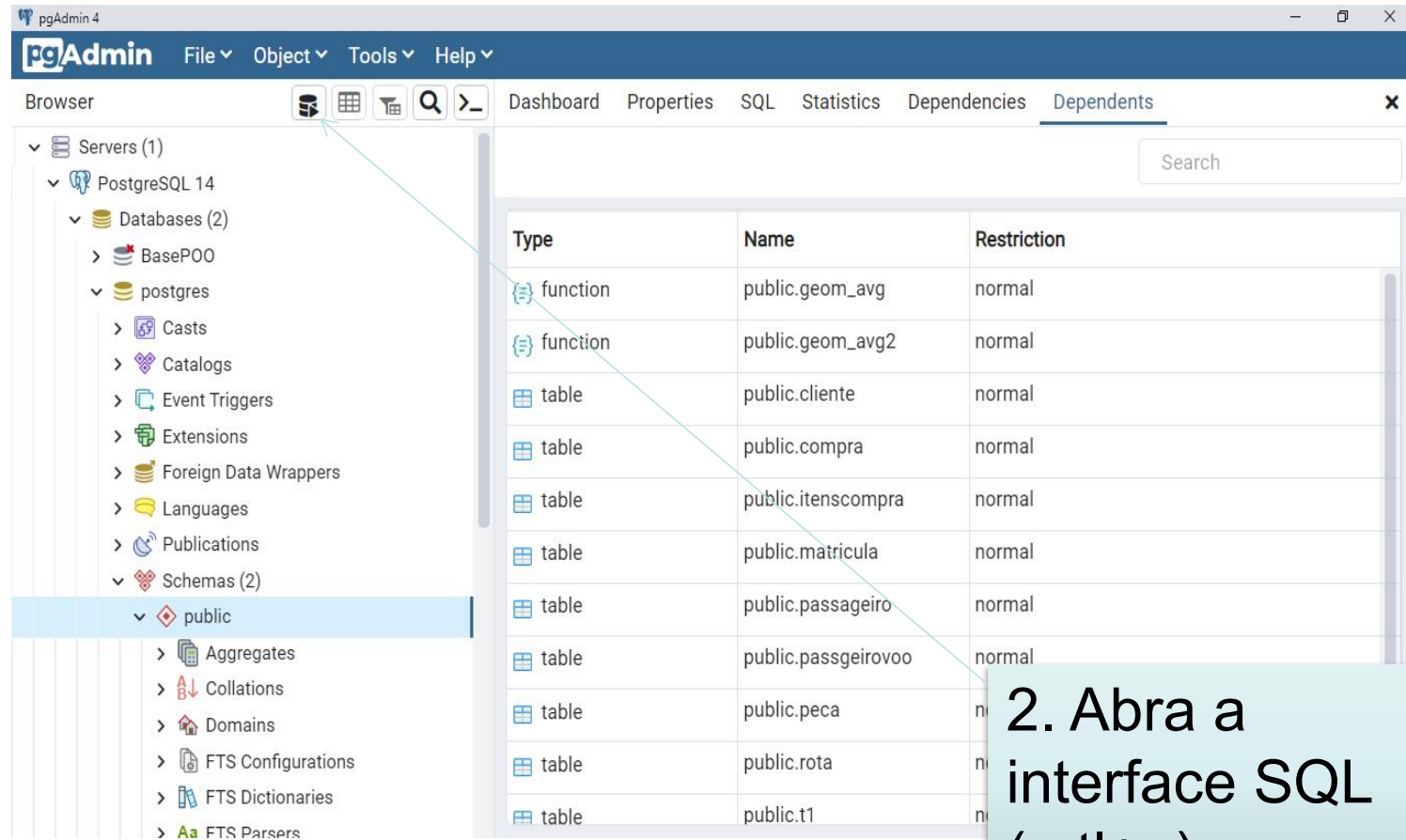
```
set search_path to universidade;  
set search_path to farmacia;  
set search_path to oficina;
```

-- apagando **todas** as tabelas e o esquema

```
drop schema cinema cascade;  
drop schema grupo01 cascade;  
drop schema public cascade;
```

Utilizando o pgAdmin III

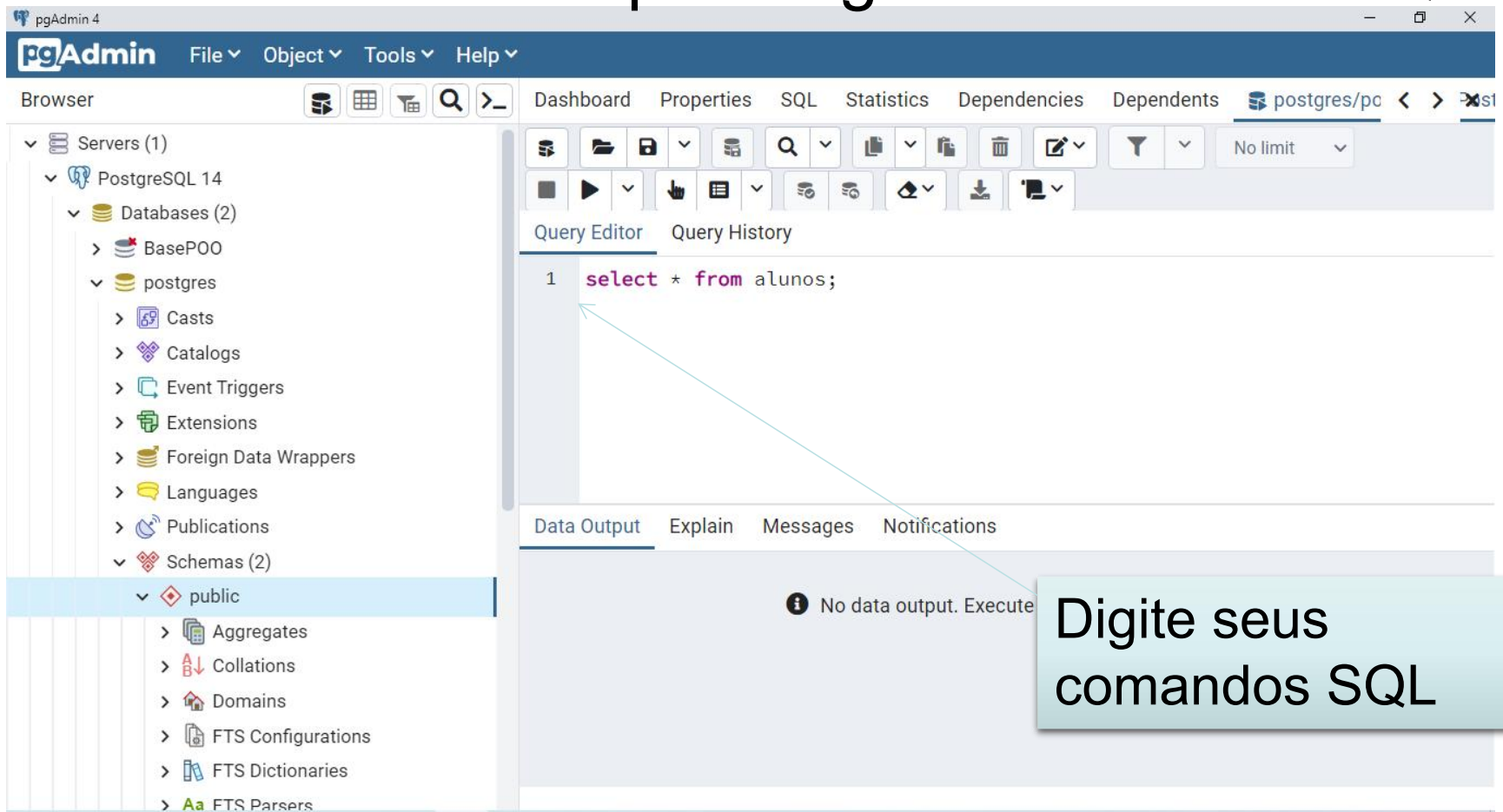
- Abra a interface para digitar comandos SQL



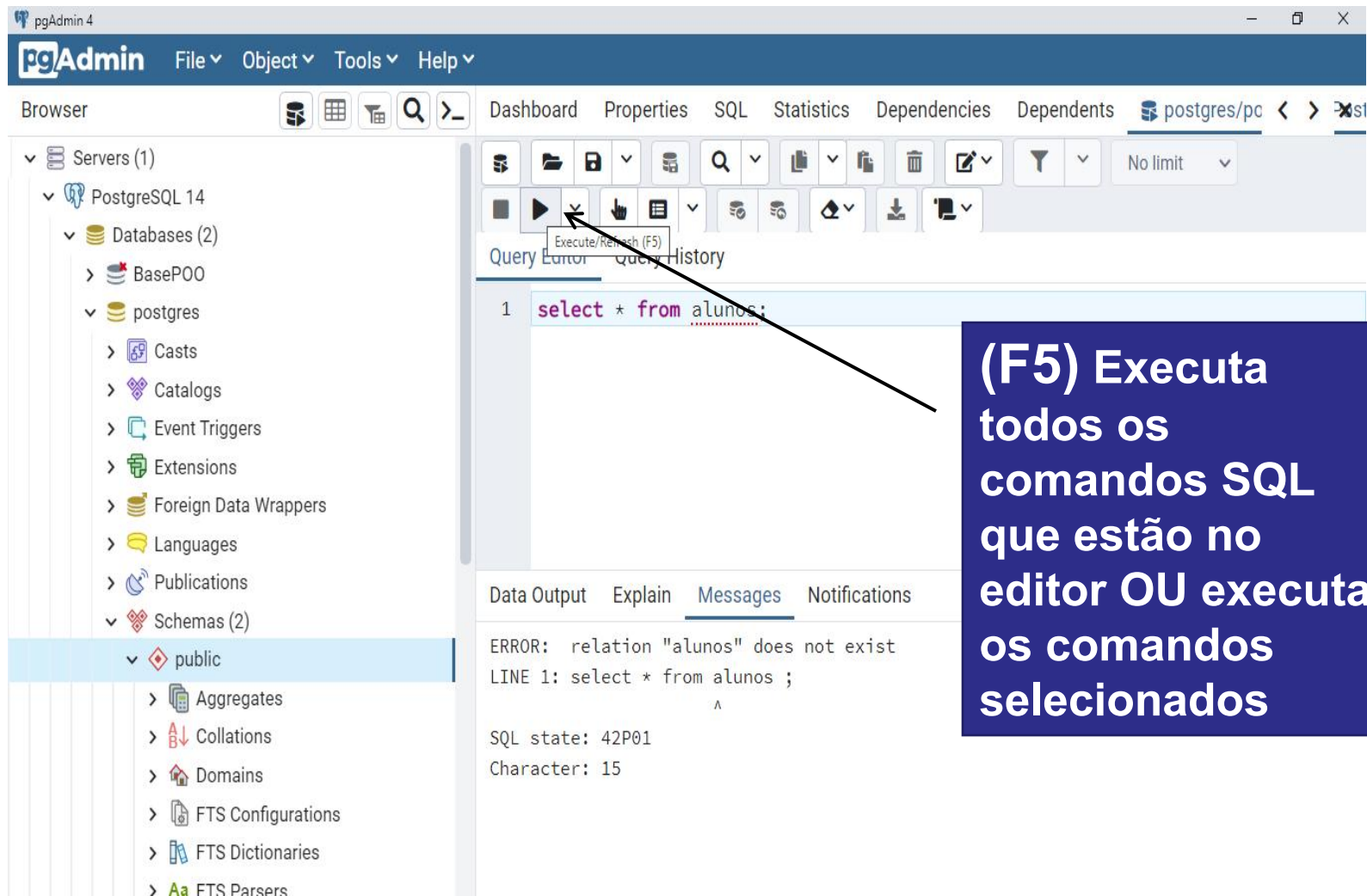
2. Abra a interface SQL (ctrl+e)

Utilizando o pgAdmin III

- Abra a interface para digitar comandos SQL



Dicas para execução pgAdmin III



The screenshot displays the pgAdmin 4 web interface. On the left, the 'Browser' pane shows a tree structure of database objects, with 'public' schema selected under 'postgres' database. The main area is the 'Query Editor', which contains a single SQL query: `select * from alunos;`. A tooltip 'Execute/Refresh (F5)' is visible over the 'Execute' button in the toolbar. Below the query editor, the 'Messages' tab shows an error message: 'ERROR: relation "alunos" does not exist LINE 1: select * from alunos ;'. The 'SQL state' is 42P01 and the 'Character' is 15.

(F5) Executa todos os comandos SQL que estão no editor OU executa os comandos selecionados

SQL

Structured Query Language

- Desenvolvida e implementada pelo laboratório de pesquisa da IBM em San Jose – início da década de 70
- Inicialmente chamada de SEQUEL (*Structured English QUery Language*)
- Criada como interface entre usuários e o primeiro SGBDR – SYSTEM R

SQL

Structured Query Language

- Uma das mais importantes linguagens relacionais
- Exemplos de SGBD que utilizam SQL
 - Oracle
 - Informix
 - Ingress
 - MS SQL Server
 - Interbase/Firebird
 - Sybase
 - DB2
 - MySQL
 - PostgreSQL

SQL

Structured Query Language

- Atrativo: pequena quantidade de comandos para realizar todas as operações necessárias para definição e manipulação de relações
 - Simplicidade
 - Grande poder de consulta
- Padrão facilita migração

SQL

Structured Query Language

- O padrão SQL
 - *American National Standard Institute (ANSI) e International Organization for Standardization (ISO)*
 - Versão mais recente:
 - SQL 2016
 - Versões anteriores
 - SQL 3 → SQL 99
 - SQL 2 → SQL 92
 - SQL 1 → SQL 86

Composição do SQL

- Linguagem de Definição dos Dados (DDL)
 - comandos para a definição, a modificação e a remoção de relações, além da criação e da remoção de índices
- Linguagem Interativa de Manipulação dos Dados (DML)
 - comandos para a consulta, a inserção, a remoção e a modificação de tuplas no banco de dados

Composição do SQL

- Linguagem de Manipulação dos Dados Embutida
 - pode ser utilizada a partir de linguagens de programação de propósito geral
- Definição de visões
 - SQL DDL inclui comandos para a criação e a remoção de visões
- Restrições de integridade
 - SQL DDL possui comandos para a especificação de restrições de integridade

Composição do SQL

- Autorização
 - SQL DDL inclui comandos para a especificação de direitos de acesso a relações e visões
- Gerenciamento de transações
 - introduz comandos para a especificação do início e do fim das transações
- Recuperação de falhas
 - introduz comandos para utilização do arquivo de *log*

Composição do SQL

- Linguagem de Definição dos Dados (DDL)
 - CREATE
 - ALTER
 - DROP

DDL – Comandos para definição de dados

- Comandos DDL
 - **CREATE** – Cria uma definição
 - **CREATE TABLE** tab ...
 - **ALTER** – Altera uma definição
 - **ALTER TABLE** tab ADD ...
 - **DROP** – Exclui uma definição
 - **DROP TABLE** tab

DDL – Comandos para definição de dados

- CREATE TABLE

Exemplo:

/ Cria tabela departamento */*

```
CREATE TABLE Departamento (  
    DNOME VARCHAR(15) NOT NULL,  
    DNUMERO INT NOT NULL,  
    GERSSN CHAR(9) NOT NULL,  
    GERDATAINICIO DATE,  
    CONSTRAINT dptopk PRIMARY KEY  
        (DNUMERO),  
    UNIQUE(DNOME),  
    CONSTRAINT dptogerfk FOREIGN KEY  
        (GERSSN) REFERENCES EMPREGADO(SSN)  
);
```

Identificadores

- Iniciam com letras (a-z) ou underscore (_)– Caracteres subsequentes: letras, dígitos (0-9), _
- Identificadores e palavras-chave não são *case-sensitive*
 - UPDATE MY_TABLE SET A = 5;
 - uPDaTE my_Table SeT a = 5;
- Convenção adotada
 - Palavras-chave em maiúscula
 - Identificadores em minúsculo
 - UPDATE my_table SET a = 5;
- Identificadores com aspas
 - Aceitam quaisquer caracteres
 - UPDATE "my_table" SET "a" = 5;

Identificadores

- Ao colocar aspas em um identificador ele torna-se *case-sensitive*
- Identificadores sem aspas são sempre transformados em minúsculo
- Se você criar um esquema ou tabela usando a interface gráfica do pgAdmin e, caso o identificador deste objeto não seja composto por letras minúsculas, o objeto será identificado somente por meio de aspas.
 - Faça o teste, criando esquemas e tabelas por meio da interface gráfica e utilizando letras maiúsculas.
- Mais informações e referência:
 - <http://www.postgresql.org/docs/8.4/static/sql-syntax-lexical.html>

DDL – Comandos para definição de dados

- CREATE TABLE – cria uma tabela, seus campos e as restrições de campo

```
CREATE TABLE <nome da tabela> (  
    <definição de coluna 1>  
    ....  
    ....  
    <definição de coluna N>  
    <restrições de integridade>  
);
```

Onde <definição de coluna> pode ser
<nome atributo> <tipo de dado> <restrições de integridade>

Tipos de dados

- Lógico

Table B-1. *PostgreSQL Logical Data Type*

SQL Name	PostgreSQL Alternative Name	Notes
boolean	bool	Holds a truth value. Will accept values such as TRUE, 't', 'true', 'y', 'yes', and '1' as true. Uses 1 byte of storage, and can store NULL, unlike a few proprietary databases.

Tipos de dados

Table B-2. *postgresql Exact Number Types*

SQL Name	PostgreSQL Alternative Name	Notes
smallint	int2	A signed 2-byte integer that can store -32768 to +32767.
integer, int	int4	A signed 4-byte integer that can store -2147483648 to +2147483647.
bigint	int8	A signed 8-byte integer, giving approximately 18 digits of precision.
bit	bit	Stores a single bit, 0 or 1. To insert into a table, use syntax such as INSERT INTO ... VALUES(B'1');.
bit varying	varbit(n)	Stores a string of bits. To insert into a table, use syntax such as INSERT INTO ... VALUES(B'011101');.

Tipos de dados

- Números aproximados

Table B-3. *PostgreSQL Approximate Number Types*

SQL Name	PostgreSQL Alternative Name	Notes
<code>numeric (precision, scale)</code>		Stores an exact number to the precision specified. The user guide states there is no limit to the precision that may be specified.
<code>real</code>	<code>float4</code>	A 4-byte, single-precision, floating-point number.
<code>double precision</code>	<code>float8</code>	An 8-byte, double-precision, floating-point number.
<code>money</code>		Equivalent to <code>numeric(9,2)</code> , storing 4 bytes of data. Its use is discouraged, as it is deprecated and support may be dropped in the future.

Tipos de dados

- Dados temporais

Table B-4. *PostgreSQL Types for Date and Time*

SQL Name	PostgreSQL Alternative Name	Notes
timestamp	datetime	Stores dates and times from 4713 BC to 1465001 AD, with a resolution of 1 microsecond. You may also see <code>timestampz</code> used sometimes in PostgreSQL, which is a shorthand for <code>timestamp</code> with time zone.
interval	interval	Stores an interval of approximately $\pm 178,000,000$ years, with a resolution of 1 microsecond.
date	date	Stores dates from 4713 BC to 32767 AD, with a resolution of 1 day.
time	time	Stores a time of day, from 0 to 23:59:59.99, with a resolution of 1 microsecond.

Tipos de dados

- Caracteres

Table B-5. *PostgreSQL Character Types*

SQL Name	PostgreSQL Alternative Name	Notes
char, character	bpchar	Stores a single character.
char(n)	bpchar(n)	Stores exactly <i>n</i> characters, which will be padded with blanks if fewer characters are actually stored.
character varying(n)	varchar(n)	Stores a variable number of characters, up to a maximum of <i>n</i> characters, which are not padded with blanks. This is the standard choice for character strings.
	text	A PostgreSQL-specific variant of varchar, which does not require you to specify an upper limit on the number of characters.

Tipos de dados

- Existem outros tipos de dados além dos apresentados anteriormente. Consulte o manual do PostgreSQL:
- <http://www.postgresql.org/docs/8.4/static/datatype.html>
- Livro: Beginning databases with PostgreSQL: Matthew and Stones, 2nd ed. Apress

DDL – Comandos para definição de dados

- CREATE TABLE

Exemplo:

/ Cria tabela departamento */*

```
CREATE TABLE Departamento (  
    DNOME VARCHAR(15) NOT NULL,  
    DNUMERO INT NOT NULL,  
    GERSSN CHAR(9) NOT NULL,  
    GERDATAINICIO DATE,  
    CONSTRAINT dptopk PRIMARY KEY  
        (DNUMERO),  
    UNIQUE(DNOME),  
    CONSTRAINT dptogerfk FOREIGN KEY  
        (GERSSN) REFERENCES EMPREGADO(SSN)  
);
```

Create Table

- Sintaxe completa: consultar manual PostgreSQL
- <https://www.postgresql.org/docs/9.5/static/sql-createtable.html>

```
CREATE [[GLOBAL|LOCAL] {TEMPORARY|TEMP}|UNLOGGED] TABLE [IF NOT EXISTS]
    table_name ( [
        { column_name data_type [COLLATE collation] [column_constraint [ ... ]]
          | table_constraint
          | LIKE source_table [ like_option ... ] }
        [, ... ]
    ] )
[ INHERITS ( parent_table [, ... ] ) ]
[ WITH ( storage_parameter [= value] [, ... ] ) | WITH OIDS|WITHOUT OIDS]
[ ON COMMIT { PRESERVE ROWS | DELETE ROWS | DROP } ]
[ TABLESPACE tablespace_name ]
```

(continua no próximo slide)

Create Table - *column_constraint*

- Especificando a restrição em frente à coluna

where *column_constraint* is:

```
[ CONSTRAINT constraint_name ]  
{ NOT NULL |  
  NULL |  
  CHECK ( expression ) [ NO INHERIT ] |  
  DEFAULT default_expr |  
  UNIQUE index_parameters |  
  PRIMARY KEY index_parameters |  
  REFERENCES reftable [ ( refcolumn ) ]  
    [MATCH FULL|MATCH PARTIAL|MATCH SIMPLE ]  
    [ ON DELETE action ] [ ON UPDATE action ]  
}  
[DEFERRABLE | NOT DEFERRABLE][INITIALLY DEFERRED | INITIALLY IMMEDIATE]
```

(continua no próximo slide)

DDL – Comandos para definição de dados

- Restrição de chave primária (PRIMARY KEY) na coluna
- Restrição de chave estrangeira (FOREIGN KEY) na coluna
 - Observe que a palavra chave REFERENCES é usada
- Restrição de unicidade (UNIQUE) coluna

Exemplo:

/ Cria tabela departamento */*

```
CREATE TABLE Departamento (  
    DNOME VARCHAR(15) NOT NULL UNIQUE,  
    DNUMERO INT NOT NULL PRIMARY KEY ,  
    GERSSN CHAR(9) NOT NULL REFERENCES  
        EMPREGADO(SSN),  
    GERDATAINICIO DATE  
);
```

DDL – Comandos para definição de dados

- Adicionando um nome à restrição

Exemplo:

/ Cria tabela departamento */*

```
CREATE TABLE Departamento (  
    DNOME VARCHAR(15) NOT NULL CONSTRAINT  
        uqdnome UNIQUE,  
    DNUMERO INT NOT NULL CONSTRAINT dptopk  
        PRIMARY KEY ,  
    GERSSN CHAR(9) NOT NULL  
        CONSTRAINT gerssnfk REFERENCES  
        EMPREGADO(SSN),  
    GERDATAINICIO DATE,  
);
```

Create Table - *table_constraint*

- Especificando a restrição na **tabela**
 - Observe a mudança na sintaxe de algumas restrições (de chave primária, chave estrangeira)

and *table_constraint* is:

```
[ CONSTRAINT constraint_name ]  
{ CHECK ( expression ) [ NO INHERIT ] |  
  UNIQUE ( column_name [, ... ] ) index_parameters |  
  PRIMARY KEY ( column_name [, ... ] ) index_parameters |  
  FOREIGN KEY ( column_name [, ... ] )  
    REFERENCES reftable [ ( refcolumn [, ... ] ) ]  
    [MATCH FULL| MATCH PARTIAL | MATCH SIMPLE ]  
    [ ON DELETE action ] [ ON UPDATE action ]  
}  
[DEFERRABLE|NOT DEFERRABLE][INITIALLY DEFERRED|INITIALLY  
IMMEDIATE]
```

(continua no próximo slide)

DDL – Comandos para definição de dados

- Restrição de chave primária (PRIMARY KEY) na tabela
- Restrição de chave estrangeira (FOREIGN KEY) na tabela
- Restrição de unicidade (UNIQUE) na tabela

Exemplo:

/ Cria tabela departamento */*

```
CREATE TABLE Departamento (  
    DNOME VARCHAR(15) NOT NULL,  
    DNUMERO INT NOT NULL,  
    GERSSN CHAR(9) NOT NULL,  
    GERDATAINICIO DATE,  
    PRIMARY KEY (DNUMERO),  
    UNIQUE(DNOME),  
    FOREIGN KEY (GERSSN) REFERENCES  
        EMPREGADO(SSN)  
);
```

DDL – Comandos para definição de dados

- Adicionando um nome à restrição

Exemplo:

/ Cria tabela departamento */*

```
CREATE TABLE Departamento (  
    DNOME VARCHAR(15) NOT NULL,  
    DNUMERO INT NOT NULL,  
    GERSSN CHAR(9) NOT NULL,  
    GERDATAINICIO DATE,  
    CONSTRAINT dptopk PRIMARY KEY (DNUMERO),  
    CONSTRAINT uqdnome UNIQUE(DNOME),  
    CONSTRAINT dptogerfk FOREIGN KEY (GERSSN)  
        REFERENCES EMPREGADO(SSN)  
);
```

Como ler a sintaxe

Convenção	
UPPERCASE (maiúsculo)	Palavra-chave SQL.
lowercase (minúsculo)	Identificadores ou constantes SQL informadas pelo usuário
<i>itálico</i>	Nome de um bloco de sintaxe. Essa convenção é usada para indicar blocos longos de sintaxe que podem ser usados em mais de um local.
(barra vertical)	Separa elementos opcionais da sintaxe dentro de colchetes ou chaves. Somente um dos itens pode ser escolhido.
[] (colchetes)	Item de sintaxe opcional. Os colchetes não fazem parte do comando.
{ } (chaves)	Item da sintaxe obrigatório. As chaves não fazem parte do comando.
[,...]	O item precedente pode ser repetido N vezes. A separação entre os itens é feita por uma vírgula
[...]	O item precedente pode ser repetido N vezes. A separação entre os itens é feita por um espaço em branco.

DDL – Comandos para definição de dados

- **ALTER TABLE** – Altera as definições de campos e de restrições.

ALTER TABLE <nome da tabela>

ADD <definição de Coluna>

ADD <Restrição de integridade> -- Chaves primárias, Estrangeiras

ALTER <definição de Coluna>

ALTER <definição de Coluna> **DEFAULT** <default-value>

ALTER <definição de Coluna> [**NOT**] **NULL**

DROP <definição de Coluna>

DROP CONSTRAINT <nome da restrição> -- Remove uma restrição

RENAME TO <novo nome> -- Renomeia a tabela

RENAME <Atributo> **TO** <novo atributo>

Onde <definição de coluna> pode ser:

<Nome Atributo> <Tipo de Dado> [**NULL**] |

[**DEFAULT** default-value] -- nao vale [**NOT NULL**]

DDL – Comandos para definição de dados

```
ALTER TABLE EMPREGADO ADD COLUMN CorCabelos CHAR(25)  
DEFAULT 'Branco';
```

```
ALTER TABLE EMPREGADO ADD Altura INT DEFAULT NULL;
```

```
ALTER TABLE EMPREGADO DROP Altura;
```

```
ALTER TABLE EMPREGADO ALTER TYPE CorCabelos CHAR(30);
```

```
ALTER TABLE DEPARTAMENTO ADD VICESSN CHAR(9)  
FOREIGN KEY (VICESSN) REFERENCES EMPREGADO (SSN)  
ON UPDATE CASCADE ON DELETE SET NULL
```


- Sintaxe ALTER TABLE
- <http://www.postgresql.org/docs/8.4/static/sql-altertable.html>

```
ALTER TABLE [ ONLY ] name [ * ]  
    action [, ... ]  
ALTER TABLE [ ONLY ] name [ * ]  
    RENAME [ COLUMN ] column TO  
new_column  
ALTER TABLE name  
    RENAME TO new_name  
ALTER TABLE name  
    SET SCHEMA new_schema
```

- Sintaxe ALTER TABLE

where *action* is one of:

```
ADD [ COLUMN ] column type [ column_constraint [ ... ] ]
DROP [ COLUMN ] column [ RESTRICT | CASCADE ]
ALTER [ COLUMN ] column [ SET DATA ] TYPE type [ USING expression ]
ALTER [ COLUMN ] column SET DEFAULT expression
ALTER [ COLUMN ] column DROP DEFAULT
ALTER [ COLUMN ] column { SET | DROP } NOT NULL
ALTER [ COLUMN ] column SET STATISTICS integer
ALTER [ COLUMN ] column SET STORAGE { PLAIN | EXTERNAL | EXTENDED | MAIN }
ADD table_constraint
DROP CONSTRAINT constraint_name [ RESTRICT | CASCADE ]
DISABLE TRIGGER [ trigger_name | ALL | USER ]
ENABLE TRIGGER [ trigger_name | ALL | USER ]
ENABLE REPLICA TRIGGER trigger_name
ENABLE ALWAYS TRIGGER trigger_name
```

- (continuação)

DISABLE RULE rewrite_rule_name
ENABLE RULE rewrite_rule_name
ENABLE REPLICA RULE rewrite_rule_name
ENABLE ALWAYS RULE rewrite_rule_name
CLUSTER ON index_name
SET WITHOUT CLUSTER
SET WITH OIDS
SET WITHOUT OIDS
SET (storage_parameter = value [, ...])
RESET (storage_parameter [, ...])
INHERIT parent_table
NO INHERIT parent_table
OWNER TO new_owner
SET TABLESPACE new_tablespace

DDL – Comandos para definição de dados

DROP TABLE – Exclui uma tabela existente de um banco de dados. Não pode ser excluída a tabela que possui alguma referência. Neste caso, deve-se primeiro excluir a tabela que possui algum campo que a está referenciando e depois excluir a tabela inicial.

DROP TABLE <nome da tabela>

Exemplo:

/ Apaga tabela Departamento */*

DROP TABLE Departamento;

- Sintaxe DROP

DROP TABLE [IF EXISTS] name [, ...] [CASCADE |
RESTRICT]

DDL – Comandos para definição de dados

- **ALTER TABLE** – Altera as definições de campos e de restrições.

ALTER TABLE <nome da tabela>

ADD <definição de Coluna>

ADD <Restrição de integridade> -- Chaves primária, Secund. Estrang.

ALTER <definição de Coluna>

ALTER <definição de Coluna> **DEFAULT** <default-value>

ALTER <definição de Coluna> [**NOT**] **NULL**

DROP <definição de Coluna>

DROP CONSTRAINT <nome da restrição>

RENAME <novo nome>

RENAME <Atributo> **TO** <novo atributo>

Onde <definição de coluna> pode ser:

<Nome Atributo> <Tipo de Dado> [**NULL**] |

[**DEFAULT** default-value]

Removendo/Adicionando uma restrição

ALTER TABLE Empregado **DROP CONSTRAINT**
ChaveEmpregado

ALTER TABLE Empregado **ADD CONSTRAINT**
ChaveEmpregado **PRIMARY KEY** (SSN);

ALTER TABLE

CREATE TABLE tabela (valor **FLOAT**);

INSERT INTO tabela **VALUES** (1.23),(1.50),(1.49),(12.74);

SELECT * FROM tabela;

	valor double precision	
1		1.23
2		1.5
3		1.49
4		12.74

ALTER TABLE tabela **ALTER COLUMN** valor **TYPE INT**;

SELECT * FROM tabela;

	valor integer	
1		1
2		2
3		1
4		13

Se o Sistema consegue converter os tipos, ao alterar o domínio ele altera os valores que estão na tabela

ALTER TABLE

CREATE TABLE tabela (valor **FLOAT**);

INSERT INTO tabela **VALUES** (1.23),(1.50),(1.49),(12.74);

SELECT * FROM tabela;

	valor	
	double precision	🔒
1	1.23	
2	1.5	
3	1.49	
4	12.74	

ALTER TABLE tabela **ALTER COLUMN** valor **VARCHAR**;

SELECT * FROM tabela;

	valor	
	character varying	🔒
1	1.23	
2	1.5	
3	1.49	
4	12.74	

Se o Sistema consegue converter os tipos, ao alterar o domínio ele altera os valores que estão na tabela

ALTER TABLE

CREATE TABLE tabela (valor **FLOAT**);

INSERT INTO tabela **VALUES** (1.23),(1.50),(1.49),(12.74);

SELECT * FROM tabela;

	valor	
	double precision	🔒
1		1.23
2		1.5
3		1.49
4		12.74

ALTER TABLE tabela **ALTER COLUMN** valor **DATE**;

SELECT * FROM tabela;

```
ERROR: column "valor" cannot be cast automatically to type date
HINT: You might need to specify "USING valor::date".
SQL state: 42804
```

Caso não consiga
alterar
automaticamente, é
gerada uma
mensagem de erro

Exercício

- Criar um banco de dados usando os recursos visuais do PostgreSQL. Tabelas:
 - Alunos: NroMatricula, cpf, nome endereço, telefone, ano nascimento
 - Disciplina: CodDisciplina, nome, codigo do curso a qual ela pertence
 - Matricula: NroMatricula, CodDisciplina, Semestre, Ano, Nota, NroFaltas

Escolha os tipos adequados para cada um dos campos e as chaves primárias

Coloque dados nas tabelas

Aula 8 – SBD

Postgres

SQL: DML

Profa. Elaine Faria

UFU

Agradecimentos

- Agradecimentos
 - Ao professor Bruno Travençolo que gentilmente cedeu seus slides

SQL

- Linguagem estruturada de consulta
- Duas classes importantes de comandos
 - DDL: Data definition language
 - Comandos para a definição de dados
 - DML: Data manipulation language
 - Comandos para a manipulação de dados
 - Exemplos: INSERT, DELETE, UPDATE, SELECT

SQL INSERT INTO

- Tabela Exemplo

```
CREATE TABLE produtos (  
    cod_prod integer,  
    nome      text,  
    preco     numeric  
);
```

- Inserindo uma tupla (linha)

```
INSERT INTO produtos VALUES (1, 'Queijo', 9.99);
```

- Dados colocados na ordem de criação das colunas da tabela
- É necessário, neste caso, conhecer a ordem das colunas
 - Fique atento pois esta ordem pode ser alterada!

Fonte: <http://pgdocptbr.sourceforge.net/pg80/dml.html>

SQL INSERT INTO

```
CREATE TABLE produtos  
(  
    cod_prod integer,  
    nome text,  
    preco numeric);
```

- Inserindo uma tupla (linha)

```
INSERT INTO produtos (cod_prod, nome, preco) VALUES (1, 'Queijo', 9.99);
```

```
INSERT INTO produtos (nome, preco, cod_prod) VALUES ('Queijo', 9.99, 1);
```

- Neste caso, indica-se as colunas
 - Essa sintaxe é fortemente recomendada
 - A ordem das colunas de uma tabela pode ser alterada em uma eventual manutenção no BD.
- Pode-se omitir uma coluna

```
INSERT INTO produtos (cod_prod, nome) VALUES (1, 'Queijo');
```

- As colunas não indicadas são preenchidas com o seus respectivos valores DEFAULT

SQL INSERT INTO

```
CREATE TABLE produtos (  
  cod_prod integer,  
  nome text,  
  preco numeric);
```

- Especificando um valor DEFAULT

```
INSERT INTO produtos (cod_prod, nome, preco) VALUES (1, 'Queijo', DEFAULT);
```

- Inserindo toda a linha com valores DEFAULT

```
INSERT INTO produtos DEFAULT VALUES;
```

- Especificando mais de uma linha

```
INSERT INTO produtos (cod_prod, nome, preco) VALUES (1, 'Queijo', DEFAULT),  
                                                       (2, 'Goiabada', 10.00)
```

Inserindo dados no banco

- Comando SQL: INSERT

```
INSERT INTO table [ ( column [, ...] ) ]  
    { DEFAULT VALUES | VALUES ( { expression |  
DEFAULT } [, ...] ) [, ...]  
    | query }  
    [ RETURNING * | output_expression [ [ AS ]  
output_name ] [, ...] ]
```

<http://www.postgresql.org/docs/8.4/static/sql-insert.html>

Modificação de dados

```
CREATE TABLE produtos (  
  cod_prod integer,  
  nome      text,  
  preco     numeric);
```

- A operação de modificação/atualização de dados que já estão no banco de dados é conhecida como UPDATE
- Para atualizar alguma tupla é necessário informar
 - O nome da *tabela* e da *coluna* a ser atualizada
 - O novo valor para a *coluna*
 - Quais tuplas (*linhas*) serão atualizadas
- Exemplo

```
UPDATE produtos SET preco = 10 WHERE preco = 5;
```

- Atualiza a coluna preco da tabela produtos, modificando o preço para 10 das linhas que possuem preço igual a 5

Modificação de dados

```
CREATE TABLE produtos (  
  cod_prod integer,  
  nome text,  
  preco numeric);
```

- Exemplo

```
UPDATE produtos SET preco = preco*1.10
```

- Atualiza a coluna preco da tabela produtos, aumentando o preço em 10% para *todos* os produtos

- Mais de uma coluna pode ser atualizada

```
UPDATE minha_tabela SET a = 5, b = 3, c = 1 WHERE a > 0;
```

SQL UPDATE

- Comando SQL: UPDATE

```
UPDATE [ ONLY ] table [ [ AS ] alias ]  
    SET { column = { expression | DEFAULT } |  
        ( column [, ...] ) = ( { expression | DEFAULT } [, ...] ) } [, ...]  
    [ FROM fromlist ]  
    [ WHERE condition | WHERE CURRENT OF cursor_name ]  
    [ RETURNING * | output_expression [ [ AS ] output_name ] [, ...] ]
```

- **CUIDADO:** Sempre teste a clausura WHERE de um comando UPDATE antes de executá-lo. Um simples erro no WHERE pode resultar na alteração de muitas, senão todas, linhas da tabela, atribuindo a elas o mesmo valor

Remoção de dados

- Para deletar dados do BD usa-se o comando DELETE
- Exemplo

DELETE FROM produtos **WHERE** preco = 10;

- Todas as tuplas cujos produtos custam 10 serão eliminadas

- Removendo todas as tuplas da tabela produtos

DELETE FROM produtos

- **CUIDADO:** é muito fácil apagar os dados das tabelas

SQL DELETE

- Comando SQL: DELETE

```
DELETE FROM [ ONLY ] table [ [ AS ] alias ]  
    [ USING usinglist ]  
    [ WHERE condition | WHERE CURRENT OF  
cursor_name ]  
    [ RETURNING * | output_expression [ [ AS ]  
output_name ] [, ...] ]
```

Observações

Inserindo campos de data

```
DROP TABLE IF EXISTS d;
```

```
CREATE TABLE d (  
  d DATE);
```

```
INSERT INTO d(d) VALUES ('11/12/2010'); -- ok funciona, mas qual o mês? 11  
ou 12?
```

```
-- vamos tentar inserir 30 de dez de 2010
```

```
INSERT INTO d(d) VALUES ('30/12/2010'); -- ok!, no meu computador  
funcionou. Pode ser que no seu não funcione este comando, mas funcione  
o próximo
```

```
-- agora vou tentar inserir a mesma data, só que em outro formato (mês, dia,  
ano)
```

```
INSERT INTO d(d) VALUES ('12/30/2010'); -- ERRO: valor do campo  
date/time está fora do intervalo: "12/30/2010"
```


Observações

Inserindo campos de data

- descobrindo o formato atual configurado no postgresql
- ver arquivo postgresql.conf. O meu é datestyle = 'iso, dmy'
- formato ISO: YYYY-MM-DD (ano,mês,dia)
- formato dmy: DD/MM/YYYYYY

SHOW datestyle;

- mudando a configuração do sistema para aceitar a data no formato anterior

SET datestyle **TO** 'mdy';

Obs: para o povoamento disponível no Teams o formato deve ser 'mdy';

- conferindo a mudança

SHOW datestyle;

- tentando inserir a data que deu erro anteriormente

INSERT INTO d(d) **VALUES** ('12/30/2010'); -- agora está ok!

Observações

Inserindo campos de data

-- É possível inserir dados independentemente do tipo configurado no
-- postgresql. Para isso use o comando `to_date`

-- inserindo o dia 30 de dezembro em dois formatos diferentes

```
INSERT INTO d(d) VALUES (to_date('12/30/2010','MM/DD/YYYY'));
```

```
INSERT INTO d(d) VALUES (to_date('30/12/2010','DD/MM/YYYY'));
```

-- outra opção (a melhor, pois não é ambígua)

```
INSERT INTO d(d) VALUES ('2010-12-30');
```

- Comando **EXTRACT**

- Para obter somente o ano de um campo de data

```
SELECT extract(year from datanasc)
```

```
FROM empregado
```

Lidando com o desconhecido: NULLs

- Quando não sabemos qual o valor preencher em um determinado atributo, podemos utilizar um valor especial chamado **NULL**
 - **NULL** significa que o valor é desconhecido no momento
 - Ex: hora da chegada de um voo que ainda está no ar
 - Ou pode significar algo não relevante para uma determinada tupla
 - Ex: Número do apartamento no cadastro do endereço. Muitas pessoas não moram em prédios.
- **NULL** não quer dizer 0 (zero)
- **NULL** não quer dizer "" (string vazia)

NULL

- Qual o resultado da comparação de dois valores NULL?
 - É indeterminado, ou seja NULL
- **SELECT NULL = NULL**
- Algumas operações com NULL também são indefinidas
 - **SELECT 2 + NULL**
- O SQL provê uma maneira para fazer testes com NULL, por meio do teste IS NULL e IS NOT NULL
 - Ex: buscar no banco quais as pessoas que não possuem nome do meio.
 - **FORMA ERRADA:**
SELECT *
FROM empregado
WHERE minicial = **NULL**
 - (consulta retorna vazio)

NULL

– FORMA CORRETA:

SELECT *

FROM empregado

WHERE minicial **IS NULL**

– Obtendo a lista de pessoas que possuem o nome do meio

– FORMA CORRETA:

SELECT *

FROM empregado

WHERE minicial **IS NOT NULL**

SEQUENCES

- SEQUENCES são usados para gerar automaticamente números sequenciais únicos.
 - Ex: 1,2,3,4,5,....
- Podemos usar essas sequências para atribuir valores automaticamente para atributos de tabelas
 - Ex: chave primária.
- SINTAXE

```
CREATE [ TEMPORARY | TEMP ] SEQUENCE name  
[ INCREMENT [ BY ] increment ]  
[ MINVALUE minvalue | NO MINVALUE ]  
[ MAXVALUE maxvalue | NO MAXVALUE ]  
[ START [ WITH ] start ] [ CACHE cache ]  
[ [ NO ] CYCLE ]
```

SEQUENCES

/ cria uma sequencia iniciando em 1000 e incrementada em 1 */*

-- DROP SEQUENCE Seq

CREATE SEQUENCE Seq

START WITH 1001

INCREMENT BY 1

-- testando a sequencia (rodar várias vezes)

SELECT NEXTVAL('Seq');

-- testando o valor ATUAL DA sequencia

SELECT CURRVAL('Seq'); -- CURRENT VAL

-- usando com INSERT INTO

CREATE TABLE teste(n int);

INSERT INTO teste VALUES (NEXTVAL('Seq'));

INSERT INTO Aluno VALUES('JOSE DA SILVA',
NEXTVAL('Seq'), 21, 'Araguari');

Relembrando a sintaxe do **CREATE TABLE**

- Sintaxe completa: consultar manual PostgreSQL
- <http://www.postgresql.org/docs/8.4/static/sql-createtable.html>

Synopsis

CREATE [[GLOBAL | LOCAL] { TEMPORARY | TEMP }] **TABLE**

table_name (

[{ column_name data_type

[DEFAULT default_expr] -- *aula de hoje*

[column_constraint [...]] - *ver próximo slide*

| table_constraint -- *ver 2 slides a frente*

| LIKE parent_table [{ INCLUDING | EXCLUDING } { DEFAULTS | CONSTRAINTS | INDEXES }] ... } [, ...])

[INHERITS (parent_table [, ...])]

[WITH (storage_parameter [= value] [, ...]) | WITH OIDS | WITHOUT OIDS]

[ON COMMIT { PRESERVE ROWS | DELETE ROWS | DROP }]

[TABLESPACE tablespace]

(continua no próximo slide)

Relembrando a sintaxe do **CREATE TABLE**

- Sintaxe completa: consultar manual PostgreSQL
- <http://www.postgresql.org/docs/8.4/static/sql-createtable.html>

where *column_constraint* is:

```
[ CONSTRAINT constraint_name ]  
{ NOT NULL |  
  NULL |  
  UNIQUE index_parameters |  
  PRIMARY KEY index_parameters |  
  CHECK ( expression ) | -- aula de hoje  
  REFERENCES reftable [ ( refcolumn ) ] [ MATCH FULL | MATCH  
PARTIAL | MATCH SIMPLE ] [ ON DELETE action ] [ ON UPDATE  
action ] } -- aula de hoje  
  
[ DEFERRABLE | NOT DEFERRABLE ] [ INITIALLY DEFERRED | INITIALLY IMMEDIATE ]  
(continua no próximo slide)
```

Relembrando a sintaxe do **CREATE TABLE**

- Sintaxe completa: consultar manual PostgreSQL
- <http://www.postgresql.org/docs/8.4/static/sql-createtable.html>

and *table_constraint* is:

```
[ CONSTRAINT constraint_name ]  
{ UNIQUE ( column_name [, ... ] ) index_parameters |  
  PRIMARY KEY ( column_name [, ... ] ) index_parameters |  
  CHECK ( expression ) |  
  FOREIGN KEY ( column_name [, ... ] ) REFERENCES reftable [ ( refcolumn [, ... ] ) ]  
    [ MATCH FULL | MATCH PARTIAL | MATCH SIMPLE ]  
    [ ON DELETE action ] [ ON UPDATE action ] }  
[ DEFERRABLE | NOT DEFERRABLE ] [ INITIALLY DEFERRED | INITIALLY IMMEDIATE ]
```

index_parameters in UNIQUE and PRIMARY KEY constraints are:

```
[ WITH ( storage_parameter [= value] [, ... ] ) ]  
[ USING INDEX TABLESPACE tablespace ]
```

(continua no próximo slide)

Valores DEFAULT

- Associa um valor padrão (*default*) a uma coluna
- Quando uma nova tupla é inserida e nenhum valor é passado para algumas das colunas, estas serão preenchidas com o seu respectivo valor *default*
- Se nenhum valor *default* é fornecido, o sistema usa o NULL
- O valor default pode ser indicado na declaração da tabela:

```
CREATE TABLE Produtos (  
    id integer,  
    nome text,  
    preco numeric DEFAULT 9.99  
);
```

Valores DEFAULT

- Expressões podem ser avaliadas

```
CREATE TABLE Produtos (  
    id integer DEFAULT  
        nextval('products_product_no_seq'),  
    nome text,  
    preco numerico DEFAULT 9.99  
);
```

Valores DEFAULT

```
CREATE TABLE log (  
  t TIMESTAMP DEFAULT now(),  
  b text  
)
```

	t timestamp without time zone	b text
1	2010-06-09 22:36:16.032	texto
2	2010-06-09 22:37:55.755	texto2
3	2010-06-09 22:37:55.755	texto3
4	2010-06-09 22:38:35.243	texto4
5	2010-06-09 22:38:38.356	texto5
6	2010-06-09 22:38:39.82	texto6

```
INSERT INTO log (b) values ('texto');  
INSERT INTO log (b) values ('texto2');  
INSERT INTO log (b) values ('texto3');  
INSERT INTO log (b) values ('texto4');  
INSERT INTO log (b) values ('texto5');  
INSERT INTO log (b) values ('texto6');
```

Valores DEFAULT

- Tipo **serial**

CREATE TABLE *tablename* (*colname* SERIAL);

É equivalente a

CREATE SEQUENCE *tablename_colname_seq*;

CREATE TABLE *tablename* (
 colname integer **DEFAULT**
 nextval('tablename_colname_seq') **NOT NULL**);

Valores Default

- Tipo **serial**

```
CREATE TABLE produtos (  
    id integer DEFAULT nextval('products_product_no_seq'),  
    nome text,  
    preco numerico DEFAULT 9.99  
);
```



```
CREATE TABLE produtos (  
    id SERIAL,  
    nome text,  
    preco numeric DEFAULT 9.99  
);
```

Valores Default

- Como chamar o 'nextval' de um tipo serial?

```
CREATE TABLE produtos (  
    id SERIAL,  
    nome text,  
    preco numeric DEFAULT 9.99  
);
```

- COMANDO INSERT INTO sem a coluna referente ao tipo SERIAL ou atribuindo o valor DEFAULT a essa coluna.
 - **INSERT INTO PRODUTOS VALUES (DEFAULT,'milho',DEFAULT)**
 - **INSERT INTO PRODUTOS VALUES (DEFAULT,'queijo')**
 - **INSERT INTO PRODUTOS(nome) VALUES ('arroz')**
 - **INSERT INTO PRODUTOS(nome,preco) VALUES ('goiabada',15.00)**
- Obs: SERIAL possui 4 bytes; BIGSERIAL 8 bytes.
- Valor inicial: 1

Restrição CHECK

- É uma restrição que verifica se os valores em uma determinada coluna satisfazem uma expressão booleana

```
CREATE TABLE produtos (  
    product_no integer,  
    nome text,  
    valor numeric CHECK (valor > 0)  
);
```

Restrição CHECK

- Pode-se nomear a restrição

```
CREATE TABLE produtos (  
    product_no integer,  
    nome text,  
    valor numeric,  
    CONSTRAINT valor_positivo CHECK (valor > 0)  
);
```

Restrição CHECK

- Pode-se trabalhar com mais de uma coluna

```
CREATE TABLE produtos (  
  product_no integer,  
  nome text,  
  valor numeric CHECK (valor > 0) ,  
  valor_com_desconto numeric ,  
                                CHECK (valor_com_desconto > 0) ,  
                                CHECK (valor > valor_com_desconto )  
);
```

Constraints

- Violações de chave estrangeira
 - Os SGBDs verificam automaticamente as restrições de chave estrangeira para evitar inconsistências na base
 - Exemplo: tente eliminar um funcionário

-- deletando um empregado

DELETE FROM empregado

WHERE SSN= '122';

É possível?

Create Table

- Sintaxe completa: consultar manual PostgreSQL
- <http://www.postgresql.org/docs/8.4/static/sql-createtable.html>

Synopsis

```
CREATE [ [ GLOBAL | LOCAL ] { TEMPORARY | TEMP } ] TABLE table_name (
    [ { column_name data_type [ DEFAULT default_expr ]
      [ column_constraint [ ... ] ] -- ver próximo slide
      | table_constraint -- ver 2 slides a frente
      | LIKE parent_table [ { INCLUDING | EXCLUDING }
                           { DEFAULTS | CONSTRAINTS | INDEXES } ] ... }
    [, ... ] )
```

```
[ INHERITS ( parent_table [, ... ] ) ]
[ WITH ( storage_parameter [= value] [, ... ] ) | WITH OIDS | WITHOUT OIDS ]
[ ON COMMIT { PRESERVE ROWS | DELETE ROWS | DROP } ]
[ TABLESPACE tablespace ]
```

(continua no próximo slide)

Create Table - *column_constraint*

- Sintaxe completa: consultar manual PostgreSQL
- <http://www.postgresql.org/docs/8.4/static/sql-createtable.html>

where *column_constraint* is:

```
[ CONSTRAINT constraint_name ]  
{ NOT NULL |  
  NULL |  
  UNIQUE index_parameters |  
  PRIMARY KEY index_parameters |  
  CHECK ( expression ) |  
  REFERENCES reftable [ ( refcolumn ) ] [ MATCH FULL | MATCH  
PARTIAL | MATCH SIMPLE ]  
  [ ON DELETE action ] [ ON UPDATE action ] }  
[ DEFERRABLE | NOT DEFERRABLE ] [ INITIALLY DEFERRED |  
INITIALLY IMMEDIATE ]
```

(continua no próximo slide)

Create Table - *table_constraint*

- Sintaxe completa: consultar manual PostgreSQL
- <http://www.postgresql.org/docs/8.4/static/sql-createtable.html>

and *table_constraint* is:

```
[ CONSTRAINT constraint_name ]  
{ UNIQUE ( column_name [, ... ] ) index_parameters |  
  PRIMARY KEY ( column_name [, ... ] ) index_parameters |  
  CHECK ( expression ) |  
  FOREIGN KEY ( column_name [, ... ] ) REFERENCES reftable [ ( refcolumn [, ... ] ) ]  
    [ MATCH FULL | MATCH PARTIAL | MATCH SIMPLE ] [ ON DELETE action ] [ ON  
UPDATE action ] }  
[ DEFERRABLE | NOT DEFERRABLE ] [ INITIALLY DEFERRED | INITIALLY IMMEDIATE ]
```

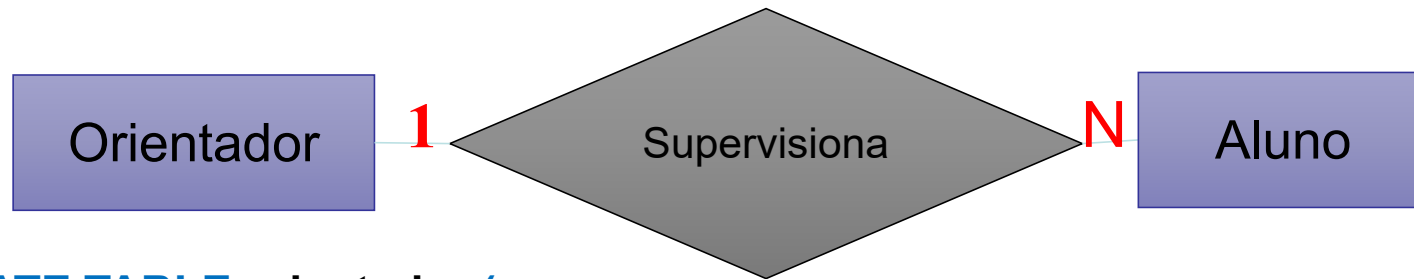
index_parameters in UNIQUE and PRIMARY KEY constraints are:

```
[ WITH ( storage_parameter [= value] [, ... ] ) ]  
[ USING INDEX TABLESPACE tablespace ]
```

(continua no próximo slide)

Chaves estrangeiras

- Ações: ON DELETE e ON UPDATE
 - **NOACTION**: A ação não é executada caso haja referência ao registro afetado
 - **CASCADE**: Pode-se propagar a alteração feita em uma tabela para as outras tabelas que a referenciam.
 - ON DELETE CASCADE
 - ON UPDATE CASCADE
 - **SET NULL/DEFAULT**: Pode-se atribuir NULL | DEFAULT a uma coluna que referencia um registro que será apagado ou alterado
 - ON DELETE SET NULL ; ON DELETE SET DEFAULT
 - ON UPDATE SET NULL ; ON UPDATE SET DEFAULT



```
CREATE TABLE orientador (  
  id INT PRIMARY KEY,  
  nome VARCHAR(255)  
);
```

```
CREATE TABLE aluno (  
  matricula INT PRIMARY KEY,  
  nome VARCHAR(255),  
  orientador_id INT REFERENCES orientador(id)  
);
```

/ Povoando as tabelas*/*

```
INSERT INTO orientador VALUES (1,'Prof. José'), (2,'Profa. Maria');  
INSERT INTO aluno VALUES (1,'Carlos',NULL), (2,'Roberto',2),  
(3,'Ailton',NULL)
```

```
CREATE TABLE aluno (  
  matricula INT PRIMARY KEY,  
  nome VARCHAR(255),  
  orientador_id INT REFERENCES  
orientador(id)  
);
```



```
CREATE TABLE aluno  
(  
  matricula integer NOT NULL,  
  nome character varying(255),  
  orientador_id integer,  
  CONSTRAINT aluno_pkey PRIMARY KEY (matricula),  
  CONSTRAINT aluno_orientador_id_fkey FOREIGN  
KEY (orientador_id)  
  REFERENCES orientador (id) MATCH SIMPLE  
  ON UPDATE NO ACTION ON DELETE NO ACTION  
)
```

	matricula integer	nome character varying(255)	orientador_id integer
1	1	Carlos	
2	2	Roberto	2
3	3	Ailton	

	id integer	nome character varying(255)
1	1	Prof. José
2	2	Profa. Maria

```
DELETE
FROM aluno
WHERE nome = 'Carlos';
```

	matricula integer	nome character varying(255)	orientador_id integer
1	2	Roberto	2
2	3	Ailton	

	id integer	nome character varying(255)
1	1	Prof. José
2	2	Profa. Maria

	matricula integer	nome character varying(255)	orientador_id integer
1	1	Carlos	
2	2	Roberto	2
3	3	Ailton	

	id integer	nome character varying(255)
1	1	Prof. José
2	2	Profa. Maria

```
DELETE
FROM aluno
WHERE nome = 'Roberto';
```

	matricula integer	nome character varying(255)	orientador_id integer
1	1	Carlos	
2	3	Ailton	

	id integer	nome character varying(255)
1	1	Prof. José
2	2	Profa. Maria

	matricula integer	nome character varying(255)	orientador_id integer
1	1	Carlos	
2	2	Roberto	2
3	3	Ailton	

	id integer	nome character varying(255)
1	1	Prof. José
2	2	Profa. Maria

```
DELETE
FROM orientador
WHERE nome = 'Prof. José';
```

	matricula integer	nome character varying(255)	orientador_id integer
1	1	Carlos	
2	2	Roberto	2
3	3	Ailton	

	id integer	nome character varying(255)
1	2	Profa. Maria

	matricula integer	nome character varying(255)	orientador_id integer
1	1	Carlos	
2	2	Roberto	2
3	3	Ailton	

	id integer	nome character varying(255)
1	1	Prof. José
2	2	Profa. Maria

```
DELETE
FROM orientador
WHERE nome = 'Profa. Maria';
```

ERRO: atualização ou exclusão em tabela "orientador" viola restrição de chave estrangeira "aluno_orientador_id_fkey" em "aluno"
DETAIL: Chave (id)=(2) ainda é referenciada pela tabela "aluno".

1	1	Carlos	
2	2	Roberto	2
3	3	Ailton	

1	1	Prof. Jose
2	2	Profa. Maria

```

CREATE TABLE aluno
(
  matricula integer NOT NULL,
  nome character varying(255),
  orientador_id integer,
  CONSTRAINT aluno_pkey PRIMARY KEY (matricula),
  CONSTRAINT aluno_orientador_id_fkey FOREIGN KEY (orientador_id)
    REFERENCES orientador (id) MATCH SIMPLE
    ON UPDATE NO ACTION ON DELETE NO ACTION
)

DELETE
FROM orientador
WHERE nome = 'Profa. Maria';

```

ERRO: atualização ou exclusão em tabela "orientador" viola restrição de chave estrangeira "aluno_orientador_id_fkey" em "aluno"
 DETAIL: Chave (id)=(2) ainda é referenciada pela tabela "aluno".

1	1	Carlos	
2	2	Roberto	2
3	3	Ailton	

1	1	Prof. José
2	2	Profa. Maria

ALTER TABLE aluno

DROP CONSTRAINT aluno_orientador_id_fkey;

ALTER TABLE aluno

ADD CONSTRAINT fk_orientador **FOREIGN KEY** (orientador_id)
REFERENCES orientador(id)

ON UPDATE NO ACTION

ON DELETE CASCADE

	matricula integer	nome character varying(255)	orientador_id integer
1	1	Carlos	
2	2	Roberto	2
3	3	Ailton	

	id integer	nome character varying(255)
1	1	Prof. José
2	2	Profa. Maria

```
DELETE
FROM orientador
WHERE nome = 'Profa. Maria';
```

	matricula integer	nome character varying(255)	orientador_id integer
1	1	Carlos	
2	3	Ailton	

	id integer	nome character varying(255)
1	1	Prof. José

```
ALTER TABLE aluno  
  DROP CONSTRAINT fk_orientador;
```

```
ALTER TABLE aluno  
  ADD CONSTRAINT fk_orientador FOREIGN KEY (orientador_id)  
  REFERENCES orientador(id)  
    ON UPDATE NO ACTION  
    ON DELETE SET NULL;
```

	matricula integer	nome character varying(255)	orientador_id integer
1	1	Carlos	
2	2	Roberto	2
3	3	Ailton	

	id integer	nome character varying(255)
1	1	Prof. José
2	2	Profa. Maria

```
DELETE
FROM orientador
WHERE nome = 'Profa. Maria';
```

	matricula integer	nome character varying(255)	orientador_id integer
1	1	Carlos	
2	3	Ailton	
3	2	Roberto	

	id integer	nome character varying(255)
1	1	Prof. José

Chaves estrangeiras

- Ações: ON DELETE e ON UPDATE
 - **NO ACTION**: A ação não é executada caso haja referência ao registro afetado
 - **CASCADE**: Pode-se propagar a alteração feita em uma tabela para as outras tabelas que a referenciam.
 - ON DELETE CASCADE
 - ON UPDATE CASCADE
 - **SET NULL/DEFAULT**: Pode-se atribuir NULL | DEFAULT a uma coluna que referencia uma registro que será apagado ou alterado
 - ON DELETE SET NULL ; ON DELETE SET DEFAULT
 - ON UPDATE SET NULL ; ON UPDATE SET DEFAULT

Referências

- <http://www.postgresql.org/docs/8.1/interactive/ddl-default.html>

Aula 9 – SBD

Álgebra Relacional

Profa. Elaine Faria
UFU

Agradecimentos

- Agradecimentos
 - Ao professor Bruno Travençolo que gentilmente cedeu seus slides

Introdução

- Relembrando:

- Um modelo de dados inclui um conjunto de **OPERAÇÕES** para manipular um banco de dados além dos CONCEITOS de modelagem necessários para a estruturação do BD.
- **OPERAÇÕES**: Linguagem de Consulta

- O que é **LINGUAGEM DE CONSULTA**?

- É uma linguagem por meio da qual os usuários obtém informações do banco de dados
- Linguagens de mais **ALTO NÍVEL** que as linguagens de programação tradicionais
 - Exemplo - SQL - Structured Query Language

- O que é **LINGUAGEM FORMAL DE CONSULTA** ?

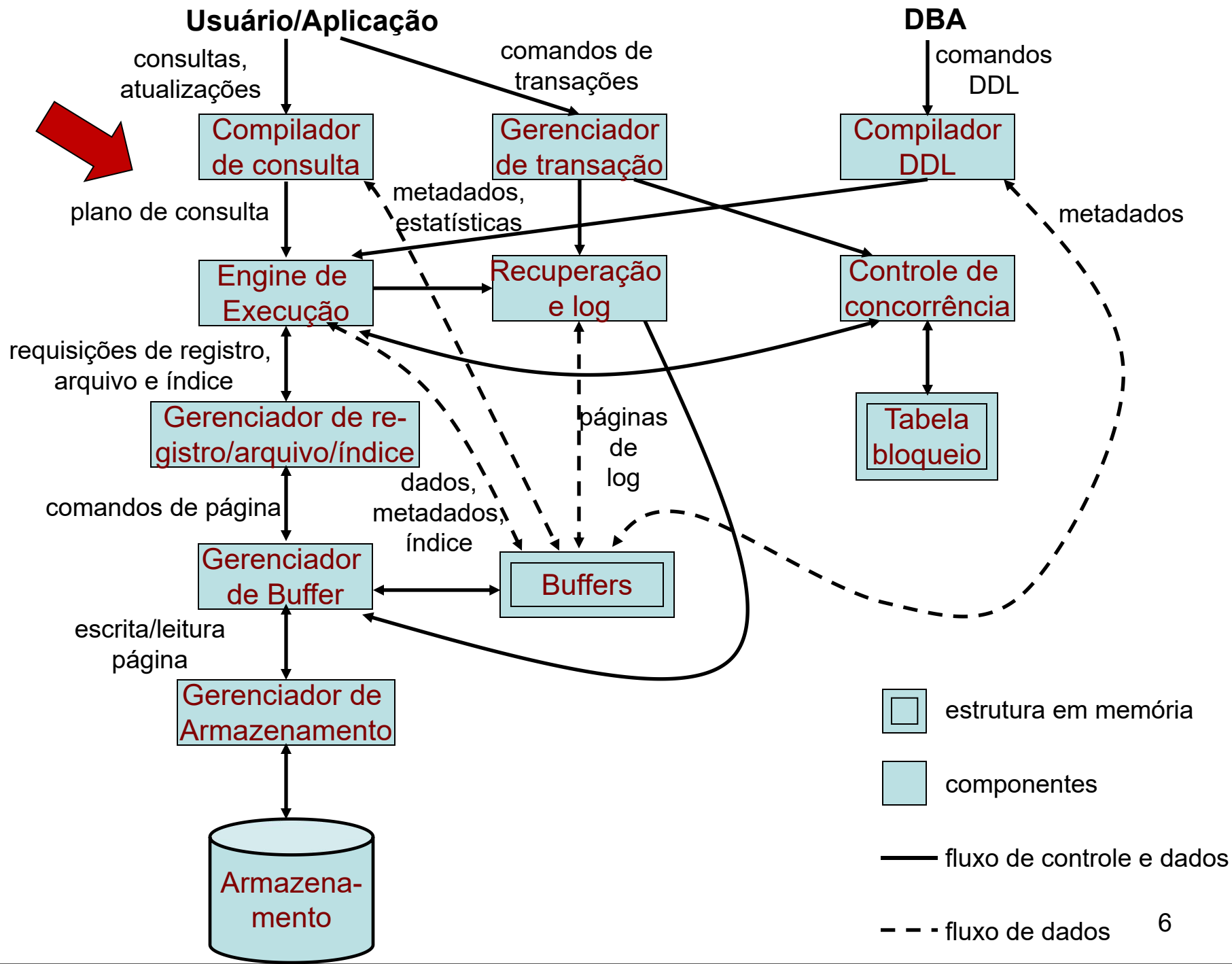
Introdução

O Modelo Relacional possui duas linguagens formais:

- **Álgebra Relacional (procedural)**
 - Na linguagem **PROCEDURAL** o usuário deve ensinar ao sistema uma sequência de operações no BD para obter o resultado desejado
- **Cálculo Relacional (não-procedural)**
 - Na linguagem **NÃO-PROCEDURAL**, o usuário descreve a informação desejada sem fornecer um procedimento específico para a obtenção dessas informações

Álgebra Relacional

- Modelo de dados inclui
 - Conceitos para a definição das restrições e estrutura do BD
 - **Conjunto de operações** para manipular o BD
- Álgebra relacional
 - Maneira teórica de se manipular o BD relacional
- Importância
 - Fundamento formal para as operações no modelo relacional
 - Base para implementar e otimizar consultas em SGBDR
 - Introduz conceitos incorporados na SQL



Álgebra Relacional

- Linguagem de consulta procedural
 - usuários especificam os dados necessários e como obtê-los
- Consiste de um conjunto de operações
 - entrada: uma ou mais relações e restrições
 - saída: uma nova relação resultado

Operações

- Fundamentais

- seleção
- projeção
- produto cartesiano
- renomear
- união
- diferença de conjuntos

- Adicionais

- intersecção de conjuntos
- junção natural
- divisão
- atribuição

- podem ser geradas a partir das operações fundamentais
- facilitam a construção de consultas

Classificação das Operações

- Unárias

- seleção
- projeção
- renomear

operam sobre uma
única relação

- Binárias

- produto cartesiano
- união
- diferença de conjuntos
- intersecção de conjuntos
- junção natural

operam sobre duas
relações

Esquema relacional

cliente (nro_cli, nome_cli, end_cli,
saldo, vendedor)

vendedor (cod_vend, nome_vend)

pedido (nro_ped, data, nro_cliente)

pedido_peça (nro_ped, nro_peça)

peça (nro_peça, descrição_peça)

Seleção σ sigma

- Seleciona tuplas da relação argumento que satisfaçam à condição de seleção

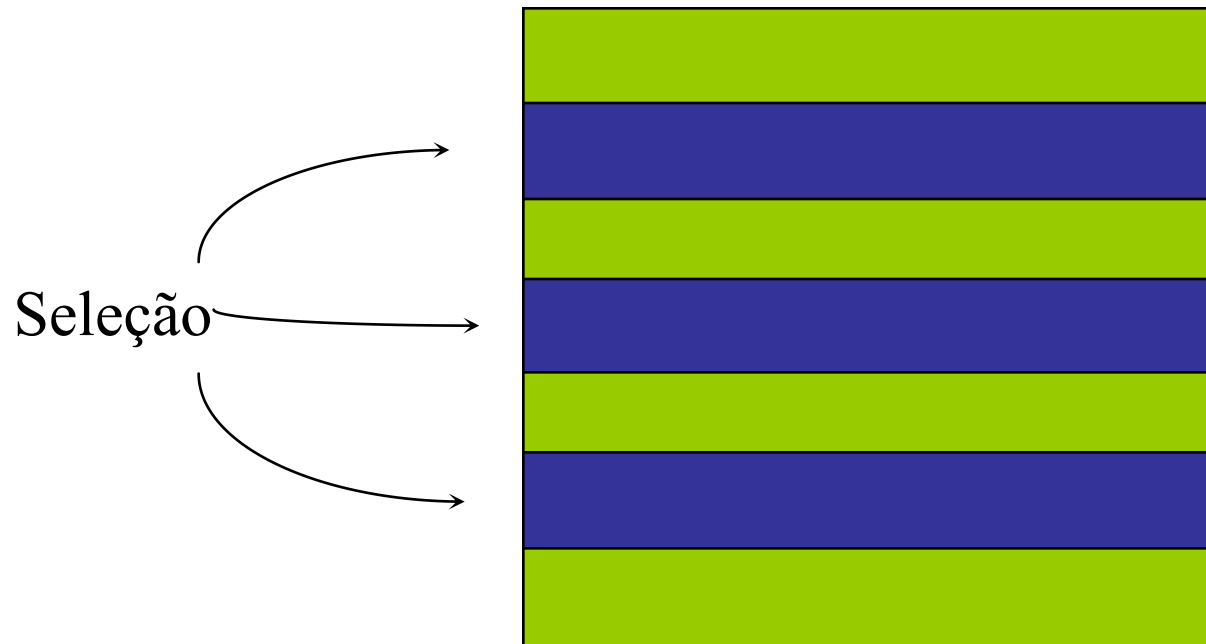
$\sigma_{\text{condição_seleção}}$ (relação argumento)

- pode envolver operadores de comparação
(=, <, ≤, >, ≥, ≠)
- pode combinar condições usando-se $\wedge$, $\vee$, $\neg$

- relação
- resultado de alguma operação da álgebra relacional

Seleção σ sigma

- Produz um subconjunto **horizontal** de uma relação



Relação Cliente

cliente (nro_cli, nome_cli, end_cli, saldo, cod_vend)

nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
2	Cristina	Avenida 1	10,00	1
3	Manoel	Avenida 3	234,00	1
4	Rodrigo	Rua X	137,00	2

Consulta 1

- Liste toda a informação da relação cliente referente ao cliente de número 4.

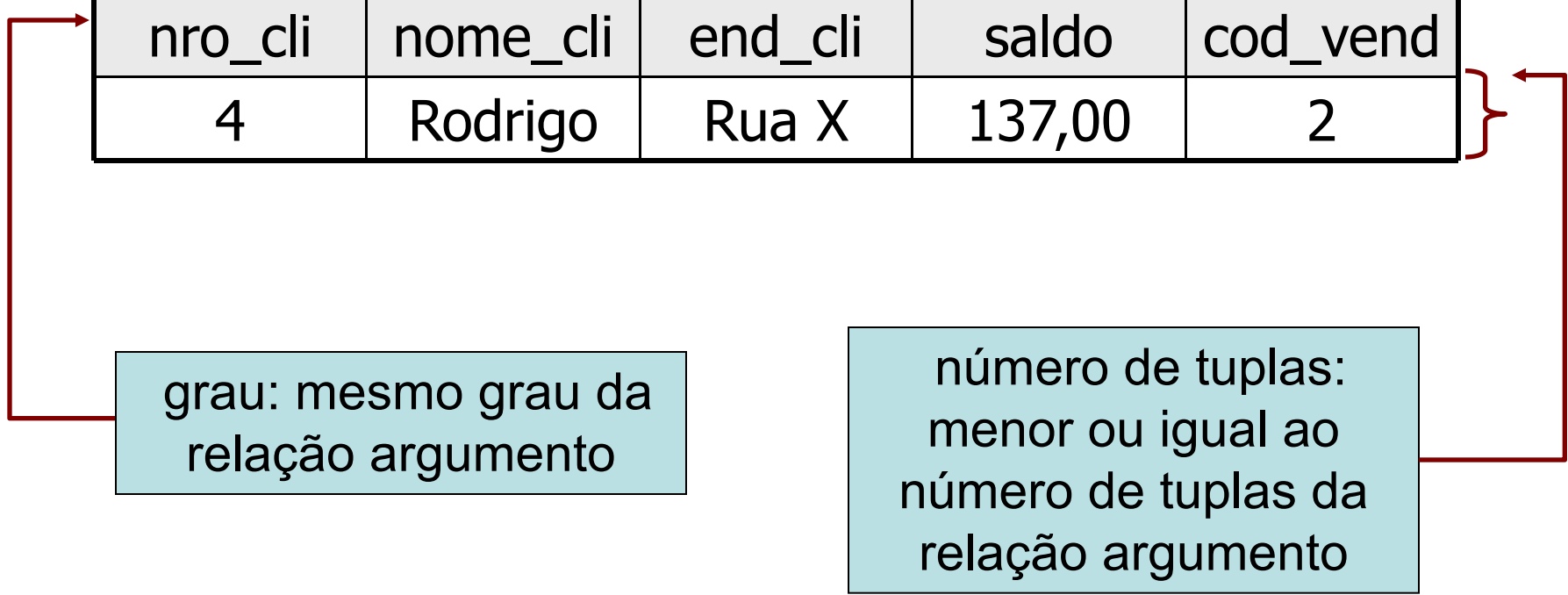
$\sigma_{\text{nro_cli} = 4} (\text{cliente})$



nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
2	Cristina	Avenida 1	10,00	1
3	Manoel	Avenida 3	234,00	1
4	Rodrigo	Rua X	137,00	2

Consulta 1

◆ Relação resultado



nro_cli	nome_cli	end_cli	saldo	cod_vend
4	Rodrigo	Rua X	137,00	2

grau: mesmo grau da
relação argumento

número de tuplas:
menor ou igual ao
número de tuplas da
relação argumento

Consulta 2

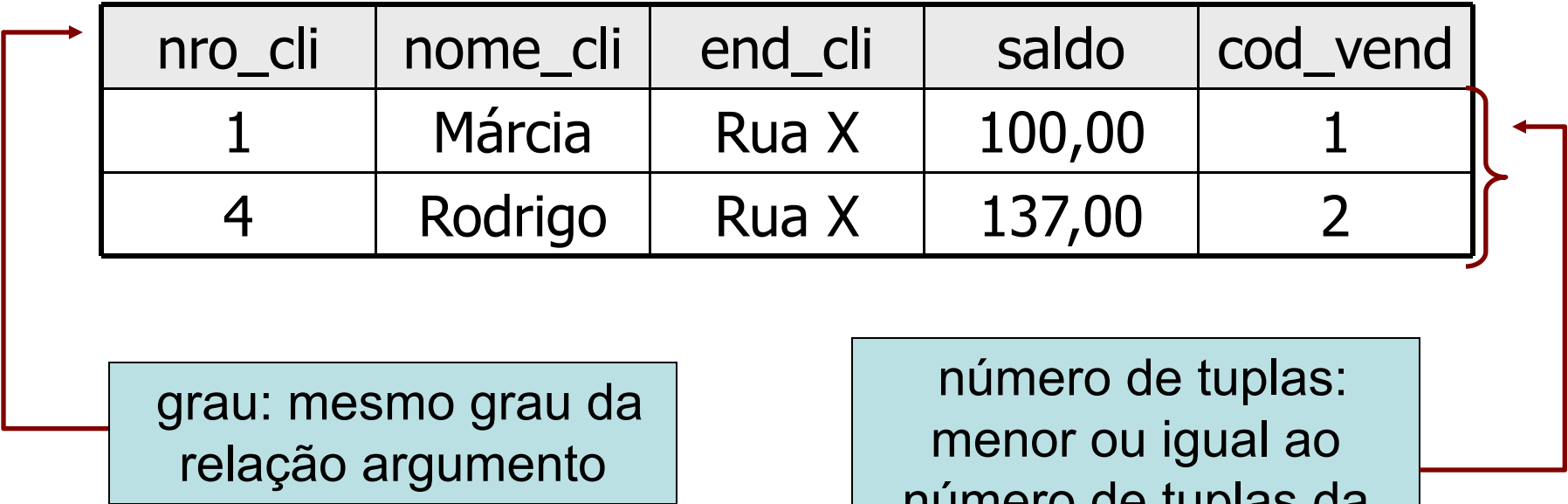
- Liste toda a informação da relação cliente para clientes que possuam saldo inferior a R\$ 200,00 e que morem na Rua X.

$\sigma_{\text{saldo} < 200,00 \wedge \text{end_cli} = \text{"Rua X"}}(\text{cliente})$

nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
2	Cristina	Avenida 1	10,00	1
3	Manoel	Avenida 3	234,00	1
4	Rodrigo	Rua X	137,00	2

Consulta 2

◆ Relação resultado



nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
4	Rodrigo	Rua X	137,00	2

grau: mesmo grau da
relação argumento

número de tuplas:
menor ou igual ao
número de tuplas da
relação argumento

Vale lembrar...

- As condições booleanas $\wedge$ (and), $\vee$ (or) e $\neg$ (not) têm sua interpretação conforme segue:
 - $(\text{cond1} \wedge \text{cond 2})$
 - é **verdadeira** se ambas cond1 e cond2 forem verdadeiras
 - caso contrário é **falsa**
 - $(\text{cond1} \vee \text{cond 2})$
 - **verdadeira** se cond1 ou cond2 forem verdadeiras
 - caso contrário é **falsa**
 - $(\neg \text{cond})$
 - **verdadeira** se cond for falsa
 - caso contrário é **falsa**

Projeção

- Produz uma nova relação contendo um subconjunto vertical da relação argumento, sem duplicações

π lista_atributos (relação argumento)

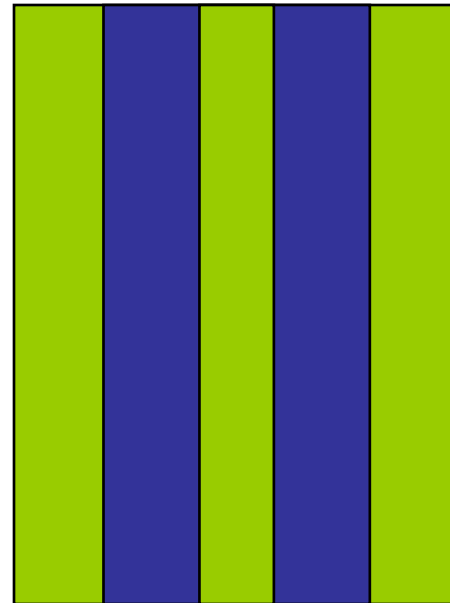
- lista de atributos
- os atributos são separados por vírgula

- relação
- resultado de alguma operação da álgebra relacional

Projeção

- Extrai atributos (Colunas) específicos de uma relação específica
- Produz um subconjunto **vertical** de uma relação

Projeção

 π pi

Consulta 3

- Liste o número e o nome de todos os clientes

$\pi_{\text{nro_cli, nome_cli}}(\text{cliente})$



nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
2	Cristina	Avenida 1	10,00	1
3	Manoel	Avenida 3	234,00	1
4	Rodrigo	Rua X	137,00	2

Consulta 3

◆ Relação resultado

grau: número
de atributos
listados em
lista_atributos



nro_cli	nome_cli
1	Márcia
2	Cristina
3	Manoel
4	Rodrigo



número de
tuplas: menor ou
igual ao número
de tuplas da
relação
argumento

Consulta 4

- Liste o endereço de todos os clientes

$\pi_{\text{end_cli}}(\text{cliente})$



end_cli
Rua X
Avenida 1
Avenida 3

Relação Resultante
- sem repetição



nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
2	Cristina	Avenida 1	10,00	1
3	Manoel	Avenida 3	234,00	1
4	Rodrigo	Rua X	137,00	2

Operações

Propriedades dos operadores unários

- A operação de Seleção é comutativa
 - $\sigma_{\langle \text{condição-A} \rangle}(\sigma_{\langle \text{condição-B} \rangle})$
- Uma sequência de seleções pode ser executada em qualquer ordem, ou pode ser transformada em uma única seleção com uma condição conjuntiva (termos cujo valor é VERDADEIRO ou FALSO, ligados pelo operador $\wedge$ (AND))
 - $\sigma_{\langle \text{condição-1} \rangle}(\sigma_{\langle \text{condição-2} \rangle}(\dots(\sigma_{\langle \text{condição-n} \rangle}(R))))$
 - $\sigma_{\langle \text{condição-1} \rangle} \wedge \langle \text{condição-2} \rangle \wedge \dots \langle \text{condição-n} \rangle (R)$

Operações

Propriedades dos operadores unários

- A operação de Projeção não é comutativa
- Se $\langle \text{lista_atribos_B} \rangle$ contém $\langle \text{lista_atribos_A} \rangle$, então ambas as expressões seguintes são corretas, e vale a igualdade:

$$- \pi_{\langle \text{lista_atribos_A} \rangle} (\pi_{\langle \text{lista_atribos_B} \rangle} R) = \pi_{\langle \text{lista_atribos_A} \rangle} R$$

Consulta 5

- Liste o número e o nome de todos os clientes que possuam saldo devedor inferior a 200,00 reais e morem na Rua X.

nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
2	Cristina	Avenida 1	10,00	1
3	Manoel	Avenida 3	234,00	1
4	Rodrigo	Rua X	137,00	2

Consulta 5

- Passos
 - realizar uma operação de seleção para criar uma nova relação que contém somente aqueles clientes com o saldo e o endereço apropriados;
 - realizar uma projeção sobre a relação resultante do passo anterior, restringindo o resultado desejado às colunas indicadas.

Consulta 5

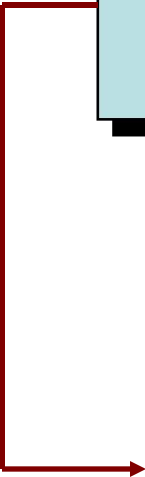
- Primeiro passo

$\sigma_{\text{saldo_dev} < 200,00 \wedge \text{end_cli} = \text{"Rua X"}}(\text{cliente})$

Consulta 5

- Segundo passo

$\pi_{\text{nro_cli, nome_cli}}$ (primeiro passo)



nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
4	Rodrigo	Rua X	137,00	2

Consulta 5

- Liste o número e o nome de todos os clientes que possuam saldo inferior a R\$ 200,00 e que morem na Rua X.

$\pi_{\text{nro_cli}, \text{nome_cli}} (\sigma_{\text{saldo_dev} < 200,00 \wedge \text{end_cli} = \text{"Rua X"}} (\text{cliente}))$

– Relação Resultado

nro_cli	nome_cli
1	Márcia
4	Rodrigo

Atribuição

- Funcionalidades
 - associa uma relação argumento a uma relação temporária
 - permite o uso da relação temporária em expressões subsequentes

relação temporária ← relação argumento

Consulta 5

- Liste o número e o nome de todos os clientes que possuam saldo inferior a R\$ 200,00 e que morem na Rua X.

$\pi_{\text{nro_cli}, \text{nome_cli}} (\sigma_{\text{saldo_dev} < 200,00 \wedge \text{end_cli} = \text{"Rua X"}} (\text{cliente}))$

- Usando atribuição
 - $\text{temp} \leftarrow \sigma_{\text{saldo_dev} < 200,00 \wedge \text{end_cli} = \text{"Rua X"}} (\text{cliente})$
 - $\pi_{\text{nro_cli}, \text{nome_cli}} (\text{temp})$

Atribuição

- Características adicionais
 - permite renomear os atributos de relações intermediárias e final
 - $R(\text{código}, \text{nome}) \leftarrow \pi_{\text{nro_cli}, \text{nome_cli}}(\text{temp})$
- Observações
 - não adiciona semântica adicional à álgebra relacional
 - geralmente utilizada para expressar consultas complexas

Renomear ($\rho = \hat{r}$)

- Renomeia
 - nome da relação
 - nomes dos atributos da relação
 - nome da relação e nomes dos atributos

$\rho_S(B_1, B_2, \dots, B_n)$ (relação)

nome *novo* da relação

nome *antigo* da relação

nomes *novos* dos atributos

Renomear

- Exemplos

- $\rho_{\text{comprador}}$ (cliente)

- $\rho_{(\text{código, nome, rua, saldo, vendedor})}$ (cliente)

- $\rho_{\text{comprador}(\text{código, nome, rua, saldo, vendedor})}$ (cliente)

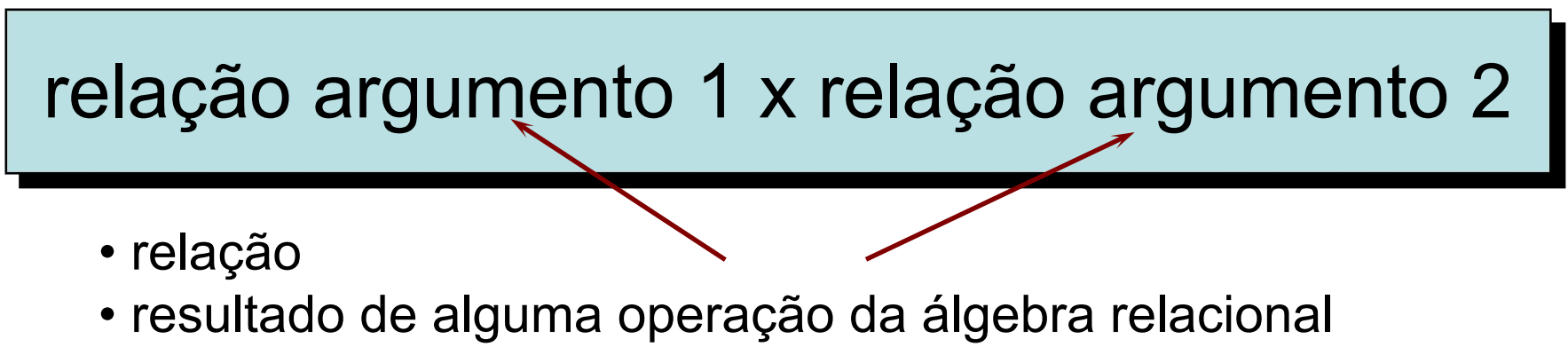
- Observação

- indicada para ser utilizada quando uma relação é usada mais do que uma vez para responder à consulta

Produto Cartesiano

- Combina tuplas de duas relações (quaisquer)
- Tuplas da relação resultante
 - todas as combinações de tuplas possíveis entre as relações participantes

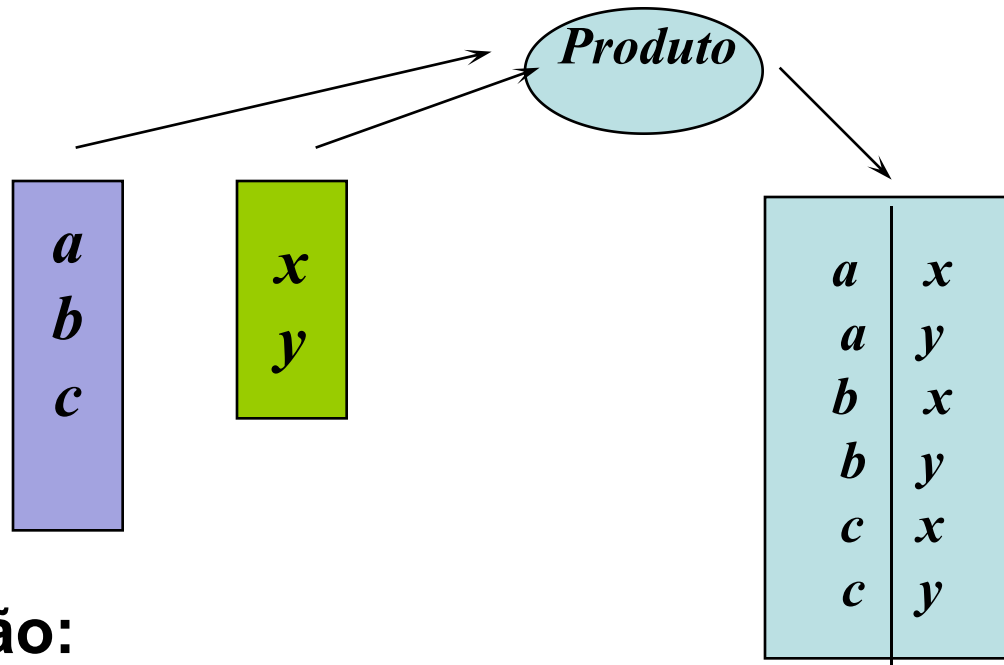
relação argumento 1 x relação argumento 2



- relação
- resultado de alguma operação da álgebra relacional

Produto Cartesiano

- Utilizado quando se necessita obter dados presentes em duas ou mais relações



Notação:

<relação 1> **X** <relação 2>

Relações Cliente e Vendedor

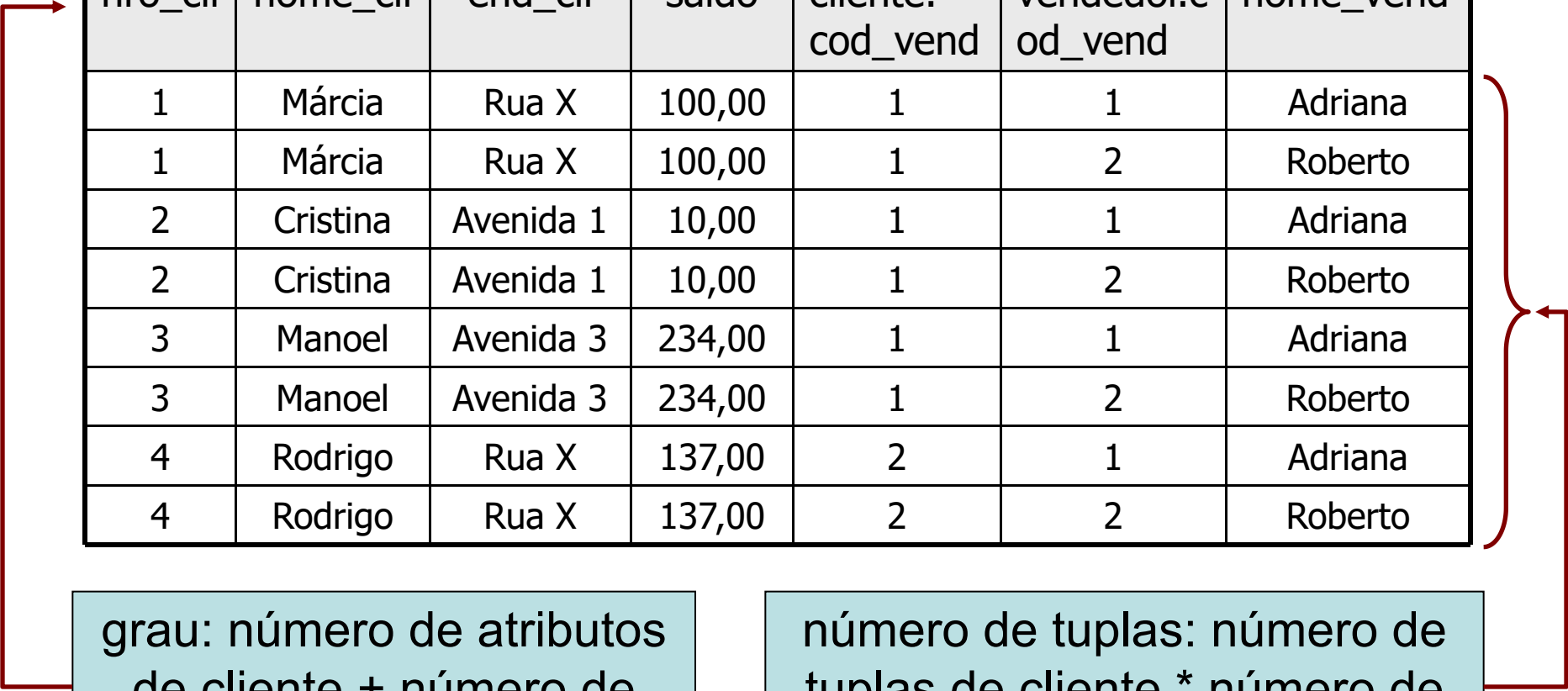
cliente (nro_cli, nome_cli, end_cli, saldo, cod_vend)

nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
2	Cristina	Avenida 1	10,00	1
3	Manoel	Avenida 3	234,00	1
4	Rodrigo	Rua X	137,00	2

vendedor (cod_vend, nome_vend)

cod_vend	nome_vend
1	Adriana
2	Roberto

Cliente x Vendedor



nro_cli	nome_cli	end_cli	saldo	cliente. cod_vend	vendedor.c od_vend	nome_vend
1	Márcia	Rua X	100,00	1	1	Adriana
1	Márcia	Rua X	100,00	1	2	Roberto
2	Cristina	Avenida 1	10,00	1	1	Adriana
2	Cristina	Avenida 1	10,00	1	2	Roberto
3	Manoel	Avenida 3	234,00	1	1	Adriana
3	Manoel	Avenida 3	234,00	1	2	Roberto
4	Rodrigo	Rua X	137,00	2	1	Adriana
4	Rodrigo	Rua X	137,00	2	2	Roberto

grau: número de atributos
de cliente + número de
atributos de vendedor

número de tuplas: número de
tuplas de cliente * número de
tuplas de vendedor

Exemplo 1

- Considere as seguintes relações
 - usuário (cliente_nome, gerente_nome)
 - cliente (cliente_nome, rua, cidade)

cliente_nome	gerente_nome
Márcia	Manoel
Rodrigo	Maria
Bruno	Manoel

cliente_nome	rua	cidade
Márcia	Rua X	Itambé
Rodrigo	Rua X	Maringá
Bruno	Rua Z	Uberlândia

- Liste o nome de todos os usuários atendidos pelo gerente Manoel, assim como as cidades nas quais eles vivem

Solução

- Primeiro passo
 - determinar quem são os usuários atendidos pelo gerente Manoel

$\text{temp}_1 \leftarrow \pi_{\text{cliente_nome}} (\sigma_{\text{gerente_nome} = \text{"Manoel"}} (\text{usuário}))$

- relação resultado temp_1

cliente_nome
Márcia
Bruno

Solução

- Segundo passo
 - realizar o produto cartesiano das relações
$$\text{temp}_2 \leftarrow \text{temp}_1 \times \text{cliente}$$
 - relação resultado temp_2

Solução

$\text{temp}_2 \leftarrow \text{temp}_1 \times \text{cliente}$

temp_1 . cliente_nome	cliente. cliente_nome	rua	cidade
Márcia	Márcia	Rua X	Itambé
Márcia	Rodrigo	Rua X	Maringá
Márcia	Bruno	Rua Z	Uberlândia
Bruno	Márcia	Rua X	Itambé
Bruno	Rodrigo	Rua X	Maringá
Bruno	Bruno	Rua Z	Uberlândia

Solução

- Terceiro passo

- eliminar informações inconsistentes

$\text{temp}_3 \leftarrow \sigma_{\text{temp1.cliente_nome} = \text{cliente.cliente_nome}} (\text{temp}_2)$

- relação resultado temp_3

temp ₁ . cliente_nome	cliente. cliente_nome	rua	cidade
Márcia	Márcia	Rua X	Itambé
Bruno	Bruno	Rua Z	Uberlândia

Solução

- Quarto passo
 - exibir as informações solicitadas

$\pi_{\text{temp1.cliente_nome, cidade}}(\text{temp3})$

- relação resultado

temp ₁ . cliente_nome	cidade
Márcia	Itambé
Bruno	Uberlândia

Exemplo 2

- Considere a seguinte relação
 - cliente (cliente_nome, rua, cidade)

cliente_nome	rua	cidade
Márcia	Rua X	Itambé
Rodrigo	Rua X	Maringá
Cristina	Rua XTZ	Maringá
Sofia	Rua X	Maringá
Ricardo	Rua AAA	Itambé

- Liste o nome dos clientes que moram na mesma rua e na mesma cidade que Rodrigo (exceto o próprio Rodrigo)

Solução

- Primeiro passo
 - determinar o nome da rua e o nome da cidade na qual Rodrigo mora

$\text{temp}_1 \leftarrow \pi_{\text{rua, cidade}} (\sigma_{\text{cliente_nome} = \text{"Rodrigo"}} (\text{cliente}))$

- relação resultado temp_1

rua	cidade
Rua X	Maringá

Solução

◆ Segundo passo

- realizar o produto cartesiano das relações

$$\text{temp}_2 \leftarrow \text{temp}_1 \times \text{cliente}$$

- relação resultado temp_2

$\text{temp}_1.\text{rua}$	$\text{temp}_1.\text{cidade}$	cliente_nome	cliente.rua	cliente.cidade
Rua X	Maringá	Márcia	Rua X	Itambé
Rua X	Maringá	Rodrigo	Rua X	Maringá
Rua X	Maringá	Cristina	Rua XTZ	Maringá
Rua X	Maringá	Sofia	Rua X	Maringá
Rua X	Maringá	Ricardo	Rua AAA	Itambé

Solução

◆ Terceiro passo

- eliminar informações indesejadas

$$\text{temp}_3 \leftarrow \sigma_{\text{cliente_nome} \neq \text{"Rodrigo"}}(\text{temp}_2)$$

- relação resultado temp_3

$\text{temp}_1.\text{rua}$	$\text{temp}_1.\text{cidade}$	cliente_nome	cliente.rua	cliente.cidade
Rua X	Maringá	Márcia	Rua X	Itambé
Rua X	Maringá	Cristina	Rua XTZ	Maringá
Rua X	Maringá	Sofia	Rua X	Maringá
Rua X	Maringá	Ricardo	Rua AAA	Itambé

Solução

◆ Quarto passo

- exibir as informações solicitadas

$\pi_{\text{cliente_nome}} (\sigma_{\text{temp}_1.\text{rua} = \text{cliente.rua} \wedge \text{temp}_1.\text{cidade} = \text{cliente.cidade} (\text{temp}_3))$

- relação resultado

cliente_nome
Sofia

Discussão

◆ Solução proposta

$\text{temp}_1 \leftarrow \pi_{\text{rua}, \text{cidade}} (\sigma_{\text{cliente_nome} = \text{"Rodrigo"}} (\text{cliente}))$

$\text{temp}_2 \leftarrow \text{temp}_1 \times \text{cliente}$

$\text{temp}_3 \leftarrow \sigma_{\text{cliente_nome} \neq \text{"Rodrigo"}} (\text{temp}_2)$

$\pi_{\text{cliente_nome}} (\sigma_{\text{temp}_1.\text{rua} = \text{cliente.rua} \wedge \text{temp}_1.\text{cidade} = \text{cliente.cidade}} (\text{temp}_3))$

◆ Operação de atribuição

$\text{temp}_1(\text{rua_rodrigo}, \text{cidade_rodrigo}) \leftarrow$

$\pi_{\text{rua}, \text{cidade}} (\sigma_{\text{cliente_nome} = \text{"Rodrigo"}} (\text{cliente}))$

$\text{temp}_2 \leftarrow \text{temp}_1 \times \text{cliente}$

$\text{temp}_3 \leftarrow \sigma_{\text{cliente_nome} \neq \text{"Rodrigo"}} (\text{temp}_2)$

$\pi_{\text{cliente_nome}} (\sigma_{\text{rua_rodrigo} = \text{rua} \wedge \text{cidade_rodrigo} = \text{cidade}} (\text{temp}_3))$

Discussão

◆ Operação renomear (1)

$\text{temp}_1 \leftarrow \pi_{\text{rua}, \text{cidade}} (\sigma_{\text{cliente_nome} = \text{"Rodrigo"}} (\text{cliente}))$

$\text{temp}_2 \leftarrow \rho_{(\text{rua_rodrigo}, \text{cidade_rodrigo})} (\text{temp}_1) \times \text{cliente}$

$\text{temp}_3 \leftarrow \sigma_{\text{cliente_nome} \neq \text{"Rodrigo"}} (\text{temp}_2)$

$\pi_{\text{cliente_nome}} (\sigma_{\text{rua_rodrigo} = \text{rua} \wedge \text{cidade_rodrigo} = \text{cidade}} (\text{temp}_3))$

◆ Operação renomear (2)

$\text{temp}_1 \leftarrow \pi_{\text{rua}, \text{cidade}} (\sigma_{\text{cliente_nome} = \text{"Rodrigo"}} (\text{cliente}))$

$\text{temp}_2 \leftarrow \rho_{\text{dados_rodrigo}(\text{rua_rodrigo}, \text{cidade_rodrigo})} (\text{temp}_1) \times \text{cliente}$

$\text{temp}_3 \leftarrow \sigma_{\text{cliente_nome} \neq \text{"Rodrigo"}} (\text{temp}_2)$

$\pi_{\text{cliente_nome}} (\sigma_{\text{rua_rodrigo} = \text{rua} \wedge \text{cidade_rodrigo} = \text{cidade}} (\text{temp}_3))$

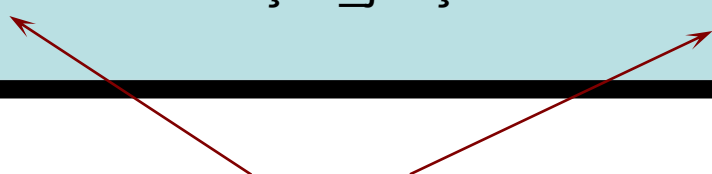
Junção

- Concatena tuplas relacionadas de duas relações em tuplas únicas
- Simplifica consultas que requerem produto cartesiano
 - forma um produto cartesiano dos argumentos

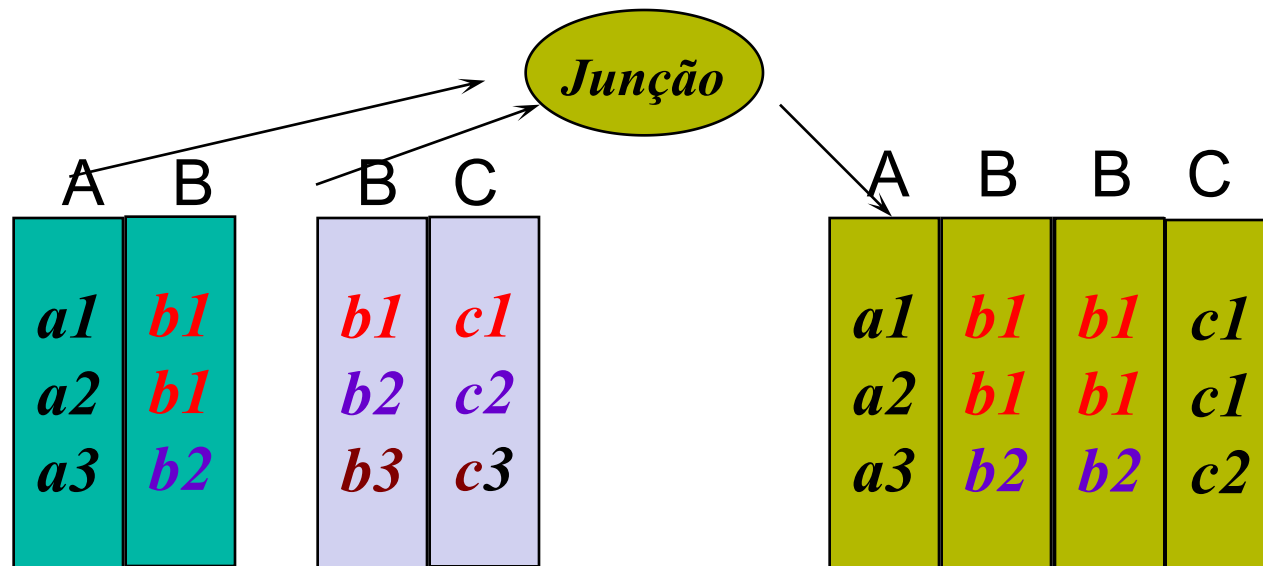
Junção

relação argumento 1 $\bowtie$ condição_junção relação argumento 2

relação



Junção



Note que a **coluna comum** na duas relações é a coluna “**b**”

Exemplo de esquema relacional

cliente (nro_cli, nome_cli, end_cli,
saldo, vendedor)

vendedor (cod_vend, nome_vend)

pedido (nro_ped, data, nro_cliente)

pedido_peça (nro_ped, nro_peça)

peça (nro_peça, descrição_peça)

cliente (nro_cli, nome_cli, end_cli, saldo, cod_vend)

nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
2	Cristina	Avenida 1	10,00	1
3	Manoel	Avenida 3	234,00	1
4	Rodrigo	Rua X	137,00	2

vendedor (cod_vend, nome_vend)

cod_vend	nome_vend
1	Adriana
2	Roberto

Cliente  vendedor
 cliente.cod_vend = vendedor.cod_vend

nro_cli	nome_cli	end_cli	saldo	cod_vend	cod_vend	nome_vend
1	Márcia	Rua X	100,00	1	1	Adriana
2	Cristina	Avenida 1	10,00	1	1	Adriana
3	Manoel	Avenida 3	234,00	1	1	Adriana
4	Rodrigo	Rua X	137,00	2	2	Roberto

grau: número de
atributos de cliente +
número de atributos de
vendedor

número de tuplas: entre
zero e o (número de
tuplas de cliente * número
de tuplas de vendedor)

Discussão

- ◆ Para os anciosos em ver uma solução em SQL:

```
select c.nome
  from cliente c, cliente cr
 where c.rua = cr.rua
    and c.cidade = cr.cidade
    and cr.nome = 'Rodrigo'
    and c.nome <> 'Rodrigo'
```

Próxima aula...

- Junções
- União
- Intersecção
- Diferença
- Subconsulta
- Divisão
- Agregação

Junção

- Condição de junção
 - $\langle \text{condição} \rangle \wedge \langle \text{condição} \rangle \wedge \dots \wedge \langle \text{condição} \rangle$
- $A_i \theta B_j$
 - A_i : atributo da relação argumento 1
 - B_j : atributo da relação argumento 2
 - A_i e B_j tem o mesmo domínio
 - θ (theta) é um operador de comparação $\{=, <, \leq, >, \geq, \neq\}$
 - existem diversas variações sobre a operação de junção

Junção

- **Junção theta**

- na qual pode ser usada qualquer operador θ válido no domínio dos atributos comparados
 - $\{=, <, \leq, >, \geq, \neq\}$
- os atributos envolvidos na comparação aparecem ambos na relação resultado (gerando pares de atributos), mas tuplas com valores nulos nos atributos envolvidos na junção não aparecem no resultado
- variação mais genérica

Junção

- **Junção theta**

cliente (nro_cli, nome_cli, end_cli, saldo, vendedor)

vendedor (cod_vend, nome_vend)

cliente ⋈_{vendedor ≠ cod_vend} vendedor

grau da relação resultante é igual a 7

Junção

- **Equi-Junção**

- uso mais comum de junção
- o operador θ é a igualdade
 - $\{=\}$
- os atributos envolvidos na comparação aparecem ambos na relação resultado (gerando pares de atributos com valores idênticos), mas tuplas com valores nulos nos atributos envolvidos na comparação não aparecem no resultado

Junção

- **Equi-Junção**

cliente (nro_cli, nome_cli, end_cli, saldo, vendedor)

vendedor (cod_vend, nome_vend)

cliente ⋈ vendedor = cod_vend

grau da relação resultante é igual a 7

Junção

- **Junção Natural – $R * S$**
 - semelhante à operação de Equi-Junção
 - porém, dos atributos da junção, apenas os originários de uma das relações operadas aparecem na relação resultado
 - requer que os atributos comparados tenham **nomes iguais** nas duas relações
 - tuplas com valores nulos nos atributos envolvidos na comparação também não aparecem no resultado

Junção

- **Junção Natural**

cliente (nro_cli, nome_cli, end_cli, saldo, cod_vend)

vendedor (cod_vend, nome_vend)



vendedor * cliente

grau da relação resultante é igual a 6

Junção

- **Junção Natural**

cliente (nro_cli, nome_cli, end_cli, saldo, vendedor)

vendedor (cod_vend, nome_vend)



vendedor * $\rho_{(nro_cli, nome_cli, end_cli, saldo, cod_vend)}$ cliente

grau da relação resultante é igual a 6

Junção

R

A	B	C
1	a	x
2	b	y
3	a	y
4	c	y

S

A	D
1	d
2	d
5	e

$R \bowtie S$

A	B	C	A	D
1	a	x	1	d
2	b	y	2	d

- Interna (inner join)
 - somente as tuplas de R que têm tuplas correspondentes em S, e vice-versa, aparecem no resultado

Junção

R

A	B	C
1	a	x
2	b	y
3	a	y
4	c	y

S

A	D
1	d
2	d
5	e

$R \bowtie S$

A	B	C	A	D
1	a	x	1	d
2	b	y	2	d
3	a	y	Null	Null
4	c	y	Null	Null

- Externa à esquerda (left outer join)
 - mantém cada tupla de R em $R \bowtie S$
 - preenche com valores nulos os atributos de S que não correspondem às tuplas em R

Junção

R

A	B	C
1	a	x
2	b	y
3	a	y
4	c	y

S

A	D
1	d
2	d
5	e

$R \bowtie S$

A	B	C	A	D
1	a	x	1	d
2	b	y	2	d
Null	Null	Null	5	e

- Externa à direita (right outer join)
 - mantém cada tupla de S em $R \bowtie S$
 - preenche com valores nulos os atributos de R que não correspondem às tuplas em S

Junção

R			S		$R \bowtie S$				
A	B	C	A	D	A	B	C	A	D
1	a	x	1	d	1	a	x	1	d
2	b	y	2	d	2	b	y	2	d
3	a	y	5	e	3	a	y	Null	Null
4	c	y			4	c	y	Null	Null
					Null	Null	Null	5	e

- Externa completa (full outer join)
 - mantém as tuplas de R e S em $R \bowtie S$
 - preenche com valores nulos os atributos que não correspondem à coluna de junção

Exercicio 1

- Considere as seguintes relações
 - usuário (cliente_nome, gerente_nome)
 - cliente (cliente_nome, rua, cidade)

usuário

cliente_nome	gerente_nome
Márcia	Manoel
Rodrigo	Maria

cliente

cliente_nome	rua	cidade
Márcia	Rua X	Itambé
Rodrigo	Rua X	Maringá

- Liste o nome de todos os usuários atendidos pelo gerente Manoel, assim como as cidades nas quais eles vivem

Exemplo 1

- ◆ Considere as seguintes relações
 - usuário (cliente_nome, gerente_nome)
 - cliente (cliente_nome, rua, cidade)
- ◆ Liste o nome de todos os usuários atendidos pelo gerente Manoel, assim como as cidades nas quais eles vivem

$\text{temp}_1 \leftarrow \pi_{\text{cliente_nome}} (\sigma_{\text{gerente_nome} = \text{"Manoel"}} (\text{usuário}))$

$\text{temp}_2 \leftarrow \text{temp}_1 \times \text{cliente}$

$\text{temp}_3 \leftarrow \sigma_{\text{temp1.cliente_nome} = \text{cliente.cliente_nome}} (\text{temp}_2)$

$\pi_{\text{temp1.cliente_nome}, \text{cidade}} (\text{temp}_3)$

Exemplo 1

- ◆ Considere as seguintes relações
 - usuário (cliente_nome, gerente_nome)
 - cliente (cliente_nome, rua, cidade)
- ◆ Liste o nome de todos os usuários atendidos pelo gerente Manoel, assim como as cidades nas quais eles vivem

$\text{temp}_1 \leftarrow \pi_{\text{cliente_nome}} (\sigma_{\text{gerente_nome} = \text{"Manoel"}} (\text{usuário}))$

$\text{temp}_2 \leftarrow \text{temp}_1 * \text{cliente}$

$\pi_{\text{temp1.cliente_nome}, \text{cidade}} (\text{temp2})$

Exemplo 2

- Considere a seguinte relação
 - cliente (cliente_nome, rua, cidade)

cliente_nome	rua	cidade
Márcia	Rua X	Itambé
Rodrigo	Rua X	Maringá
Cristina	Rua XTZ	Maringá
Sofia	Rua X	Maringá
Ricardo	Rua AAA	Itambé

- Liste o nome dos clientes que moram na mesma rua e na mesma cidade que Rodrigo

Exemplo 2

- ◆ Considere a seguinte relação
 - cliente (cliente_nome, rua, cidade)
- ◆ Liste o nome dos clientes que moram na mesma rua e na mesma cidade que Rodrigo

$\text{temp}_1 \leftarrow \pi_{\text{rua}, \text{cidade}} (\sigma_{\text{cliente_nome} = \text{"Rodrigo"}} (\text{cliente}))$

$\text{temp}_2 \leftarrow \text{temp}_1 \times \text{cliente}$

$\text{temp}_3 \leftarrow \sigma_{\text{cliente_nome} \neq \text{"Rodrigo"}} (\text{temp}_2)$

$\pi_{\text{cliente_nome}} (\sigma_{\text{temp}_1.\text{rua} = \text{cliente.rua} \wedge \text{temp}_1.\text{cidade} = \text{cliente.cidade}} (\text{temp}_3))$

Exemplo 2

- ◆ Considere a seguinte relação
 - cliente (cliente_nome, rua, cidade)
- ◆ Liste o nome dos clientes que moram na mesma rua e na mesma cidade que Rodrigo

$\text{temp}_1 \leftarrow \pi_{\text{rua}, \text{cidade}} (\sigma_{\text{cliente_nome} = \text{"Rodrigo"}} (\text{cliente}))$

$\text{temp}_2 \leftarrow \text{temp}_1 * \text{cliente}$

$\text{temp}_3 \leftarrow \sigma_{\text{cliente_nome} \neq \text{"Rodrigo"}} (\text{temp}_2)$

$\pi_{\text{cliente_nome}} (\text{temp}_3)$

Operações sobre Conjuntos

- Operações

- união
- intersecção
- diferença

- Características

- atuam sobre relações compatíveis
- eliminam tuplas duplicadas da relação resultado

Duas relações são compatíveis quando:

- possuem o mesmo grau
- seus atributos possuem os mesmos domínios (os domínios dos i -ésimos atributos de cada relação são os mesmos)

União de Conjuntos

- Une duas relações R e S compatíveis em uma relação que contém todas as tuplas pertencentes a R, a S, ou a ambas R e S

relação argumento 1 $\cup$ relação argumento 2



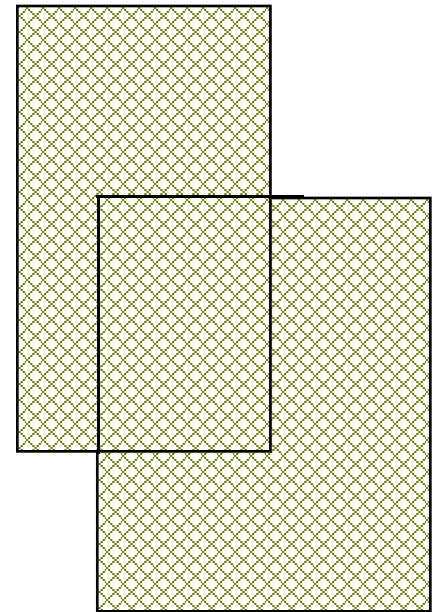
- relação
- resultado de alguma operação da álgebra relacional

União de Conjuntos

- relações cujos domínios dos atributos são iguais, na mesma ordem de definição das colunas

Notação:

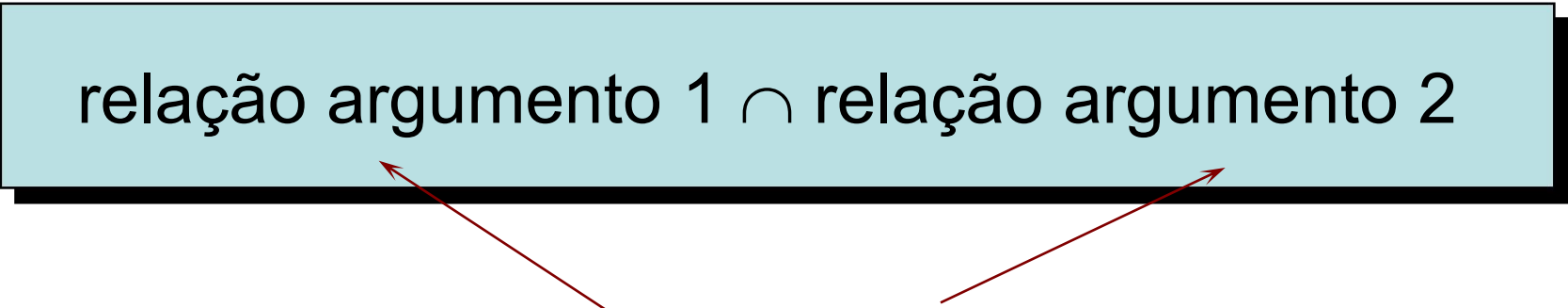
$\langle \text{relação1} \rangle \cup \langle \text{relação2} \rangle$



Intersecção de Conjuntos

- Une duas relações R e S compatíveis em uma relação que contém todas as tuplas pertencentes tanto a R quanto a S

relação argumento 1 $\cap$ relação argumento 2

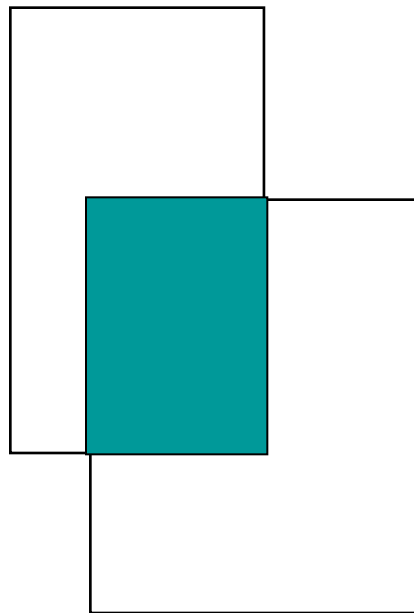


- relação
- resultado de alguma operação da álgebra relacional

Intersecção de Conjuntos

Notação:

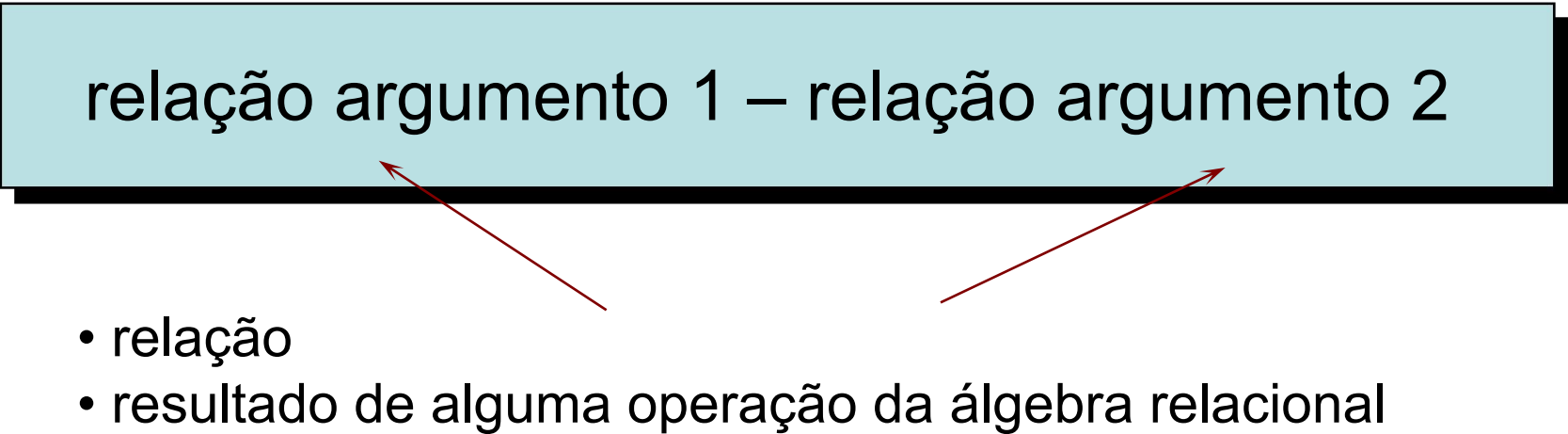
$\langle \text{relação1} \rangle \cap \langle \text{relação2} \rangle$



Diferença de Conjuntos

- Une duas relações R e S compatíveis em uma relação que contém todas as tuplas pertencentes a R que não pertencem a S

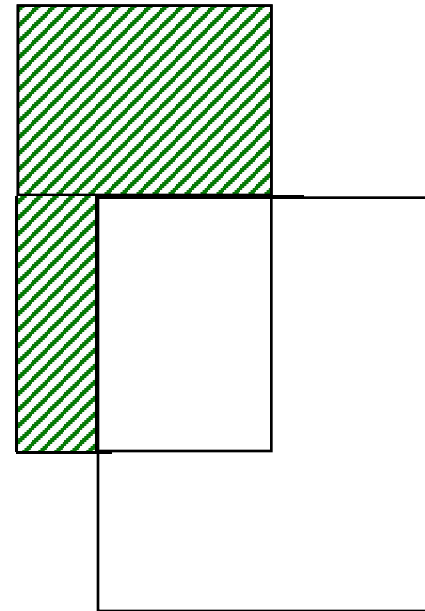
relação argumento 1 – relação argumento 2



- relação
- resultado de alguma operação da álgebra relacional

Diferença de Conjuntos

Notação: $\langle \text{relação1} \rangle - \langle \text{relação2} \rangle$



Exemplo : Listar os vendedores que não atendem nenhum cliente, ou seja, que estão na tabela Vendedor mas que não estão na tabela de “Clientes”

Relações Cliente e Pedido

cliente (nro_cli, nome_cli, end_cli, saldo, cod_vend)

nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
2	Cristina	Avenida 1	10,00	1
3	Manoel	Avenida 3	234,00	1
4	Rodrigo	Rua X	137,00	2

pedido (nro_ped, data, nro_cliente)

nro_ped	data	nro_cliente
1	10/12/2004	1
2	11/12/2004	4

Consultas 5, 6 e 7

- Liste os números dos clientes que
 5. ou têm pedido, ou foram atendidos pelo vendedor 2, ou ambos
 6. têm pedido, e que foram atendidos pelo vendedor 2
 7. têm pedido, mas que não foram atendidos pelo vendedor 2

cliente (nro_cli, nome_cli, end_cli, saldo, cod_vend)

nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
2	Cristina	Avenida 1	10,00	1
3	Manoel	Avenida 3	234,00	1
4	Rodrigo	Rua X	137,00	2

pedido (nro_ped, data, nro_cliente)

nro_ped	data	nro_cliente
1	10/12/2004	1
2	11/12/2004	4

cliente (nro_cli, nome_cli, end_cli, saldo, cod_vend)

nro_cli	nome_cli	end_cli	saldo	cod_vend
1	Márcia	Rua X	100,00	1
2	Cristina	Avenida 1	10,00	1
3	Manoel	Avenida 3	234,00	1
4	Rodrigo	Rua X	137,00	2

pedido (nro_ped, data, nro_cliente)

nro_ped	data	nro_cliente
1	10/12/2004	1
2	11/12/2004	4

Consultas 5, 6 e 7

- Liste os números dos clientes que têm pedido.

$$\text{Temp}_1 \leftarrow \pi_{\text{nro_cliente}}(\text{pedido})$$

- Liste os números dos clientes que foram atendidos pelo vendedor 2.

$$\text{temp}_2 \leftarrow \pi_{\text{nro_cli}}(\sigma_{\text{cod_vend} = 2}(\text{cliente}))$$

Consulta 5

- 10 Liste os números dos clientes que ou têm pedido, ou foram atendidos pelo vendedor 2, ou ambos.

temp₁

nro_cliente
1
4

temp₂

nro_cli
4

temp₁ $\cup$ temp₂

nro_cliente
1
4

Consulta 6

- 10 Liste os números dos clientes que têm pedido, e que foram atendidos pelo vendedor 2 .

$temp_1$

nro_cliente
1
4

$temp_2$

nro_cli
4

$temp_1 \cap temp_2$

nro_cliente
4

Consulta 7

- 10 Liste os números dos clientes que têm pedido, mas que não foram atendidos pelo vendedor 2.

temp₁

nro_cliente
1
4

temp₂

nro_cli
4

temp₁ – temp₂

nro_cliente
1

Exercício

- Considere a seguinte relação
 - conta (nro_conta, saldo)

nro_conta	saldo
01-010101-01	100,00
01-020202-02	200,00
01-030303-03	300,00
01-040404-04	400,00

- Liste o maior saldo

Solução

- Primeiro passo
 - realizar o produto cartesiano da relação conta com ela mesma

$$\text{temp}_1 \leftarrow \text{conta} \times \rho_{\text{conta2}}(\text{conta})$$

- relação resultado temp_1

conta.nro_conta	conta.saldo	conta2.nro_conta	conta2.saldo
01-010101-01	100,00	01-010101-01	100,00
01-010101-01	100,00	01-020202-02	200,00
01-010101-01	100,00	01-030303-03	300,00
01-010101-01	100,00	01-040404-04	400,00
01-020202-02	200,00	01-010101-01	100,00
01-020202-02	200,00	01-020202-02	200,00
01-020202-02	200,00	01-030303-03	300,00
01-020202-02	200,00	01-040404-04	400,00
01-030303-03	300,00	01-010101-01	100,00
01-030303-03	300,00	01-020202-02	200,00
01-030303-03	300,00	01-030303-03	300,00
01-030303-03	300,00	01-040404-04	400,00
01-040404-04	400,00	01-010101-01	100,00
01-040404-04	400,00	01-020202-02	200,00
01-040404-04	400,00	01-030303-03	300,00
01-040404-04	400,00	01-040404-04	400,00

Solução

- Segundo passo
 - listar os saldos que não são os mais altos

$$\text{temp}_2 \leftarrow \pi_{\text{conta.saldo}} (\sigma_{\text{conta.saldo} < \text{conta2.saldo}} (\text{temp}_1))$$

- relação resultado temp_2

conta.saldo
100,00
200,00
300,00

Solução

- Terceiro passo
 - listar todos os saldos da relação conta

$$\text{temp}_3 \leftarrow \pi_{\text{saldo}}(\text{conta})$$

- relação resultado temp_3

saldo
100,00
200,00
300,00
400,00

Solução

- Quarto passo
 - fazer a diferença entre “todos os saldos da relação conta” e “os saldos que não são os mais altos”

$$\text{temp}_3 - \text{temp}_2$$

- relação resultado

saldo
400,00

Exercício

- Considere as seguintes relações
 - Aluno = (Nome, Idade, Curso)
 - Professor = (Nome, Idade, Depto)
 - Matricula = (NomeA, Disciplina, Nota)
 - Aulas = (NomeProf, Disciplina)
- 1. Listar a idade e o nome dos alunos e professores do banco de dados
- 2. Listar as disciplinas em que os alunos de Computação se matricularam
- 3. Listar o departamento dos professores que não ministram a disciplina de Banco de Dados

Exercício

1) Listar a idade e o nome dos alunos e professores do banco de dados

$Aux1 \leftarrow \pi_{\{Nome, Idade\}}(Aluno)$

$Aux2 \leftarrow \pi_{\{Nome, Idade\}}(Professor)$

$Pessoas \leftarrow Aux1 \cup Aux2$

2) Listar as disciplinas em que os alunos de computação se matricularam

$\pi_{\{Disciplina\}}(\sigma_{(curso = "computação")}(Aluno * \rho_{(Nome, Disciplina, Nota)} Matricula))$

3) Listar o departamento dos professores que não ministram a disciplina de Banco de Dados

$Aux1 \leftarrow \pi_{Nome}(\sigma_{disciplina = "Banco de Dados"}(Professor * \rho_{(Nome, Disciplina)} Aulas))$

$Aux2 \leftarrow \pi_{Nome}(Professor)$

$Departamento \leftarrow \pi_{Depto}(Aux2 - Aux1) * Professor$

Atividades complementares

- Leitura para casa
 - Capítulo 6, “Álgebra e cálculo relacional” do livro: Elmasri, Ramez; Navathe, Shamkant B. Sistemas de banco de dados, 6ª. edição (2011).

Aula 10 – SBD

SQL: Consulta Simples

Profa. Elaine Faria

UFU

Agradecimentos

- Agradecimentos
 - Ao professor Bruno Travençolo que gentilmente cedeu seus slides

Consultas

- O processo de obtenção ou o comando para a obtenção de dados de um banco de dados é chamado **consulta**
- Em SQL, o comando SELECT é usado para especificar consultas
- As consultas simples em SQL correspondem às operações de SELEÇÃO, PROJEÇÃO e JUNÇÃO da álgebra relacional

Sintaxe SELECT

```
SELECT [ ALL | DISTINCT [ ON ( expressão [, ...] ) ] ]  
* | expressão [ AS nome_de_saída ] [, ...]  
[ FROM item_do_from [, ...] ]  
[ WHERE condição ]  
[ GROUP BY expressão [, ...] ]  
[ HAVING condição [, ...] ]  
[ { UNION | INTERSECT | EXCEPT } [ ALL ] seleção ]  
[ ORDER BY expressão [ ASC | DESC | USING operador ]  
[, ...] ]  
[ LIMIT { contador | ALL } ]  
[ OFFSET início ]  
[ FOR UPDATE [ OF nome_da_tabela [, ...] ] ]
```

Itens FROM

onde item_do_from pode ser um entre:

[ONLY] nome_da_tabela [*] [[AS] alias [(alias_de_coluna
[, ...])]]
(seleção) [AS] alias [(alias_de_coluna [, ...])]
nome_da_função ([argumento [, ...]]) [AS] alias
[(alias_de_coluna [, ...]
| definição_de_coluna [, ...])]
nome_da_função ([argumento [, ...]]) AS
(definição_de_coluna [, ...])
item_do_from [NATURAL] tipo_de_junção item_do_from [ON
condição_de_junção
| USING (coluna_de_junção [, ...])]

Comando SELECT

Exemplos

/ Seleciona todos os campos na tabela ALUNO */*

```
SELECT *  
FROM ALUNO;
```

/ Selecionar todos os nomes e nros. de matrícula da tabela ALUNO */*

```
SELECT NMat, Nome  
FROM ALUNO;
```

/ Selecionar todos os nomes e nros. de matrícula da tabela ALUNO, renomeando a coluna nome para “aluno” */*

```
SELECT NMat, Nome AS Aluno  
FROM ALUNO;
```

Comando SELECT

Exemplos

DISTINCT – remove as tuplas duplicadas da resposta

/ Selecciona todas as cidades */*



SELECT cidade
FROM filiais;

Resultado correto, mas
com muitas repetições

Cidade
São Paulo
São Paulo
São Paulo
Rio de Janeiro
Rio de Janeiro
Angra dos Reis
Belo Horizonte
Belo Horizonte
Uberlândia
Uberlândia

/ Seleccionar todas as cidades*/*

SELECT DISTINCT cidade
FROM filiais;



Cidade
São Paulo
Belo Horizonte
Uberlândia
Rio de Janeiro
Angra dos Reis

Comando SELECT

CLÁUSULA WHERE

WHERE - Especifica quais registros das tabelas listadas na cláusula FROM são afetados por uma instrução SELECT, UPDATE ou DELETE. Se você não especificar uma cláusula WHERE, a consulta retornará todas as linhas da tabela.

EXEMPLO:

/ Seleciona nome/idade dos alunos cuja idade seja maior que 22 */*

```
SELECT Nome, Idade  
FROM ALUNO  
WHERE Idade > 22;
```


Comando SELECT CLÁUSULA WHERE

Exemplos :

/ Selecciona os números de matrícula dos alunos matriculados a partir de 30/07/2002 */*

```
SELECT NMat  
FROM MATRICULA  
WHERE Data > '30/07/2002';
```

/ Selecciona os números de matrícula dos alunos matriculados a partir de 30/07/2002 */*

```
SELECT M.NMat  
FROM MATRICULA M  
WHERE M.Data > '30/07/2002';
```

CLÁUSULA WHERE

Sintaxe

WHERE *condition*

Onde *condition* é qualquer expressão que retorne um tipo booleano. Qualquer linha que não satisfaça a condição *condition* será eliminada do resultado

Exemplos :

/ Seleciona os números de matrícula dos alunos matriculados a partir de 30/07/2002 */*

```
SELECT NMat  
FROM MATRICULA  
WHERE Data > '30/07/2002';
```

/ Seleciona os números de matrícula dos alunos matriculados a partir de 30/07/2002 */*

```
SELECT M.NMat  
FROM MATRICULA M  
WHERE M.Data > '30/07/2002';
```

CLÁUSULA WHERE

OPERADOR BETWEEN...AND

BETWEEN...AND: Determina se o valor de uma expressão se situa dentro de um intervalo especificado de valores. Se o valor de *expr* estiver entre *<valor1>* e *<valor2>* (inclusive), o operador BETWEEN...AND retornará True ; caso contrário, retornará False.

EXEMPLO:

/ Listar todos alunos com idade entre 20 e 22 anos*/*

SELECT Nome, Idade

FROM Aluno

WHERE Idade **BETWEEN** 20 **AND** 22;

CLÁUSULA WHERE

OPERADOR IN

IN: Determina se o valor de uma expressão é igual a algum dos vários valores em uma lista especificada. Se *expr* for encontrado na lista de valores, o operador IN retornará True; caso contrário, retornará False.

EXEMPLO:

/ Listar todos os alunos provenientes de São Paulo, São Carlos ou Rio de Janeiro*/*

SELECT *

FROM ALUNO

WHERE UPPER(Cidade) IN ('SAO PAULO', 'SAO CARLOS', 'RIO DE JANEIRO'); -- UPPER transforma os caracteres do atributo Cidade em maiúsculo

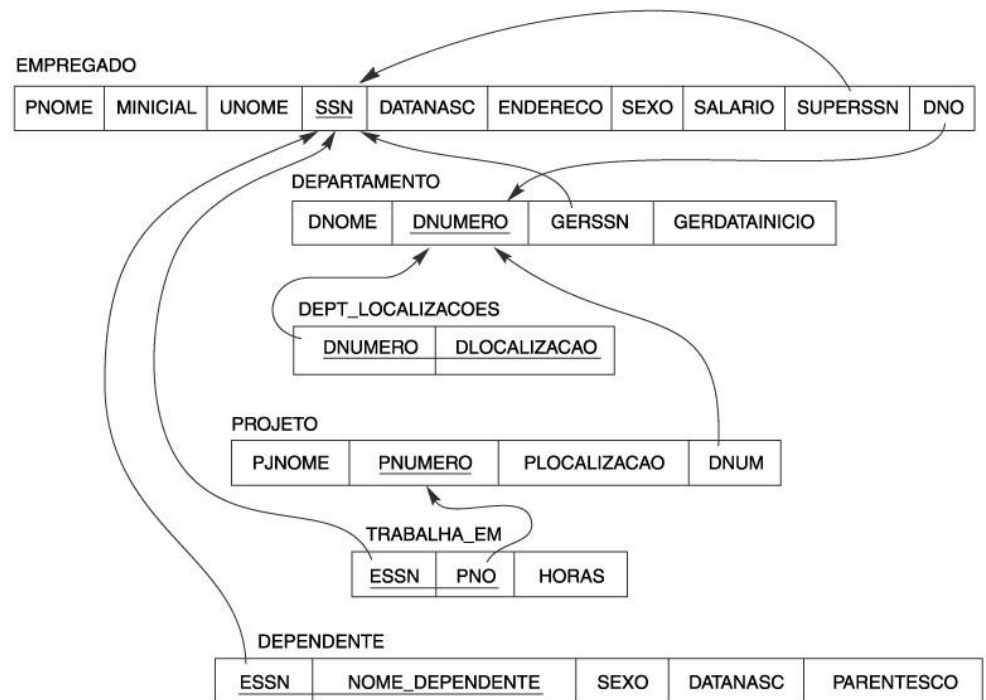
CLÁUSULA WHERE

OPERADOR IN

IN + subconsulta: O operador IN pode trabalhar com subconsulta

/ Listar os nomes funcionários que trabalham em projetos'*/*

```
SELECT nome  
FROM empregado  
WHERE ssn IN (  
  SELECT essn  
  FROM trabalha_em )
```

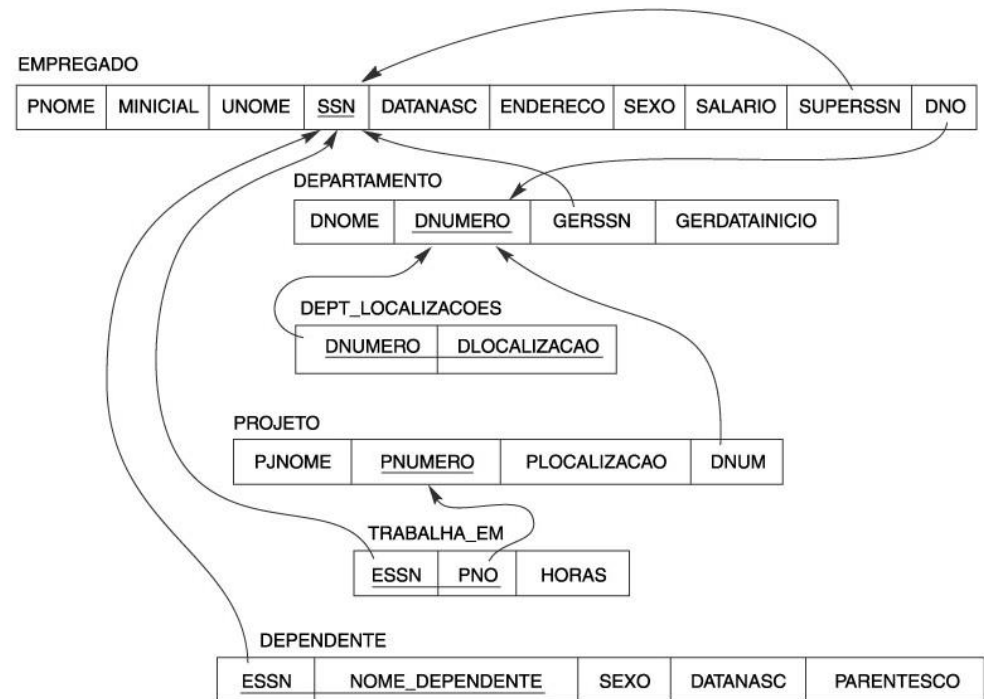


CLÁUSULA WHERE OPERADOR IN

NOT IN + subconsulta: O operador IN pode trabalhar com subconsulta.

/ Listar os nomes funcionários que não trabalham em nenhum projeto'*/*

```
SELECT nome  
FROM empregado  
WHERE ssn NOT IN (  
  SELECT essn  
  FROM trabalha_em)
```



Comando SELECT

Consultas aninhadas

- Muitas vezes é preciso buscar uma informação no banco para, então, usá-la na condição de comparação de uma consulta
- Isso pode ser feito por meio de consultas aninhadas:
 - Forma-se um bloco SELECT-FROM-WHERE dentro da cláusula WHERE de outra consulta.
 - Essa outra consulta é chamada de consulta interna.

/ Listar os projetos dos funcionários de nome 'José'*/*

```
SELECT projeto_pnumero  
FROM trabalha_em  
WHERE essn IN (  
    SELECT ssn  
    FROM empregado  
    WHERE pnome = 'José')
```



Consulta aninhada
(consulta interna)

Comando SELECT

Consultas aninhadas – Operador IN

/ Listar os nomes funcionários que não trabalham em nenhum projeto'*/*

```
SELECT nome  
FROM empregado  
WHERE ssn NOT IN (  
    SELECT essn  
    FROM trabalha_em )
```


Comando SELECT

Consultas aninhadas – Operador IN

Tabela Filiais

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG
Internet		

```
CREATE TABLE ProximasCidades(  
  cidade varchar(50));  
DELETE FROM ProximasCidades;  
INSERT INTO ProximasCidades  
  Select Distinct cidade  
  from filiais  
  where cidade is not null;  
INSERT INTO ProximasCidades VALUES ('Araguari'),('Ituiutaba');  
SELECT * FROM proximasCidades
```



/ Qual o resultado? */*

```
SELECT *  
FROM filiais  
WHERE cidade IN (  
  SELECT cidade  
  FROM ProximasCidades)
```

	cidade character varying(50)
1	Rio de Janeiro
2	Belo Horizonte
3	Uberlândia
4	Angra dos Reis
5	São Paulo
6	Araguari
7	Ituiutaba

Comando SELECT

Consultas aninhadas – Operador IN

/ Qual o resultado?*

*São as cidades que já temos
filiais mas que ainda estão na
lista de próximas cidade
E a filial da Internet? */*

```
SELECT *  
FROM filiais  
WHERE cidade IN (  
    SELECT cidade  
    FROM ProximasCidades)
```

	nome character varying(30)	cidade character varying(50)	uf character(2)
1	Penha	São Paulo	SP
2	Limão	São Paulo	SP
3	Brás	São Paulo	SP
4	Glória	Belo Horizonte	MG
5	Pampulha	Belo Horizonte	MG
6	Centro	Uberlândia	MG
7	Tiberv	Uberlândia	MG
8	Barra	Rio de Janeiro	RJ
9	Tijuca	Rio de Janeiro	RJ
10	Angra	Angra dos Reis	RJ

Comando SELECT

Consultas aninhadas – Operador IN

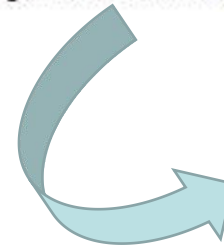
Tabela Filiais

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG
Internet		

```
CREATE TABLE ProximasCidades(  
  cidade varchar(50));  
DELETE FROM ProximasCidades;  
INSERT INTO ProximasCidades  
  Select Distinct cidade  
  from filiais  
  where cidade is not null;  
INSERT INTO ProximasCidades VALUES ('Araguari'),('Ituiutaba');  
SELECT * FROM proximascidades
```

/ Qual o resultado? */*

```
SELECT *  
FROM ProximasCidades  
WHERE cidade IN (  
  SELECT cidade  
  FROM filiais)
```



	cidade character varying(50)
1	Rio de Janeiro
2	Belo Horizonte
3	Uberlândia
4	Angra dos Reis
5	São Paulo
6	Araguari
7	Ituiutaba

Comando SELECT

Consultas aninhadas – Operador IN

/ Qual o resultado?*

*São as cidades que já temos
filiais*/*

```
SELECT *  
FROM ProximasCidades  
WHERE cidade IN (  
    SELECT cidade  
    FROM filiais)
```

	cidade character varying(50)
1	Rio de Janeiro
2	Belo Horizonte
3	Uberlândia
4	Angra dos Reis
5	São Paulo

Comando SELECT

Consultas aninhadas – Operador IN

Tabela Filiais

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG
Internet		

```
CREATE TABLE ProximasCidades(  
  cidade varchar(50));  
DELETE FROM ProximasCidades;  
INSERT INTO ProximasCidades  
  Select Distinct cidade  
  from filiais  
  where cidade is not null;  
INSERT INTO ProximasCidades VALUES ('Araguari'),('Ituiutaba');  
SELECT * FROM proximascidades
```

/ Qual o resultado? */*

```
SELECT *  
FROM ProximasCidades  
WHERE cidade NOT IN (  
  SELECT cidade  
  FROM filiais)
```



	cidade character varying(50)
1	Rio de Janeiro
2	Belo Horizonte
3	Uberlândia
4	Angra dos Reis
5	São Paulo
6	Araguari
7	Ituiutaba

Comando SELECT

Consultas aninhadas – Operador IN

/ Qual o resultado? */*

```
SELECT *  
FROM ProximasCidades  
WHERE cidade NOT IN (  
  SELECT cidade  
  FROM filiais)
```



	cidade character varying(50)

```
SELECT *  
FROM ProximasCidades  
WHERE cidade NOT IN (  
  SELECT cidade  
  FROM filiais  
  WHERE cidade IS NOT NULL)
```



	cidade character varying(50)
1	Araquari
2	Ituiutaba

Operadores booleanos

a	b	a AND b	a OR b
TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
TRUE	NULL	NULL	TRUE
FALSE	FALSE	FALSE	FALSE
FALSE	NULL	FALSE	NULL
NULL	NULL	NULL	NULL

a	NOT a
TRUE	FALSE
FALSE	TRUE
NULL	NULL

			Resultado
Araguari	=	São Paulo	False
Araguari	=	São Paulo	False
Araguari	=	São Paulo	False
Araguari	=	Rio de Janeiro	False
Araguari	=	Rio de Janeiro	False
Araguari	=	Angra dos Reis	False
Araguari	=	Belo Horizonte	False
Araguari	=	Belo Horizonte	False
Araguari	=	Uberlândia	False
Araguari	=	Uberlândia	False
Araguari	=		NULL

FALSE OR NULL => NULL

*lembre que NULL é **desconhecido**, ou seja, desconhecemos o resultado da operação:

se o *desconhecido* fosse TRUE, temos que

FALSE OR TRUE => TRUE;

se o *desconhecido* fosse FALSE, temos que

FALSE OR FALSE => FALSE

Assim, desconhecemos o resultado da operação, ou seja, o resultado é NULL

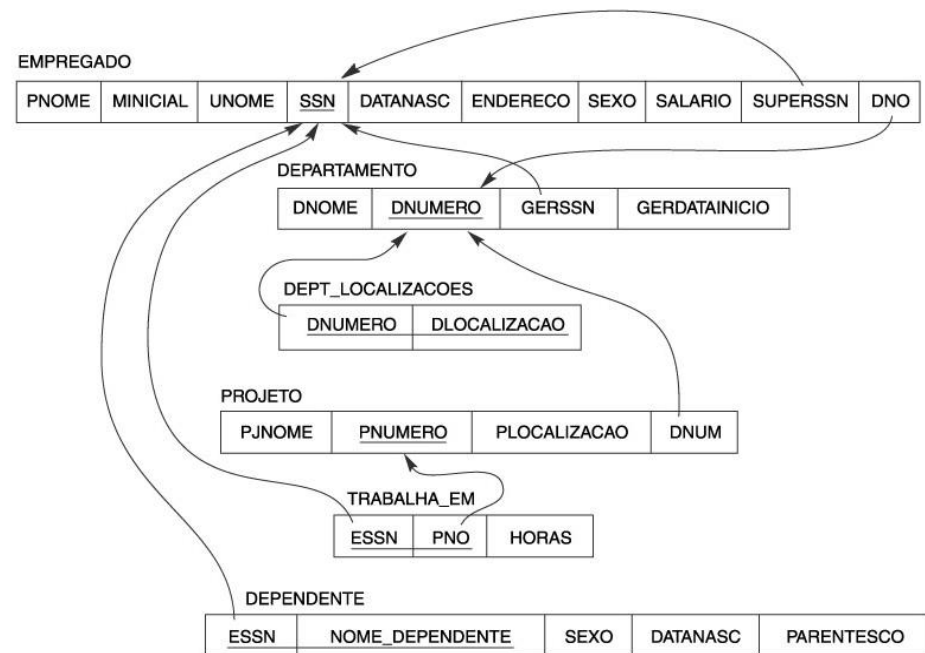
Comando SELECT

Consultas aninhadas

- Se um único atributo com uma única tupla é retornado da consulta externa, então podemos usar o operador =

/ Listar o funcionário com o maior salário */*

```
SELECT pnome,unome,salario
FROM empregado
WHERE salario = (
  SELECT max(salario)
  FROM empregado)
```



CLÁUSULA WHERE OPERADOR IS NULL

IS NULL: Determina se o valor de uma expressão é Nula

EXEMPLO:

/ Listar todas as disciplinas que NÃO possuem pré-requisito */*

SELECT Nome

FROM Discip

WHERE SiglaPreReq = **NULL**;

-- forma errada de comparar nulos

CLÁUSULA WHERE OPERADOR IS NULL

IS NULL: Determina se o valor de uma expressão é Nula

EXEMPLO:

/ Listar todas as disciplinas que NÃO possuem pré-requisito */*

SELECT Nome

FROM Discip

WHERE SiglaPreReq **IS NULL**;

-- observe que o teste não é SiglaPreReq = NULL

CLÁUSULA WHERE

OPERADOR LIKE

LIKE: Compara uma expressão de seqüência com um padrão em uma expressão SQL. Para padrão, você pode utilizar caracteres curinga (por exemplo, Like 'MAK%', para 'MAKROMBOOKS') ou utilizar caracteres isolados (por exemplo, Like '_OSE', para 'JOSE' e 'ROSE')

EXEMPLO:

/ Listar todos os alunos cujo nome termina em 'ina',
ignorando as 3 primeiras letras */*

SELECT Nome

FROM Aluno

WHERE UPPER(Nome) LIKE '___INA'; -- São 3 '_'. Procure
também substituir os 3 '_' por um '%'

Comando SELECT

CLÁUSULA ORDER BY

ORDER BY: Classifica os registros resultantes de uma consulta em um campo ou campos especificados, em ordem crescente ou decrescente. Os registros são classificados pelo primeiro campo listado após **ORDER BY**. Os registros que têm valores iguais naquele campo serão então classificados pelo valor no segundo campo listado e assim por diante.

EXEMPLO:

/ Listar todos alunos ordenado-os descentemente por idade e depois por nome */*

SELECT Nome, Idade

FROM Aluno

ORDER BY Idade **DESC**, Nome **DESC**; -- Tente também
trocar **DESC** por **ASC**

Referências

- Slides adaptados da aula da Profa. Josiane M. Bueno (*in memoriam*)
- Slides Prof. Humberto Razente
- Slides Profa. Sandra de Amo

Aula 11 – SBD

SQL: Consulta com Junção

Profa. Elaine Faria
UFU

Agradecimentos

- Agradecimentos
 - Ao professor Bruno Travençolo que gentilmente cedeu seus slides

Sintaxe SELECT

```
SELECT [ ALL | DISTINCT [ ON ( expressão [, ...] ) ] ]  
* | expressão [ AS nome_de_saída ] [, ...]  
[ FROM item_do_from [, ...] ] -- aula de hoje  
[ WHERE condição ]  
[ GROUP BY expressão [, ...] ]  
[ HAVING condição [, ...] ]  
[ { UNION | INTERSECT | EXCEPT } [ ALL ] seleção ]  
[ ORDER BY expressão [ ASC | DESC | USING operador ]  
[, ...] ]  
[ LIMIT { contador | ALL } ]  
[ OFFSET início ]  
[ FOR UPDATE [ OF nome_da_tabela [, ...] ] ]
```

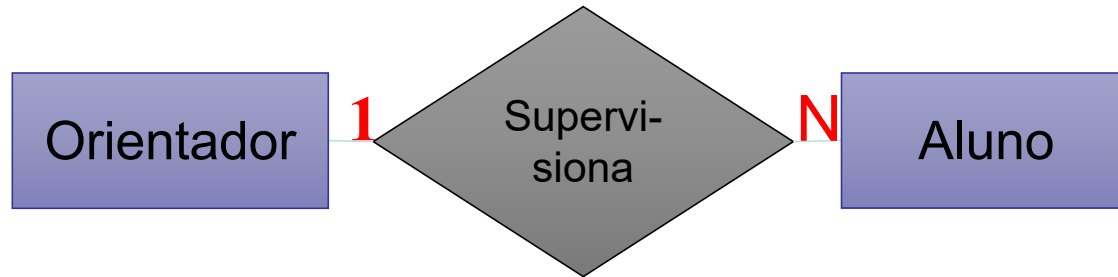

Itens FROM

onde item_do_from pode ser um entre:

```
[ ONLY ] nome_da_tabela [ * ] [ [ AS ] alias [ ( alias_de_coluna [, ...] ) ] ]  
( seleção ) [ AS ] alias [ ( alias_de_coluna [, ...] ) ]  
nome_da_função ( [ argumento [, ...] ] ) [ AS ] alias [ ( alias_de_coluna [, ...]  
    | definição_de_coluna [, ...] ) ]  
nome_da_função ( [ argumento [, ...] ] ) AS ( definição_de_coluna [, ...] )  
item_do_from [ NATURAL ] tipo_de_junção item_do_from [ ON  
condição_de_junção  
    | USING ( coluna_de_junção [, ...] ) ]
```

Onde tipo_de_junção pode ser:

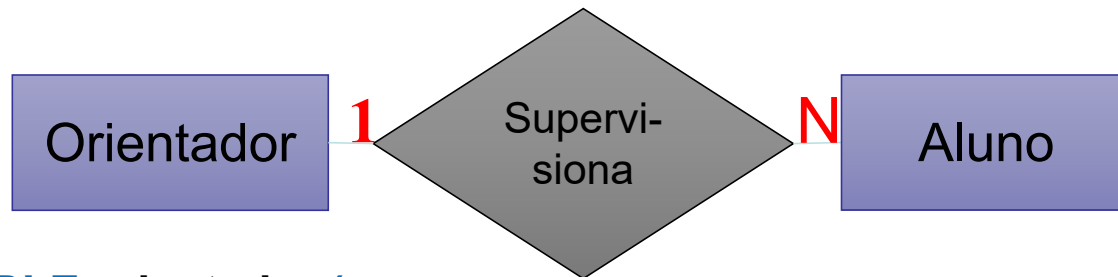
```
[ INNER ] JOIN  
LEFT [ OUTER ] JOIN  
RIGHT [ OUTER ] JOIN  
FULL [ OUTER ] JOIN  
CROSS JOIN
```



/ Criando as tabelas*/*

```
CREATE TABLE orientador (  
  id INT PRIMARY KEY,  
  nome VARCHAR(255)  
);
```

```
CREATE TABLE aluno (  
  matricula INT PRIMARY KEY,  
  nome VARCHAR(255),  
  orientador_id INT REFERENCES orientador(id)  
);
```

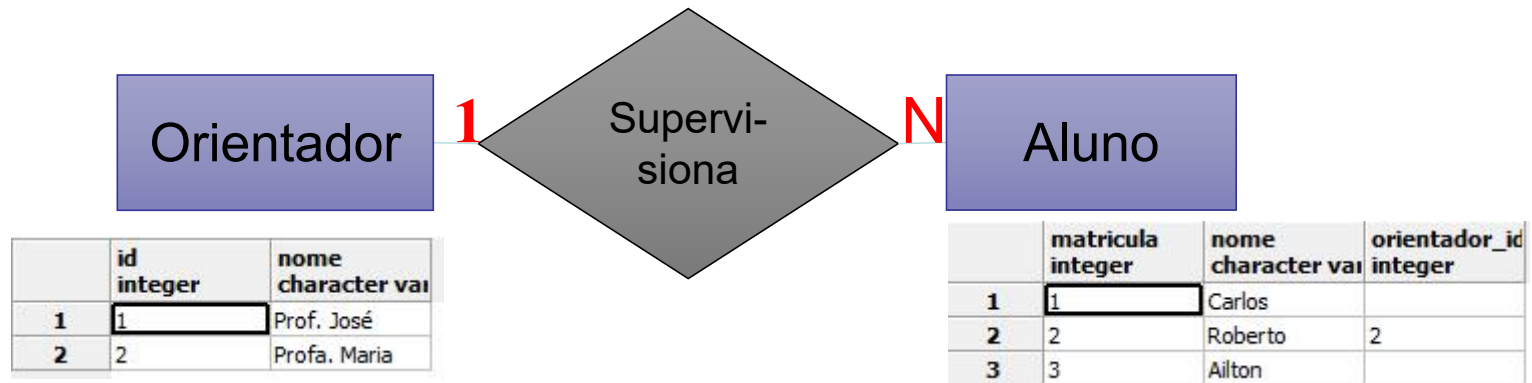


```
CREATE TABLE orientador (  
  id INT PRIMARY KEY,  
  nome VARCHAR(255)  
);
```

```
CREATE TABLE aluno (  
  matricula INT PRIMARY KEY,  
  nome VARCHAR(255),  
  orientador_id INT REFERENCES orientador(id)  
);
```

/ Povoando as tabelas*/*

```
INSERT INTO orientador VALUES (1,'Prof. José'), (2,'Profa. Maria');  
INSERT INTO aluno VALUES (1,'Carlos',NULL), (2,'Roberto',2),  
(3,'Ailton',NULL)
```

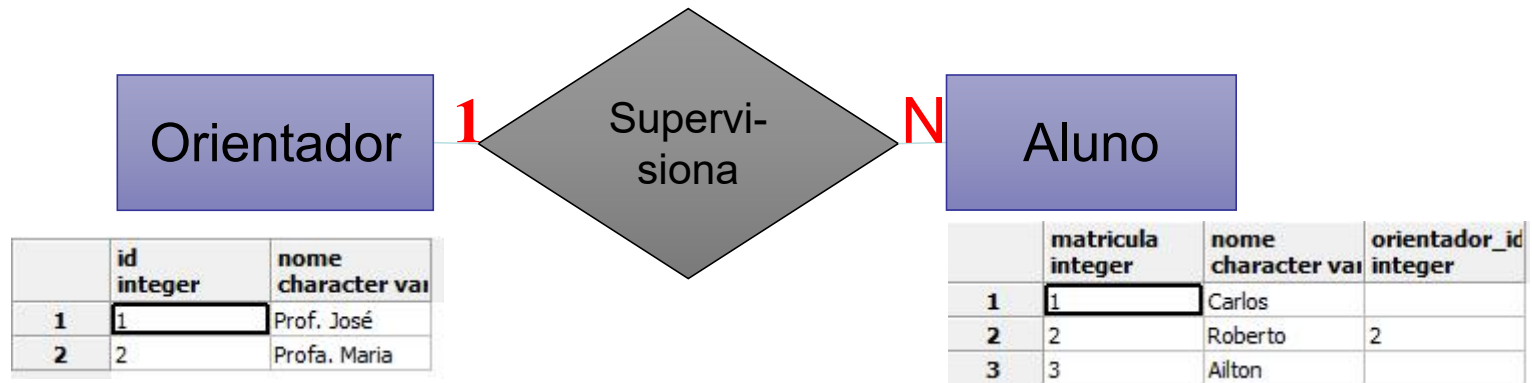


```

SELECT *
FROM orientador,aluno
WHERE aluno.orientador_id = orientador.id;

```

	id integer	nome character vai	matricula integer	nome character varying	orientador_id integer
1	2	Profa. Maria	2	Roberto	2

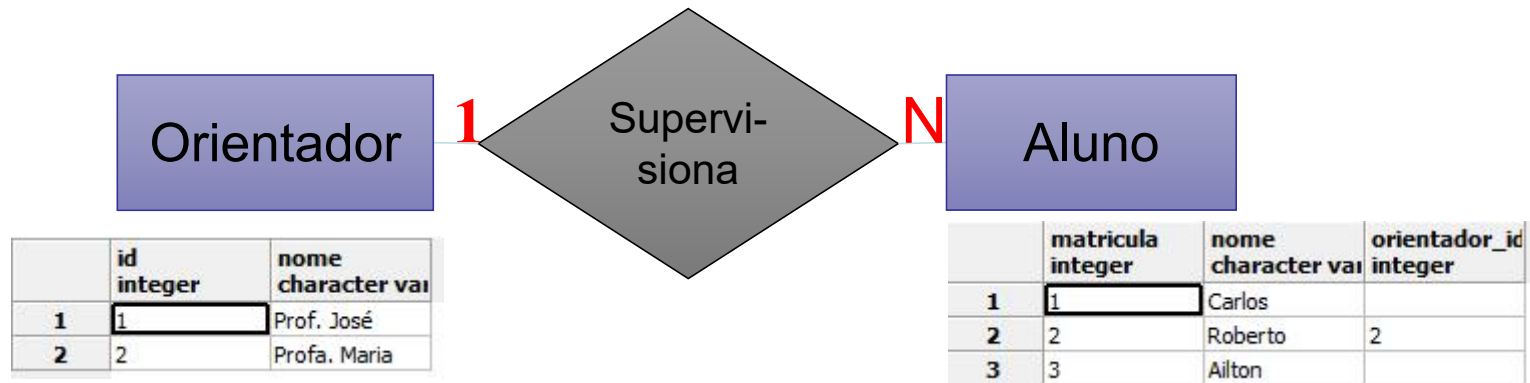


```

SELECT *
FROM orientador INNER JOIN aluno
ON aluno.orientador_id = orientador.id;

```

	id integer	nome character vai	matricula integer	nome character varying	orientador_id integer
1	2	Profa. Maria	2	Roberto	2

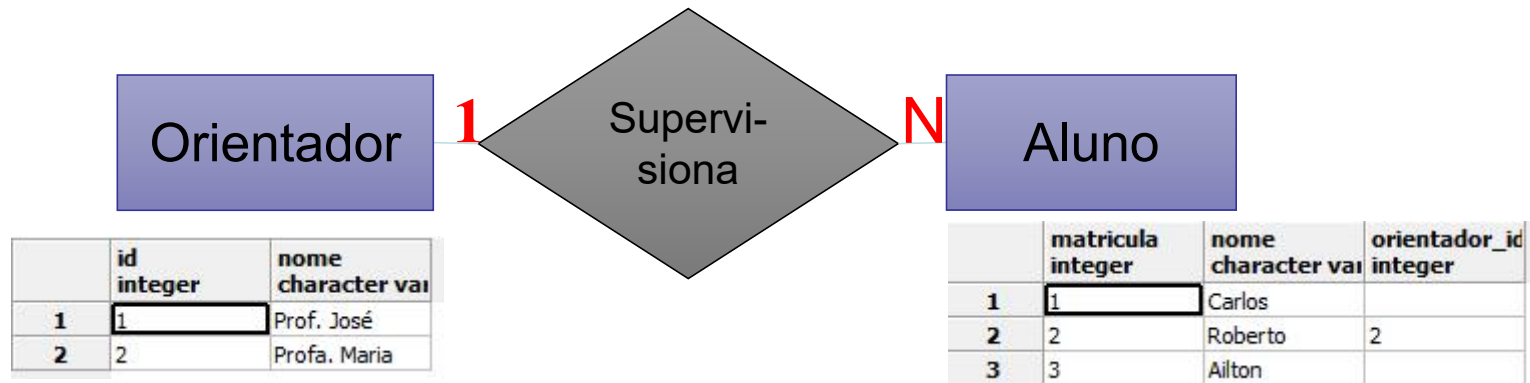


```

SELECT *
FROM orientador LEFT OUTER JOIN aluno
ON aluno.orientador_id = orientador.id;

```

	id integer	nome character vai	matricula integer	nome character varying	orientador_id integer
1	1	Prof. José			
2	2	Profa. Maria	2	Roberto	2

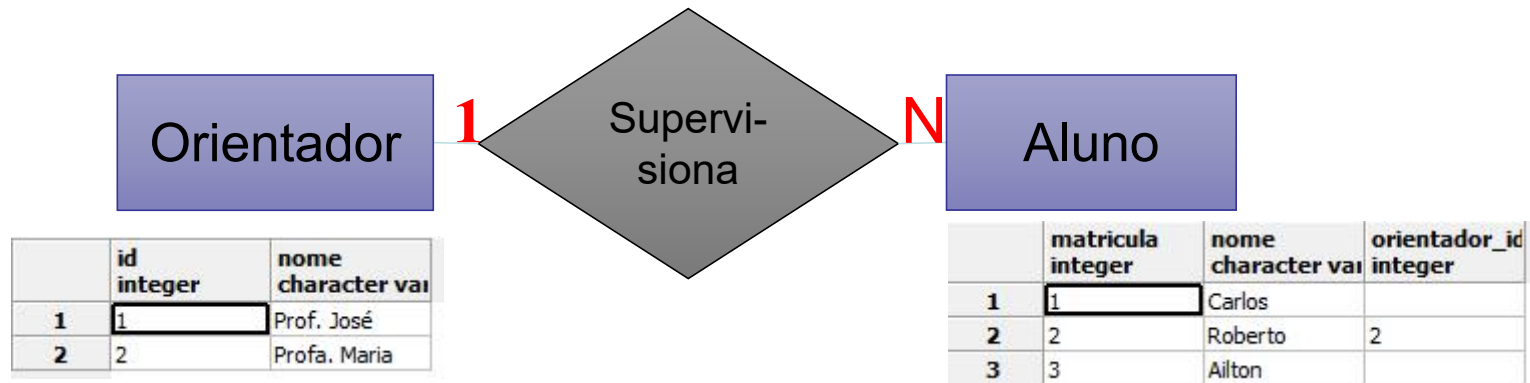


```

SELECT *
FROM orientador RIGHT OUTER JOIN aluno
ON aluno.orientador_id = orientador.id;

```

	id integer	nome character vai	matricula integer	nome character varying	orientador_id integer
1			1	Carlos	
2	2	Profa. Maria	2	Roberto	2
3			3	Ailton	

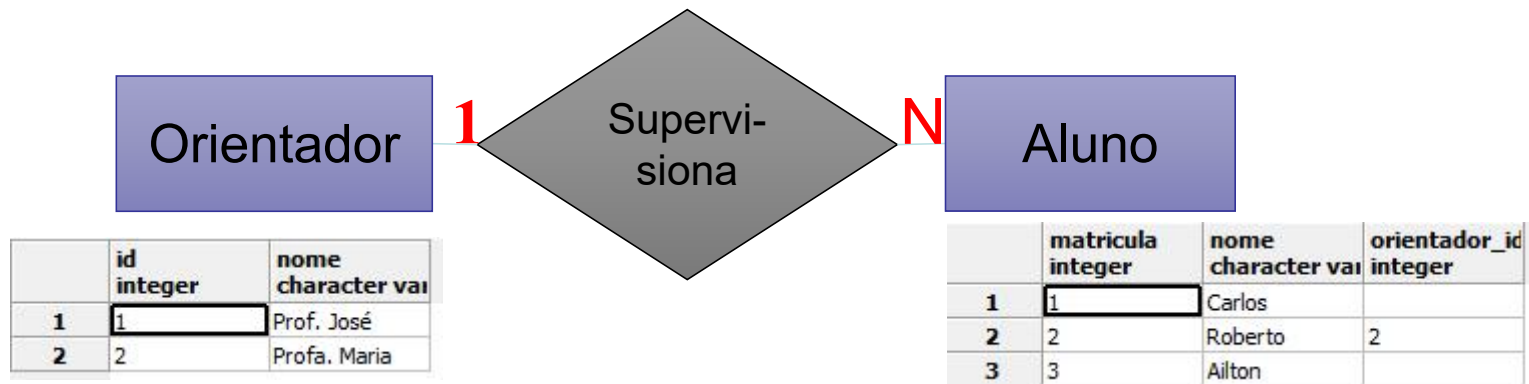


```

SELECT *
FROM orientador FULL OUTER JOIN aluno
ON aluno.orientador_id = orientador.id;

```

	id integer	nome character vai	matricula integer	nome character vai	orientador_id integer
1	1	Prof. José			
2	2	Profa. Maria	2	Roberto	2
3			1	Carlos	
4			3	Ailton	



SELECT *
FROM orientador CROSS JOIN aluno;

	id integer	nome character vai	matricula integer	nome character vai	orientador_id integer
1	1	Prof. José	1	Carlos	
2	1	Prof. José	2	Roberto	2
3	1	Prof. José	3	Ailton	
4	2	Profa. Maria	1	Carlos	
5	2	Profa. Maria	2	Roberto	2
6	2	Profa. Maria	3	Ailton	

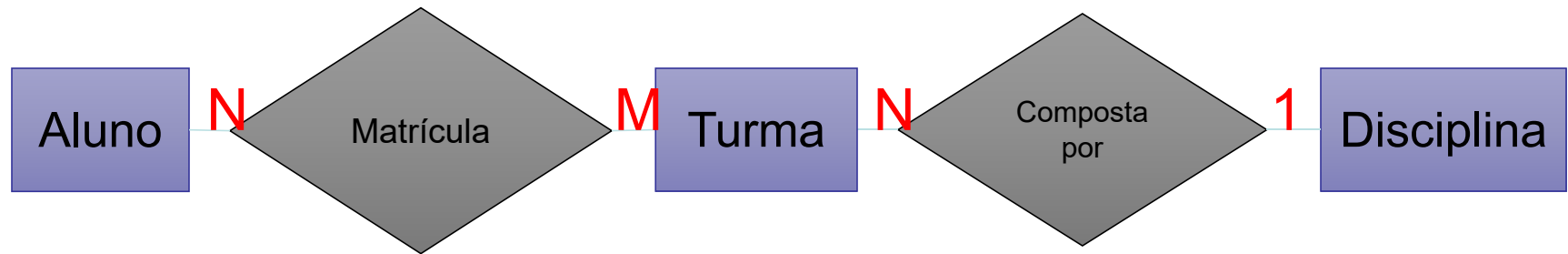
OUTER JOINS

- Variante da operação de JOIN que baseia-se em valores NULL. O resultado de um OUTER JOIN é igual a de um INNER JOIN mas com a inclusão das tuplas que não satisfazem a condição de JOIN.
- Três variantes:
 - LEFT OUTER JOIN
 - As tuplas da tabela à esquerda que não obedecem a condição do JOIN aparecem na resposta
 - RIGHT OUTER JOIN
 - As tuplas da tabela à direita que não obedecem a condição do JOIN aparecem na resposta
 - FULL OUTER JOIN
 - As tuplas das duas tabelas que não obedecem a condição do JOIN aparecem na resposta

Mais exemplos..

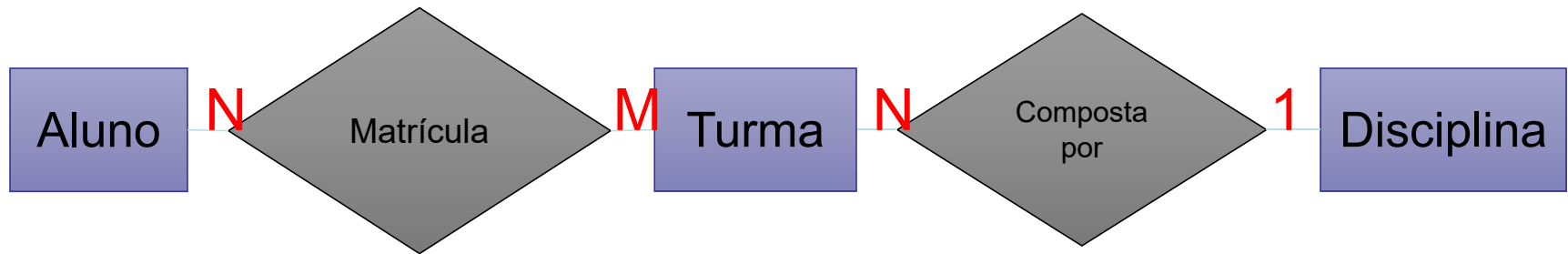
```
set search_path to universidade;
```

```
SELECT *  
FROM professor p  
    left join  
        (estudante e  
         inner join  
           faculdade f  
             on f.sigla = e.fac_est)  
    on p.id = e.tutor
```



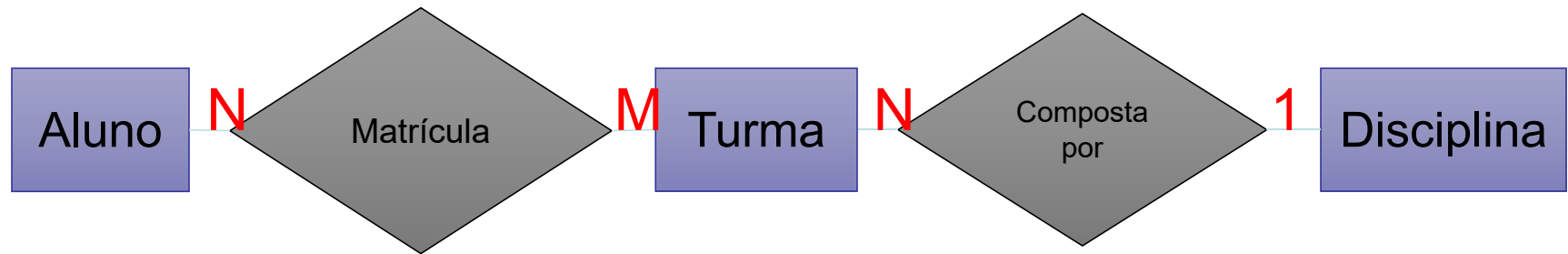
/ Liste as matriculas efetuadas em cada turma de cada disciplina */*

```
SELECT Aluno.NMat, Aluno.NOME, Matricula.CODIGOTURMA,  
Turma.SIGLA, Turma.NUMERO, Discip.NOME  
FROM Discip  
    INNER JOIN  
Turma ON Discip.SIGLA = Turma.SIGLA  
    INNER JOIN  
Matricula ON Turma.Codigo = Matricula.CODIGOTURMA  
    INNER JOIN  
Aluno ON Aluno.NMat = Matricula.NMat      ;
```



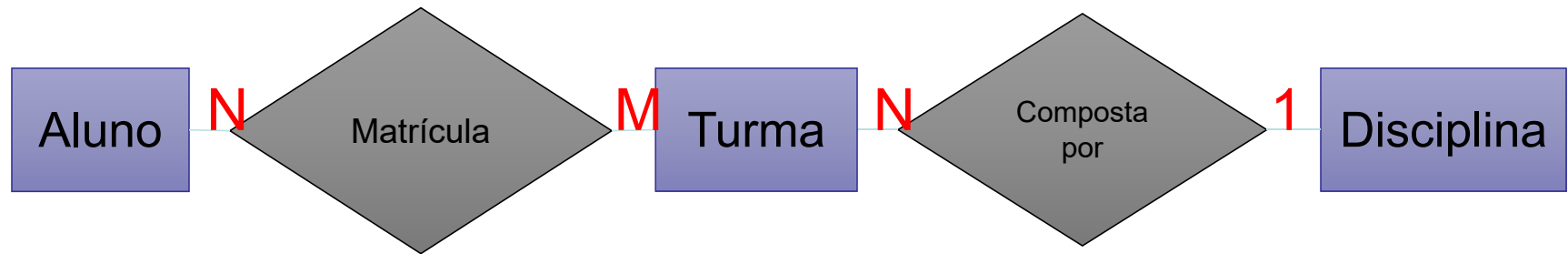
/ Liste as matriculas efetuadas em cada turma de cada disciplina */*

```
SELECT Aluno.NMat, Aluno.NOME,  
Matricula.CODIGOTURMA,  
Turma.SIGLA, Turma.NUMERO, Discip.NOME  
FROM Discip INNER JOIN  
    (Turma INNER JOIN  
        (Matricula INNER JOIN Aluno  
            ON Matricula.Nmat = Aluno.NMat )  
        ON Turma.Codigo = Matricula.CODIGOTURMA)  
    ON Discip.SIGLA = Turma.SIGLA;
```



/ Liste as matriculas efetuadas em cada turma de cada disciplina – alterando a ordem das junções*/*

```
SELECT Aluno.NMat, Aluno.NOME,  
Matricula.CODIGOTURMA,  
Turma.SIGLA, Turma.NUMERO, Discip.NOME  
FROM Aluno INNER JOIN  
    (Matricula INNER JOIN  
        (Turma INNER JOIN Discip  
            ON Turma.SIGLA = Discip.SIGLA)  
        ON Matricula.CODIGOTURMA = Turma.Codigo)  
    ON Aluno.NMat = Matricula.NMat;
```



/ Liste as matriculas efetuadas em cada turma de cada disciplina –
Comando equivalente usando WHERE*/*

```
SELECT A.NMat, A.NOME, M.CODIGOTURMA, T.SIGLA,  
        T.NUMERO, D.NOME  
FROM Aluno A, Matricula M, Turma T, DiscipD  
WHERE T.SIGLA = D.SIGLA AND  
        M.CODIGOTURMA = T.Codigo AND  
        A.NMat = M.NMat;
```

Union, Intersect e Except

- SQL fornece 3 construtores de manipulação de conjuntos
 - União → UNION
 - Interseção → INTERSECT
 - Diferença → EXCEPT

Exemplo

- C1: Encontre os nomes das disciplinas que são da FACOM ou possuem carga horaria maior que 60

```
SELECT D.nome  
FROM disciplina D  
WHERE D.fac_disc = 'FACOM' OR  
       D.ch > 60
```

Exemplo

- C1: Encontre os nomes das disciplinas que são da FACOM ou possuem carga horaria maior que 60

```
SELECT D.nome  
FROM disciplina D  
WHERE D.fac_disc = 'FACOM'  
UNION  
SELECT D.nome  
FROM disciplina D  
WHERE D.ch > 60
```

Exemplo

- C2: Encontre os nomes das disciplinas que tem são da FACOM e possuem carga horaria maior que 60

```
SELECT D.nome  
FROM disciplina D  
WHERE D.fac_disc = 'FACOM' AND  
       D.ch > 60
```

Exemplo

- C2: Encontre os nomes das disciplinas que são da FACOM e possuem carga horaria maior que 60

```
SELECT D.nome  
FROM disciplina D  
WHERE D.fac_disc = 'FACOM'  
INTERSECT  
SELECT D.nome  
FROM disciplina D  
WHERE D.ch > 60
```

Exemplo

- C3: Encontre os nomes das disciplinas que são da FACOM e que não possuem carga horaria maior que 60

```
SELECT D.nome  
FROM disciplina D  
WHERE D.fac_disc = 'FACOM'  
EXCEPT  
SELECT D.nome  
FROM disciplina D  
WHERE D.ch > 60
```

Exemplo

- C4: Encontre os nomes das disciplinas que tem como pré-requisito 'PP1' ou 'POO'

```
SELECT D.nome
FROM disciplina D, pre_requisito P, disciplina D2
WHERE D.codigo = P.cod_disc AND
      P.cod_pre = D2.codigo AND
      (D2.nome='PP1' OR D2.nome = 'POO')
```

Exemplo

- C5: Encontre os nomes das disciplinas que tem como pré-requisito 'PP 1' e 'POO'

```
SELECT D.nome
FROM disciplina D, disciplina D1, disciplina D2,
      pre_requisito P1, pre_requisito P2
WHERE D.codigo = P1.cod_disc AND
      D.codigo = P2.cod_disc AND
      P1.cod_pre = D1.codigo AND
      P2.cod_pre = D2.codigo AND
      D1.nome='PP1' AND D2.nome = 'POO'
```

Exemplo

- Consultas C4 e C5 podem ser melhor escritas usando UNION e INTERSECT

Exemplo

- C4

```
SELECT D.nome
  FROM disciplina D, disciplina D1,
        pre_requisito P
 WHERE D.codigo = P.cod_disc AND
        P.cod_pre = D1.codigo AND D1.nome='PP1'
UNION
SELECT D.nome
  FROM disciplina D, disciplina D1,
        pre_requisito P
 WHERE D.codigo = P.cod_disc AND
        P.cod_pre = D1.codigo AND D1.nome='POO'
```

Exemplo

- C5

```
SELECT D.nome
  FROM disciplina D, disciplina D1,
        pre_requisito P
 WHERE D.codigo = P.cod_disc AND
        P.cod_pre = D1.codigo AND D1.nome='PP1'
INTERSECT
SELECT D.nome
  FROM disciplina D, disciplina D1,
        pre_requisito P
 WHERE D.codigo = P.cod_disc AND
        P.cod_pre = D1.codigo AND D1.nome='POO'
```

Exemplo

A solução proposta para a consulta C5 usando INTERSECT contém um erro

Qual é esse erro?

Exemplo

- C6: Encontre os códigos das disciplinas que tem como pré-requisito 'PP 1', mas não 'POO'

```
SELECT P.cod_disc  
FROM disciplina D, pre_requisito P  
WHERE P.cod_pre = D.codigo AND D.nome='PP1'  
EXCEPT  
SELECT P.cod_disc  
FROM disciplina D, pre_requisito P  
WHERE P.cod_pre = D.codigo AND D.nome='POO'
```

- Observações

- UNION, INTERSECT E EXCEPT eliminam duplicatas

- Para reter as duplicatas use

- UNION ALL

- O número de cópias de uma linha no resultado é $m+n$, onde m e n são os números de vezes que a linha aparece nas duas partes da união

- INTERSECT ALL

- O número de cópias de uma linha no resultado é $\min(m,n)$

- EXCEPT ALL

- O número de cópias de uma linha no resultado é $m - n$ onde m corresponde à primeira relação

Aula 12 – SBD

SQL: Consultas - Agregação

Profa. Elaine Faria
UFU

Agradecimentos

- Agradecimentos
 - Ao professor Bruno Travençolo que gentilmente cedeu seus slides

Funções agregadas

- Funções agregadas calculam um único valor a partir de um conjunto de valores de entrada

Function	Argument Type	Return Type	Description
<code>array_agg(expression)</code>	any	array of the argument type	input values concatenated into an array
<code>avg(expression)</code>	smallint, int, bigint, real, double precision, numeric, or interval	numeric for any integer-type argument, double precision for a floating-point argument, otherwise the same as the argument data type	the average (arithmetic mean) of all input values
<code>bit_and(expression)</code>	smallint, int, bigint, or bit	same as argument data type	the bitwise AND of all non-null input values, or null if none
<code>bit_or(expression)</code>	smallint, int, bigint, or bit	same as argument data type	the bitwise OR of all non-null input values, or null if none
<code>bool_and(expression)</code>	bool	bool	true if all input values are true, otherwise false
<code>bool_or(expression)</code>	bool	bool	true if at least one input value is true, otherwise false
<code>count(*)</code>		bigint	number of input rows
<code>count(expression)</code>	any	bigint	number of input rows for which the value of <i>expression</i> is not null
<code>every(expression)</code>	bool	bool	equivalent to <code>bool_and</code>
<code>max(expression)</code>	any array, numeric, string, or date/time type	same as argument type	maximum value of <i>expression</i> across all input values
<code>min(expression)</code>	any array, numeric, string, or date/time type	same as argument type	minimum value of <i>expression</i> across all input values
<code>sum(expression)</code>	smallint, int, bigint, real, double precision, numeric, or interval	bigint for smallint or int arguments, numeric for bigint arguments, double precision for floating-point arguments, otherwise the same as the argument data type	sum of <i>expression</i> across all input values
<code>xmlagg(expression)</code>	xml	xml	concatenation of XML values (see also Section 9.14.1.7)

Mais funções: <http://www.postgresql.org/docs/8.4/static/functions-aggregate.html>

Comando SELECT

FUNÇÃO MIN() / MAX()

MIN() : Retorna o mínimo de um conjunto de valores contido em um campo especificado em uma consulta.

MAX() : Retorna o máximo de um conjunto de valores contido em um campo especificado em uma consulta.

EXEMPLO:

/ Verificar qual é a idade do aluno mais velho */*

```
SELECT MAX(Idade)  
FROM ALUNO;
```

/ Verificar o(s) nome(s) e a idade do(s) aluno(s) mais novo(s) */*

```
SELECT Nome, Idade  
FROM ALUNO  
WHERE Idade IN ( SELECT MIN(Idade)  
                  FROM ALUNO);
```

Comando SELECT

FUNÇÃO SUM()

SUM(): Retorna a soma de um conjunto de valores contido em um campo especificado em uma consulta. A função Sum ignora os registros que contenham campos Null.

EXEMPLO:

/ Calcular a quantidade de créditos oferecidos */*

```
SELECT SUM(NNCred) AS TotalCreditos  
FROM DISCIP;
```

/ Calcular o gasto da companhia com os salários*/*

```
SELECT SUM(salario) AS TotalSalario  
FROM empregado;
```

Comando SELECT

FUNÇÃO AVG()

AVG(): Calcula a média aritmética de um conjunto de valores contido em um campo especificado em uma consulta.

EXEMPLO:

/ Calcular a idade média de alunos cadastrados */*

```
SELECT AVG(Idade) AS Media  
FROM ALUNO
```

/ Calcular a média dos salários dos func. do departamento
2 */*

```
SELECT AVG(salario) AS Media  
FROM empregado  
WHERE dno=2;
```

Comando SELECT

FUNÇÃO COUNT()

COUNT(): Calcula o número de registros retornado por uma consulta. A função **COUNT** **não conta** registros que tenham campos *NULL*, exceto quando *expr* for o caractere curinga asterisco (*).

EXEMPLO:

NMAT	Nome	cpf_orientador
1	Carlos	222.222.222-22
2	José	
3	Maria	555.555.222-12
4	André	222.222.222-22

/ Contar quantos alunos estão cadastrados*/*

SELECT COUNT(*)

FROM alunos;

/ Contar quantos alunos fazem iniciação científica (IC) – Observe que registros onde cpf_orientador é NULL não são considerados*/*

SELECT COUNT(cpf_orientador)

FROM aluno;

/ Contar quantos orientadores de IC existem. Note que um orientador pode orientar mais de um aluno */*

SELECT COUNT(DISTINCT cpf_orientador)

FROM aluno;

COUNT()

- É possível usar a palavra chave DISTINCT com o count. Desta forma, os valores duplicados nas colunas não são contados.

```
/* Qual o número de estados  
atendidos pela empresa */  
SELECT COUNT (DISTINCT uf )  
FROM filiais
```

Resposta: 3

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG

Observações Funções Agregadas

- Uma função agregada não pode aparecer com um atributo da tabela

/ Verificar qual é a idade e o nome do aluno mais velho */*

```
SELECT nome, MAX(Idade)  
FROM ALUNO;
```

- Funções agregadas podem aparecer com atributos quando é utilizado o operador **GROUP BY**, explicado mais adiante

Agrupamentos

SELECT [ALL | DISTINCT [ON (expressão [, ...])]]
* | expressão [AS nome_de_saída] [, ...]
[FROM item_do_from [, ...]]
[WHERE condição]
[GROUP BY expressão [, ...]]
[HAVING condição [, ...]]
[{ UNION | INTERSECT | EXCEPT } [ALL] seleção]
[ORDER BY expressão [ASC | DESC | USING operador] [, ...]]
[LIMIT { contador | ALL }]
[OFFSET início]
[FOR UPDATE [OF nome_da_tabela [, ...]]]

Agrupamentos

- Podemos dividir o conjunto de tuplas de uma relação em grupos de acordo com algum critério, baseado nos valores dos atributos
 - Por exemplo, na tabela *dependentes* as tuplas podem ser agrupadas de acordo com o parentesco do dependente
 - Exemplo: pai, filho, irmã, irmão, tio, avó

Felipe	108	M	25/02/1997	Filho
Felipe	115	M	15/12/2004	Filho
Felipe	116	M	29/03/2001	Filho
Felipe	117	M	30/04/2000	Filho
Renato JR	1000	M	24/05/1968	Filho
Joaquim	1006	M	30/06/2005	Filho
Joao	101	M	11/10/1992	Filho
Paula	200	F	25/06/1975	Irmã
Maria	1201	F	20/06/1988	Irmã
Alberto	1202	M	15/02/1958	Irmão
João	1201	M	28/09/1992	Irmão
Tania	208	F	10/06/1950	Mãe
Maria_Inês	1203	F	15/11/1940	Mãe
Dolores Martins	704	F	20/05/1956	Mãe
Maria Junger	403	F	05/10/1954	Mãe
Jose Moreno	204	M	15/08/1970	Marido
Joao	207	M	05/05/1950	Pai

Agrupamentos

- Por exemplo, na tabela *dependentes* as tuplas podem ser agrupadas de acordo com o parentesco do dependente
- Exemplo: pai, filho, irmã, irmão, tio, avó

Felipe	108	M	25/02/1997	Filho
Felipe	115	M	15/12/2004	Filho
Felipe	116	M	29/03/2001	Filho
Felipe	117	M	30/04/2000	Filho
Renato JR	1000	M	24/05/1968	Filho
Joaquim	1006	M	30/06/2005	Filho
Joao	101	M	11/10/1992	Filho
Paula	200	F	25/06/1975	Irmã
Maria	1201	F	20/06/1988	Irmã
Alberto	1202	M	15/02/1958	Irmão
João	1201	M	28/09/1992	Irmão
Tania	208	F	10/06/1950	Mãe
Maria_Inês	1203	F	15/11/1940	Mãe
Dolores Martins	704	F	20/05/1956	Mãe
Maria Junger	403	F	05/10/1954	Mãe
Jose Moreno	204	M	15/08/1970	Marido
Joao	207	M	05/05/1950	Pai

Agrupamentos

/ parentesco de todos os dependentes*/*

```
SELECT parentesco  
FROM dependentes  
GROUP BY parentesco;
```



Marido
Irmã
Filho
Pai
Irmão
Mãe

- Observe que na cláusula **SELECT** só podem constar os atributos presentes no **GROUP BY**
- Isso faz sentido pois, por exemplo, existem 2 irmãs cadastradas, qual delas apareceria no resultado?

/ parentesco de todos os dependentes*/*

```
SELECT nome_dependente,parentesco  
FROM dependentes  
GROUP BY parentesco;
```



ERRO: coluna
"dependentes.nome_dependente"
deve aparecer na cláusula
GROUP BY ou ser utilizada em
uma função de agregação

Agrupamentos

- Mais de um atributo pode ser usado no agrupamento

/ agrupando as filiais por cidade/estado*/*

```
SELECT cidade, uf  
FROM filiais  
GROUP BY cidade, uf
```

Tabela Filiais:

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG

Resultado da
Consulta com
GROUP BY



Rio de Janeiro	RJ
São Paulo	SP
Uberlândia	MG
Belo Horizonte	MG
Angra dos Reis	RJ

Funções agregadas + Agrupamentos

- As funções agregadas (e.g., COUNT, MIN, MAX, AVG) podem ser usadas para cálculos com subgrupos de tuplas definidos pela cláusula GROUP BY

/ Qual o número de filiais por estado */*

SELECT uf, **COUNT**(uf) -- poderia ser **COUNT**(*)

FROM filiais

GROUP BY uf

Tabela Filiais:

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG

Resultado da
Consulta com
GROUP BYs



	estado character(2)	count bigint
1	RJ	3
2	SP	3
3	MG	4

Agrupamentos

- Inserindo uma nova filial:
Internet

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG
Internet		

/ agrupando as filiais por estado*/*

```
SELECT UF  
FROM filiais  
GROUP BY UF
```

Resultado da
Consulta com
GROUP BY



	uf character(2)
1	
2	MG
3	RJ
4	SP



Campos Nulos
também são
agrupados

Agrupamentos

- Inserindo uma nova filial:
Internet

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG
Internet		

/ agrupando as filiais por estado e
contando o número por estado*/*

```
SELECT UF, COUNT(*)  
FROM filiais  
GROUP BY UF
```

Resultado da
Consulta com
GROUP BY



	uf character(2)	count bigint
1		1
2	MG	4
3	RJ	3
4	SP	3



Campos Nulos
também são
agrupados e
contados

Agrupamentos

- Inserindo uma nova filial:
Internet

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG
Internet		

/ agrupando as filiais por estado e
contando o número por estado*/*

```
SELECT UF, COUNT(UF)  
FROM filiais  
GROUP BY UF
```

Resultado da
Consulta com
GROUP BY



	uf character(2)	count bigint
1		0
2	MG	4
3	RJ	3
4	SP	3



Campos Nulos
também são
agrupados

Agrupamentos

- Inserindo uma nova filial:
Internet

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG
Internet		

/ agrupando as filiais por estado e
contando o número por estado*/*

```
SELECT UF, COUNT(Nome)
FROM filiais
GROUP BY UF
```

Resultado da
Consulta com
GROUP BY



	uf character(2)	count bigint
1		1
2	MG	4
3	RJ	3
4	SP	3



Campos Nulos
também são
agrupados e
contados

Agrupamentos

- Inserindo uma nova filial:
Internet

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG
Internet		

/ agrupando as filiais por estado e
contando o número por estado*/*

```
SELECT UF, COUNT(Cidade)  
FROM filiais  
GROUP BY UF
```

Resultado da
Consulta com
GROUP BY



	uf character(2)	count bigint
1		0
2	MG	4
3	RJ	3
4	SP	3



Campos Nulos
também são
agrupados

Agrupamentos

- Inserindo uma nova filial:
Internet

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG
Internet		

/ agrupando as filiais por estado e
contando o número por estado*/*

```
SELECT UF, COUNT(DISTINCT Cidade)  
FROM filiais  
GROUP BY UF
```



	uf character(2)	count bigint
1	MG	2
2	RJ	2
3	SP	1
4		0

Cada cidade
só é contada
uma vez



Campos Nulos
também são
agrupados

Funções agregadas + Agrupamentos

- As funções agregadas (e.g., COUNT, MIN, MAX, AVG) podem ser usadas para cálculos com subgrupos de tuplas definidos pela cláusula GROUP BY

/ Qual o número de filiais por estado */*

SELECT uf, **COUNT**(uf) -- poderia ser **COUNT**(*)

FROM filiais

GROUP BY uf

Tabela Filiais:

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG

Resultado da
Consulta com
GROUP BY

	estado character(2)	count bigint
1	RJ	3
2	SP	3
3	MG	4

Comando SELECT

CLÁUSULA HAVING

- **HAVING:** é semelhante à cláusula WHERE. HAVING elimina tuplas *agrupadas* que não satisfazem a uma determinada condição.
- Diferença com WHERE: WHERE filtra tuplas *individuais* antes da aplicação do GROUP BY, enquanto HAVING filtra grupo de tuplas criadas por GROUP BY. As condições de filtragem do HAVING devem ser feitas baseando-se nos atributos agrupados por GROUP BY

EXEMPLO:

/ Listar os estados com
mais de 3 filiais*/*

```
SELECT uf  
FROM filiais  
GROUP BY uf  
HAVING COUNT(*) >3
```

Tabela Filiais:

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG



	estado character(2)
1	MG

Comando SELECT

CLÁUSULA HAVING

EXEMPLO:

/ Listar os estados com*

mais de 3 filiais renomeando o atributo de saída como nfiliais/*

SELECT UF, Count(*) AS nfiliais

FROM filiais

GROUP BY UF

HAVING COUNT(*) >3

Tabela Filiais:

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG



	uf character(2)	nfiliais bigint
1	MG	4

Comando SELECT

CLÁUSULA HAVING

EXEMPLO:

/ Listar todos os estados, exceto SP, com
mais de 3 filiais renomeando o atributo de saída como nfiliais*/*

SELECT UF, Count(*) AS nfiliais

FROM filiais

WHERE UF <> 'SP'

GROUP BY UF

HAVING COUNT(*) >3

Tabela Filiais:

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG



	uf character(2)	nfiliais bigint
1	MG	4

Comando SELECT

CLÁUSULA HAVING

EXEMPLO:

/ Listar todos os estados, exceto SP, com
mais de 3 filiais renomeando o atributo de saída como nfiliais*/*

SELECT UF, Count(*) AS nfiliais

FROM filiais

GROUP BY UF

HAVING COUNT(*) >3

AND UF <> 'SP'

Qual a
diferença entre
esta consulta e
a anterior?

Tabela Filiais:

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ
Barra	Rio de Janeiro	RJ
Angra	Angra dos Reis	RJ
Pampulha	Belo Horizonte	MG
Glória	Belo Horizonte	MG
Tibery	Uberlândia	MG
Centro	Uberlândia	MG



	uf character(2)	nfiliais bigint
1	MG	4

Comando SELECT

CLÁUSULA HAVING

EXEMPLO: Erro comum

/ Listar os estados com
mais de 3 filiais*/*

SELECT UF, Count(*) AS nfiliais

FROM filiais

GROUP BY UF

HAVING nfiliais >3

Tabela Filiais:

Nome	Cidade	UF
Brás	São Paulo	SP
Limão	São Paulo	SP
Penha	São Paulo	SP
Tijuca	Rio de Janeiro	RJ



```
ERRO:  coluna "nfiliais" não existe  
LINE 4: HAVING nfiliais >3
```

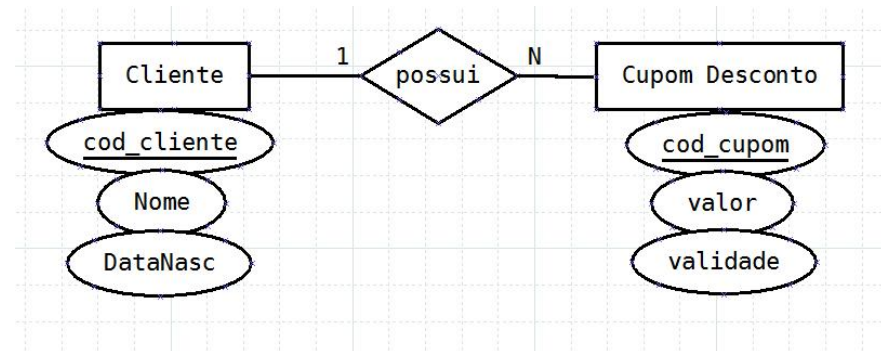
^

Centro	Uberlândia	MG
--------	------------	----

Exemplo

```
SELECT * FROM cliente;
```

cod_cliente	nome	data_nasc
1	Maria	2000-01-05
2	Ana	1998-05-03
3	Carlos	1990-02-02



```
SELECT * FROM cupom_desconto;
```

num_cupom	valordesconto	validade	cod_cliente
1	0.3	2020-10-20	1
2	0.4	2020-10-20	1
3	0.15	2020-10-20	1
4	0.15	2020-10-20	2
5	0.15	2020-10-20	2

```
SELECT * FROM cliente NATURAL JOIN cupom_desconto;
```

cod_cliente	nome	data_nasc	num_cupom	valordesconto	validade
1	Maria	2000-01-05	1	0.3	2020-10-20
1	Maria	2000-01-05	2	0.4	2020-10-20
1	Maria	2000-01-05	3	0.15	2020-10-20
2	Ana	1998-05-03	4	0.15	2020-10-20
2	Ana	1998-05-03	5	0.15	2020-10-20

Atributos no SELECT com GROUP BY

cod_cliente	nome	data_nasc	num_cupom	valordesconto	validade
1	Maria	2000-01-05	1	0.3	2020-10-20
1	Maria	2000-01-05	2	0.4	2020-10-20
1	Maria	2000-01-05	3	0.15	2020-10-20
2	Ana	1998-05-03	4	0.15	2020-10-20
2	Ana	1998-05-03	5	0.15	2020-10-20

EXEMPLO:

/ Listar a quantidade de cupons por cliente */*

SELECT cliente.cod_cliente, **COUNT**(num_cupom) **AS**

qte_cupom

FROM cliente **INNER JOIN** cupom_desconto

ON cliente.cod_cliente = cupom_desconto.cod_cliente

GROUP BY cliente.cod_cliente

cod_cliente	qte_cupom
1	3
2	2

Atributos no SELECT com GROUP BY

cod_cliente	nome	data_nasc	num_cupom	valordesconto	validade
1	Maria	2000-01-05	1	0.3	2020-10-20
1	Maria	2000-01-05	2	0.4	2020-10-20
1	Maria	2000-01-05	3	0.15	2020-10-20
2	Ana	1998-05-03	5	0.15	2020-10-20

Somente atributos que estão no GROUP BY podem aparecer no SELECT

EXEMPLO:

/ Listar a quantidade de cupons por cliente */*

```
SELECT cliente.cod_cliente, COUNT(num_cupom) AS  
qte_cupom  
FROM cliente INNER JOIN cupom_desconto  
ON cliente.cod_cliente = cupom_desconto.cod_cliente  
GROUP BY cliente.cod_cliente
```

cod_cliente	qte_cupom
1	3
2	2

Atributos no SELECT com GROUP BY

cod_cliente	nome	data_nasc	num_cupom	valordesconto	validade
1	Maria	2000-01-05	1	0.3	2020-10-20
1	Maria	2000-01-05	2	0.4	2020-10-20
1	Maria	2000-01-05	3	0.15	2020-10-20
2	Ana	1998-05-03	4	0.15	2020-10-20

Os outros atributos podem constar dentro de funções de agregação

EXEMPLO:

/ Listar a quantidade de cupons por cliente */*

```
SELECT cliente.cod_cliente, COUNT(num_cupom) AS  
qte_cupom  
FROM cliente INNER JOIN cupom_desconto  
ON cliente.cod_cliente = cupom_desconto.cod_cliente  
GROUP BY cliente.cod_cliente
```

cod_cliente	qte_cupom
1	3
2	2

Atributos no SELECT com GROUP BY

cod_cliente	nome	data_nasc	num_cupom	valordesconto	validade
1	Maria	2000-01-05	1	0.3	2020-10-20

Somente atributos que estão no GROUP BY podem aparecer no SELECT.

No exemplo abaixo, mesmo que

`cupom_desconto.cod_cliente = cliente.cod_cliente`

um erro ocorrerá

EXEMPLO:

/ Listar a quantidade de cupons por cliente */*

`SELECT cupom_desconto.cod_cliente, COUNT(num_cupom) AS
qte_cupom`

`FROM cliente INNER JOIN cupom_desconto`

`ON cliente.cod_cliente = cupom_desconto.cod_cliente`

`GROUP BY cliente.cod_cliente`

ERROR: column "cupom_desconto.cod_cliente" must appear in the GROUP BY clause or be used in an aggregate function

LINE 1: SELECT cupom_desconto.cod_cliente, COUNT(num_cupom) AS
qte_c...

Atributos no SELECT com GROUP BY

cod_cliente	nome	data_nasc	num_cupom	valordesconto	validade
1	Maria	2000-01-05	1	0.3	2020-10-20
1	Maria	2000-01-05	2	0.4	2020-10-20
					20-10-20
					20-10-20
					20-10-20

A partir do SQL 1999, é possível colocar outros atributos que não estão agrupados no SELECT desde que o atributo usado no agrupamento seja uma chave primária

EXEMPLO:

/ Listar a quantidade de cupons por cliente */*

```
SELECT cliente.cod_cliente, cliente.nome, COUNT(num_cupom) AS  
qte_cupom  
FROM cliente INNER JOIN cupom_desconto  
ON cliente.cod_cliente = cupom_desconto.cod_cliente  
GROUP BY cliente.cod_cliente
```

PRIMARY KEY
no agrupamento

cod_cliente	nome	qte_cupom
1	Maria	3
2	Ana	2

Consultas aninhadas

EXISTS e NOT EXISTS

- EXISTS (subconsulta)
 - retorna TRUE se existir ao menos uma tupla no resultado da subconsulta
- NOT EXISTS (subconsulta)
 - retorna TRUE se a subconsulta retornar um conjunto vazio (zero tuplas)

- Exemplo:

```
SELECT col1
FROM tab1
WHERE EXISTS (SELECT 1
              FROM tab2
              WHERE col2 = tab1.col2);
```

Consultas aninhadas

EXISTS e NOT EXISTS

- O otimizador do SGBD pode executar a consulta apenas até determinar que tem ao menos uma tupla como resultado
- Como o resultado depende apenas de se há tuplas no resultado, uma convenção comum é escrever as cláusulas exists na forma:
 - ... EXISTS(SELECT 1 FROM... WHERE ...)

Exemplo

```
SELECT * FROM produto;
```

cod_produto	nome	valor
1	Queijo	15.00
2	Goiabada	8.00
3	Doce de leite	7.00

```
SELECT * FROM cliente;
```

cod_cliente	nome	data_nasc
1	Maria	2000-01-05
2	Ana	1998-05-03
3	Carlos	1990-02-02

```
SELECT * FROM compra;
```

cod_cliente	cod_produto	datahora
1	1	2016-10-20 00:00:00
1	2	2016-10-20 00:00:00
1	3	2016-10-20 00:00:00
2	1	2016-10-20 00:00:00
2	2	2016-10-21 00:00:00
2	2	2016-10-22 00:00:00

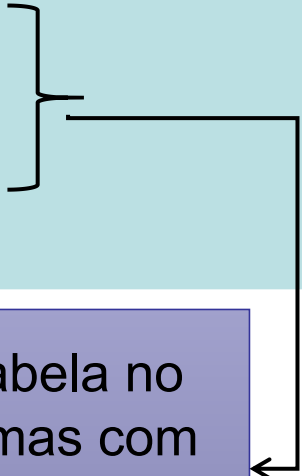
-- Mostrar os clientes que fizeram compras

```
SELECT *  
FROM cliente  
WHERE EXISTS  
    ( SELECT 1  
      FROM compra  
      WHERE compra.cod_cliente =  
            cliente.cod_cliente)
```

Exemplo

-- Mostrar os clientes que fizeram compras

```
SELECT *  
FROM cliente  
WHERE EXISTS  
    ( SELECT 1  
      FROM compra  
      WHERE compra.cod_cliente =  
cliente.cod_cliente)
```



Subconsulta com uma tabela no FROM (tabela compra) mas com comando no WHERE que usa uma tabela da consulta externa

Exemplo

-- *Mostrar os clientes que fizeram compras*

Para cada tupla da consulta externa, a consulta interna é feita

```
SELECT *  
FROM cliente  
WHERE EXISTS  
    ( SELECT 1  
      FROM compra  
      WHERE compra.cod_cliente = cliente.cod_cliente)
```

cod_cliente	nome	data_nasc
1	Maria	2000-01-05
2	Ana	1998-05-03
3	Carlos	1990-02-02

Exemplo

-- Mostrar os clientes que fizeram compras

```
SELECT *  
FROM cliente  
WHERE EXISTS  
    ( SELECT 1  
      FROM compra  
      WHERE compra.cod_cliente = cliente.cod_cliente)
```

cod_cliente	nome	data_nasc
1	Maria	2000-01-05
2	Ana	1998-05-03
3	Carlos	1990-02-02

Como temos 3 clientes, a subconsulta é executada 3 vezes

Exemplo

-- Mostrar os clientes que fizeram compras

```
SELECT *  
FROM cliente  
WHERE EXISTS  
    ( SELECT 1  
      FROM compra  
      WHERE compra.cod_cliente = cliente.cod_cliente)
```

cod_cliente	nome	data_nasc
1	Maria	2000-01-05
2	Ana	1998-05-03
3	Carlos	1990-02-02



Para cada execução da subconsulta o **EXISTS** verifica se a subconsulta retornou algum resultado

Exemplo

```
SELECT * FROM produto;
```

cod_produto	nome	valor
1	Queijo	15.00
2	Goiabada	8.00
3	Doce de leite	7.00

```
SELECT * FROM cliente;
```

cod_cliente	nome	data_nasc
1	Maria	2000-01-05
2	Ana	1998-05-03
3	Carlos	1990-02-02

```
SELECT * FROM compra;
```

cod_cliente	cod_produto	datahora
1	1	2016-10-20 00:00:00
1	2	2016-10-20 00:00:00
1	3	2016-10-20 00:00:00
2	1	2016-10-20 00:00:00
2	2	2016-10-21 00:00:00
2	2	2016-10-22 00:00:00

-- Mostrar os clientes que NÃO fizeram compras

```
SELECT *  
FROM cliente  
WHERE NOT EXISTS  
    ( SELECT 1  
      FROM compra  
      WHERE compra.cod_cliente =  
            cliente.cod_cliente)
```

Exemplo

```
SELECT * FROM produto;
```

cod_produto	nome	valor
1	Queijo	15.00
2	Goiabada	8.00
3	Doce de leite	7.00

*-- Mostrar os clientes que compraram
todos os produtos*

```
SELECT * FROM cliente;
```

cod_cliente	nome	data_nasc
1	Maria	2000-01-05
2	Ana	1998-05-03
3	Carlos	1990-02-02

```
SELECT * FROM compra;
```

cod_cliente	cod_produto	datahora
1	1	2016-10-20 00:00:00
1	2	2016-10-20 00:00:00
1	3	2016-10-20 00:00:00
2	1	2016-10-20 00:00:00
2	2	2016-10-21 00:00:00
2	2	2016-10-22 00:00:00

Exemplo

```
SELECT * FROM produto;
```

cod_produto	nome	valor
1	Queijo	15.00
2	Goiabada	8.00
3	Doce de leite	7.00

```
SELECT * FROM cliente;
```

cod_cliente	nome	data_nasc
1	Maria	2000-01-05
2	Ana	1998-05-03
3	Carlos	1990-02-02

```
SELECT * FROM compra;
```

cod_cliente	cod_produto	data
1	1	2016-10-2
1	2	2016-10-2
1	3	2016-10-2
2	1	2016-10-2
2	2	2016-10-2
2	2	2016-10-2

*-- Mostrar os clientes que compraram
todos os produtos*

```
SELECT *  
FROM cliente  
WHERE NOT EXISTS
```

```
(SELECT cod_produto  
FROM produto
```

```
EXCEPT
```

```
SELECT cod_produto  
FROM compra  
WHERE compra.cod_cliente =  
cliente.cod_cliente
```

```
)
```


-- *Mostrar os clientes que compraram **todos** os produtos*

```
SELECT *  
FROM cliente  
WHERE NOT EXISTS  
  
    (SELECT cod_produto  
     FROM produto  
  
     EXCEPT  
  
     SELECT cod_produto  
     FROM compra  
     WHERE compra.cod_cliente = cliente.cod_cliente  
    )
```

Subconsulta
correlacionada: ele é
executada para cada
cliente existente na
base

-- *Mostrar os clientes que compraram **todos** os produtos*

```
SELECT *  
FROM cliente  
WHERE NOT EXISTS  
  
  (SELECT cod_produto  
   FROM produto  
  
   EXCEPT  
  
   SELECT cod_produto  
   FROM compra  
   WHERE compra.cod_cliente = cliente.cod_cliente  
  )
```

Todos os produtos

-- *Mostrar os clientes que compraram **todos** os produtos*

```
SELECT *  
FROM cliente  
WHERE NOT EXISTS  
  
  (SELECT cod_produto  
   FROM produto  
  
   EXCEPT  
  
   SELECT cod_produto  
   FROM compra  
   WHERE compra.cod_cliente = cliente.cod_cliente  
  )
```

Produtos para
o cliente “atual”

-- *Mostrar os clientes que compraram **todos** os produtos*

```
SELECT *  
FROM cliente  
WHERE NOT EXISTS  
  
  (SELECT cod_produto  
   FROM produto  
  
   EXCEPT  
   )  
  SELECT cod_produto  
  FROM compra  
  WHERE compra.cod_cliente = cliente.cod_cliente  
)
```

Ao subtrair o conjunto de todos os produtos dos produtos que o cliente comprou, teremos uma resposta vazia caso ele tenha comprado todos

-- *Mostrar os clientes que compraram **todos** os produtos*

```
SELECT *  
FROM cliente  
WHERE NOT EXISTS }  
  
    (SELECT cod_produto  
     FROM produto  
  
     EXCEPT  
  
     SELECT cod_produto  
     FROM compra  
     WHERE compra.cod_cliente = cliente.cod_cliente  
    )
```

NOT EXISTS retornará TRUE caso a subconsulta seja vazia, ou seja, caso o cliente tenha comprado todos os produtos

Comando SELECT

Consultas aninhadas correlacionadas

- Consultas aninhadas correlacionadas ocorrem quando o resultado da subconsulta (consulta interna) muda de acordo com a tupla que está sendo avaliada na consulta externa.
 - Cuidado: isso pode ser muito lento, pois a subconsulta é reavaliada para cada linha da consulta externa. Se houver forma de evitar isso, sua consulta pode ser mais rápida

Comando SELECT

Itens do FROM

- É possível realizar consultas sobre os resultados obtidos em outras consultas. Isso pode ser feito adicionando a consulta na cláusula FROM
 - Relembrando a sintaxe

onde item_do_from pode ser um entre:

```
[ ONLY ] nome_da_tabela [ * ] [ [ AS ] alias [ ( alias_de_coluna [, ...] ) ] ]  
( seleção ) [ AS ] alias [ ( alias_de_coluna [, ...] ) ]  
nome_da_função ( [ argumento [, ...] ] ) [ AS ] alias [ ( alias_de_coluna [, ...]  
| definição_de_coluna [, ...] ) ]  
nome_da_função ( [ argumento [, ...] ] ) AS ( definição_de_coluna [, ...] )  
item_do_from [ NATURAL ] tipo_de_junção item_do_from [ ON  
condição_de_junção  
| USING ( coluna_de_junção [, ...] ) ]
```

Comando SELECT

Itens do FROM

- Exemplo

/ Listar os estados com 3 filiais (sem usar having)*/*

```
SELECT *  
FROM (  
  SELECT estado, count(*) as nfiliais  
  FROM filiais  
  GROUP BY estado) AS t  
WHERE t.nfiliais = 3
```


Referências

- Slides adaptados da aula da Profa. Josiane M. Bueno (*in memoriam*)
- Slides Prof. Humberto Razente
- Slides Profa. Sandra de Amo

Aula 13 – SBD

SQL: Consultas - Revisão

Profa. Elaine Faria
UFU

Sobre os exemplos

- Todos os exemplos usarão a seguinte base

Marinheiros(id_marin: integer, nome_marin: string, avaliação: integer, idade: real)

Barcos(id_barco: integer, nome_barco: string, cor: string)

Reservas(id_marin: integer, id_barco: integer, dia: date)

id_marin	nome_marin	avaliação	idade
22	Daniel	7	45
29	Bruno	1	33
31	Lucas	8	55,5
32	Alceu	8	25,5
58	Raul	10	35
64	Homero	7	35
71	Ze	10	16
74	Honorato	9	35
85	Americo	3	25,5
95	Bob	3	63,5

Instância M3 de Marinheiros

id_marin	id_barco	dia
22	101	10/10/96
22	102	10/10/98
22	103	10/08/98
22	104	10/07/98
31	102	11/10/98
31	103	11/06/98
31	104	11/12/98
64	101	09/05/98
64	102	09/05/98
74	103	09/08/98

Instância R2 de Reservas

id_barco	nome_barco	cor
101	Interlagos	azul
102	Interlagos	Vermelho
103	Clipper	Verde
104	Marinha	Vermelho

Instância B1 de Barcos

Exercício

- C1: Encontre os nomes de marinheiros que reservaram o barco 103.

```
SELECT M.nome_marin  
FROM Marinheiros M, Reservas R  
WHERE M.id_marin=R.id_marin AND  
       R.id_barco = 103
```

Exercício

- C2: Encontre os nomes dos marinheiros que reservaram um barco vermelho

```
SELECT M.nome_marin  
FROM Marinheiros M, Reserva R, Barcos B  
WHERE M.id_marin = R.id_marin AND  
R.id_barco = B.id_barco AND B.cor='vermelho'
```

Exercício

- C3: Compute incrementos das avaliações de pessoas que manobraram dois barcos diferentes no mesmo dia

```
SELECT M.nome_marin, M.avaliacao+1 AS avaliação
FROM Marinheiros M, Reservas R1, Reservas R2
WHERE M.id_marin = R1.id_marin AND
      M.id_marin = R2.id_marin AND
      R1.dia = R2.dia AND
      R1.id_barco <> R2.id_barco
```

Exercício

- C4: Encontre os nomes dos marinheiros que reservaram um barco vermelho ou um barco verde

```
SELECT M.nome_marin  
FROM Marinheiros M, Reservas R, Barcos B  
WHERE M.id_marin=R.id_marin AND  
      R.id_barco = B.id_barco AND  
      (B.cor='vermelho' OR B.cor='verde')
```


Exercício

- C4

```
SELECT M.nome_marin
FROM Marinheiros M, Reservas R, Barcos B
WHERE M.id_marin=R.id_marin AND
       R.id_barco = B.id_barco AND B.cor = 'vermelho'
UNION
SELECT M2.nome_marin
FROM Marinheiros M2, Barcos B2, Reservas R2
WHERE M2.id_marin=R2.id_marin AND
       R2.id_barcos=B2.id_barco AND B2.cor='verde'
```

Exercício

- C5: Encontre os nomes dos marinheiros que reservaram um barco vermelho e um barco verde

```
SELECT M.nome_marin
FROM Marinheiros M, Reservas R1, Barcos B1,
     Reservas R2, Barcos B2
WHERE M.id_marin = R1.id_marin AND
      R1.id_barco = B1.id_barco AND
      M.id_marin = R2.id_marin AND
      R2.id_barco = B2.id_barco
      B1.cor='vermelho' AND B2.cor = 'verde'
```

Exercício

- C5

```
SELECT M.nome_marin
FROM Marinheiros M, Reservas R, Barcos B
WHERE M.id_marin=R.id_marin AND
       R.id_barco = B.id_barco AND B.cor = 'vermelho'
INTERSECT
SELECT M2.nome_marin
FROM Marinheiros M2, Barcos B2, Reservas R2
WHERE M2.id_marin=R2.id_marin AND
       R2.id_barcos=B2.id_barco AND B2.cor='verde'
```

Exercício

- C5 usando INTERSECT

```
SELECT M.nome_marin
FROM Marinheiros M
WHERE M.id_marin IN ((SELECT R.id_marin
                      FROM Barcos B, Reservas R
                      WHERE R.id_barco = B.id_barco
                      AND B.cor = 'vermelho')
INTERSECT
      (SELECT R2.id_marin
       FROM Barcos B2, Reservas R2
       WHERE R2.id_barco = B2.id_barco
       AND B2.cor='verde'))
```

Exercício

- C7: Encontre os nomes dos marinheiros que não reservaram um barco vermelho

```
SELECT M.nome_marin  
FROM Marinheiros M  
WHERE M.id_marin NOT IN (SELECT R.id_marin  
                          FROM Reservas R  
                          WHERE R.id_barco IN (  
                            SELECT B.id_barco  
                            FROM Barcos B  
                            WHERE B.cor='vermelho'))
```

Exercício

- C8: Encontre os nomes dos marinheiros que reservaram todos os barcos

```
SELECT M.nome_marin  
FROM Marinheiros M  
WHERE NOT EXISTS ((SELECT B.id_barco  
                    FROM Barcos B)  
                  EXCEPT  
                  (SELECT R.id_barco  
                    FROM Reservas  
                    WHERE R.id_marin = M.id_marin))
```

Exercício

- C9: Encontre a idade do marinheiro mais jovem que pode votar (ou seja, tenha no mínimo 18 anos) para cada nível de avaliação com no mínimo dois marinheiros desse tipo

```
SELECT M.avaliacao, Min(M.idade) as minIdade  
FROM Marinheiros M  
WHERE M.idade >=18  
GROUP BY M.avaliacao  
HAVING COUNT(*) > 1
```

Exercício

- C10: Para cada barco vermelho, encontre o número de reservas desse barco

```
SELECT B.id-barco, COUNT(*) AS contagemR  
FROM Barcos B, Reservas R  
WHERE R.id-barco = B.id-barco AND  
       B.cor='vermelho'  
GROUP BY B.id-barco
```


Aula 14 – SBD

Dependências Funcionais e Normalização

Profa. Elaine Faria
UFU

Agradecimentos

Este material é baseado

- No livro R. Ramakrishnan e J. Gehrke, *Database Management Systems*
- Nos slides do prof. Bruno Nassif

- Agradecimentos

- Ao professor Bruno Travençolo que gentilmente cedeu seus slides

Refinamento de Esquema

- Problemas causados pela redundância
 - Armazenamento redundante
 - Algumas informações são armazenadas repetidamente
 - Anomalias de atualização
 - Se uma cópia dos dados redundantes é atualizada, é gerada uma inconsistência
 - Anomalias de inserção
 - Pode não ser possível armazenar certas informações, a não ser que outras informações não relacionadas sejam armazenadas
 - Anomalias de exclusão
 - Pode não ser possível excluir certas informações sem perder também algumas informações não relacionadas

Refinamento de Esquema

- Problemas causados pela redundância
 - Ex: Funcion_Horista (cpf, nome, vaga, avaliacao, salario_hora, horas_trabalhadas)
 - RI: salario_hora é determinado pela avaliação → é uma **dependência funcional**
 - Essa dependência funcional gera redundância

cpf	nome	vaga	avaliacao	salario_hora	horas_trabalhadas
123456	Artur	48	8	10	40
234567	Sandro	22	8	10	30
345678	Saulo	35	5	7	30
456789	Gustavo	35	5	7	32
567890	Manoel	35	8	10	40

Instância da relação Funcion_Horistas

- Se o mesmo valor aparece na coluna avaliacao de duas tuplas, a RI nos informa que o mesmo valor deve aparecer na coluna salario_hora
- Consequências negativas
 - Armazenamento redundante
 - Anomalias de atualização
 - Anomalias de inserção
 - Anomalias de exclusão

Refinamento de Esquema

- Valores Nulos
 - Podem tratar anomalias de inserção e de exclusão
 - Ex: Relação Funcion_Horistas
 - Uso de nulos na anomalia de inserção
 - Inserir uma tupla de funcionários com valores nulos no campo de salario_hora
 - Não pode-se registrar o salario_hora para uma avaliação a não ser que exista um funcionário com essa avaliação
→ cpf não pode ser nulo

Decomposições

- Redundância
 - Surge quando um esquema relacional impõe uma associação entre atributos que não é natural
 - Dependências funcionais podem ser usadas para identificar tais situações
 - Podem sugerir refinamentos para o esquema
- Ideia básica: muitos problemas provenientes da redundância podem ser resolvidos substituindo uma relação por um conjunto de relações “menores”

Decomposições

- Decomposição de um esquema de relação R
 - Consiste na substituição do esquema de relação por dois (ou mais) esquemas de relação, cada um contendo um subconjunto de atributos de R e, juntos, incluindo todos os atributos de R
 - Ex: Decompor Funcion_Horista em:

Funcion_Horista2(cpf, nome, vaga, avaliacao, horas_trabalhadas)
Salarios(avaliacao, salario_hora)

cpf	nome	vaga	avaliação	horas_trabalhadas
123456	Artur	48	8	40
234567	Sandro	22	8	30
345678	Saulo	35	5	30
456789	Gustavo	35	5	32
567890	Manoel	35	8	40

avaliacao	salario_hora
8	10
5	7

É possível:

- Registrar o salario_hora para qualquer avaliação
- Mudar o salario_hora associado a uma avaliacao apenas atualizando uma única tupla

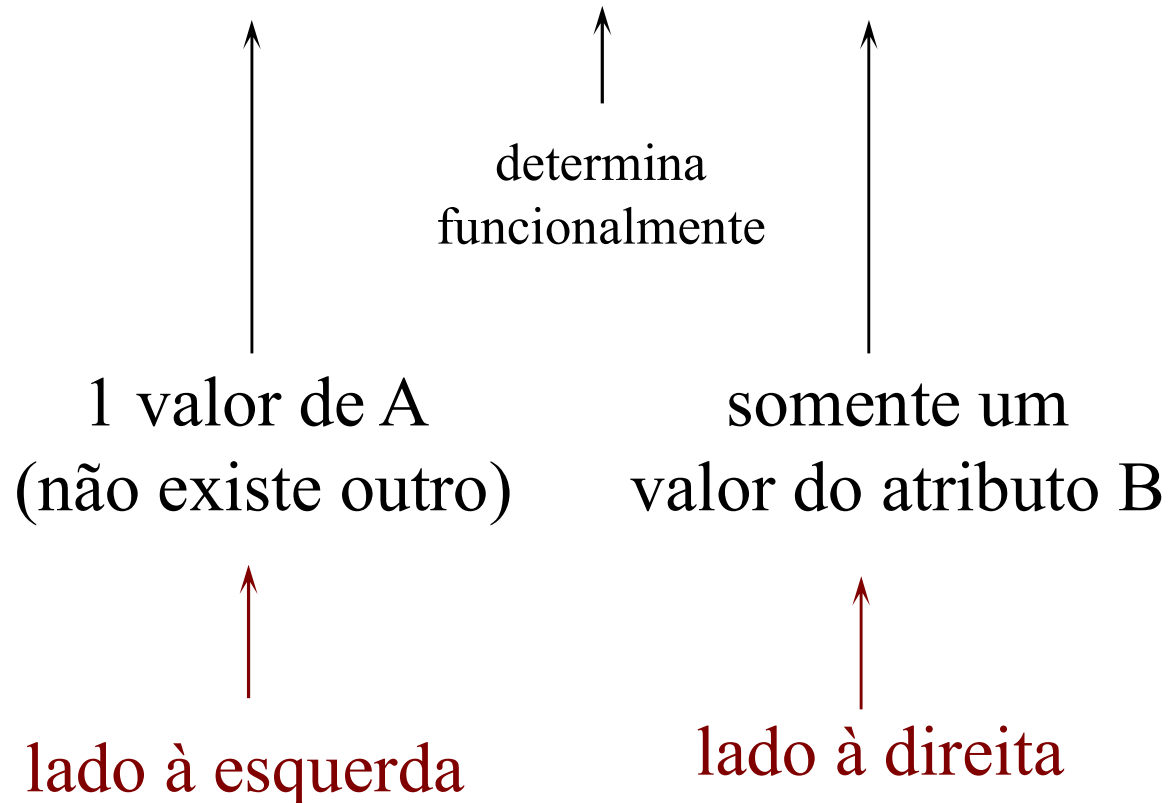
Dependência Funcional

- É uma **restrição** entre dois conjuntos de atributos de um esquema de relação R
 - restrição $\rightarrow$ impõe uma limitação nos valores possíveis de tuplas que formem um estado da relação r de R
- É uma propriedade da **semântica** ou do **significado dos atributos**
 - são derivadas pelo projetista do BD na análise da especificação de requisitos

Dependência Funcional

relação R (atributo A , atributo B , atributo C)

$\text{atributo } A \rightarrow \text{atributo } B$



Dependências Funcionais

- Um atributo Y de um esquema de relação R é funcionalmente dependente de um outro atributo X de R , se um valor de X determina um único valor de Y em qualquer momento
- Notação: $X \rightarrow Y$
- Se Y é funcionalmente dependente de X , então X determina funcionalmente Y

Dependências Funcionais

- Uma dependência funcional é uma *propriedade do esquema da relação R* , não de um estado particular válido da relação r de R
- $X \rightarrow Y$ diz que se duas tuplas concordam nos valores dos atributos X , elas também devem concordar nos valores dos atributos Y

Se $t1.X = t2.X$, então $t1.Y = t2.Y$

- Se $X \rightarrow Y$ em R , isso não implica necessariamente que $Y \rightarrow X$ em R

Dependências Funcionais

- Dependência Funcional (DF)
 - Certas DFs podem ser especificadas sem recorrer a uma relação específica, mas pelas propriedades de seus atributos
 - Os exemplos abaixo deveriam ser válidos para qualquer advogado ou engenheiro no Brasil:
 $\{\text{ESTADO}, \text{OAB}\} \rightarrow \text{NOME_ADVOGADO}$
 $\{\text{ESTADO}, \text{CREA}\} \rightarrow \text{NOME_ENGENHEIRO}$

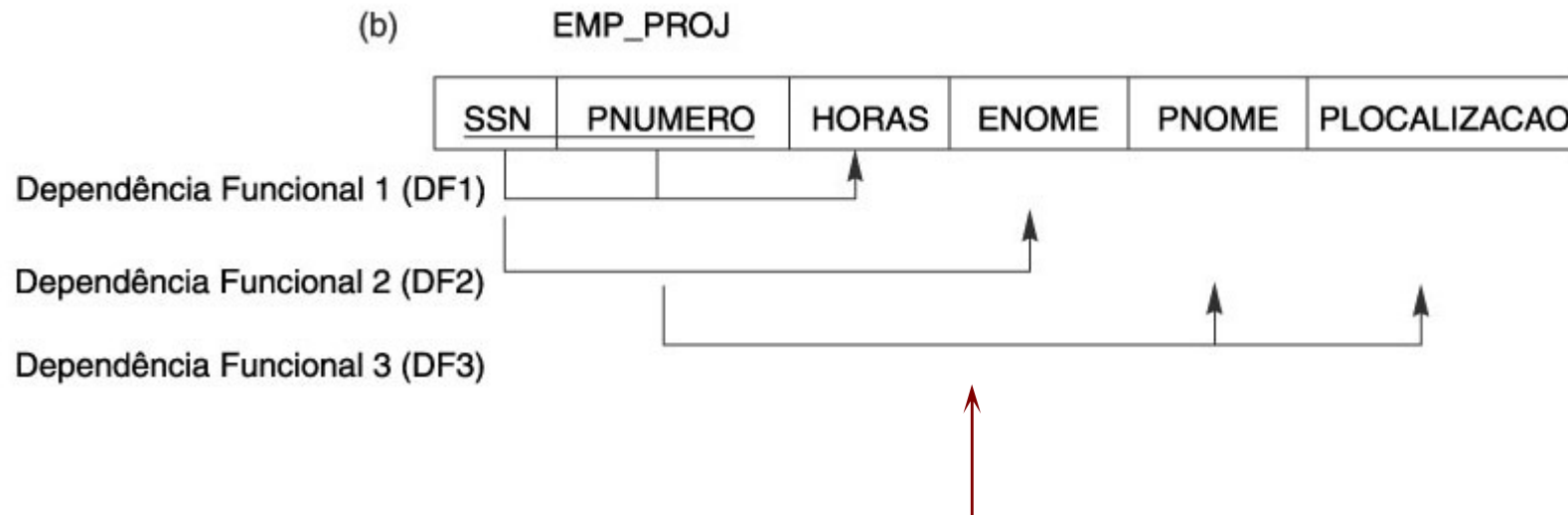
Dependências Funcionais

- Dependência Funcional (DF)
 - Também é possível que algumas DF possam deixar de existir
 - Exemplo:

PRIMEIROS_QUATRO_DÍGITOS → OPERADORA_CELULAR
com a portabilidade essa DF passou a não ser mais verdadeira

Dependência Funcional

- Exemplo:
 - $\{SSN, PNUMERO\} \rightarrow HORAS$
 - $SSN \rightarrow ENOME$
 - $PNUMERO \rightarrow \{PNOME, PLOCALIZACAO\}$



Notação diagramática para DF

Dependência Funcional

- Dada a seguinte relação
cliente (nro_cliente, nome, endereço)

As seguintes dependências são corretas?

- $\text{nro_cliente} \rightarrow \text{nome}$  OK!
- $\text{nro_cliente} \rightarrow \text{endereço}$ 
- $\text{nome} \rightarrow \text{endereço}$
- ~~$\text{endereço} \rightarrow \text{nome}$~~ Não!

Exercício 2

- Dada a seguinte relação (e suas instâncias)
linha_pedido (nro_pedido, nro_peça
qtidade_comprada, preço_cotado)

nro_pedido	nro_peça	qtidade_ comprada	preço_cotado
101	P01	3	30,00
101	P02	4	70,00
102	P01	8	80,00
102	P02	3	20,00

	nro_pedido	nro_peça	qtidade_comprada	preço_cotado
E	101	P01	3	30.00
	101	P02	4	70,00
	102	P01	8	80,00
	102	P02	3	20,00

- As seguintes dependências são verdadeiras?

~~– nro_pedido $\rightarrow$ qtidade_comprada~~

~~– nro_peça $\rightarrow$ qtidade_comprada~~

~~– nro_pedido $\rightarrow$ preço_cotado~~

~~– nro_peça $\rightarrow$ preço_cotado~~

Não!

OK!

- {nro_pedido, nro_peça} $\rightarrow$ qtidade_comprada
- {nro_pedido, nro_peça} $\rightarrow$ preço_cotado
- {nro_pedido, nro_peça} $\rightarrow$ {qtidade_comprada, preço_cotado}

Exemplo

DF: $AB \rightarrow C$

A	B	C	D
a1	b1	c1	d1
a1	b1	c1	d2
a1	b2	c2	d1
a2	b1	c3	d1

Observações

- DF não é o mesmo que restrição de chave (vide tuplas 1 e 2)
- Se duas tuplas diferem no campo A ou B, elas podem diferir no C
- Se for adicionada a tupla (a1,b1,c2,d1), a DF é violada

Dependências Funcionais

- Restrição de chave primária
 - É um caso especial de DF
 - Os atributos na chave desempenham o papel de X e o conjunto de todos os atributos na relação desempenha o papel de Y

Obs.:

A definição de uma DF não exige que o conjunto X seja mínimo

Normalização

- Processo de Normalização
 - Inicia com um esquema de relação ou conjunto de esquemas de relação
 - Produz uma nova coleção de esquemas de relação
 - Equivalente à coleção original (representa a mesma informação)
 - Livre de problemas

Problemas relacionadas à decomposição

- É preciso decompor uma relação?
 - Várias formas normais foram propostas
 - Se a relação está em uma dessas formas normais, certos tipos de problemas não podem surgir
- Propriedades das decomposições
 - Junção sem perda → recuperar qualquer instância da relação decomposta
 - Preservação da dependência → garante que cada dependência funcional será representada em algum esquema de relação resultante da decomposição

Problemas relacionadas à decomposição

- Desempenho
 - Consultas na relação original podem exigir junção das relações decompostas → penalidade no desempenho
 - Se a maioria das consultas e atualizações examinam apenas uma das relações decompostas → pode melhorar o desempenho

Formas Normais

- Formas normais (FN)
 - Dado um esquema é necessário decidir se ele é um bom projeto ou se é preciso decompô-lo em relações menores
 - Se um esquema de relação está em uma dessas FNs sabe-se que certos tipos de problemas não podem surgir
- Formas Normais baseadas em DFs
 - Primeira forma normal (1FN)
 - Segunda forma normal (2FN)
 - Terceira forma normal (3FN)
 - Forma normal de *Boyce-Codd* (FNBC)

Revisão: Chaves e Superchaves

- Uma superchave de uma relação R é um conjunto de atributos S contido em R
 - no qual não haverá duas tuplas t_1 e t_2 cujo
$$t_1[S] = t_2[S]$$

Revisão: Chaves e Superchaves

- Uma chave K é uma superchave com a propriedade adicional de que a remoção de qualquer atributo da chave fará com que K não identifique mais unicamente cada tupla da relação
 - a diferença é que uma chave tem que ser mínima

Revisão: Chaves e Superchaves

EMPREGADO			chave estrangeira (f.k.)	
ENOME	<u>SSN</u>	DATANASC	ENDERECO	DNUMERO
chave primária (p.k.)				

- Exemplo:
 - {SSN} é uma chave de empregado
 - Superchaves
 - {SSN, Enome}
 - {SSN, Enome, Datnasc}
 - {SSN, Enome, Datnasc, Endereço}
 - {SSN, Enome, Datnasc, DNumero}

Revisão: Chaves e Superchaves

- Chave candidata:
 - Se um esquema de relação tiver mais de uma chave, cada uma delas é chamada chave candidata
 - Uma delas é arbitrariamente designada para ser chave primária
- Um atributo de um esquema de relação R é chamado atributo primário se for membro de alguma chave candidata

Revisão: Chaves e Superchaves

EMPREGADO			chave estrangeira (f.k.)	
ENOME	<u>SSN</u>	DATANASC	ENDERECO	DNUMERO
chave primária (p.k.)				

- Exemplo:
 - {SSN} é a única chave candidata de empregado, portanto também é a chave primária

Revisão: Chave Primária

- Um atributo A (ou coleção de atributos) é a chave primária para um esquema de relação R se
 - todos os atributos em R são funcionalmente dependentes de A
 - não existe um subconjunto próprio de A que determina funcionalmente os atributos em R

Primeira Forma Normal (1FN)

- Uma relação está na primeira forma normal
 - Se todo campo contém apenas valores atômicos e monovalorados → nenhuma lista nem conjuntos
 - Requisito implícito na definição de modelo relacional usada nessa disciplina

Primeira Forma Normal (1FN)

- Exemplo

- cliente (nro_cli, nome, {end_entrega})

repetição

nro_cli	nome	end_entrega
124	João dos Santos	{Rua 10, 1024 Rua 24, 1356}
311	José Ferreira Neves	{Rua 46, 1344 Rua 98, 4456}

- cliente nem mesmo pode ser qualificado como uma relação ...

Métodos para Corrigir o Problema

- Método 1
 - Gerar uma nova relação contendo o grupo de repetição e a chave primária da relação original

Cliente_nome

<u>nro_cli</u>	nome
124	João dos Santos
311	José Ferreira Neves

Cliente_entrega

<u>nro_cli</u>	<u>end_entrega</u>
124	Rua 10, 1024
124	Rua 24, 1356
311	Rua 46, 1344
311	Rua 98, 4456

Métodos para Corrigir o Problema

- Método 2
 - substituir o grupo de repetição pelo número máximo de valores estabelecido para o grupo
 - **abordagem menos genérica e que pode introduzir muitos valores *null***

<u>nro cli</u>	nome	end_entrega1	end_entrega2
124	João dos Santos	Rua 10, 1024	Rua 24, 1356
311	José Ferreira Neves	Rua 46, 1344	Rua 98, 4456
025	Cecília Neves	Rua 77, 275	<i>null</i>

Primeira Forma Normal (1FN)

- Problema

- cliente (nro_cli, nome, {end_entrega})

Corrigindo o problema ...

- Solução 1

- cliente_nome (nro_cli, nome)

- cliente_entrega (nro_cli[↖], end_entrega)

- Solução 2

- cliente (nro_cli, nome, entrega1, entrega2)

Segunda forma normal (2FN)s

- Uma relação está na 2FN se
 - Está na 1FN
 - Todo atributo não chave deve ser totalmente dependentes da chave primária (dependente de toda chave e não de parte dela)

Segunda Forma Normal (2FN)

- Exemplo:
 - pedido (nro-pedido, data, nro-peça, descrição, quantidade_comprada, preço_cotado)

nro-pedido → data

nro-peça → descrição

{nro-pedido, nro-peça} → {quantidade_comprada, preço_cotado}

Segunda Forma Normal (2FN)

- Método para corrigir o problema:
 - Para cada sub-conjunto do conjunto de atributos que constitui a chave primária, começar uma relação com esse sub-conjunto como sua chave primária
 - Incluir os atributos da relação original na relação correspondente à chave primária apropriada, isto é, colocar cada atributo junto com a coleção mínima da qual ele depende, atribuindo um nome a cada relação

Segunda Forma Normal (2FN)

- Problema: pedido (nro-pedido, data, nro-peça,
descrição, qtdade_comprada,
preço_cotado)

Corrigindo o problema ...

- Solução:
pedido (nro-pedido, data)
peça (nro_peça, descrição)
pedido_peça (nro_pedido, nro_peça,
qtdade_comprada, preço_cotado)

Terceira Forma Normal

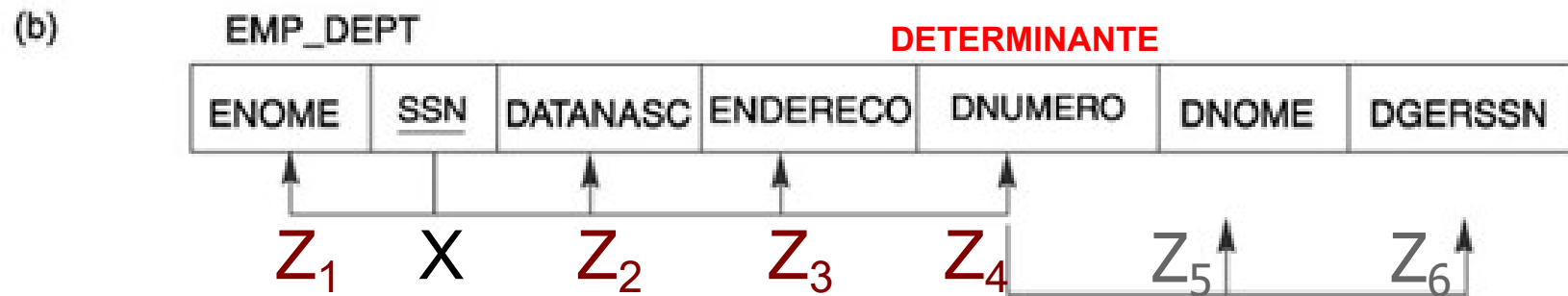
- Uma relação R está na 3FN se
 - Está na 2FN
 - Não existem atributos não chave que sejam dependentes de outros atributos não chave
 - Dependência transitiva

Terceira Forma Normal (3FN)

- Dependência transitiva $X \rightarrow Y$ em R
 - se $(X \rightarrow Z)$ e $(Z \rightarrow Y)$ e $(Z$ não for nem a chave candidata nem um subconjunto de qualquer chave de R)

Terceira Forma Normal (3FN)

- Exemplo de dependência transitiva



- DNOME e DGERSSN dependem funcionalmente de DNUMERO ($Z_4 \rightarrow \{Z_5, Z_6\}$)
- DNUMERO depende funcionalmente de SSN ($X \rightarrow \{Z_1, \dots, Z_4\}$)
 - DNUMERO não é chave, nem parte de chave
- DNOME e DGERSSN dependem transitivamente de SSN

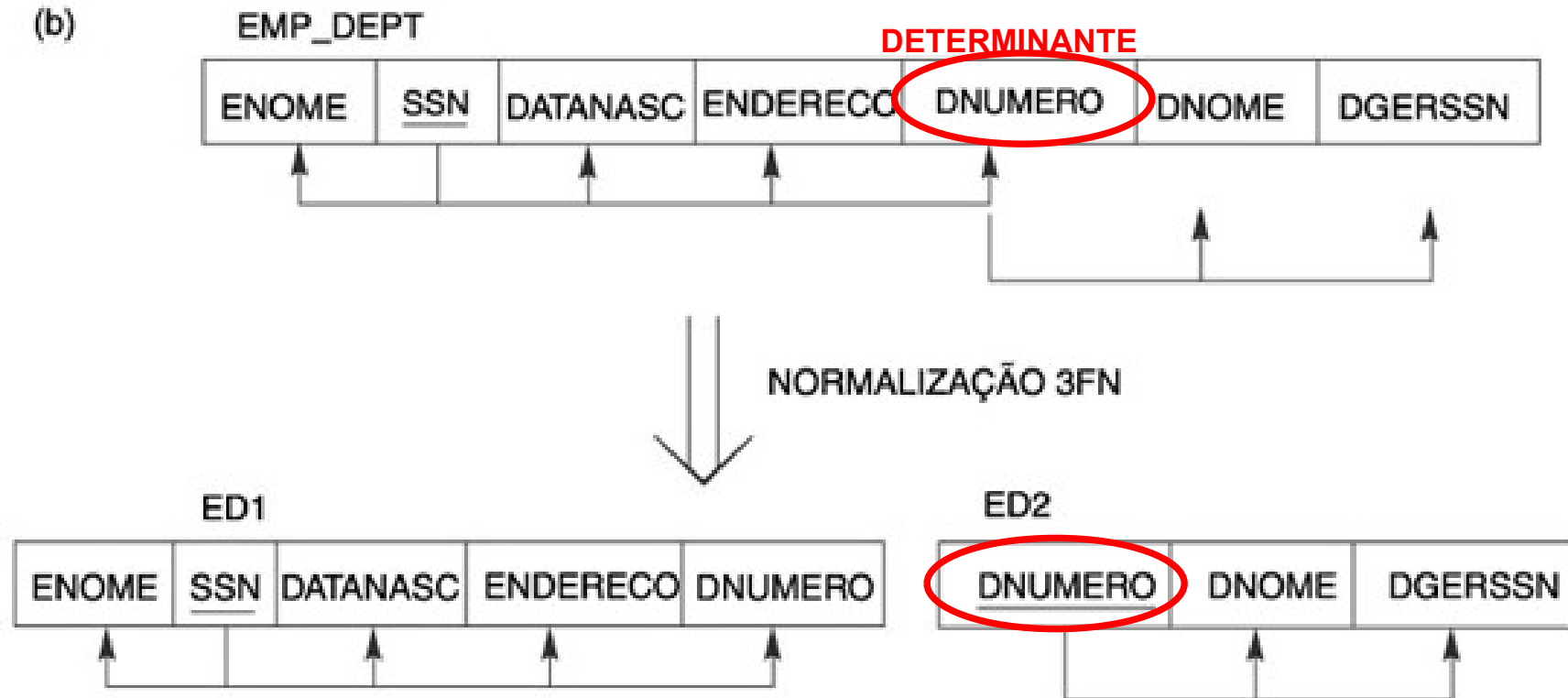
Terceira Forma Normal (3FN)

- Método para corrigir o problema:
 - para cada **determinante** que não é uma chave candidata, remover da relação os atributos que dependem desse determinante
 - criar uma nova relação contendo todos os atributos da relação original que dependem desse determinante
 - tornar o determinante a chave primária da nova relação

Terceira Forma Normal (3FN)

- Exemplo 1:

(b)



Terceira Forma Normal (3FN)

- Exemplo 2:
 - cliente (nro-cliente, nome-cliente, end-cliente, nro-vendedor, nome-vendedor)

nro-cliente → nome-cliente, end-cliente,
nro_vendedor

nro-vendedor → nome_vendedor

Terceira Forma Normal (3FN)

- Problema: cliente (nro-cliente, nome-cliente, end-cliente, nro-vendedor, nome-vendedor)

Corrigindo o problema ...

- Solução:
cliente (nro-cliente, nome-cliente, end-cliente, **nro-vendedor**)
vendedor (**nro-vendedor**, nome-vendedor)

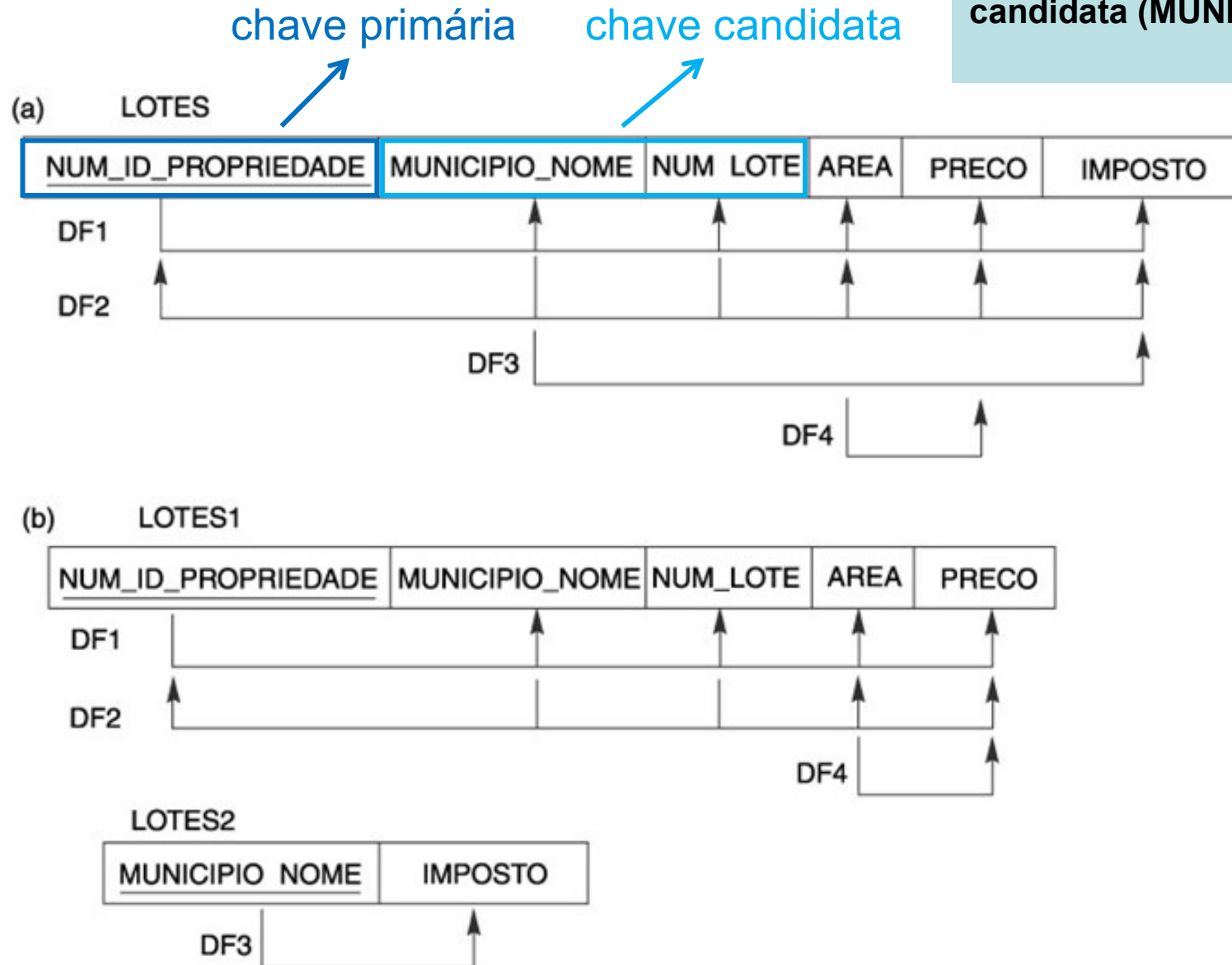
Definição Genérica

- Segunda forma normal
 - Um esquema de relação R está na 2FN se cada atributo não primário de R não for parcialmente dependente de nenhuma chave de R (chave primária ou chaves candidatas)

Definição Genérica

Segunda forma normal

Não está na 2FN: imposto depende de parte da chave candidata (MUNICIPIO_NOME)

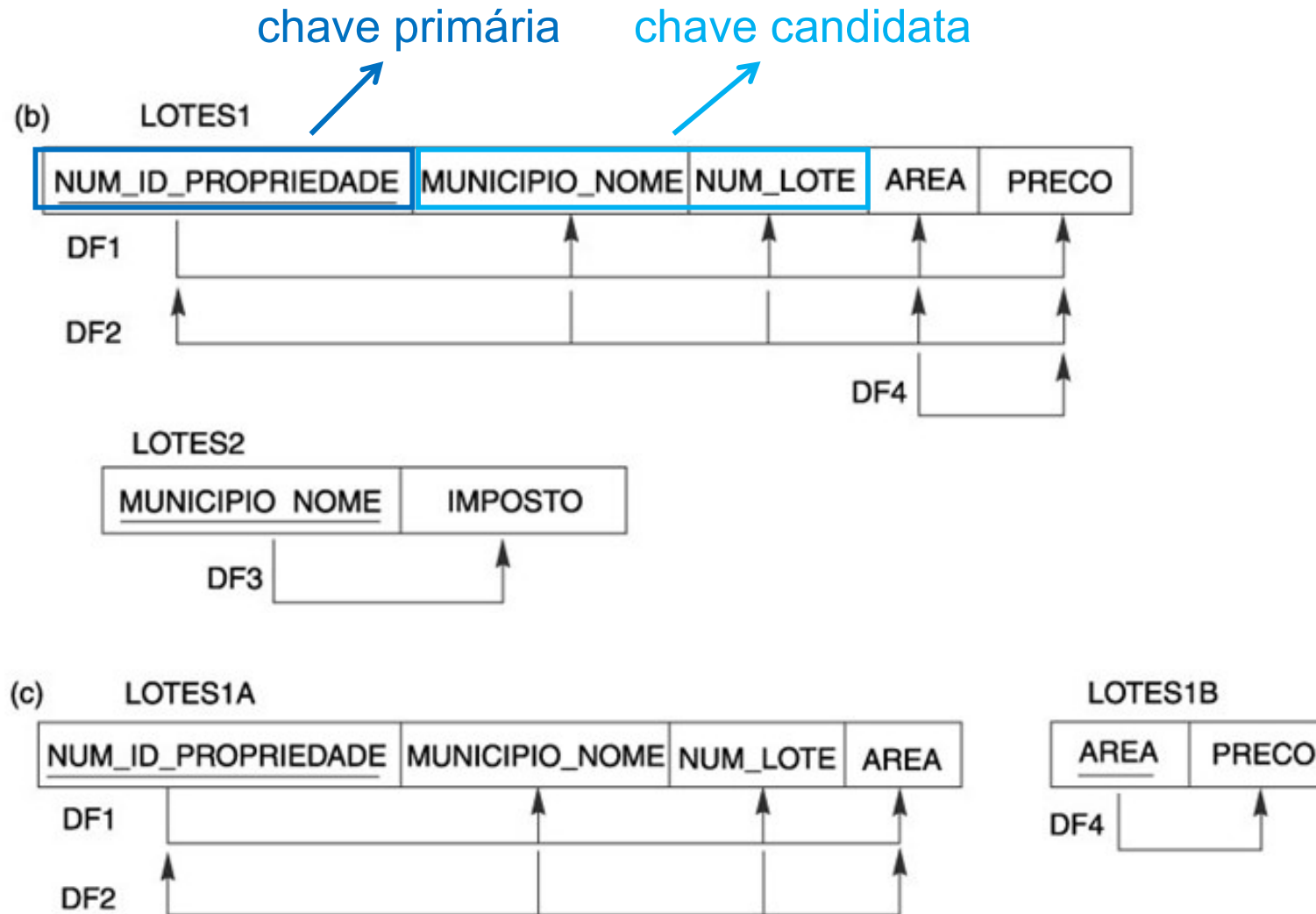


Definição Genérica

- Terceira forma normal
 - Um esquema de relação R está na 3FN se para cada dependência funcional $X \rightarrow A$, X é uma superchave de R ou A é um atributo primário de R

Definição Genérica

Terceira forma normal



Não está na 3FN: **PRECO** depende de **AREA** (ambos não chave)

Terceira Forma Normal

- Na 3FN é possível ocorrer alguma redundância

Se houver “Uma dependência não trivial $X \rightarrow A$ e X não for superchave e A for parte da chave”

Ex: Pedido com atributos CPDK (cliente, pedido, data e cartão) e DF: $C \rightarrow K$

- C não é chave e K não faz parte de nenhuma chave
 - A chave é CPD
 - A relação não está na 3FN e os pares (C, K) são armazenados de forma redundante
- Se DF: $K \rightarrow C$, ou seja, KPD também é chave
- $K \rightarrow C$ não viola a 3FN e Reservas está na 3FN
 - Contudo em todas as tuplas contendo o mesmo valor de K , o mesmo par (C, K) é gravado de forma redundante

Forma Normal de Boyce-Codd

- R está na forma normal de Boyce-Codd (FNBC) se
 - Para toda DF $X \rightarrow A$ em F , X é uma superchave, isto é $X \rightarrow R$.

Forma Normal de Boyce-Codd

- Em uma relação na FNBC
 - As únicas dependências são aquelas em que uma chave determina algum(ns) atributo(s)
 - É garantido que nenhuma redundância pode ser detectada usando-se apenas as informações das DFs
 - É a forma normal mais desejável do ponto de vista de redundância

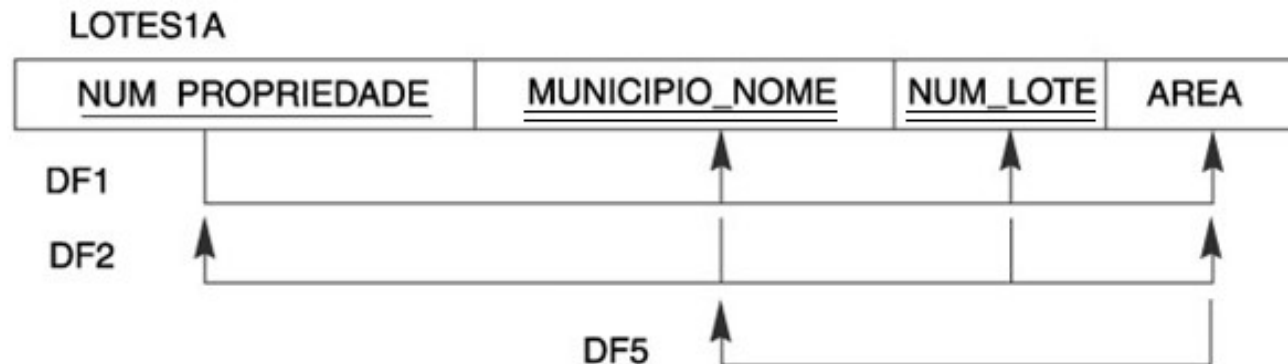
Exemplo

- Considere o esquema LOTES, que descreve lotes à venda em vários municípios
- Considere as chaves:
 - NUM_PROPRIEDADE
 - {MUNICIPIO_NOME, NUM_LOTE}



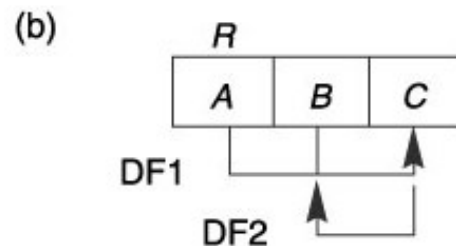
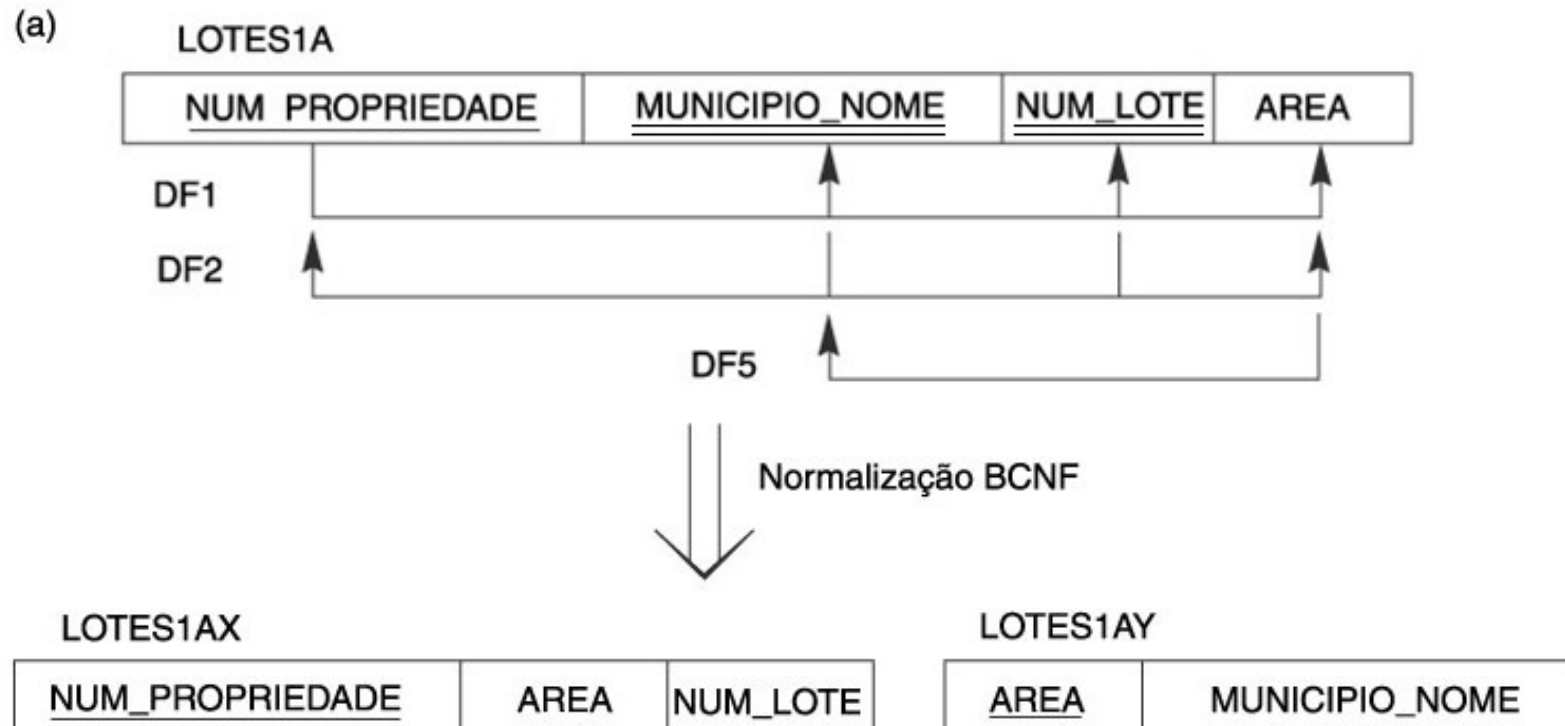
Exemplo

- Consideremos que há milhares de lotes, mas apenas nos municípios de Uberlândia e Araguari
 - em Uberlândia só existem lotes com 100, 200 e 300 m²
 - em Araguari só existem lotes com 150, 250 e 350 m²
- Nesse caso
 - DF5: AREA → MUNICIPIO_NOME



Forma Normal de Boyce-Codd

Exemplo



Uma Relação R esquemática com DF. Está na 3FN mas não na BCNF

Forma Normal de Boyce-Codd

X	Y	A
---	---	---

x	y1	a
x	y2	?

- Suponha a DF $X \rightarrow A$
 - O que podemos inferir sobre a coluna A na 2ª tupla?
R: usando a DF, conclui-se que a 2ª tupla também tem o valor a na coluna A
 - Uma situação assim pode surgir em uma relação na FNBC?
R: Não! Se uma relação está na FNBC porque A é diferente de X, segue-se que X deve ser uma chave
Se X é uma chave, então $y1 = y2$, o que significa que as duas tuplas são idênticas

Forma Normal

- Cada FN engloba a FN anterior:
 - Toda relação em 2FN está na 1FN
 - Toda relação em 3FN está na 2FN
 - Toda relação em BCNF está na 3FN
- Existem relações que estão na 3FN mas não em BCNF
- A meta é alcançar a BCNF (ou 3FN) em todas as relações

Considerações Finais

- A normalização para as FN apoiadas em DF se atinge com a separação dos atributos em duas ou mais relações
 - Isso aumenta o número de relações
 - Requer operações de junção na recuperação de informações
- Normalizar evita inconsistências nas relações, porém obriga a execução de operações de junção nas consultas

Considerações Finais

- Normalizar ou não uma relação?
 - O que é mais importante
 - garantir a eliminação de inconsistências no banco de dados ou a eficiência de acesso?
 - Se a consistência não for um fator fundamental pode-se abrir mão da normalização
 - em casos muito especiais
 - por exemplo em relações com uma quantidade crítica de linhas

Exercícios

- Diga em que forma normal (Nenhuma, 1 FN, 2 FN ou 3 FN) está cada relação abaixo, justificando sua resposta. Depois, se necessário, indique os passos que devem ser realizados para normalizar para a forma normal mais restrita possível.
- LIVROS = {Título, Autor, Tipo, Preço, {FiliaçãoDoAutor}, Editora}
 - Título → {Editora, Tipo}
 - Tipo → Preço
 - Autor → FiliaçãoDoAutor
- FORNECEDOR = {CNPJ, RazãoSocial, NomeFantasia, Contato}
 - CNPJ → {RazãoSocial, NomeFantasia, Contato}
- CLIENTE = {CPF, Nome, NroAgência, NroConta, TipoConta}
 - CPF → {Nome, NroAgência, NroConta, TipoConta}
 - {NroAgência, NroConta} → {CPF, Nome, TipoConta}
 - TipoConta → NroAgência

Exercícios

- LIVROS = {Título, Autor, Tipo, Preço, {FiliaçãoDoAutor}, Editora}
 - DF:
 - Título → {Editora, Tipo}
 - Tipo → Preço
 - Autor → FiliaçãoDoAutor

1) Não está na 1FN (atributo multivalorado)

LIVROS = {Título, Autor, Tipo, Preço, Pai, Mãe, Editora}

2) Não está na 2FN (existência de dependência parcial)

- LIVROS = {Título, Autor}
- DADOSLIVROS = {Título, Tipo, Preço, Editora}
- AUTOR = {Autor, Pai, Mãe}

3) Não está na 3FN (existência de dependência transitiva)

- LIVROS = {Título, Autor}
- DADOSLIVROS = {Título, Tipo, Editora}
- TIPO_LIVRO = {Tipo, Preço}
- AUTOR = {Autor, Pai, Mãe}

Exercícios

- ◆ FORNECEDOR = {CNPJ, RazãoSocial, NomeFantasia, Contato}
 - DF: CNPJ $\rightarrow$ {RazãoSocial, NomeFantasia, Contato}
 - 1FN, ok! Todos atributos atômicos
 - 2FN, ok! A chave primária é composta de apenas 1 atributo
 - 3FN, ok! Não existe dependência transitiva

Exercícios

- CLIENTE = {CPF, Nome, NroAgência, NroConta, TipoConta}
 - CPF $\rightarrow$ {Nome, NroAgência, NroConta, TipoConta}
 - {NroAgência, NroConta} $\rightarrow$ {CPF, Nome, TipoConta}
 - TipoConta $\rightarrow$ NroAgência
 - 1FN, ok! Todos atributos atômicos
 - 2FN, ok! Não existe dependência parcial
 - 3FN, ok! Não existe dependência transitiva

Exercícios

1. Diga em que forma normal (Nenhuma, 1FN, 2FN, 3FN) está cada relação abaixo, justificando sua resposta. Depois, se necessário, indique os passos que devem ser realizados para normalizar para a forma normal mais restrita possível.

a) CARROSVENDIDOS = {Carro, DataVenda, Vendedor, Comissão, Desconto}

■ DF:

- ◆ Carro → DataVenda
- ◆ DataVenda → Desconto
- ◆ Vendedor → Comissão

b) FILIAL = {CodF, País, Cidade, Continente, Língua, NomeGerente, FusoHorário, Nível}

■ DF:

- ◆ CodF → {País, Cidade, NomeGerente, Nível, FusoHorário}
- ◆ País → {Continente, Língua}

Exercícios

c. ProjetoEmpresa = {CodProj, Tipo, Descr, CodEmp, Nome, Cat, Sal, DataIni, TempAl}

– DF:

- { CodProj, CodEmp } $\rightarrow$ {DataIni, TempAl}
- CodProj $\rightarrow$ {Tipo, Descr}
- CodEmp $\rightarrow$ {Nome, Cat, Sal}
- Cat $\rightarrow$ Sal

d. Matricula = {CodAluno, CodTurma, CodDisciplina, CargaHoraria, Oferecimento, NomeDisciplina, NomeAluno, CodLocalNascAluno, NomeLocalNascAluno, Nota, Faltas}

– DF:

- CodAluno $\rightarrow$ {NomeAluno, CodLocalNascAluno, NomeLocalNascAluno}
- CodLocalNascAluno $\rightarrow$ NomeLocalNascAluno
- CodTurma $\rightarrow$ {CodDisciplina, NomeDisciplina, CargaHoraria, Oferecimento}
- CodDisciplina $\rightarrow$ {NomeDisciplina, CargaHoraria}
- {CodAluno, CodTurma} $\rightarrow$ {Nota, Faltas}

Referências

- R. Ramakrishnan e J. Gehrke, *Database Management Systems*, 3a Edição, McGraw-Hill, 2003.

Aula 15 – SBD

Stored Procedure

Profa. Elaine Faria
UFU

Agradecimentos

- Agradecimentos
 - Ao professor Bruno Travençolo que gentilmente cedeu seus slides

FUNCTIONS

- O PostgreSQL permite ao desenvolvedor estender as funcionalidade do servidor de BD por meio da criação de novas funções utilizando a linguagem C e carregando-as quando o servidor é iniciado.
- Uma extensão pode ser uma função bem simples, ou complexa como uma linguagem de programação. Essas linguagens permitem criar nossas próprias funções, conhecidas como **stored procedures**, de forma mais rápida e fácil que se fossem escritas em C.

FUNCTIONS

- Comando SQL: **CREATE FUNCTION**

CREATE FUNCTION *nome-da-função* ([<tipo_dado> [, ...]])

RETURNS <tipo_dado>

AS <definição>

LANGUAGE 'nome_linguagem'

<definição> é dada por uma string que pode conter várias linhas e pode ser escrita em qualquer linguagem suportada pelo PostgreSQL e carregada como uma linguagem procedural. A linguagem **PL/pgSQL** foi desenvolvida especialmente para criação de *stored procedures* no PostgreSQL

FUNCTIONS

- Outras linguagens procedurais que podem ser usadas.
Exemplos:
 - PL/Tcl
 - PL/Perl
 - PL/Python
- Quando uma função é criada, sua definição é armazenada no BD. Quando a função é chamada pela primeira vez, a definição é compilada e executada. Isso implica **que o programador não será avisado de erros enquanto não usar a função** (exceto para certos erros de sintaxe).

Exemplo

```
CREATE LANGUAGE plpgsql;
```

```
CREATE FUNCTION add_one (int4)
```

```
RETURNS int4 as '
```

```
BEGIN
```

```
    return $1 + 1; -- $1 é a representação para o 1o. argumento
```

```
END;' language 'plpgsql';
```

```
-- Como chamar o store procedure SELECT + nome +  
    parâmetros
```

```
SELECT add_one(47);
```

PL/pgSQL *stored procedures*

- PL/pgSQL é uma linguagem estruturada por blocos. Sintaxe

```
[<<label>>]  
[DECLARE declarations]  
BEGIN  
    statements  
END;
```

Isso é um bloco. Ele contém um rótulo (*label* – opcional), uma seção de declaração de variáveis (DECLARE), e a declaração de início e fim de bloco (BEGIN-END)

-- For PostgreSQL 8.0 and later

```
CREATE FUNCTION name ( [ [arg] ftype [, ...] ] )
```

```
RETURNS rtype AS
```

```
$$
```

```
block definition
```

```
$$ LANGUAGE plpgsql;
```

Argumentos da Função

- Uma função pode ter 0 ou mais parâmetros
- No corpo da função os parâmetros são referidos como \$1, \$2,...

-- geom_avg

-- Obtém a média geométrica de dois inteiros

CREATE FUNCTION geom_avg(int4, int4)

RETURNS float8 AS

\$\$

BEGIN

 RETURN SQRT(\$1 * \$2::float8);

END;

\$\$ language plpgsql;

Argumentos da Função

- A partir do Postgre 8.0 é possível dar nomes para os parâmetros
- Toda *store procedure* deve retornar um valor
 - Mas pode retornar VOID

-- *geom_avg*

-- *Obtém a média geométrica de dois inteiros*

```
CREATE FUNCTION geom_avg(valor1 int4, valor2 int4)
```

```
RETURNS float8 AS
```

```
$$
```

```
BEGIN
```

```
    RETURN SQRT(valor1 * valor2::float8);
```

```
END;
```

```
$$ language plpgsql;
```

Declarações de variáveis

- Declaração:

nome [CONSTANT] tipo [NOT NULL] [{ DEFAULT | := } value];

- Exemplo

- n **INTEGER** := 2; -- declaração com inicialização
- Valor_de_PI **CONSTANT FLOAT8** := pi();

Declarações de variáveis - TIPOS

- Copiando tipos
- O tipo da variável pode ser especificado utilizando um tipo de outro item da base de dados

*nome_da_variável***%TYPE**

- Exemplos

Variável declarada anteriormente

n **INTEGER** := 2

Declarando outra variável do mesmo tipo

n2 **n%TYPE**; -- n2 é do mesmo tipo tipo n, ou seja, inteiro.

Declarações de variáveis - TIPOS

- O tipo da variável pode ser especificado utilizando um tipo já definido em um atributo de uma tabela

*nome_da_tabela.nome_da_coluna***%TYPE**

- Exemplos

Tabela empregado

```
CREATE TABLE empregado(nome VARCHAR(255),  
                          salario DECIMAL(10,2)..
```

Declarando uma variável baseando-se no tipo de um atributo

```
Soma_Salarios empregado.salario%TYPE; -- Soma_Salarios é do  
mesmo tipo do atributo salário da tabela empregado, ou seja,  
DECIMAL(10,2)
```


Declarações de variáveis

```
DECLARE
```

```
n1 integer;
```

```
n2 integer;
```

```
BEGIN
```

```
    -- pode-se usar n1 e n2 aqui
```

```
    n2 := 1;
```

```
    DECLARE
```

```
    n2 integer; -- esconde a variável n2 definida anteriormente
```

```
    n3 integer;
```

```
    BEGIN
```

```
        -- n1, n2 e n3 estão disponíveis aqui
```

```
        n2 := 2;
```

```
    END;
```

```
    -- n3 não está mais disponível
```

```
    -- n2 ainda possui valor 1
```

```
END;
```

SELECT INTO

- Variação do comando SELECT que permite atribuir o resultado de uma consulta a uma variável
- Vale lembrar que essa variação é própria para *store procedures*
- *Sintaxe:*
SELECT expressão INTO variável [FROM ..];

Obs: existe o comando SELECT INTO fora do pl/pgSQL. Ele serve para criar tabelas a partir de uma consulta. Mas não é recomendado. Veja mais em:

<http://www.postgresql.org/docs/8.1/static/sql-selectinto.html>

Exemplo 2

-- obtém o próximo número de matrícula (NMat)

CREATE OR REPLACE FUNCTION NOVO_NMat ()

RETURNS int4 AS

\$\$

DECLARE UltimoNumero INTEGER;

BEGIN

SELECT max(NMat)

FROM ALUNO

INTO UltimoNumero; -- joga max(Nmat) na variável UltimoNumero

RETURN (UltimoNumero + 1);

END;

\$\$ language 'plpgsql';

Exemplo 2

- Return: retorna um valor de uma função:
RETURN expression;

```
-- obtém o próximo número de matrícula (NMat)
CREATE FUNCTION NOVO_NMat ()
RETURNS int4 AS
$$
DECLARE UltimoNumero INTEGER;
BEGIN
    SELECT max(NMat) FROM ALUNO INTO UltimoNumero;
    RETURN (UltimoNumero + 1);
END;
$$ language 'plpgsql';
```

Exceções e Mensagens

- Ao encontrar uma condição que impossibilita continuar a execução de uma *store procedure* pode-se usar exceções

Table 10-11. *PostgreSQL Exception Levels*

Level	Behavior
DEBUG, LOG, INFO	Writes a message in the log (usually suppressed)
NOTICE, WARNING	Writes a message in the log and sends it to the application
EXCEPTION	Writes a message in the log and terminates the stored procedure

Exceções e Mensagens

- Exemplo
 - DEBUG

-- Escreve o valor da variável n em um *log*

```
CREATE FUNCTION scope()  
RETURNS integer AS  
$$  
DECLARE n INTEGER := 4;  
BEGIN  
    RAISE DEBUG 'O valor de N é %', n;  
    RETURN (n);  
END;  
$$ language 'plpgsql';
```

Exceções e Mensagens

- Exemplo

- NOTICE

- Mostra o valor da variável n (envia a mensagem para o aplicativo) e continua a execução do procedure

```
CREATE OR REPLACE FUNCTION scope()
```

```
RETURNS integer AS
```

```
$$
```

```
DECLARE n INTEGER := 4;
```

```
BEGIN
```

```
    RAISE NOTICE 'O valor de N é %', n;
```

```
    RETURN (n);
```

```
END;
```

```
$$ language 'plpgsql';
```

Exceções e Mensagens

- Exemplo

- EXCEPTION

-- Mostra o valor de n (escreve no log) e TERMINA A EXECUÇÃO NO PONTO

CREATE OR REPLACE FUNCTION scope()

RETURNS integer **AS**

\$\$

DECLARE n **INTEGER** := 4;

BEGIN

RAISE EXCEPTION 'O valor de N é %', n;

RETURN (n);

END;

\$\$ language 'plpgsql';

Condicionais

- **IF-THEN-ELSE**

```
IF expression
THEN
    statements
[ELSE
    statements]
END IF;
```
- **NULLIF(*input*, *value*)**
 - *retorna NULL se input=value*
- **CASE**

```
CASE
    WHEN expression
    THEN expression
    ...
ELSE expression
END;
```

```
res := CASE
    WHEN n2 = 1
    THEN 5
    WHEN n2 = 2
    THEN 6
    ELSE 7
END;
```

Condicionais

IF *boolean-expression* THEN

statements

[ELSIF *boolean-expression* THEN

statements

[ELSIF *boolean-expression* THEN

statements

...]]

[ELSE

statements]

END IF;

LOOPS

- LOOP

[<<*label*>>]

LOOP

statements

END LOOP;

- Para sair do loop

EXIT [*label*] [*WHEN expression*];

- Exemplo

<<indefinite>>

LOOP

n := n + 1;

EXIT indefinite WHEN n >= 10;

END LOOP;

LOOPS

- WHILE

```
[<<label>>]  
WHILE expression  
LOOP  
    statements  
END LOOP;
```

- FOR

```
FOR name IN [REVERSE] from .. to  
LOOP  
    statements  
END LOOP;
```

```
[ <<label>> ]  
FOR target IN query  
LOOP  
    statements  
END LOOP [ label ];
```

LOOPS

- FOR – exemplo para percorrer todas as tuplas

DECLARE

tupla record;

cid integer;

...

FOR cid IN 1 .. 15

LOOP

SELECT * INTO tupla – joga o resultado em tupla

FROM clientes

WHERE cliente_id = cid;

-- processamento sobre o cliente

RAISE NOTICE 'Cliente: %', tupla.nome;

...

END LOOP;

LOOPS

- FOR – exemplo para percorrer todas as tuplas

DECLARE

tupla record;
cid integer

...

FOR cid

LOOP

SE

* IN

FR

clien

W

cliente_id =

-- p

amento sobre

RAIS

CE

'Cliente:

nome;

...

END LOOP;

- joga o resultado em tupla

Não use assim!

LOOPS

- FOR – exemplo para percorrer todas as tuplas

```
DECLARE
```

```
    tupla record; n_cliente integer
```

```
....
```

```
SELECT COUNT(*)
```

```
FROM clientes
```

```
INTO n_clientes -- obtém o número de clientes
```

```
FOR cid IN 1 .. 15n_clientes
```

```
LOOP
```

```
    SELECT * INTO tupla-- joga o resultado em tupla
```

```
    FROM clientes
```

```
    WHERE cliente_id = cid;
```

```
    -- processamento sobre o cliente
```

```
    RAISE NOTICE 'Cliente: %', tupla.nome;
```

```
...
```

```
END LOOP;
```

LOOPS

- FOR – exemplo para percorrer todas as tuplas

DECLARE

tupla record; n_cliente integer

....

SELECT CO

FROM cli

INTO n_

FOR c

LOOP

SEL

FR

WH

-- proc

RAISE NO

...

END LOOP;

Não use assim!

LOOPS

- FOR – exemplo para percorrer todas as tuplas

DECLARE

tupla record;

....

~~FOR cid IN 1 .. n_clientes~~

FOR tupla IN SELECT * FROM clientes

LOOP

~~— SELECT * INTO tupla — joga o resultado em tupla~~

~~— FROM clientes~~

~~— WHERE cliente_id = cid;~~

-- processamento sobre o cliente

RAISE NOTICE 'Cliente: %', tupla.nome;

...

END LOOP;

LOOPS

- FOR – exemplo

-- listar o nome e idade de todos os alunos via mensagem no programa aplicativo

```
DECLARE
```

```
    tupla record;
```

```
BEGIN
```

```
FOR tupla IN SELECT * FROM aluno
```

```
LOOP
```

```
    RAISE NOTICE 'Nome: %, Idade: %', tupla.nome, tupla.idade;
```

```
END LOOP;
```

```
END;
```

LOOPS

- OBS: no FOR com SELECT, o comando SELECT é feito somente uma vez e não a cada passo do LOOP. Você pode, inclusive apagar os dados da tabela que estão no SELECT do loop e continuar o loop normalmente